Learn RISC-V CPU Implementation and BSV

RISC-V: a major open-standard Instruction Set Architecture
BSV: a modern, open-source High-Level Hardware Design Language

Rishiyur S. Nikhil

Bluespec, Inc.



© 2023-2025 Rishiyur S.Nikhil

DRAFT: June 28, 2026
Please do not circulate without the author's permission.

Acknowledgements: BSV

The original ideas for a rule-based computation model for hardware design were developed at MIT by James Hoe (currently Professor, Carnegie Mellon University) and Arvind (Professor, MIT). The ideas behind CRegs (concurrent registers) were also developed later at MIT by Daniel Rosenband and Arvind.

The idea of embedding this computation model in a Haskell-like language (including types, typeclasses, static elaboration computation model, higher-order programming, and monadic static elaboration) was due to Lennart Augustsson.

The embedding in SystemVerilog-like syntax, technical refinement of the formal semantics of BSV, extension to multiple clock domains and implementation and improvements of the *bsc* compiler is due to several employees (including the author) of Sandburst, Inc. and Bluespec, Inc. since 2003.

Thanks to all the employees of Bluespec, Inc. and to all the users of BSV over the years— commercial, academic and independent— for their feedback and insights on BSV and how to teach it.

Thanks to Bluespec, Inc., for agreeing to open-source BSV with its *bsc* compiler and libraries in 2020.

Acknowledgements: RISC-V

Thanks to the team from Universithy of California, Berkeley led by Professor Krste Asanović for their tremendous gift to the world of the free and open RISC-V specification, as a sophisticated, industrial-strength, ISA.

Thanks to Bluespec, Inc., for supporting the author in creating previous RISC-V CPU BSV designs (Flute, Piccolo, and Magritte), C simulators for RISC-V (Cissr V1 and V2) and FPGA-based RISC-V Systems (including Catamaran), all of which have greatly informed the new designs Fife and Drum in this book.

Short Table of Contents

Detailed Table of Contents vi

List of Figures xiv

1 Introduction 1-1

2 RISC-V: Overview of the RISC-V ISA 2-1

3 BSV: Overview of the BSV program structure 3-1

4 BSV: Combinational circuits for the RISC-V Stage functions 4-1

5 BSV: Struct types and tuples; and
RISC-V: Memory requests & responses 5-1

6 RISC-V: Core functions for ISA execution (used in Drum and Fife) 6-1

7 BSV: Modules and Interfaces: Registers, Register Files and FIFOs 7-1

8 RISC-V: Modules for GPRs and CSRs 8-1

9 BSV: Finite State Machines (FSMs) using `StmtFSM` 9-1

10 RISC-V: the Drum CPU
(unpipelined; FSM-sequenced) 10-1

11 BSV: Verifying BSV designs 11-1

12 RISC-V: Functional verification of CPUs 12-1

13 BSV: Rules and their Semantics 13-1

14 RISC-V: the Drum unpipelined CPU (using Rules instead of `StmtFSM`) 14-1

15 RISC-V: the Fife pipelined CPU: Principles 15-1

16 RISC-V: the Fife pipelined CPU code 16-1

17 PBT: Property-based Testing 17-1

18 BSV: Rules and Methods II: Improved performance with CRegs (Concurrent Registers) 18-1

19 RISC-V: Optimizing Drum and Fife 19-1

20 BSV: Suggested further study 20-1

21 RISC-V: Suggested further study 21-1

A Resources: Documents and Tools A-1

B Why BSV? B-1

C Glossary C-1

D BSV: Importing C/C++ functions into BSV simulations D-1

E Exercises E-1
BSV Index INDEX-BSV-1
RISC-V Index INDEX-RISC-V-1
Bibliography BIB-1

Detailed Table Of Contents

Detailed Table of Contents vi
List of Figures xiv
1 Introduction 1-1
1.1 Goals . . . 1-1
1.2 The plan, in more detail . . . 1-2
1.3 **Drum** and **Fife**, our two implementations . . . 1-4
1.3.1 The Design Space for CPU implementations . . . 1-4
1.3.2 ISA executed by **Drum** and **Fife** . . . 1-5
1.3.3 **Drum** and **Fife** . . . 1-6
1.3.4 **Drum** and **Fife** source codes . . . 1-7
1.4 About **BSV**, the HL-HDL we use for our designs . . . 1-8
1.5 Basic model of digital hardware needed for this book . . . 1-8
1.5.1 Clocks and State Elements . . . 1-9
1.5.2 Bit-vectors, values and buses . . . 1-9
1.5.3 Digital circuits . . . 1-9
1.5.4 Computer memories . . . 1-10
1.5.4.1 Memory hierarchies . . . 1-11
1.5.4.2 Memory technologies for the memory hierarchy . . . 1-12
1.5.4.3 Split-phase memory access . . . 1-12
1.5.4.4 Memory pipelining and memory ordering . . . 1-12
1.6 Additional Resources for this Textbook/Course . . . 1-13
2 RISC-V: Overview of the RISC-V ISA 2-1
2.1 Introduction . . . 2-1
2.2 What is an ISA? . . . 2-1
2.3 Why choose RISC-V? . . . 2-3
2.4 Overview of the RISC-V ISA . . . 2-4
2.5 Instruction encodings . . . 2-6
2.6 Unprivileged ISA RV32I . . . 2-7
2.6.1 "Upper Immediate" instructions LUI and AUIPC . . . 2-7
2.6.2 Conditional BRANCH instructions . . . 2-9
2.6.3 LOAD and STORE memory-access instructions . . . 2-10
2.6.4 Register-Register Arithmetic and Logic instructions . . . 2-10
2.6.5 Register-Immediate Arithmetic and Logic instructions . . . 2-11
2.6.6 Unconditional Jump instructions . . . 2-11
2.6.7 FENCE . . . 2-12
2.7 Traps due to illegal instructions and other exceptions, and CSRs . . . 2-12
2.7.1 ECALL and EBREAK instructions, and Interrupts . . . 2-14
2.7.2 CSRRxx instructions . . . 2-14
2.7.2.1 CSRs INSTRET and INSTRETH . . . 2-15
2.7.2.2 CSRs CYCLE and CYCLEH . . . 2-16
2.7.2.3 CSRs TIME and TIMEH, and memory-mapped location MTIME 2-16
2.7.2.4 Measuring CPU performance . . . 2-16
2.8 RV64I differences from RV32I . . . 2-17
2.9 Abstract algorithm for interpreting an ISA . . . 2-17

2.10 Continued Evolution of the RISC-V ISA (with your contribution?) . . . 2-19
3 BSV: Overview of the BSV program structure 3-1
3.1 Introduction . . . 3-1
3.2 A Minimal **BSV** program . . . 3-1
3.2.1 More about `$display()` . . . 3-3
3.3 Packages and files . . . 3-3
3.3.1 What's in a Package? . . . 3-3
3.3.2 Extending our example minimal **BSV** program . . . 3-5
3.3.3 Visibility of names, exports and imports . . . 3-7
3.3.4 Resolving ambiguous imports . . . 3-7
3.3.5 Exporting types abstractly . . . 3-8
3.4 Interface and Module Declarations . . . 3-8
3.4.1 What's in an interface declaration? . . . 3-8
3.4.2 What's in a module declaration? . . . 3-9
3.4.3 Extending our example minimal **BSV** program further . . . 3-10
3.5 Rules and Interface Definitions . . . 3-11
3.5.1 What's in a rule? . . . 3-11
3.5.2 What's in an interface definition? . . . 3-12
3.6 Static Elaboration and Hardware Module Structure . . . 3-13
3.6.1 Module interaction *via* methods . . . 3-14
3.7 Conclusion . . . 3-14
4 BSV: Combinational circuits for the RISC-V Stage functions 4-1
4.1 Introduction . . . 4-1
4.2 Hexadecimal and Binary Notation for literal integers . . . 4-2
4.3 Syntax of Identifiers . . . 4-2
4.4 Syntax of comments . . . 4-3
4.5 Introduction to Types . . . 4-3
4.6 Bit vectors, and declaring identifiers . . . 4-5
4.6.1 "`let`" bindings—having *bsc* infer the identifier's type . . . 4-5
4.6.2 Don't-care values . . . 4-6
4.6.3 Built-in Operators on Bit Vectors . . . 4-7
4.7 The `Bool` type and boolean values . . . 4-8
4.7.1 Caution: `Bool`, `Bit#(1)` and `Int#(1)` are distinct types . . . 4-8
4.8 Integer types . . . 4-8
4.9 Functions . . . 4-9
4.9.1 More about **BSV** functions . . . 4-10
4.9.1.1 **BSV** functions describe both structure and computation . . . 4-10
4.9.1.2 **BSV** functions are inlined and have zero incremental hardware cost 4-10
4.9.1.3 **BSV** functions can be recursive . . . 4-11
4.9.2 **BSV** function argument and result types are not just for data . . . 4-11
4.10 Example: recognizing legal RISC-V BRANCH instructions . . . 4-11
4.10.1 Combinational circuits and primitives . . . 4-12
4.10.1.1 Combinational circuits have no side-effects (are "pure") . . . 4-13
4.10.2 Encapsulating the code in functions . . . 4-13
4.11 Pure functions vs. functions with side-effects (`Action`, `ActionValue`) . . . 4-14
4.11.1 Combinational circuits = "doesn't have `Action` or `ActionValue` type" . . . 4-15
4.11.2 Using `ActionValue` on pure functions for `$display` debugging . . . 4-15
4.12 `StmtFSM`: a useful facility for testbenches . . . 4-15
4.13 User-defined types: `enum` types . . . 4-16
4.13.1 `deriving (Bits)` . . . 4-16
4.13.2 `deriving (Eq)` . . . 4-16
4.13.3 `deriving (FShow)` . . . 4-17
4.14 if-then-else statements and hardware multiplexers . . . 4-17
4.14.1 Parallel multiplexers and MUX synthesis . . . 4-19

4.15 Case-expressions . . . 4-19
4.16 Sharing code for RV32 and RV64 *via* parameterization . . . 4-20
4.16.1 Using Type Synonyms and Macros to Parameterize for RV32 and RV64 . . . 4-21
4.16.1.1 Type synonyms . . . 4-21
4.16.1.2 Macros, preprocessing, and conditional compilation . . . 4-22
5 BSV: Struct types and tuples; and RISC-V: Memory requests & responses 5-1
5.1 RISC-V: structs communicated between stages . . . 5-1
5.2 BSV: `struct` types . . . 5-2
5.2.1 Creating (constructing) struct values . . . 5-2
5.2.2 Selecting struct fields . . . 5-3
5.2.3 Updating struct fields using assignment . . . 5-3
5.2.4 Representation of a struct in bits . . . 5-4
5.2.5 Printing a struct for logging/debugging . . . 5-4
5.2.6 Memory request type . . . 5-5
5.2.7 Memory request size . . . 5-6
5.3 BSV: Tuples and the `match` statement . . . 5-6
5.4 RISC-V: Memory Requests and Responses; IMem and DMem . . . 5-7
5.4.1 Separation of IMem and DMem (Harvard Architecture) . . . 5-7
5.4.2 Address Alignment . . . 5-8
5.4.3 Memory Responses . . . 5-9
6 RISC-V: Core functions for ISA execution (used in Drum and Fife) 6-1
6.1 Introduction . . . 6-1
6.2 The function `fn_Fetch` . . . 6-1
6.3 The `fn_Decode` function . . . 6-3
6.4 The `fn_Dispatch` function after reading input registers . . . 6-8
6.5 The `fn_EX_Control` function . . . 6-12
6.6 The `fn_EX_Int` function . . . 6-13
6.7 No shared functions for Execute DMem and Retire . . . 6-17
7 BSV: Modules and Interfaces: Registers, Register Files and FIFOs 7-1
7.1 Introduction . . . 7-1
7.2 Modules: state, interfaces and behavior . . . 7-1
7.2.1 Internal behavior (*rules*) . . . 7-2
7.2.2 Interface declarations . . . 7-2
7.2.2.1 Hardware for an interface . . . 7-3
7.2.3 Module definitions . . . 7-4
7.2.4 Module instantiation and method invocation . . . 7-4
7.3 **BSV** Library Modules: Registers . . . 7-4
7.3.1 `Reg#(t)`, the register interface from the **BSV** library . . . 7-5
7.3.2 Registers are strongly typed . . . 7-5
7.3.3 `mkReg(`v`)` and `mkRegU`: register module (constructor) from the **BSV** library . . . 7-5
7.3.4 Syntactic shorthands for register access . . . 7-6
7.4 **BSV** Library Modules: Register files . . . 7-7
7.4.1 The register file interface `RegFile#(index_t,data_t)` . . . 7-7
7.4.2 `mkRegFileFull`, a register file module (constructor) . . . 7-7
7.5 **BSV** Library Modules: FIFOs . . . 7-8
7.5.1 `FIFOF#(`t`)`, the FIFO interface type . . . 7-8
7.5.1.1 `pop`: a useful function combining `first` and `deq` . . . 7-8
7.5.2 FIFO modules (constructors): `mkFIFOF` and more . . . 7-9
7.5.2.1 `mkPipelineFIFOF` and `mkBypassFIFOF` . . . 7-9
7.5.2.2 `mkSizedFIFOF`: explicitly depth-sized FIFOs . . . 7-9
7.5.2.3 Initial state of FIFOs after reset . . . 7-10
7.5.3 FIFOs are strongly typed . . . 7-10
7.5.4 Semi-FIFO interfaces for each end of a FIFO . . . 7-10

7.5.5 Interface-transformer functions . . . 7-11
7.5.6 Connecting FIFOs . . . 7-11
7.5.7 Connecting separately compiled pipeline stages . . . 7-13
7.6 Polymorphic and Monomorphic Types . . . 7-14
7.6.1 Polymorphic Modules and Synthesizability into Verilog . . . 7-14
8 RISC-V: Modules for GPRs and CSRs 8-1
8.1 Introduction . . . 8-1
8.2 A register file for GPRs, with special treatment of register 0 (`x0`) . . . 8-1
8.2.1 Inlined *vs.* separate module `mkGPRs` . . . 8-3
8.3 A register file for RISC-V CSRs . . . 8-3
9 BSV: Finite State Machines (FSMs) using `StmtFSM` 9-1
9.1 Introduction . . . 9-1
9.1.1 Sequential FSMs, Concurrent FSMs, and Digital Hardware . . . 9-2
9.1.2 Communicating between FSM states and between concurrent FSMs . . . 9-3
9.2 `StmtFSM`: structured FSM processes in **BSV** . . . 9-4
9.3 Actions and the `Action` type . . . 9-5
9.3.1 `Action` blocks: composing actions into larger actions . . . 9-5
9.3.2 Binding names in `Action` blocks . . . 9-6
9.4 `StmtFSM`: sequences of actions . . . 9-6
9.5 `StmtFSM`: conditionals (if-then-else) . . . 9-7
9.6 `StmtFSM`: while-loops . . . 9-7
9.7 `StmtFSM`: pausing until some condition holds . . . 9-7
9.8 `StmtFSM`: `mkAutoFSM`: a simple FSM module constructor . . . 9-8
9.9 `StmtFSM` in testbenches . . . 9-8
9.10 `StmtFSM`: many more features . . . 9-8
9.11 A historical note about "structured programming" . . . 9-9
10 RISC-V: the Drum CPU (unpipelined; FSM-sequenced) 10-1
10.1 Introduction . . . 10-1
10.2 The **Drum** CPU module interface . . . 10-1
10.3 The **Drum** CPU module . . . 10-2
10.4 Help-functions for the **Drum** CPU module behavior . . . 10-4
10.5 The main behavior actions in the **Drum** CPU module . . . 10-5
10.5.1 FSM action for Fetch . . . 10-5
10.5.1.1 **BSV**: `Action` *vs.* zero-argument `Action` function . . . 10-5
10.5.2 FSM action for Decode . . . 10-6
10.5.3 FSM action for Dispatch . . . 10-6
10.5.4 FSM actions for Execute and Retire . . . 10-7
10.5.4.1 FSM actions in Direct flow of Execute and Retire . . . 10-8
10.5.4.2 Counting retired instructions in CSR `minstret` . . . 10-9
10.5.4.3 FSM actions in Execute and Retire Control flow . . . 10-10
10.5.4.4 FSM actions in Execute and Retire Integer flow . . . 10-11
10.5.4.5 FSM actions in Execute and Retire DMem flow . . . 10-11
10.5.5 FSM actions for exceptions . . . 10-12
10.6 The **Drum** CPU module behavior . . . 10-13
10.7 Conclusion . . . 10-14
10.7.1 But **Drum** code looks just like C!? Why not code it in C? . . . 10-15
10.7.2 Adding more RISC-V register-to-register instructions to **Drum** . . . 10-15
10.7.3 Adding RISC-V Atomic Memory Op instructions to **Drum** . . . 10-16
11 BSV: Verifying BSV designs 11-1
11.1 Introduction . . . 11-1
11.2 Testbenches and DUTs . . . 11-1
11.3 **printf**-style Debugging . . . 11-2
11.3.1 `FShow` for "pretty-printing" enums and structs . . . 11-3

11.3.2 `Fmt` formatted values . . . 11-4
11.4 Dynamic assertions . . . 11-5
11.5 Waveform-style debugging . . . 11-5
11.6 Summary . . . 11-6
12 RISC-V: Functional verification of CPUs **12-1**
12.1 Introduction . . . 12-1
12.2 Trusted functional simulators ("golden reference models") . . . 12-2
12.3 RISC-V test programs for verification . . . 12-3
12.3.1 ISA tests . . . 12-3
12.3.2 ACTs and other test suites . . . 12-4
12.3.3 What does "verified" mean? Levels of assurance and coverage . . . 12-4
12.3.4 Testing regimes . . . 12-6
12.4 A testbench for **Drum** and **Fife** . . . 12-6
12.5 Symmetric Tandem Verification of CPU implementations . . . 12-8
12.5.1 Configuration . . . 12-8
12.5.2 Level of detail in traces . . . 12-9
12.5.3 Online *vs.* offline tandem verification . . . 12-9
12.6 Asymmetric tandem verification and "full-system" verification . . . 12-10
12.6.1 Instructions with non-deterministic results . . . 12-10
12.6.2 Reading uninitialized memory . . . 12-10
12.6.3 Devices and interrupts . . . 12-10
12.6.4 Asymmetric mode . . . 12-11
12.7 Tandem verification with real hardware (FPGA or ASIC) . . . 12-12
12.8 Randomized Property-based testing (PBT) with **TestRIG** . . . 12-12
13 BSV: Rules and their Semantics **13-1**
13.1 Introduction . . . 13-1
13.2 Syntax of a rule and the data types of its components . . . 13-1
13.3 A rule's implicit condition and overall condition . . . 13-2
13.4 Semantics of a rule in isolation . . . 13-2
13.4.1 Hardware representation of a rule in isolation . . . 13-4
13.4.2 A rule firing cannot perform the same action more than once . . . 13-5
13.5 Semantics of a collection of rules . . . 13-5
13.6 Mapping rules to a clock, for real-time behavior . . . 13-6
13.6.1 Constraints on mapping rules to a clock . . . 13-7
13.6.2 The Rule Controller produced by the *bsc* compiler, and reasoning about performance . . . 13-8
13.6.3 Explicit control of rule ordering, and controller optimizations . . . 13-9
13.7 `StmtFSM` can always be translated into rules . . . 13-9
13.8 When to use `StmtFSM` and when to use rules . . . 13-10
13.8.1 Use rules for unstructured processes . . . 13-11
14 RISC-V: the Drum unpipelined CPU (using Rules instead of `StmtFSM`) **14-1**
14.1 Introduction . . . 14-1
14.2 The **Drum** CPU module behavior with Rules . . . 14-1
14.2.1 Optimizing the **Drum** rules . . . 14-3
14.3 Conclusion . . . 14-4
15 RISC-V: the Fife pipelined CPU: Principles **15-1**
15.1 Introduction . . . 15-1
15.2 Continuing to Fetch, with PC Prediction and Epochs . . . 15-2
15.2.1 PC Prediction in the Fetch Stage . . . 15-2
15.2.2 Identifying and Flushing Wrong-path Instructions . . . 15-3
15.2.3 Terminology: Speculative Instructions and Commits . . . 15-4
15.2.4 Speculative instructions should not have any side-effects . . . 15-4
15.3 Managing Register Read/Write Hazards with a Scoreboard . . . 15-4
15.3.1 Releasing Scoreboard Reservations for Uncommitted Instructions . . . 15-6

15.3.2 Bypassing . . . 15-6
15.4 Retiring outputs of the Execute Stages in Order with Tags . . . 15-6
15.4.1 Full out-of-order execution and reorder buffers . . . 15-7
15.5 Allowing Memory Ops to be Pipelined, with a Store Buffer . . . 15-8
15.5.1 What about LOADs and STOREs to non-memory-like devices (MMIO)? . . . 15-9
15.6 The Retire Stage . . . 15-9
16 RISC-V: the Fife pipelined CPU code **16-1**
16.1 Introduction . . . 16-1
16.2 The **Fife** top-level CPU module . . . 16-1
16.3 How we connect stages . . . 16-3
16.4 The Fetch stage . . . 16-4
16.4.1 Prioritizing rule `rl_Fetch_from_Retire` over `rl_Fetch_req` . . . 16-7
16.5 The Decode stage . . . 16-7
16.5.1 Balancing concurrent paths in a pipeline . . . 16-9
16.6 The Register-Read and Dispatch (and Register-Write) stage . . . 16-10
16.6.1 BSV: Vectors for the Scoreboard . . . 16-11
16.6.2 The Register-Read and Dispatch (and Register-Write) module . . . 16-12
16.7 The Execute Control stage . . . 16-15
16.8 The Execute Integer Ops stage . . . 16-16
16.9 The Execute Memory Ops stage (speculative DMem) . . . 16-17
16.10 **Fife**: the Retire stage . . . 16-17
16.10.1 Common facilities used by many rules . . . 16-20
16.10.2 Rule to retire wrong-path instructions (all paths; discard) . . . 16-21
16.10.3 Rules to retire from direct path . . . 16-22
16.10.3.1 Rule to retire CSRRxx instructions (direct path) . . . 16-22
16.10.3.2 Rule to retire MRET instructions (direct path) . . . 16-23
16.10.3.3 Rule to retire ECALL and EBREAK instructions (direct path) . . . 16-24
16.10.3.4 Rule to retire exceptions from the direct path . . . 16-24
16.10.4 Rule to retire from the Execute Control path . . . 16-25
16.10.5 Rule to retire from the Execute Integer path . . . 16-26
16.10.6 Rules to retire from the Execute DMem path, or perform deferred DMem request . . . 16-27
16.10.6.1 Retire speculative and exception from DMem . . . 16-27
16.10.6.2 Rule to handle deferred DMem requests (from Execute DMem path) . . . 16-29
16.10.6.3 Rules to retire responses for deferred DMem requests . . . 16-29
16.10.7 Common Rule to handle exceptions . . . 16-31
16.10.8 **Fife** module interface definition . . . 16-31
16.11 Conclusion . . . 16-32
17 PBT: Property-based Testing **17-1**
17.1 Introduction . . . 17-1
18 BSV: Rules and Methods II: Improved performance with CRegs (Concurrent Registers) **18-1**
18.1 Introduction . . . 18-1
18.2 A motivating example for CRegs: an up-down counter for credit-based flow-control 18-1
18.2.1 Semantic and Performance Analysis of `mkUp_Down_Counter_I` when mapped to clocked hardware . . . 18-3
18.3 Concurrent Registers `CRegs`, and a Faster Up-Down Counter . . . 18-3
18.3.1 A possible hardware implementation of a CReg . . . 18-4
18.3.2 A Faster Up-Down Counter, using a `CReg` . . . 18-4
18.4 Example: Using a CReg for the RISC-V CSR `mcycle` . . . 18-5
18.5 PipelineFIFOs and BypassFIFOs . . . 18-6
18.5.1 `mkPipelineFIFOF` . . . 18-7
18.5.2 `mkBypassFIFOF` . . . 18-8
18.5.3 Back-to-back compositions of `BypassFIFOF` and `PipelineFIFOF` . . . 18-9
18.6 Alternatives to CRegs: RWires and their variants (deprecated) . . . 18-11

19 RISC-V: Optimizing Drum and Fife **19-1**
19.1 Introduction . . . 19-1
19.2 Pipeline traces and visualization to analyze performance . . . 19-3
19.3 Optimization opportunities in Drum and Fife . . . 19-5
19.3.1 Drum and Fife: Fusing Decode and RR-Dispatch . . . 19-5
19.3.2 Drum and Fife: Fusing some Retire actions . . . 19-6
19.3.3 Drum (rules version): short-circuiting stages . . . 19-7
19.3.4 Drum: using narrower inter-stage buffers . . . 19-7
19.3.5 Fife: saving FIFO resources . . . 19-8
19.3.6 Fife: Reducing the misprediction penalty . . . 19-9
19.3.6.1 Save a tick for Fetch redirection using CRegs . . . 19-9
19.3.6.2 Save a tick for Fetch redirection by eliminating backward FIFO . 19-10
19.3.6.3 Quicker reaction to redirection by Register-Read and Dispatch . . 19-10
19.3.6.4 Better next-PC prediction . . . 19-10
19.3.7 Fife: Reducing the register-hazard penalty . . . 19-13
19.3.7.1 Fife Bypassing: Save a tick in GPRs and Scoreboard using CRegs 19-13
19.3.7.2 Save a tick for register update by eliminating backward FIFO . . 19-14
19.3.7.3 Fife: Dispatching multiple instructions that write to the same Rd 19-14
19.3.8 Drum and Fife: Reducing memory system delays . . . 19-15
19.3.8.1 TCMs (Tightly Coupled Memories) . . . 19-15
19.3.8.2 Caches . . . 19-15
19.3.8.3 Virtual Memory . . . 19-16
19.3.8.4 TLBs (Translation Lookaside Buffers) . . . 19-16
19.3.8.5 TLBs and Caches . . . 19-16
20 BSV: Suggested further study **20-1**
20.1 Introduction . . . 20-1
20.2 Alternative simulators: **Bluesim**, other Verilog sims . . . 20-1
20.3 First-class modules . . . 20-1
20.4 Polymorphism . . . 20-1
20.5 Typeclasses. Conversion of Integer literals. Bits/pack/unpack . . . 20-1
20.6 Bluecheck . . . 20-1
20.7 Tagged unions, pattern-matching . . . 20-1
20.8 Multiple Clock Domains . . . 20-1
20.9 Importing RTL . . . 20-1
20.10 BH alternative syntax . . . 20-1
21 RISC-V: Suggested further study **21-1**
21.1 Introduction . . . 21-1
21.2 Implementing RV64I instead of RV32I . . . 21-1
21.3 M Extension . . . 21-1
21.4 F and D Extensions . . . 21-1
21.5 C Extension . . . 21-1
21.6 A extention . . . 21-2
21.7 Advanced branch prediction . . . 21-2
21.8 Register renaming: towards out of order processing . . . 21-2
21.9 Advanced bypassing: towards dataflow and out-of-order processing . . . 21-2
21.10 Memory systems: TCMs, Caches, PMPs . . . 21-3
21.11 Memory Systems: Virtual Memory . . . 21-3
21.12 Performance measurement . . . 21-3
21.13 Testing . . . 21-3
21.14 Interrupts . . . 21-3
21.15 Linux and server-class capability . . . 21-4
21.15.1 Hypervisor support . . . 21-4
21.15.2 RISC-V ISA Formal Specification . . . 21-4
A Resources: Documents and Tools **A-1**

A.1 GitHub . . . A-1
A.2 RISC-V ISA (Instruction Set Architecture) Specifications . . . A-1
A.3 RISC-V Trusted Simulators and Reference Programs for Testing Implementations . A-2
A.4 RISC-V Assembly Language Manuals . . . A-2
A.5 RISC-V GNU tools, including `riscv-gcc` compiler . . . A-2
A.6 BSV . . . A-3
A.6.1 "BSV By Example" book (free downloadable PDF) . . . A-3
A.6.2 **BSV** Tutorial . . . A-4
A.6.3 MIT Course Material . . . A-4
A.6.4 University of Cambridge Examples . . . A-4
A.6.5 *bsc* download and installation; *bsc* and **BSV** manuals . . . A-5
A.7 Verilator (or other Verilog simulator) . . . A-5
A.8 Amazon AWS . . . A-6
A.9 Xilinx Vivado . . . A-6
A.10 RISC-V textbooks . . . A-7
B Why BSV? **B-1**
B.1 Why BSV instead of some other Hardware Design Language? . . . B-2
B.1.1 A better computational model . . . B-3
B.1.2 Modern language features . . . B-4
B.1.3 Comparison with C++-based High Level Synthesis . . . B-4
B.1.3.1 C++ codes need significant rewriting . . . B-5
B.1.3.2 Narrow range of applicability due to automatic parallelization . . B-5
B.1.3.3 Lack of "Algotecture": Architectural transparency and predictability B-6
B.1.3.4 Summary . . . B-6
C Glossary **C-1**
D BSV: Importing C/C++ functions into BSV simulations **D-1**
D.1 Introduction . . . D-1
D.2 In BSV code, declare a BSV version of the C function . . . D-2
D.3 Compile the BSV code with the *bsc* compiler . . . D-2
D.4 Linking . . . D-3
D.5 Recommendations for arguments and results of imported C/C++ functions . . . D-3
D.5.1 Only use BSV types corresponding to C types . . . D-3
D.5.2 Use `ActionValue#(t)` for imported C function's result . . . D-4
D.6 Example: Memory Model for Drum and Fife . . . D-4
E Exercises **E-1**
BSV Index **INDEX-BSV-1**
RISC-V Index **INDEX-RISC-V-1**
Bibliography **BIB-1**

List of Figures

1.1 Goals for this book . . . 1-1
1.2 Topics covered in this book (in red text in red box) . . . 1-3
1.3 Chapter roadmap . . . 1-4
1.4 Two CPU implementations (microarchitectures) . . . 1-6
1.5 A clock signal . . . 1-9
1.6 Digital circuits consist of state elements interconnected with acyclic combinational circuits. . . . 1-10
1.7 A typical memory hierarchy in a computer system . . . 1-11
1.8 Split-phase memory access . . . 1-12

2.1 What is an ISA? . . . 2-1
2.2 Modularity of the RISC-V ISA . . . 2-5
2.3 RISC-V Instruction Encodings . . . 2-6
2.4 Construction of 21-bit immediate from 20 "imm" bits in J-type instructions . . . 2-7
2.5 Construction of 13-bit immediate from 12 "imm" bits in B-type instructions . . . 2-7
2.6 RISC-V RV32I Instructions . . . 2-8
2.7 RISC-V LUI and AUIPC Instruction semantics . . . 2-8
2.8 Execution semantics for LUI instructions . . . 2-9
2.9 Execution semantics for AUIPC instructions . . . 2-9
2.10 JAL and JALR for subroutine call and return . . . 2-11
2.11 CSRs for handling traps . . . 2-12
2.12 Trap and return flow . . . 2-13
2.13 CSRRxx instructions (from Unprivileged Spec) . . . 2-15
2.14 CSRRxx instruction semantics . . . 2-15
2.15 RV64I instructions (in addition to RV32I) . . . 2-17
2.16 Simple interpretation of RISC-V instructions . . . 2-18

3.1 File-level view of a **BSV** program . . . 3-4
3.2 What's in a **BSV** package/file? . . . 3-4
3.3 Namespace control with imports and exports . . . 3-7
3.4 Typical components in an interface declaration . . . 3-8
3.5 Typical components in a module declaration . . . 3-9
3.6 Typical components of a rule . . . 3-11
3.7 Typical components of an interface definition . . . 3-12
3.8 Static elaboration . . . 3-13
3.9 Module interaction *via* methods . . . 3-14

4.1 The hierarchy of Values, Types and Kinds . . . 4-4
4.2 RISC-V conditional BRANCH instructions . . . 4-11
4.3 Testing for a legal BRANCH instruction . . . 4-12
4.4 If-then-else is a multiplexer . . . 4-18
4.5 Nested if-then-elses become cascaded multiplexers . . . 4-19
4.6 Nested if-then-elses using an AND-OR MUX (for mutually exclusive conditions) . . . 4-20

5.1 Simple interpretation of RISC-V instructions
(Same as Fig. 2.16, with arrows annotated with `struct` types) 5-1

6.1 Simple interpretation of RISC-V instructions 6-1

7.1 Hardware wires/buses for interface methods of `ActionValue(t)`, `Action` and value result types 7-3
7.2 How we connect **Fife** stages 7-13

9.1 Simple interpretation of RISC-V instructions (same as Fig. 5.1) 9-1
9.2 A (greatly over-simplified) FSM for controlling an elevator. 9-2
9.3 Concurrent, independent FSMs. 9-3
The grey arrows indicate communication *via* registers, FIFOs, ...
9.4 Structured FSMs, built compositionally. 9-4

10.1 Simple interpretation of RISC-V instructions (same as Fig. 5.1) 10-1
10.2 Execute and Retire actions in **Drum** 10-7
10.3 Adding more RISC-V instructions 10-16

11.1 A testbench connected to a DUT 11-1

12.1 Complexity of CPU verification 12-1
12.2 Levels of Assurance 12-5
12.3 Top-level simulation setup for the **Drum** and **Fife** CPUs 12-6
12.4 Symmetric Tandem Verification 12-8
12.5 Asymmetric tandem verification: dealing with "minor" differences, interrupts (asynchronous events), devices, *etc.* 12-11

13.1 Syntactic structure of a rule 13-1
13.2 Semantics of a rule in isolation 13-3
13.3 Hardware representation of a rule in isolation 13-4
13.4 A clock signal 13-7
13.5 Controlling rule execution (CAN_FIRE excerpt from Figure 13.3) 13-8

15.1 Pipelined interpretation of RISC-V instructions (Fig. 2.16 with some annotations) . 15-1
15.2 Using epochs to identify and discard mispredicted instructions 15-3
15.3 Using a scoreboard to manage register read/write hazards 15-5
15.4 Multiple concurrent pipelines in the Execute stage, of varying latencies 15-7
15.5 Speculative and non-speculative DMem access, with a Store Buffer 15-8
15.6 Retire actions in **Fife** 15-10

16.1 Pipelined interpretation of RISC-V instructions (Fig. 2.16 with some annotations) . 16-1
16.2 How we connect **Fife** stages 16-4
16.3 Actions in the "Retire" stage of **Fife** (same as Fig. 15.6) 16-18

18.1 Credit-based flow control with an up/down counter 18-2
18.2 A possible hardware implementation of a CReg 18-4
18.3 A producer and consumer connected with a FIFO 18-7
18.4 Combinational path through `mkPipelineFIFOF` 18-8
18.5 Combinational paths through `mkBypassFIFOF` 18-9
18.6 A producer and consumer connected with a composed BypassFIFO-PipelineFIFO. . 18-10
18.7 Pipeline-stage modularity enabled by composing a BypassFIFO and a PipelineFIFO 18-11

19.1 Visualization of the per-instruction stage events in Drum and Fife 19-4

Chapter 1

Introduction

1.1 Goals

This book has two complementary goals, illustrated in Figure 1.1.

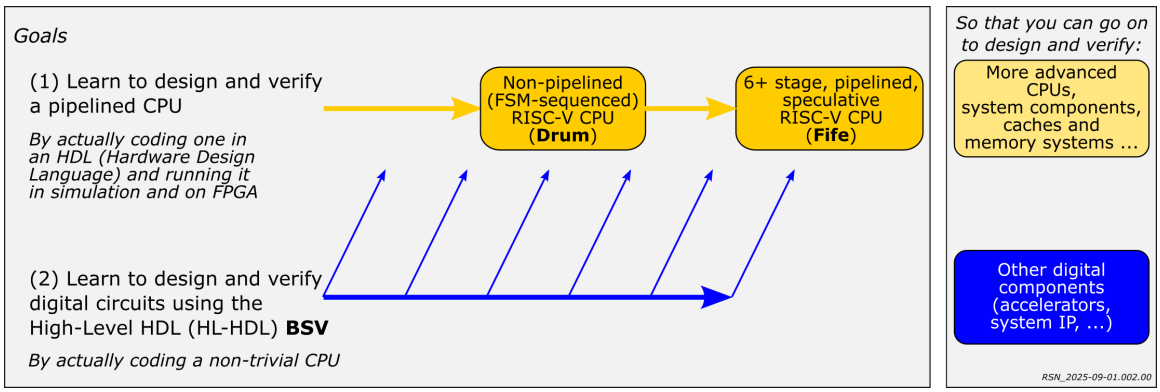


Figure 1.1: Goals for this book

First, the book should help you learn to design a non-trivial computer processor (CPU). Second, the book should help you learn modern digital hardware design, which nowadays is done using a Hardware Design Language (HDL).

The two goals are nicely complementary. One cannot really understand how to design a CPU with just textbook, pencil and paper, and whiteboard. For deeper understanding one needs really to *do the design* and actually run it (in simulation first, on FPGA second, and possibly finally on ASIC).[1]

Similarly, one cannot really learn an HDL (or any progrmming language for that matter) without actually practising writing, running and testing programs. For this, we need interesting examples; what better example than designing a CPU?

In this book we learn how to design two RISC-V CPUs, **Drum** and **Fife**, both of which can execute exactly the same RISC-V binaries (*i.e.,* RISC-V machine code produced from C programs (for example) by the *riscv-gcc* compiler (or other RISC-V compilers).

[1] FPGAs and ASICs are certain kinds of silicon hardware circuit devices. Appendix C has a Glossary with more information.

Drum is the simpler CPU, because it is not pipelined. By designing this first, we can master all the nuances of RISC-V *functionality* without being distracted by complexities due to pipelining. Then we move on to **Fife**, where we can focus exclusively on the complexities due to pipelining. This is facilitated by the fact that **Fife** *reuses most of the RISC-V functional code developed for* **Drum**[2].

Both designs are coded in the **BSV** High-Level HDL (HL-HDL). We interleave topics from **BSV** and CPU-design so that **BSV** can be learned gradually and incrementally. After each step in learning **BSV**, we apply that new knowledge to another actual sub-design of the CPUs. Each chapter title is prefixed by "RISC-V" or "**BSV**" to indicate whether the chapter is primarily about RISC-V or mostly about **BSV**.

By the end of this book, we will not only be quite fluent in **BSV** for digital design, we will also have two RISC-V CPU designs that we can run in simulation and on FPGAs, *i.e.,* which can execute RISC-V binaries.

Equally important, the student will be ready to move on to more advanced projects in digital system and subsystem design, including other CPUs, RISC-V or otherwise, of varying complexity. Of course, the topics in this book are just the tip of the iceberg; it takes years of experience and skill to be able to design a modern, advanced CPU, but this book should prepare the student well to launch into such endeavors. At the end of the book, Chapters 20 and 21 contain suggestions for further study.

1.2 The plan, in more detail

Figure 1.2 shows the broad range of topics that any CPU designer needs to know. In this book we will touch on all the topics shown in red text.

We start by reviewing the RISC-V ISA itself in Chapter 2. What are RISC-V instructions? How are they coded in bits? What does executing an instruction mean? The ISA *per se* is not a focus of this book (for which many textbooks and educational materials are available), but understanding the ISA is of course a prerequisite to designing a CPU.

The RISC-V ISA has many options, discussed in Chapter 2; our focus in this book will be on a small standard subset called "RV32I" which is adequate for small embedded systems.

NOTE:

In order to run actual RISC-V programs on our implementations, we need to undertand how to use the *riscv-gcc* compiler to compile C and RISC-V Assembly Language programs into RISC-V binaries (so-called "ELF" files). Another useful tool is *riscv-objdump*, which can disassemble the binary back into assembly-language text. This is useful for debugging our implementation, so that we can understand execution instruction-by-instruction, and diagnose anything that goes wrong.

In this book we do not discuss how to install and use compilation tools.

Most of the book is focused on designing and verifying **Drum** and **Fife**.

We code our designs in **BSV**. We use **BSV** for CPU itself, and for the top-level testbench. For simulation, the supporting "system" components—a Memory, UART and GPIO—are mostly written in C and linked into a simulation executable. For running on FPGA, however, the system components need to be written in an HDL.

[2] We are unaware of any other designs where ISA functionality *vs.* pipelining are factored in this way.

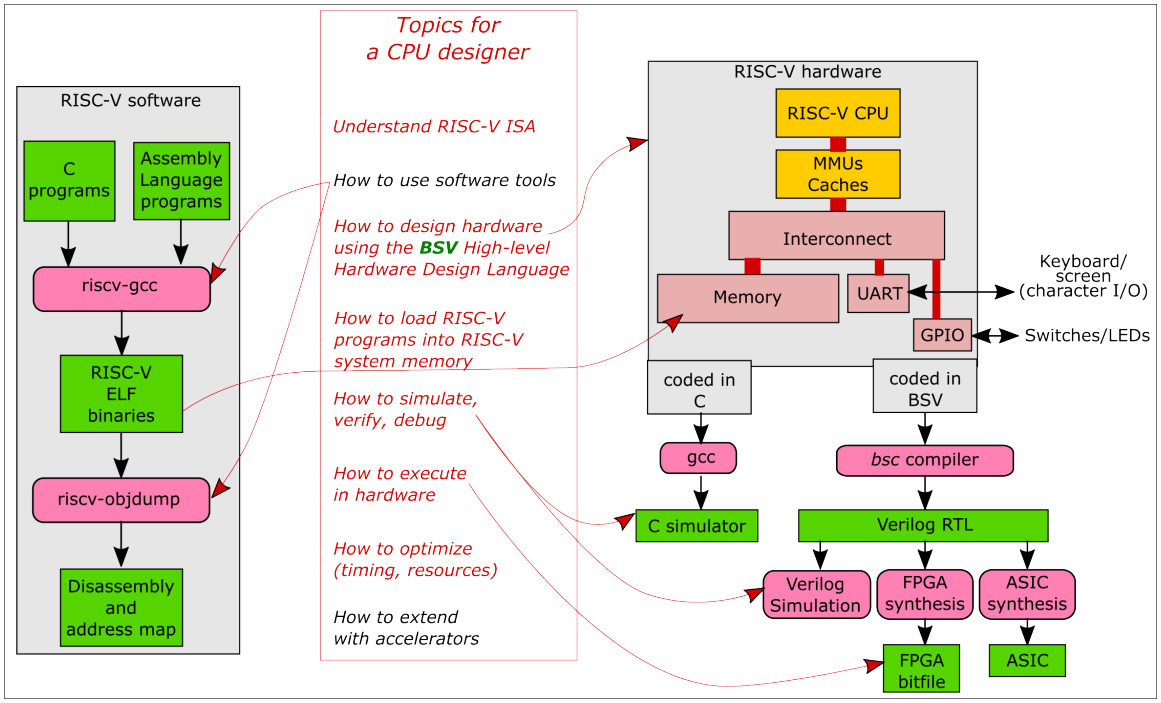


Figure 1.2: Topics covered in this book (in red text in red box)

We can simulate the CPU and supporting system on our laptop/workstation in two ways:

1. We use the *bsc* compiler to compile **BSV** code directly into a so-called **Bluesim** simulation executable.

2. We use the *bsc* compiler to translate **BSV** code into Verilog RTL, and then we use the **Verilator** tool to compile that into a Verilog simulation executable. **Verilator** is a free and open-source tool. The generated Verilog can also run on any other Verilog simulator, of which there are many, both commercial and free.

The two methods are cycle-by-cycle accurate with each other. Further, they are cycle-by-cycle accurate with running the same design on an FPGA or in an ASIC. Simulation on a laptop/workstation is invaluable for debugging the hardware design because the turnaround time to fix a problem and run a new simulation is very short (minutes) compared to creating a new version for an FPGA (several hours).

Of course, simulation (**Bluesim** or Verilog) runs much more slowly (by a factor 10,000 times or slower) compared to running on an FPGA, and so simulation is useful primarily for early debugging and analysis of the design while running small RISC-V programs.

In Chapters 11 and 12, respectively, we discuss debugging and verification of **BSV** designs in general, and CPUs in particular. These are based on *traces* that are generated during simulation.

In Chapter 17 we discuss a semi-formal mechanism called **PBT**, or Property-Based Testing, to verify designs.[3] The general principles of **PBT** are incorporated in a RISC-V CPU-specific

[3] **PBT** is the generic name for tools like Haskell's *QuickCheck* (https://en.wikipedia.org/wiki/QuickCheck).

testing framework called **TestRIG**, also discussed in this chapter.

In Chapter 18 we discuss **BSV**'s "CRegs" ("Concurrent Registers") which provide a principled way to "fuse" separate rules so that they can execute in a single clock, where previously those rules could only execute in separate clocks. This plays a central role in optimization of our RISC-V designs, for example to reduce the latency between register read and write, PC redirection and resumption, *etc.*. These techniques are described in Chapter 19.

We will discuss how to process our Verilog RTL through an FPGA synthesis tool to create an FPGA bitfile which can then be loaded into an FPGA and executed in hardware. We will use FPGAs in the Amazon AWS cloud, but it can be synthesized and run on any other FPGA platform as well.

NOTE: Our *bsc*-generated Verilog RTL can also be processed through ASIC synthesis tools targeting an ASIC design flow. We will not be discussing ASIC flows in this book.

Figure 1.3 shows a roadmap of the chapters in the book, and the interplay of **BSV** and RISC-V topics.

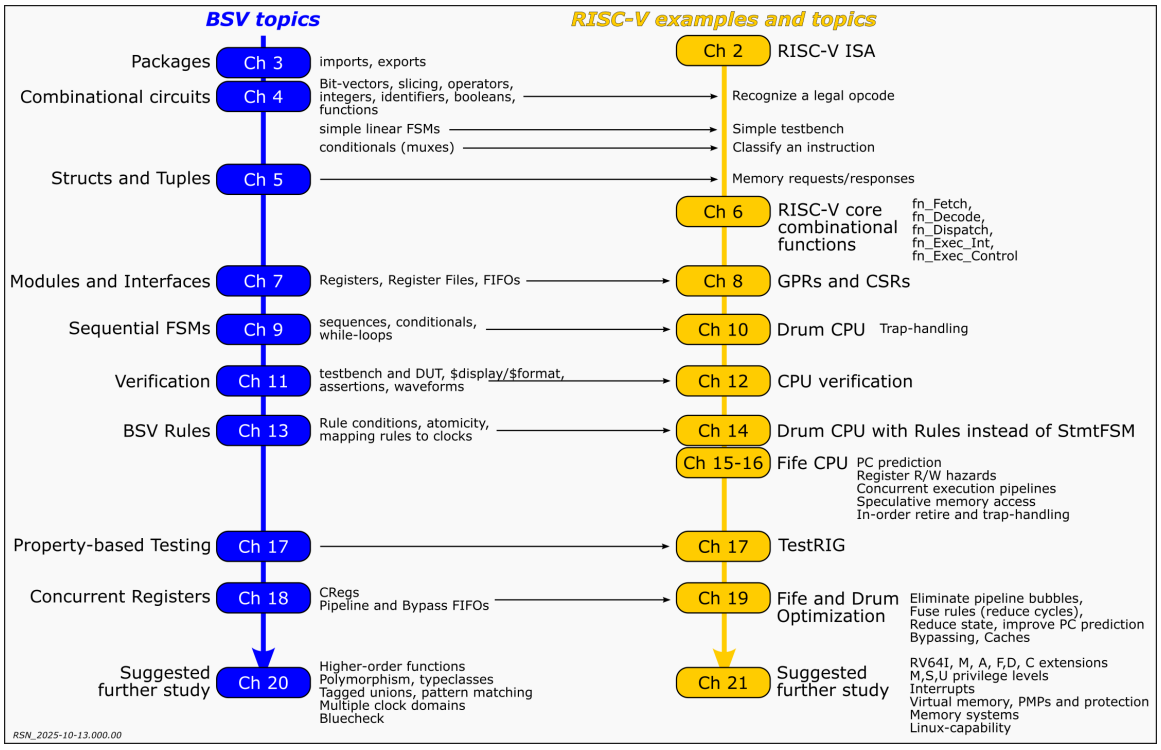


Figure 1.3: Chapter roadmap

1.3 Drum and Fife, our two implementations

1.3.1 The Design Space for CPU implementations

Any artefact/engine that executes the instructions of any ISA is an *interpreter* for that ISA. Interpreters can be implemented in software or in hardware.

ISA interpreters written in software, and running like any other program on workstations, are called "simulators". We do not discuss them in this book, although we may use them during verification of our hardware designs. Appendix A.3 has more information on, and references to open-source RISC-V simulators.

Hardware implementations (and even software simulators) can vary widely in microarchitecture. Some design options are:

- Sequential or pipelined? One full instruction at-a-time, or multiple instructions flowing through a pipe, each at a more advanced stage in its execution than the one behind it?
- Predictive/Speculative? In pipelined implementations, can we predict/speculate what instructions to fetch while a BRANCH/JUMP flows through the pipe, before we know the actual next-instruction determined by the BRANCH/JUMP?
- Superscalar/VLIW? Fetch and execute more than one instruction in parallel, taking care to preserve sequential ISA semantics?
- Out-of-order? Execute each instruction as soon as its input data is available, without waiting for prior instructions which may still be waiting for their inputs?

For the same microarchitecture, a software simulator is typically *much slower* than a hardware implementation (often by factors of tens of thousands). The first reason is because it involves (at least) two layers of simulation. The software simulator is itself a program that is being interpreted by the underlying workstation hardware. That program (the simulator), in turn, is interpreting the target ISA. Every layer of interpretation can slow down overall performance by an order of magnitude or more. Secondly, simulators typically do not exploit the natural parallelism of hardware.

Paradoxically, adding microarchitectural detail will normally slow down a software simulator but speed up a hardware implementation. This is because more microarchitetural detail usually exploits more *parallelism* and *concurrency* in RISC-V interpretation, which can be fully exploited by hardware, but requires more sequential work in a software simulator.

1.3.2 ISA executed by Drum and Fife

Drum and **Fife** execute exactly the same binaries; the only difference will be in **Fife**'s superior performance (speed).

As described in more detail in Chapter 2, the RISC-V ISA has many modular options. For **Drum** and **Fife**, we limit our attention to a small subset:

- The so-called "RV32I" ISA from the RISC-V Unprivileged ISA spec: basic integer arithmetic and logic operations; branch and jump; load and store.
- Plus: a few elements from the RISC-V Privileged ISA spec for handling exceptions: Control and Status Registers (CSRs), traps and trap-handling, and CSRRxx instructions. This allows software to recover from illegal instructions, memory access errors, and so on.

For detailed information on RV32I, please see the specification document "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA" [27]. In particular, look at:

- The first table in Chapter 24 "RV32/64G Instruction Set Listings" entitled "RV32I Base Instruction Set", showing the forty RV32I instructions (and reproduced in Figure 2.6).
- These instructions are described in more detail in that document in Chapter 2 "RV32I Base Integer Instruction Set, Version 2.1".

For detailed information on traps and trap-handling, please see the specification document "The RISC-V Instruction Set Manual Volume II: Privileged Architecture" [28].

1.3.3 Drum and Fife

Drum and **Fife** are illustrated in Figure 1.4. Each instruction is taken through the same stages, shown.

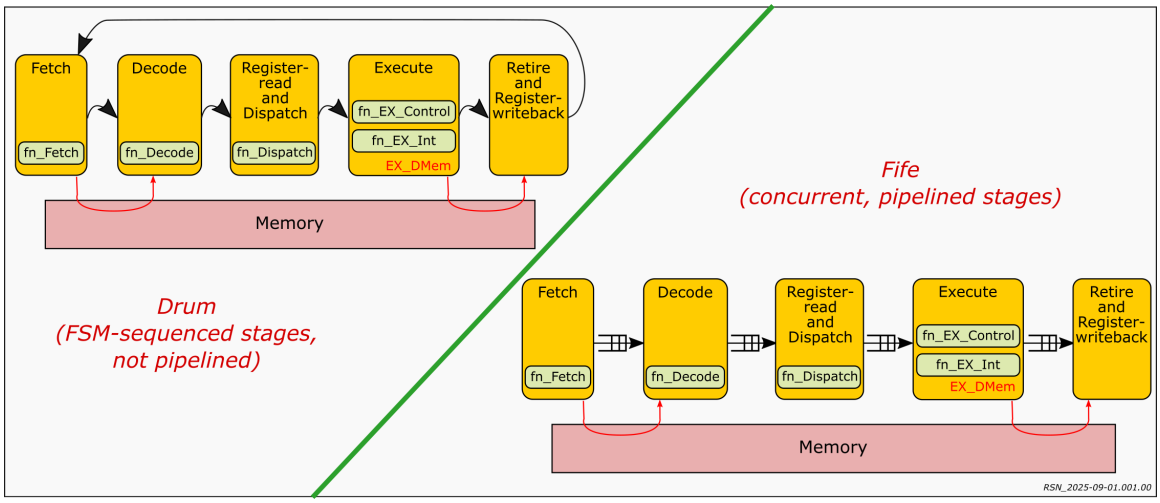


Figure 1.4: Two CPU implementations (microarchitectures)

Drum is a *non-pipelined* implementation. For each instruction, the design is *sequenced* through the stages shown in the diagram, using an FSM (Finite State Machine), completing all stages sequentially before starting on the next instruction.

Drum is almost a direct transliteration into **BSV** code of the generic ISA execution algorithm described in Section 2.9. In fact the **BSV** code will look very similar to C/C++ code for a purely functional RISC-V simulator. Being written in **BSV**, however, we can compile and run it on actual hardware (FPGAs, ASICs).

The design of **Drum** will familiarize us with all the basic RISC-V concepts and execution flows (the RISC-V ISA, preparing and running a RISC-V binary to run on the design, analysing traces), without being distracted by the complexities of pipelining.

Drum is not fast compared to **Fife** or other hardware CPUs, because of its lack of microarchitectural features, but we can still synthesize it to run at several 100 MHz on an FPGA,

which makes it faster than most software functional simulators. It will be small (silicon area, and therefore low power as well).

Drum is covered in Chapter 6 through Chapter 10.

Fife is a 6+ stage pipelined design, which is a microarchitectural choice focused on higher performance (speed) than **Drum**. All the stages in the diagram run *concurrently*, with each stage receiving input from the previous stage to its left ("upstream") and producing output for the next stage to its right ("downstream"). The design includes simple branch-prediction, register read/write hazard management, concurrent execution pipelines, in-order retirement of instructions, and speculative stores to memory using a store-buffer.

Fife is covered in Chapter 15 through Chapter 16.

Fife *reuses* most of the RISC-V functional code from **Drum** without change. Thus, **Fife** chapters can focus purely on the new issues raised by pipelining (and which are in fact applicable to any pipelined CPU design, not just RISC-V).

At the end of the book, Chapter 21 discusses how to extend **Drum** and **Fife** to handle RV64I and more Unprivileged ISA options—**M**: integer multiply/divide, **A**: atomics, **F,D**: single-and double-precision floating point, and **C**: compressed— and more privileged ISA options—Privilege levels (**M**: Machine, **S**: Supervisor and **U**:User; full complement of Control and Status Registers (CSRs); Virtual Memory). With these extensions, **Drum** and **Fife** will be able to run a full-feature Operating System (OS), such as Linux.

1.3.4 Drum and Fife source codes

The full source codes (**BSV**) for **Drum** and **Fife** are included with this book. Excerpts of the actual code are taken, as-is, for inclusion in this book. Excerpts look like this:

src_Common/CPU_IFC.bsv: line 28 ...

```
interface CPU_IFC;
   method Action init (Initial_Params initial_params);

   // IMem
   interface FIFOF_O #(Mem_Req) fo_IMem_req;
   interface FIFOF_I #(Mem_Rsp) fi_IMem_rsp;
   ...
   // DMem, non-speculative
   interface FIFOF_O #(Mem_Req) fo_DMem_req;
   interface FIFOF_I #(Mem_Rsp) fi_DMem_rsp;
   ...
endinterface
```

The label on the top border of the box indicates the file (`src_Common/CPU_IFC.bsv`) from which this has been excerpted, and the starting line number (27) in the file. The "`...`" indicates we have *elided* some lines from the source file which are not directly relevant to the current discussion.

NOTE: We strongly recommend, as you read the book, that you also keep open a text editor in which you can simultaneously view the actual sources.

If a code excerpt in the book does *not* contain a filename/line-number in the top border of the box, then it is not from the actual source code, it is just meant to illustrate some point in the text.

1.4 About BSV, the HL-HDL we use for our designs

Both designs are coded in **BSV**, a free, open-source, modern, High-Level Hardware Design Language (HL-HDL). **BSV** code can be compiled into Verilog, which can then be run on any Verilog simulator, or can be further processed by FPGA tools to run on FPGAs, or by ASIC tools for ASIC implemenetations.

BSV is a modern, high-level HDL taking inspiration from two sources in modern software programming languages. First, for data types, program structure, and static elaboration, **BSV** directly emulates the Haskell functional programming language. In fact, **BSV** is actually Haskell, with a syntax closer to SystemVerilog. For Haskell enthusiasts, there is an alternate syntax called "BH" which directly uses Haskell syntax; please see *bsc* documentation for details.

For specifying dynamic behavior, **BSV** emulates a class of formal specification languages for concurrent programming based on rules (including Term Rewriting Systems, Unity, TLA+, and Event-B).

BSV is a "universal" digital hardware design language, like Verilog and SystemVerilog. Although here we focus on CPU design, it is suitable for any digital hardware design whatsoever.

Appendix B has a more detailed justification of why we choose **BSV** over other hardware design languages like Verilog, SystemVerilog and VHDL.

1.5 Basic model of digital hardware needed for this book

"Digital Design" is traditionally taught as a separate course preceding any course on CPU design. Much time is spent on circuit diagrams and schematics, then transitioning to hardware design languages (HDLs) like Verilog, SystemVerilog or VHDL. Topics include boolean logic gates (AND, OR, NOT, XOR, ...), combinational circuits (half-adders, adders, multiplexers, ...), logic minimization, flip flops, registers, binary number representations (twos complement), FSMs (Finite State Machines), and so on. Supporting examples are often quite artificial and simplistic.

However, just like the transition in software programming languages from machine code and assembly language to high-level languages (which began in the 1960s and 1970s), most modern digital hardware design is done at a higher level of abstraction. Designs are described using HDLs like Verilog, SystemVerilog and VHDL. We express logic using traditional arithmetic and logic operators on bit-vectors; we express multiplexes using if-then-else and case statements; we use library elements for n-bit registers, FIFOs, memories, and so on. We rely on *compilers* (synthesis tools) to transform such descriptions into low-level gates, wires, flip-flops, *etc.*.

Accordingly, we will focus here on **BSV** language-based hardware design, and skip the study of detailed low-level underlying circuits. All our examples will be taken from **Drum** and **Fife** so that the motivation is clear.

In the next few sections, we cover the basic digital hardware concepts that are adequate to launch us into designing **Drum** and **Fife** with **BSV**.

1.5.1 Clocks and State Elements

Digital hardware is driven by a clock, an electrical signal that oscillates between a low voltage and a high voltage, with sharp transitions between the two, at regular intervals (it is therefore sometimes called a "square" wave). This is illustrated in Figure 1.5.

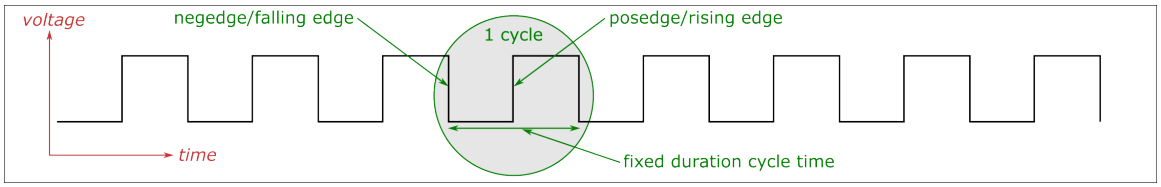


Figure 1.5: A clock signal

Nowadays, clock cycle times/periods are typically measured in nanoseconds. For example, a 100 MHz clock has a 10ns cycle time.

Any value that must be visible from one clock to the next must be *stored* in a *state element*—a component that can "remember" values, *i.e.,* has *state*, like a register, FIFOF, buffer, memory, ...

1.5.2 Bit-vectors, values and buses

All data in digital circuits are represented in binary form, as vectors of bits. In **BSV** notation, the *type* (data type) of a bit-vector is written like this: "`Bit#(`n`)`", where n is the number of bits (width of the bit-vector). A bit-vector may directly represent bits, or it may be an encoding of some higher-level construct such as an instruction, a register name, a character string, a memory request or response, and so on. Collectively we call all such constructs *values*.

In hardware, values are carried on bundles of wires called *buses*. In circuit diagrams, an n-bit-wide bus is usually drawn as a line with a small diagonal hash mark labeled with n. Values are also stored in state elements.

1.5.3 Digital circuits

Digital circuits are composed of state elements and *combinational circuits* connecting outputs of state elements to inputs of state elements, as illustrated in Figure 1.6.

Connecting the output of a combinational circuit to the input of another combinational circuit just builds a larger combinational circuit (*e.g.,* the edge from the first to the second combinational circuit in the figure).

State elements are *storage* elements. Combinational circuits are *acyclic* networks of *logic gates*. A logic gate is a circuit that "instantaneously" computes an output boolean value that is a function of its boolean inputs (such as AND, OR, NOT, XOR, ...).

Any digital circuit repeatedly performs the following:

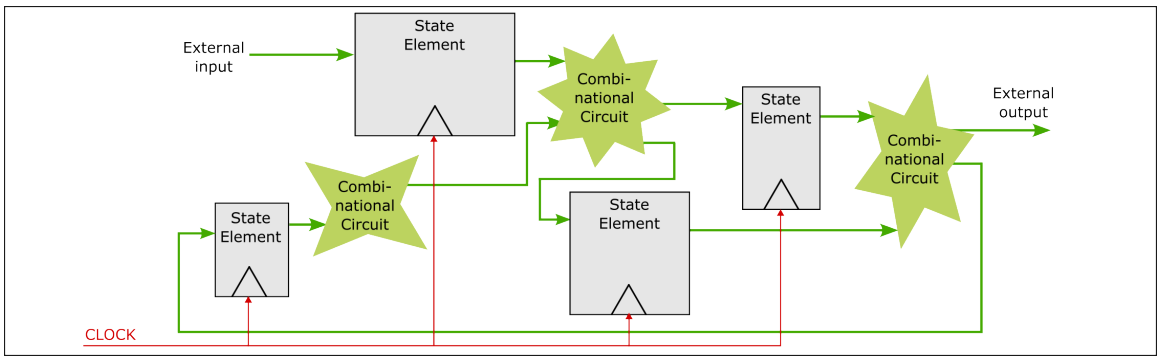


Figure 1.6: Digital circuits consist of state elements interconnected with acyclic combinational circuits.

- After each "posedge" (positive edge of a clock), the combinational circuits compute new input values for all state elements, based on the current output values from state elements.
- At the next posedge, these new input values are stored into ("clocked into") the state elements.

The above description is a simplification, or an idealization. Physics and circuit silicon technology dictates how long an electrical signal takes to propagate from the output of a state element and through a combinational circuit before it arrives at the input of a state element. For correct operation, the clock period needs to be greater than this, to allow signals that are inputs to state elements to "settle down" and be steady when they are captured on the next clock edge.

In the *Digital Abstraction* we assume this condition is met, and we idealize wire and combinational circuit delays as taking zero time.

NOTE:

- Advanced digital ciruits may have multiple clocks driving different parts of the circuit.
- Advanced digital ciruits may vary the clock speed dynamically, to match power consumption to current performance demands (higher speeds ⇒ more power consumption).
- Some digital ciruits use the negative edge ("negedge") of the clock insted of the posedge.
- Advanced digital ciruits may use both the posedge and the negedge of the clock.

1.5.4 Computer memories

A computer memory is a packaged sub-system:

- To read memory, we present a *read request* at its inputs, with an *address*; the memory provides a *read response* at its outputs, containing the addressed data.

- To write memory, we present a *write request* at its inputs, with an address and data; the memory provides a *write response* at its outputs, containing an "ok" status.

Because we access memory by presenting an address identifying a location in memory, they are called RAMs (Random Access Memories). Nowadays memories are typically byte-addressed, *i.e.,* each address identifies a distinct byte in memory. Memory access instructions in the CPU (LOAD, STORE) typically specify a *size*, such as 1, 2, 4 or 8 bytes, indicating the number of bytes to transfer between the CPU and memory.

1.5.4.1 Memory hierarchies

Figure 1.7 illustrates a typical memory hierarchy in a modern computer system. The L1

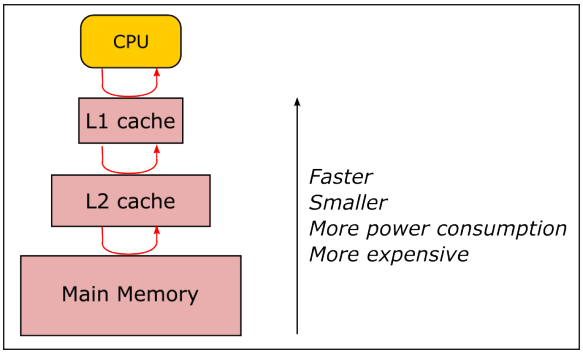


Figure 1.7: A typical memory hierarchy in a computer system

cache ("Level 1 cache"), L2 cache and Main Memory are all memory subsystems. The memory closest to the CPU (L1 cache) is the smallest, fastest (see Section 1.5.4.1) consumes most power and is the most expensive. The memory furthest from the CPU (Main Memory) is largest, slowest, consumes least power, and is the cheapest.

Main Memory contains "all the real contents" of memory. The caches contain temporary copies of a subset of locations of Main Memory.

The CPU sends its memory request to the L1 cache; if it contains a copy of the requested address, this is called a "cache hit", and the L1 cache returns the response to the CPU. If it does not contain a copy of the requested address (a "cache miss"), the L1 cache, in turn, requests it from the L2 cache which, again, may hit or miss. In the worst case, a request has to be serviced from Main Memory. When a cache retrieves a datum from a lower-level element, it retains it, so that a future access to the same address will hit.

The delay from when the CPU issues a request until it receives the corresponding response is called *memory latency*. This is typically measured in clock cycles (ticks) or in real-time units (nanoseconds, microseconds). L1 caches can typically be accessed in 1 tick if there is a hit. A cache miss can take up to several tens of ticks (the so-called "miss penalty")..

Memory latency is generally not predictable—it can depend on the address of the request, and on past history of accesses which affects what is currently available in the caches. It can also depend on technology replacement: next year's memory system may be slower/faster than today's.

Fortunately, many (if not most) programs exhibit reuse, temporal and spatial *locality*. Reuse and temporal locality means that an instruction or datum read from memory is likely to be accessed again soon. Spatial locality means that after an instruction or datum is accessed from an address in memory, the CPU's next accesses are likely to be from nearby addresses. These properties increase the likelihood that a near-future access will hit in the cache and be serviced quickly. Modern caches can achieve hit rates (the ratio of hits to hits+misses) that can approach 90% or more.

A CPU design should generally not depend on any specific memory latency, *i.e.,* it should be robust to changes and variations.

The rate at which requests and responses can be pumped through the memory system by the CPU is called *memory bandwidth*. This is usually measured in transactions-per-second, or requests-per-second, or in bytes-per-second where the bytes usually refer to data bytes transferred, excluding address bytes and other overhead bytes.

1.5.4.2 Memory technologies for the memory hierarchy

Two major categories of memory technologies are SRAMs (Static Random Access Memories) and DRAMs (Dynamic Random Access Memories). For the same size, SRAMs are more expensive (more silicon area, more power consumption) and faster (less time to access a location) compared to DRAMs. Thus, computers typically use SRAMs for L1 caches and DRAMs for main memory. L2 and other intermediate caches may use SRAM or DRAM. Cache sizes typically range from the 10s of kilobytes to 100 megabytes, whereas main memories can range into several gigabytes.

1.5.4.3 Split-phase memory access

Because memory latency is unpredictable by the CPU, we always assume memory accesses are *split-phase* transactions—a request action followed an unpredictable number of ticks later by a response action. This is illustrated in Figure 1.8. During the interval between

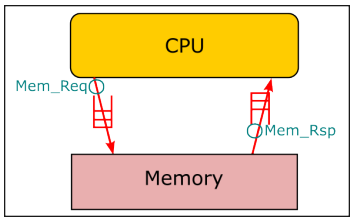


Figure 1.8: Split-phase memory access

request and response, the CPU need not be idle; it can execute other instructions, prepare the next memory access, *etc.*.

1.5.4.4 Memory pipelining and memory ordering

Memory systems are also often pipelined: the CPU can issue a continuous stream of requests, and the the memory sends back a continuous stream of responses. The CPU does not have

to wait for a memory response before issuing the next request. Thus, in Figure 1.8, we have labeled the arrows to and from memory with little red symbols representing *queues* or FIFOs (First-In-First-Out). The whole path from the CPU to memory and back can be regarded as a queue, with requests entering and responses exiting the queue.

NOTE:

> Advanced memory systems may not be FIFO-like; the CPU may receive responses in a different order from the original requests, in the interests of returning a response as soon the data is available, rather than waiting idly for its turn.
>
> *In this book we will assume FIFO-like, in-order memory accesses.*

1.6 Additional Resources for this Textbook/Course

Appendix A has a detailed listing of resources (documents and software tools) needed for this book, and for further reading.

Appendix C contains a Glossary of terms and abbreviations.

This book also contains:

- a detailed index of **BSV** topics, and Chapter 20 has suggestions for further study of **BSV**;
- a detailed index of RISC-V topics, and Chapter 21 has suggestions for further study of RISC-V and CPU design.

Chapter 2

RISC-V: Overview of the RISC-V ISA

2.1 Introduction

This entire chapter can safely be skipped by those already familiar with RISC-V concepts, or who learn it elsewhere (there are many alternate resources on the web). This chapter contains only generic information about ISAs and the RISC-V ISA in particular. It can be read as a general introduction to the RISC-V universe; none of this is specific to this book.

2.2 What is an ISA?

The acronym "ISA" stands for "Instruction Set Architecture". Figure 2.1 illustrates the role of an ISA as an intermediary between hardware (CPU implementations) and software (which runs on the CPU hardware).

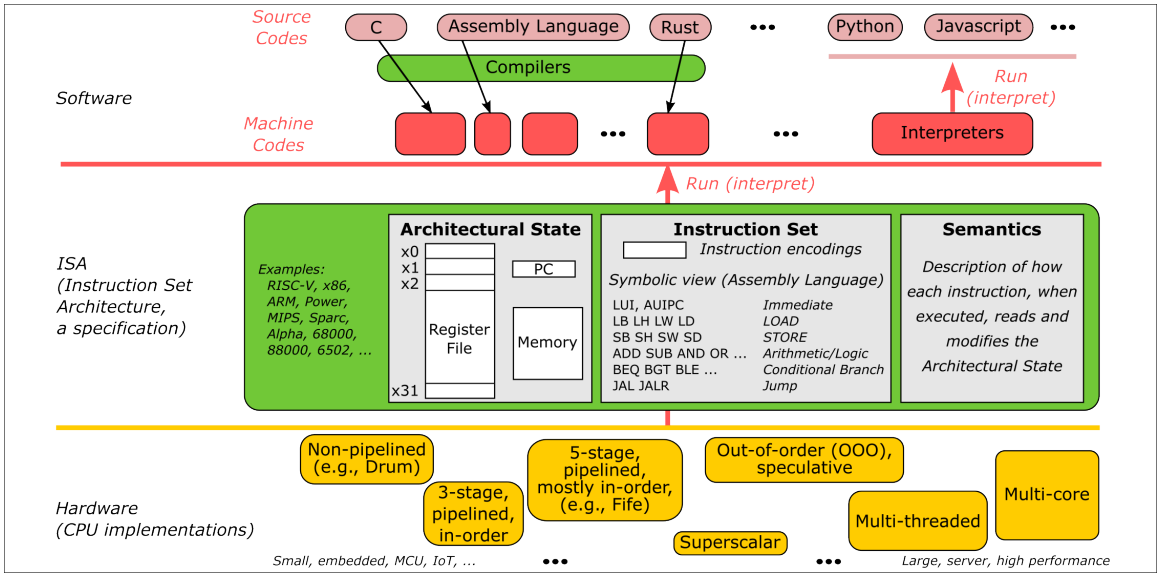


Figure 2.1: What is an ISA?

An ISA, *per se*, is neither software nor hardware. It is merely a *specification*, a document defining three things:

- *Architectural State*: the registers visible to the instructions, such as the program counter (PC) and a register file containing, say, 32 registers named x0 through x31.

  Typically (nowadays), the architectural state includes a *byte-addressed* memory (a memory where an *address* identifies a single byte, and where individual bytes may be read and written.

- An *Instruction Set*: a collection of instructions. For each instruction, the ISA specifies how it is encoded in bits. The ISA may also specify a way of writing an instruction in symbolic text, which we call Assembly Language. Instructions are typically grouped in classes, such as Immediate, LOAD/STORE, Arithmetic and Logic, Conditional Branches, Jumps, System Instructions, and so on.

- *Semantics*: a description, for each instruction about *how* it executes—what architectural state it observes, and what architectural state it updates, and how. For example, a Conditional Branch instruction typically observes two registers in the register file, compares them, and updates the PC depending on the comparison result.

  The overal semantics of a program being executed is just a sequential composition of individual instruction semantics.

The full ISA may be described in ordinary text prose and diagrams, or in a semi-formal or formal language. For example, the RISC-V ISA is described both in text prose and diagrams, and in the formal language "Sail".

Beneath the ISA level are actual *implementations* of the ISA. These range from software simulators of the ISA to silicon implementations. Silicon implementations typically vary widely in their *micro-architecture* features, such as pipelining, in-order or out-of-order, speculation, superscalarity, multi-threading, multi-core. The choice of micro-architecture is typically based on a particular target market, trading-off cost, performance (speed), energy consumption, capabilities (embedded software to full server OS with network and storage stacks), silicon technology (FPGA *vs.* ASIC, ASICs with various silicon feature sizes and techniques), and so on. Variations in micro-architectures provide product differentiation.

Each implementation runs (interprets) machine code, *i.e.,* an encoding of instructions in memory. The machine codes are typically produced by compilers, which are themselves machine code programs that transform source codes into machine codes. Some machine codes are themselves software interpreters for so-called interpreted languages such as Python, Javascript, Java, Scheme, Lisp, and so on.

A crucially important point is that *all the implementations of an ISA should respect the semantics of the ISA as specified in the ISA definition.* They can (and do) play all kinds of micro-architectural tricks under the covers, but the result of executing any program should be explainable purely based on the ISA specification.

Thus, and ISA is a *contract* or *API* between software and hardware implementations. People writing software, people writing compilers for software, people writing interpreters for interpreted languages, *etc.* need only to understand and refer to the ISA specification to do their job. They do not need to know about the specific hardware implementation

on which it will eventually run. This enables *software portability*, *i.e.,* a machine code program for an ISA should be able to run, without change, on *any* implementation of the ISA, including future next-generation laptops/servers for the ISA.

NOTE: In the rarer cases where the "bleeding edge" of software performance really matters, human coders and compilers may produce different machine codes for specific implementations of the same ISA, in order to exploit particular quirks of each implementation's micro-architecture such as branch-prediction or register-hazard penalties.

Examples of ISAs include:

- *RISC-V*: Open ISA (not proprietary). Silicon implementations from various vendors, worldwide.
- *x86*: Proprietary. Silicon Implementations from Intel and AMD, primarily, with names like Xeon, Core i9, 13th Generation Core, Alder Lake, Raptor Lake, and so on.
- *ARMv8*, *ARMv9*: Proprietary. Implementations from ARM licensees like Apple, Samsung, Broadcom and others.
- *Power*: Proprietary. Implementations from IBM and other licensees.
- *Sparc*: Proprietary. Implementations originally from Sun Microsystems, nowadays from Oracle, and also from licensees such as Fujitsu.
- *MIPS*: Proprietary. Implementations from MIPS and other licensees.
- Other famous, but now defunct ISAs: *Alpha* (from Digital Equipment Corp./Compaq/HP), *Itanium* (from Intel, HP), *68000* and *88000* (from Motorola), ...

2.3 Why choose RISC-V?

In the list of ISAs above, except for RISC-V, all the other ISAs are *proprietary*, *i.e.,*, in order to produce and sell a silicon implementation it is necessary to obtain a license (permission) from the company that "owns" the ISA. These licenses can be very expensive, in the thousands of dollars or more.

The RISC-V ISA is owned by RISC-V International ("RVI"), a non-profit corporation based in Switzerland (https://riscv.org). As of Fall 2023, RVI claimed a growing membership, including over three thousand corporate, government, university, and individual members from over 70 countries.

The ISA is "open" in that no prior permission or license from RVI is needed in order to produce and sell implementations of the ISA. If a *commercial* product is claimed publicly to implement the RISC-V ISA, the vendor needs to get official certification from RVI that it indeed does so (it must pass a battery of certification tests), but this is a one-time certification, quite different from a production license. Non-commercial products (from universities, research labs, hobbyists, *etc.*) do not need any such certification.

Separate from these commercial concerns, there are also concerns about quality of an ISA and the richness of the ecosystem supporting an ISA. The RISC-V ISA is attractive on these dimensions as well.

The RISC-V ISA was originally designed at University of California, Berkeley, by a team of researchers who have half a century of deep knowledge and experience on ISAs, computer

architectures, computer systems, and ecosystem software (the Sparc ISA also came out of Berkeley in the 1980s). As such, the RISC-V ISA can be seen as a new, clean-slate design that incorporates all the lessons learned from all previous ISAs dating all the way back to the 1950s.

An ISA is useless without a strong *ecosystem* supporting the ISA. This includes artefacts such as compilers, programming language implementations, debuggers, test and verification infrastructure, embedded operating systems, real-time operating systems, workstation and server-class operating systems, boot loaders, device drivers, and so on. It also includes services, such as tutorials, books, courses, and training materials (this book can be seen as a contribution). The RISC-V ecosystem is already quite rich, and it grows daily precisely because of the open nature of the ISA, enabling thousands of contributors worldwide to participate in the effort.

The openness of the RISC-V ISA also enables entrepreneurs and researchers to attempt *innovations* in micro-architecture and design and production techniques, something which is not practically feasible with a proprietary ISA.

The RISC-V ISA is unusually *modular* (more details in Section 2.4). It consists of a *very* small "base" Integer ISA and a number of optional standard extensions (such as Integer Multiply/Divide, floating point, Atomics, and Compressed instructions), and a systematic way of substituting or adding new, non-standard, proprietary extensions. This makes it easier for an entrepreneur to "tune" the ISA for their proprietary implementation aimed at a target market with specific requirements.

There is also a security dimension to RISC-V's attractiveness. When a customer buys a silicon implementation of an ISA, they have to trust that neither the vendor nor the supply chain inserted any secret "back-doors" by which others can monitor what the processor is doing or, worse, disable it or program it remotely. The RISC-V ISA being open, it is more possible for a customer to develop their own trusted supply chains for silicon implementations.

Silicon implementations of RISC-V are already availble from dozens of vendors located in several countries worldwide. These supply chains are only likely to grow more diverse. Many production-ready, competitive RISC-V designs are available in free and open-source form.

2.4 Overview of the RISC-V ISA

NOTE:

More detailed information on the topics of this section can be found in:

- RISC-V ISA specification documents. Please see Appendix A.2 for links.
- Formal specification of the RISC-V ISA, written in the Sail language. Please see Appendix A.2 for links.
- RISC-V Assembly Language manuals Please see Appendix A.4 for links.

The RISC-V ISA is designed to be highly modular. Figure 2.2 shows the major components. The foundations (*base* integer ISA) are RV32I and RV64I, for implementations with 32-bit and 64-bit register widths, respectively. Technically these are two separate ISAs, but all but two of the RV32I instructions have identical counterparts in RV64I, so one can think of RV64I as a superset of RV32I, as suggested in the diagram. RV32I has just 40 instructions. RV64I slightly modifies two of them and adds a few more instructions.

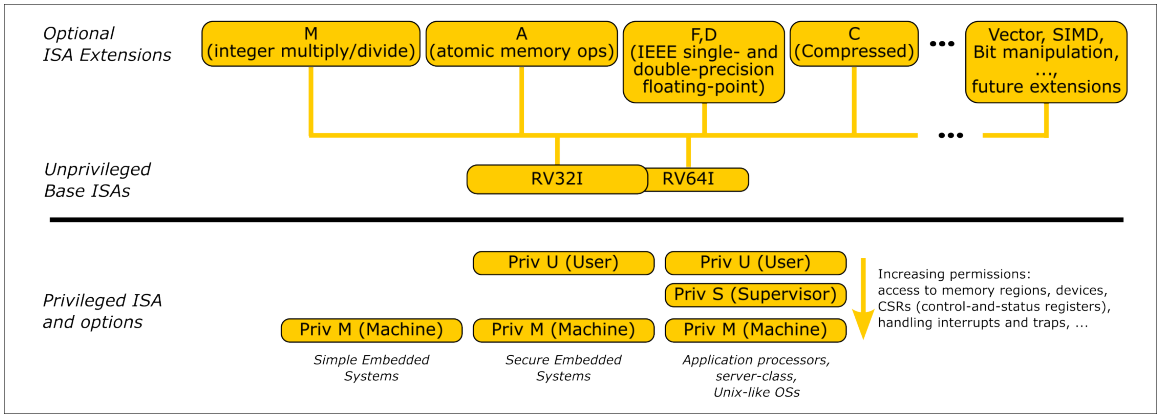


Figure 2.2: Modularity of the RISC-V ISA

To the base ISA one can add several standard optional ISA extensions:

- M: a few instructions for integer multiplication and division.
- A: a few instructions for atomic memory operations (atomic read-modify-write of locations in memory).
- F,D: several instructions for IEEE single- and double-precision floating point operations.
- C: several so-called "compressed" instructions, which are only 16-bits wide, for applications where it is important for the code size to be small.
- Vector extension for vector arithmetic for scientific, AI and high-performance computing.
- Other extensions like SIMD (Single Instruction, Multiple Data), Bit Manipulation, useful in image processing, cryptography, *etc.*

The standard Privileged ISA is shown in the bottom half of the diagram. This consists of three "privilege" levels U (User, low), S (Supervisor) and M (Machine, high), with increasing capabilities with respect what aspects of the architectural state are visible and updatable.

The transition into the Privileged ISA only happens through a few carefully managed gateways. Thus, the whole standard Privileged ISA can easily be substituted with some other, non-standard, Privileged ISA, should the implementor find that beneficial.

Simple RISC-V implementations for very small embedded systems may implement only one privilege level (M). Since there is only one privilege level, there are no relative protections—all code can access all architectural state.

Slightly more secure RISC-V implementations for embedded systems may implement two privilege levels (M and U). Now, code running at U privilege can be prevented from accessing (and damaging) certain parts of architectural state such as devices, regions of memory, *etc.*

Most medium- to large-size systems implement all three privilege levels, M, S and U. Typically, user code runs at U privilege; operating systems such as Linux run at S privilege, and low-level device-access codes run at the highest (M) privilege. When S is implemented, code running at both U and S privilege levels can also use *virtual memory*.

2.5 Instruction encodings

All RISC-V instructions are encoded in 32 bits (except for the C extension, where they are encoded in 16 bits). Although there are hundreds of instructions if we count RV32I, RV64I, M, A, F, D, C, and Privileged ISA, they are all encoded in just a few 32-bit formats, shown in the Unprivileged spec, page 130, reproduced here in Figure 2.3. The labels on the

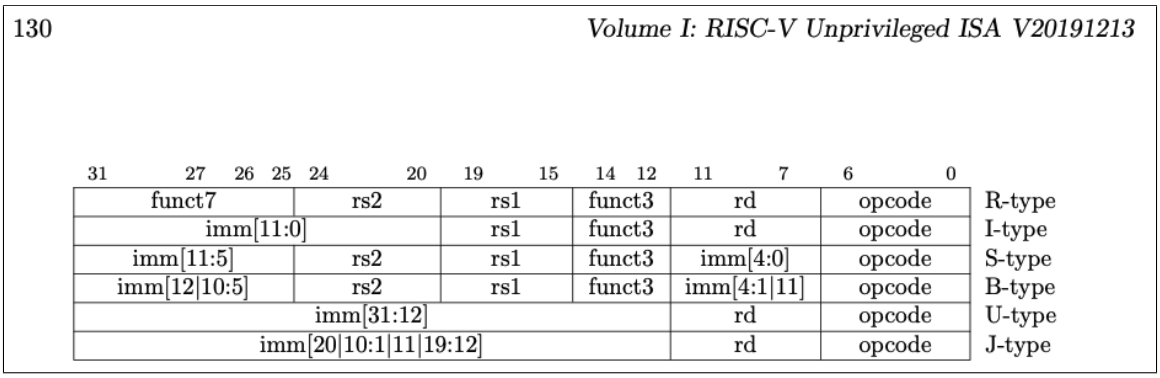


Figure 2.3: RISC-V Instruction Encodings

right-hand-side are suggestive of the class of instructions that use that coding:

- R: "register class": typically two register inputs
- I: "immediate class": typically one register and one immediate input
- S: "store class": for the STORE instructions SB, SH and SW
- B: "branch class": for the conditional branch instructions
- U: "Upper Immediate class": for LUI (Load Upper Immediate) and AUIPC (Add Upper Immediate to PC)
- J: "Jump class": for jump instructions JAL and JALR

We use standard Verilog "bit-slice" notation to refer to bit-fields of the 32-bit instruction. Thus, instr[6:0] refers to the lower 7 bits (bits 0 through 6) of the 32-bit instruction.

The overall "operation code" (opcode) of an instruction is a combination of 6-bit `opcode` field in instr[6:0] and the funct3 and funct7 fields at other positions in the instruction.

If the instruction reads at least one register, the first one is specified in the rs1 field ("register source 1"); the 5 bits in instr[15-19] specify one of the 31 registers. If the instruction reads two registers, the second one is specified in the rs2 field; the 5 bits in instr[24-20] specify one of the 31 registers. If the instruction writes a register, it is specified in the rd field ("register destination"); the 5 bits in instr[11-7] specify one of the 31 registers.

In RISC-V, reading register `x0` always yields the value 0. This is a useful convenience built into the ISA. For example, there is an instruction to test if the rs1-value is less than the rs2-value. If we specify `x0` for rs2, then we effectively test if the rs1 value is negative.

Some instructions take input data from bits in the instruction itself; these are called "immediate" fields. In Figure 2.3 we can see that there are various encodings for the immediate fields. Consider the J-type instruction. Figure 2.4 shows how the 20 "imm" bits in the instruction are permuted, with a 0 bit appended to the right, to produce a 21-bit immediate value to be used in the computation.

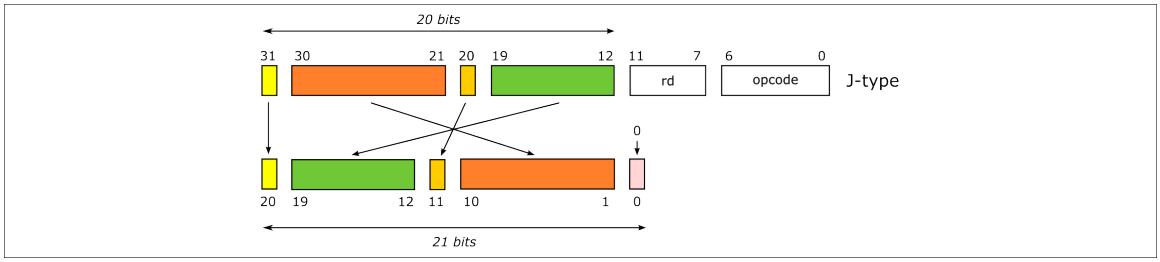


Figure 2.4: Construction of 21-bit immediate from 20 "imm" bits in J-type instructions

Similarly, Figure 2.5 shows how the 12 "imm" bits in a B-type instruction are permuted, with a 0 bit appended to the right, to produce a 13-bit immediate value to be used in the computation.

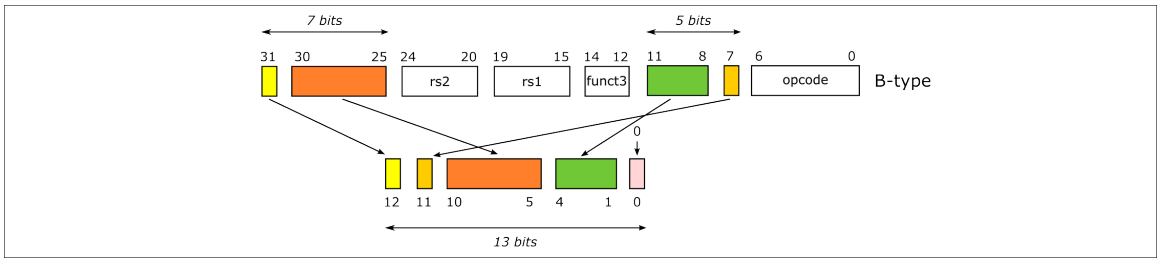


Figure 2.5: Construction of 13-bit immediate from 12 "imm" bits in B-type instructions

In both of the above, the encoding takes advantage of the fact that Jump and Branch target PCs are always at least 2-byte aligned, and therefore we do not have to use up an instruction "imm" bit to represent the "0" least-significant bit. The extra bit in the immediate value effectively doubles the "distance" that a jump or branch can span.[1]

The fact that there are so few encoding formats justifies the "RISC" in the ISA name: Reduced Instruction Set Computer. Having fewer formats, with few or zero special cases, simplifies the hardware needed to decode an instruction.

2.6 Unprivileged ISA RV32I

Figure 2.6 reproduces the table on page 130 of the RISC-V Unprivileged ISA specification, which shows all RV32I instructions. On the right-side of the diagram we have labeled different classes of instructions.

Note that there are just 40 instructions (again justifying the "RISC" name).

2.6.1 "Upper Immediate" instructions LUI and AUIPC

Figure 2.7 reproduces the top of page 19 of the RISC-V Unprivileged ISA specification, which describes the semantics of the LUI and AUIPC instructions in prose text. (Please

[1] Actually in RV32I and RV64I PC targets are 4-byte aligned. The 2-byte alignment here accomodates the optional C (Compressed) extension, where instructions may be only 2-byte aligned.

RV32I Base Instruction Set

imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20|10:1|11|19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND
fm | pred | succ		rs1	000	rd	0001111	FENCE
000000000000		00000	000	00000	1110011	ECALL
000000000001		00000	000	00000	1110011	EBREAK

"load immediate"-kind: to load constant values into a register

"jump-and-link"-kind: subroutine calls and returns; distant jumps

"conditional branch"-kind: test and possibly jump up to ~0x1000 distance

"load data from memory into register" (rs1 and imm specify address)

"store data from register rs2 to memory" (rs1 and imm specify address)

"integer arithmetic operations (register-immediate)"

"integer arithmetic operations (register-register)"

"system" operations (ignore FENCE for now)

Figure 2.6: RISC-V RV32I Instructions

SRLI is a logical right shift (zeros are shifted into the upper bits); and SRAI is an arithmetic right shift (the original sign bit is copied into the vacated upper bits).

31 12	11 7	6 0
imm[31:12]	rd	opcode
20	5	7
U-immediate[31:12]	dest	LUI
U-immediate[31:12]	dest	AUIPC

LUI (load upper immediate) is used to build 32-bit constants and uses the U-type format. LUI places the U-immediate value in the top 20 bits of the destination register *rd*, filling in the lowest 12 bits with zeros.

AUIPC (add upper immediate to `pc`) is used to build `pc`-relative addresses and uses the U-type format. AUIPC forms a 32-bit offset from the 20-bit U-immediate, filling in the lowest 12 bits with zeros, adds this offset to the address of the AUIPC instruction, then places the result in register *rd*.

Figure 2.7: RISC-V LUI and AUIPC Instruction semantics

ignore the first first two lines in the figure about the SRLI and SRAI instructions which are discussed in Section 2.6.5.)

Figure 2.8 is a pictorial depiction of the semantics of an LUI instruction. At the left of the

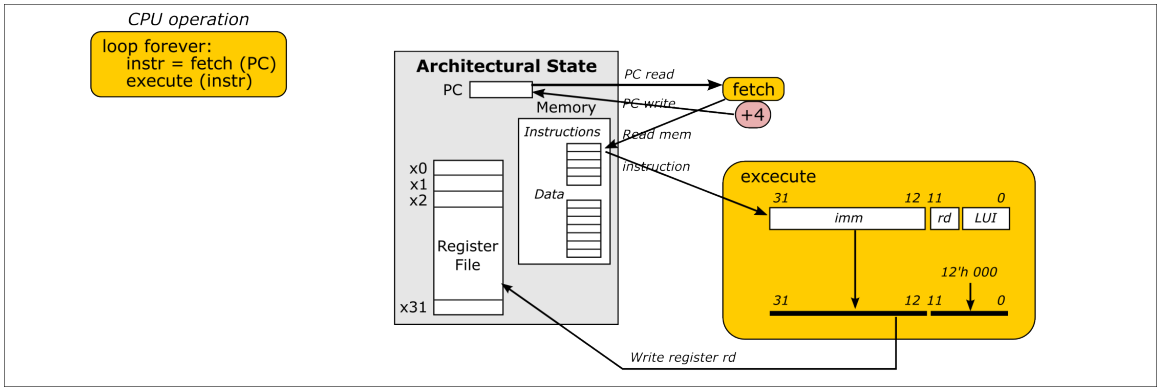


Figure 2.8: Execution semantics for LUI instructions

figure is depiction of the general behavior of a CPU, namely to loop forever, at each iteration fetching the instruction that is in memory at the address specified by the PC register, and then executing that instruction. For the LUI instruction, to the 20 bits [31:12] of the instruction (the "immediate" value) we append twelve bits of constant 0 as least-significant bits to construct a 32-bit value. This value is then written into register rd.

Figure 2.9 is a pictorial depiction of the semantics of an AUIPC instruction. As with LUI,

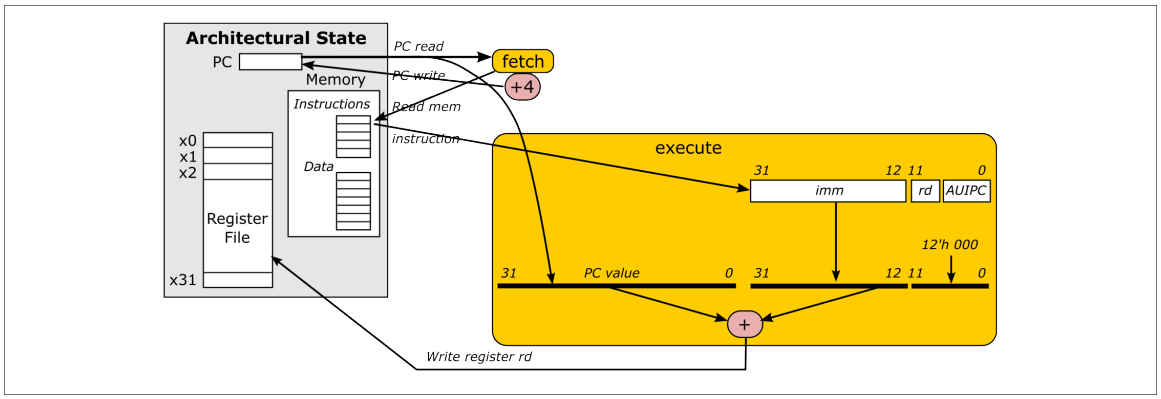


Figure 2.9: Execution semantics for AUIPC instructions

to the 20 bits [31:12] of the instruction (the "immediate" value) we append twelve bits of constant 0 as least-significant bits to construct a 32-bit value. For AUIPC we add this value to the 32-bits of the PC, and the result is then written into register rd.

2.6.2 Conditional BRANCH instructions

Conditional branch instructions compare the values in registers rs1 and rs2 for some condition: BEQ (equal), BNE (not-equal), BLT (less-than), BGE (greater-than-or-equal). If the condition is:

- false ("branch not taken"):
  - The instruction is a no-op, it just falls through to the next instruction (PC := PC+4).
- true ("branch taken"):
  - The 12 "imm" bits form a 13-bit value (see Figure 2.5).
  - This 13-bit value is sign-extended to 32 bits and added to the current PC to form a target address ("branch target").
  - The PC is updated to contain the target address.

Because of sign-extension (+/-), the branch may be forwards or backwards relative to the current PC.

For BLT and BGE, the comparison treats the two input 32-bit values as *signed* values. For BLTU and BGEU, they are treated as *unsigned* values.

2.6.3 LOAD and STORE memory-access instructions

LOAD (LB, LH, LW, LBU, LHU) instructions move data from memory into a register. STORE (SB, SH, SW) instructions move data from a register to memory. In both cases, the address is formed by sign-extending the 12-bit immediate value to 32 bits and adding it to the value from register rs1. For LOAD instructions, the value loaded is placed into register rd. For STORE instructions, the value in register rs2 is stored to memory, and there is no result value written into any register.

As with most modern ISAs, RISC-V memory is byte-addressed, *i.e.,* each address refers to a specific byte in memory. The size of the data to be loaded/stored is given by "B" (1 Byte), "H" (Halfword = 2 bytes), or "W" (Word = 4 bytes) in the instruction name.

The difference between LB and LBU is whether the 8 bits loaded (1 byte) are sign-extended or zero-extended, respectively, to 32 bits when placed in the 32-bit rd. (And similarly for LH vs. LHU.)

2.6.4 Register-Register Arithmetic and Logic instructions

These instructions read registers rs1 and rs2, perform the specified arithmetic/logic operation on the two values, and store the result into register rd.

SLT ("set if less than") tests if register[rs1] < register[rs2], and writes 0 (false) or 1 (true) into the destination register, treating the inputs as 32-bit signed values. SLTU treats inputs as unsigned values.

SRL ("shift right logical") shifts the value from register[rs1] to the right (towards the least-significant bits) by the amount in register[rs2]. Zeroes are shifted in from the left.

In SRA ("shift right arithmetic"), the register[rs1][31] is shifted in from the left, *i.e.,* the 32-bit value is treated as a signed integer (bit 31 is the sign bit).

In SLL ("shift left logical"), zeroes are shifted in from the right.

In all three shifts, since there are only 32 bits in register[rs1], only the lower 5 bits of register[rs2] are relevant (*i.e.,* bits [4:0]); the rest are ignored.

2.6.5 Register-Immediate Arithmetic and Logic instructions

These instructions read register rs1, form a signed 32-bit value from the 12-bit "imm" bits, and perform the specified arithmetic/logic operation on the two values, and store the result into register rd.

In SRLI ("shift right logical immediate"), SRAI ("shift right arithmetic immediate"), and SLLI ("shift left logical immediate"), the 5-bit `shamt` field provides the "shift amount".

2.6.6 Unconditional Jump instructions

JAL ("jump and link") forms a 21-bit value from the 20 "imm" bits (see Figure 2.4), sign-extends it to 32 bits, adds it to the current PC value and updates the PC with the result.

JALR ("jump and link register") forms a 12-bit value from the 12 "imm" bits, sign-extends it to 32 bits, adds that to the value in register[rs1], forces bit [0] to be 0, and updates the PC to that value

Both JAL and JALR store the current PC + 4 ("return address") into register[rd].

These are most often used for subroutine calls and returns. Figure2.10 illustrates the protocol.

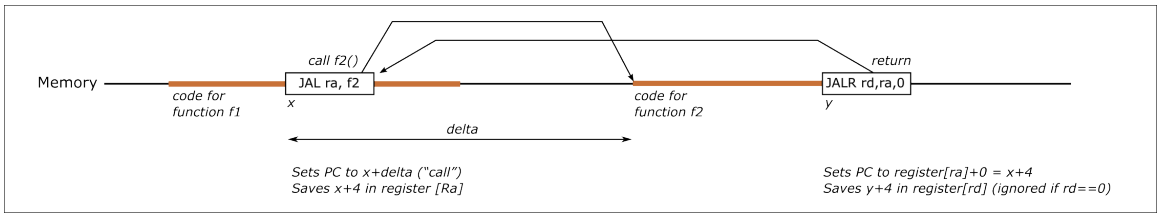


Figure 2.10: JAL and JALR for subroutine call and return

Suppose, inside function f1(), at address x, we have a `JAL ra,f2` instruction representing a subroutine call to function f2(). The compiler/linker will place "delta" in the JAL instruction's "immediate" field, where delta is the difference between x and the entry point of f2(). In RISC-V Assembly Language, "ra" ("return address") is another name for register[1], which is used to hold return addresses as part of the standard software calling convention. The JAL instruction saves x+4 (the return address) in register ra, and sets the PC to x+delta, so that the next instruction fetched will be the entry point of f2().

Inside f2(), at address y, suppose we have a `JALR 0,ra,0` instruction representing the return to the caller. It saves y+4 in register 0, but recall that writes to register 0 are always ignored. It reads the value in register ra, adds 0 to it and sets the PC to the result value, *i.e.,* the next instruction fetched will be from x+4.

JAL is used for most subroutine calls which are to a manifestly known subroutines (so the compiler/linker can compute the "delta" for the immediate value). JALR is used for subroutine calls where the "delta" is not known to the compiler/linker, for example when calling through a table of function pointers.

JALR is also used for subroutine calls and returns and conditional branches to "distant" target addresses. Remember that BRANCH instructions only have a 13-bit signed offset

from the current PC, and JAL only has a 21-bit signed offset. For more distant jumps, we can construct a full 32-bit value in a register (using one or more LUI and AUIPC instructions, for example) and then use that as the jump rs1 in a JALR instruction.

2.6.7 FENCE

The FENCE instruction is intended for RISC-V implementations that contain caches in the memory system. In such systems, data may not reach memory quickly or at all (is in the cache and is not evicted), and two items of data x_1 and x_2 may reach memory in a different order from the STORE instructions that wrote them (because the order in which their cache lines are written back to memory has no relationship to the program STORE order).

The FENCE instruction is often used to "push" data from the cache to memory, although technically the FENCE instruction only guarantees an "ordering", *i.e.,* that if there are two STOREs x_1 and x_2 before and after a FENCE, respectively, then x_1 will be visible before x_2 to any other agent in the system (another CPU, a DMA engine, an I/O device, *etc.*).

2.7 Traps due to illegal instructions and other exceptions, and CSRs

Most processors cannot afford to get "stuck" on an unrecognized instruction (illegal instruction). An illegal instruction raises one of several possible "exceptions". Other events that raise exceptions are misaligned memory access (in FETCH or LOAD/STORE), or a memory-access to an unimplemented address (in FETCH or LOAD/STORE), *etc.*.

NOTE: "Illegal" just means: outside the currently implemented subset. For example (see Figure 2.2), a legal RISC-V instruction in, say, the M extension is considered illegal in an implementation that only implements the RV32I subset.

Exceptions are treated like an "unexpected call" to a special subroutine called a "trap handler". The architectural state is extended with a few extra registers belonging to a class of registers called CSRs (Control and Status Registers), shown in Figure 2.11.

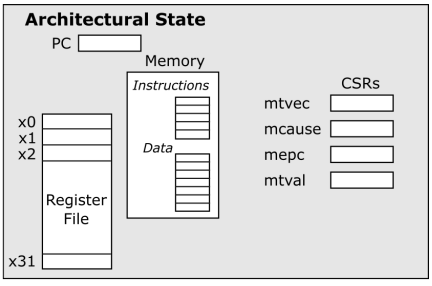


Figure 2.11: CSRs for handling traps

To "return" from a trap handler we need one more instruction: MRET, which is in the "RISC-V Privilege M ISA" (see Figure 2.2). Figure 2.12 illustrates the flow into the trap

handler and back on an exception (an ILLEGAL instruction is one possible cause of the exception).

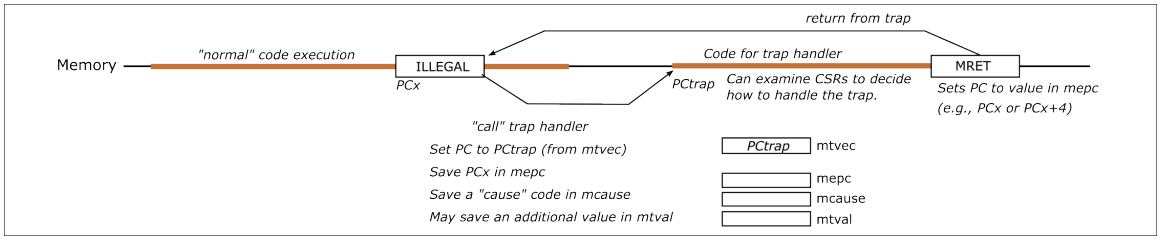


Figure 2.12: Trap and return flow

When the exception is detected, the hardware saves the faulting PC (PCx in the figure) in CSR MEPC, saves a cause-code in CSR MCAUSE and possibly one more piece of data (depending on the type of exception) in CSR MTVAL. Then, it sets the PC to the value in MTVEC, so that we start executing the trap-handler code.

When the trap-hanlder has finished, it executes an MRET instruction which copies the value in MEPC into the PC, continuing execution at that location.

Exception-cause codes relevant to this book (RV32I + exception handling) are shown in the table below (excerpted from Table 3.6 in the Privileged ISA Specification document).

Exception-Cause code	Description
0	Instruction address misaligned
1	Instruction access fault
2	Illegal instruction
3	Breakpoint
4	Load address misaligned
5	Load access fault
6	Store/AMO address misaligned
7	Store/AMO access fault
...	...
11	Environment call M-mode
...	...

The trap handler code can examine MCAUSE, MEPC and MTVAL to determine what it should do to handle the trap. Regarding MEPC, it may:

- Leave MEPC untouched, so that, on MRET, we retry the exceptional instruction.

  Example: the exception was a page-fault due to a FETCH on an umapped page. The trap handler may map the page, and MRET to the faulting instruction to be retried (which, hopefully, should not page-fault again).

- Increment MEPC by 4, so that, on MRET, we resume at the next instruction after the faulting instruction.

  Example: the faulting instruction is a legal RISC-V instruction, but has deliberatly been left unimplemented (*e.g.,* to save hardware cost). The trap handler "emulates"

the instruction (*i.e.,* performs the required computation using implemented instructions), and resumes the normal flow at the next instruction.

Example: the trap handler just records that this instruction is illegal so that it can avoid it in subsequent execution.

- Change it to something else entirely, to abandon the current "normal" flow and do something else.

  Example: in an Operating System, we save MEPC for future resumption, and change it to give another process a chance to execute.

2.7.1 ECALL and EBREAK instructions, and Interrupts

RV32I instructions ECALL and EBREAK are handled just like other exceptions; in that sense, these are not "unexpected" conditions, but exceptions deliberately induced by the program. The only notable feature is that they place certain exception-cause codes in the MCAUSE CSR (see "Environment call M-mode" and "Breakpoint", respectively, in the table above).

In systems with operating systems, ECALL is used to move in a disciplined way from lower to higher privilege levels (from User mode to Supervisor mode, and from Supervisor Mode to Machine mode).

Some RISC-V implementations contain a hardware "Debug Module", in which case EBREAK is treated differently (please see the RISC-V Debug Module specification document). It is used as part of the implementation of the "break" command in debuggers like GDB, LLDB and OpenOCD.

An external interrupt is also dispatched into the trap-handler just like exceptions, the only difference being the code placed in the MCAUSE register (see Table 3.6 in the Privileged ISA Specfication document for all the defined interrupt cause codes).

2.7.2 CSRRxx instructions

In Section 2.7 we described how various CSRs are read and written during trap-handling. These reads and writes are performed *by the hardware* as part of taking a trap or performing an MRET.

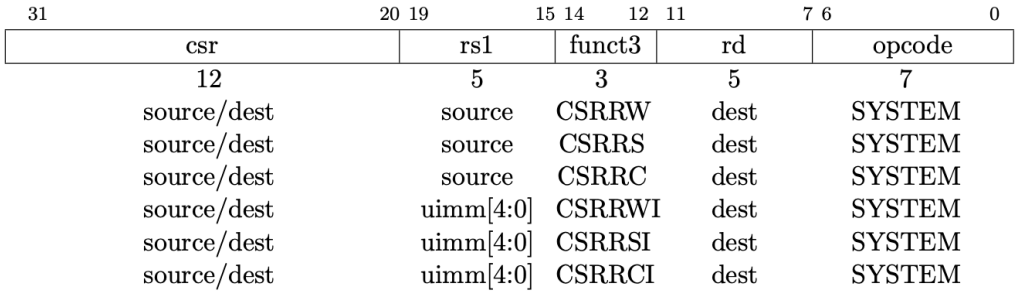
But we also need a way to read and write CSRs *programmatically*, *i.e.,* from instructions in the program. For example, before any trap is taken, some early part of the execution needs to write the trap-vector's PC into MTVEC. During trap-handling, the trap-handler needs to read (and possibly write) MEPC, MCAUSE and MTVAL.

Programmatic access to CSRs is provided by a family of six "CSRRxx" instructions. The following table is excerpted from the RISC-V Unprivileged ISA spec document (Chapter 9, on the "Zicsr" extension). Each CSRRxx instruction:

- reads a value y from the CSR (whose 12-bit address is given in `instr[31:20]`);
- and writes y into a general-purpose register (rd);
- takes a value x either by reading from a general-purpose register (rs1), or using the rs1 field itself as a literal 5-bit value and zero-extending it to XLEN bits;

9.1 CSR Instructions

All CSR instructions atomically read-modify-write a single CSR, whose CSR specifier is encoded in the 12-bit *csr* field of the instruction held in bits 31–20. The immediate forms use a 5-bit zero-extended immediate encoded in the *rs1* field.



31 — 20	19 — 15	14 — 12	11 — 7	6 — 0
csr	rs1	funct3	rd	opcode
12	5	3	5	7
source/dest	source	CSRRW	dest	SYSTEM
source/dest	source	CSRRS	dest	SYSTEM
source/dest	source	CSRRC	dest	SYSTEM
source/dest	uimm[4:0]	CSRRWI	dest	SYSTEM
source/dest	uimm[4:0]	CSRRSI	dest	SYSTEM
source/dest	uimm[4:0]	CSRRCI	dest	SYSTEM

Figure 2.13: CSRRxx instructions (from Unprivileged Spec)

- and updates the value in the CSR using x.

Figure 2.14 illustrates the semantics of CSRRxx instructions. Some important points:

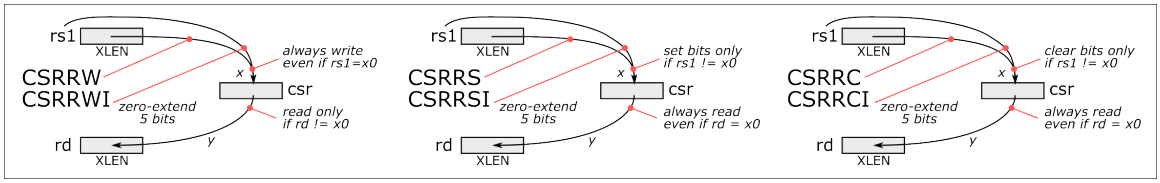


Figure 2.14: CSRRxx instruction semantics

- CSRRW and CSRRWI simply write the value x into the CSR.
- CSRRW and CSRRWI do not read the CSR if rd= 0. Specifically, even *reading* a CSR can have side-effects for certain CSRs, so avoiding this read prevents those side-effects.
- In CSRRS and CSRRSI, for each "1" bits in x, it sets the corresponding bits in the CSR to 1.
- In CSRRC and CSRRCI, for each "1" bits in x, it clears the corresponding bits in the CSR to 0.
- CSRRS, CSRRSI, CSRRC and CSRRCI update the CSR only if rs1 is not zero. Specifically, *writing* a CSR bit can have side-effects for certain CSRs (beyond just writing that bit), so avoiding this write prevents those side-effects.

For more details, please read Chapter 9 of the RISV-V Unprivileged ISA specification.

Three CSRs are often used to measure performance of a RISC-V CPU: INSTRET, CYCLE and TIME, and these are described in the following sections.

2.7.2.1 CSRs INSTRET and INSTRETH

A RISC-V CPU contains a CSR to count the number of instructions that have executed completely (the technical term for which is "retired"). The count is maintained by the

hardware, incrementing on each retired instruction, and can be ready by software using a CSRRxx instruction.

The CSR is 64 bits wide, so on RV32 systems, reading the INSTRET CSR returns the lower 32 bits and reading the INSTRETH CSR (H for "high") returns the upper 32 bits.

2.7.2.2 CSRs CYCLE and CYCLEH

A RISC-V CPU contains a CSR to count the number of clock cycles that have elapsed. The count is maintained by the hardware, incrementing on each clock cycle, and can be ready by software using a CSRRxx instruction.

The CSR is 64 bits wide, so on RV32 systems, reading the CYCLE CSR returns the lower 32 bits and reading the CYCLEH CSR (H for "high") returns the upper 32 bits.

2.7.2.3 CSRs TIME and TIMEH, and memory-mapped location MTIME

Most modern computers have a so-called "real-time clock" which is a circuit that maintains precise actual elapsed time. Unlike clock cycles, which may vary dynamically based on instantaneous performance and power-saving demands, a real-time clock advances at a fixed rate. Because real-time clock circuits can be complex and expensive, they are not required to be part of a RISC-V CPU itself. Instead, they are memory-mapped devices—the CPU can issue a LOAD instruction to a particular address called the "MTIME" address to read a number representing "ticks" of a real-time clock. By making it a memory-mapped device, a real-time clock can also be shared by multiple CPUs.

RISC-V specifies the real-time clock to be a 64-bit memory-mapped register. The exact rate at which it advances is not specified, as long as it is a constant rate. Typically it "ticks" every few nanoseconds.

In addition to accessing the real-time clock via LOAD instructions at the MTIME address, the CPU can also access it via CSRRxx instructions using the CSR address TIME. In RV64 this returns the entire 64-bit value. In RV32 it returns the lower 32 bits, and a CSRRxx access to the CSR address TIMEH returns the upper 32 bits.

2.7.2.4 Measuring CPU performance

Using the INSTRET and CYCLE CSRs, software can measure a standard metric for CPU implementations, CPI (Cycles per Instruction) or its inverse IPC (Instructions per Cycle). These measures can vary depending on the current mix of instructions, because different instructions may take a different number of clocks to complete. For example, simple integer-op instructions finish in few cycles, whereas the CPU may need more cycles to execute integer multiplication, division, and square root, and memory operations, and floating operations, and system instructions. Memory operation may take fewer cycles on a cache-hit and TLB-hit and more cycles on a cache-miss or TLB-miss. Further, modern advanced CPUs can dynamically vary their clock speeds depending on the instantaneous demand for performance and/or power-saving.

By reading the TIME CSR before and after an application, software can measure the actual elapsed time to run a particular application on a particular input data-set on a particular RISC-V CPU implementation.

2.8 RV64I differences from RV32I

The Architectural State for RV64I is just like that for RV32I, except that the PC and 32 registers are now 64-bits wide instead of 32-bits wide. Figure 2.15 reproduces the table of RV64I instructions from page 131 of the Unprivileged Spec document. RV64I starts

Volume I: RISC-V Unprivileged ISA V20191213 131

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
funct7		rs2	rs1	funct3	rd	opcode	R-type
imm[11:0]			rs1	funct3	rd	opcode	I-type
imm[11:5]		rs2	rs1	funct3	imm[4:0]	opcode	S-type

RV64I Base Instruction Set (in addition to RV32I)

imm[11:0]		rs1	110	rd	0000011	LWU
imm[11:0]		rs1	011	rd	0000011	LD
imm[11:5]	rs2	rs1	011	imm[4:0]	0100011	SD
000000	shamt	rs1	001	rd	0010011	SLLI
000000	shamt	rs1	101	rd	0010011	SRLI
010000	shamt	rs1	101	rd	0010011	SRAI
imm[11:0]		rs1	000	rd	0011011	ADDIW
0000000	shamt	rs1	001	rd	0011011	SLLIW
0000000	shamt	rs1	101	rd	0011011	SRLIW
0100000	shamt	rs1	101	rd	0011011	SRAIW
0000000	rs2	rs1	000	rd	0111011	ADDW
0100000	rs2	rs1	000	rd	0111011	SUBW
0000000	rs2	rs1	001	rd	0111011	SLLW
0000000	rs2	rs1	101	rd	0111011	SRLW
0100000	rs2	rs1	101	rd	0111011	SRAW

Figure 2.15: RV64I instructions (in addition to RV32I)

with the same forty instructions as in RV32I, except that it replaces 3 of them—SLLI, SRLI and SRAI—with slight modifications: the "shamt" field is now 6 bits instead of 5, to accommodate 64-bit shifts.

In RV64I, the LW instruction loads 32-bits from memory, sign-extends it to 64 bits and stores it in the destination register. RV64I adds the LWU instruction that does the same, except that it zero-extends the 32-bit value from memory. RV64I also adds the LD instruction to load 64-bits from memory into a register.

RV64I adds the SD instruction to store a 64-bit value from a register into memory.

In RV64I, ADDI, SLLI, SRLI, SRAI, ADD, SUB, SLL, SRL and SRA all operate on 64-bit values. RV64I adds ADDIW, SLLIW, SRLIW, SRAIW, ADDW, SUBW, SLLW, SRLW and SRAW to operate on the lower 32-bits of 64-bit register values.

2.9 Abstract algorithm for interpreting an ISA

From our previous study of the RISC-V ISA, we know that the basic integer "architectural state" of a RISC-V CPU is very simple:

- A "program counter" (PC) indicating the address in memory of the next instruction to be executed.
- A "register file" consisting of 32 general purpose registers (GPRs), each containing data.

The PC and each register are either 32-bits wide (in the RV32 option of RISC-V) or 64-bits wide (in the RV64 option). For simplicity, we'll focus on RV32 here, but everything we discuss also applies to RV64.

Interpreting a program involves the repetition of a few simple stages for each instruction, illustrated in Figure 2.16:

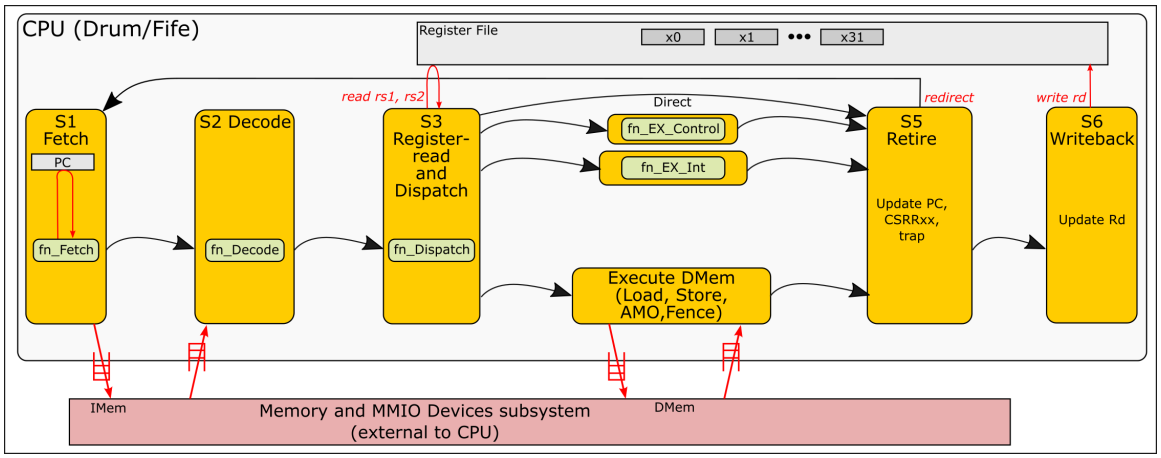


Figure 2.16: Simple interpretation of RISC-V instructions

- The "Fetch" stage reads the current value of the PC and uses that value as an address in memory from which to read an instruction. Then, we proceed to the "Decode" stage.
- The "Decode" stage examines the fetched instruction to check if it is legal, to classify its major category (such as Control, Integer Arithmetic/Logic, or Memory), and to extract some properties such as which GPRs it reads (if any) and which GPR it writes (if any). Then, we proceed to the "Register-Read and Dispatch" stage.
- The "Register-Read and Dispatch" stage reads the GPRs for the instruction's inputs. Then, we proceed to one of the "Execute" stages, based on the category of the opcode in the instruction (Branch/Jump, Integer Arithmetic/Logic, or Memory).
- The "Execute Control" stage is used for conditional-branch and jump instructions. For the former it evaluates the branch condition and, if true, and computes the branch-target PC. For jump instructions it computes the jump-target PC. Then, we proceed to the "Retire" stage.
- The "Execute Integer Arithmetic and Logic" stage is used for integer arithmetic and logic operations (addition, subtraction, boolean ops, shifts, *etc.*). Then, we proceed to the "Retire" stage.

- The "Execute Data Memory (DMem) Ops" stage calculates a memory address based on an input value (that was read from a GPR) and reads or writes memory at that address. Then, we proceed to the "Retire" stage.
- The "Direct" stage is used for all other instructions: instructions that have already trapped (Fetch memory errors, illegal/unrecognized instructions), CSRRxx instructions, *etc.*. This "stage" does not do anything is thus shown in the diagram as a direct arc to the "Retire" stage.
- The "Retire" stage performs instructions that came through the Direct path (include CSRRxx instructions) and updates the PC from where we will fetch the next instruction. It also performs all trap actions.
- The "Register-Writeback" stage updates a GPR (for those instructions that update a GPR). Then, it goes back to the Fetch stage to interpret the next instruction.

Thus, we repeat these steps forever, instruction after instruction, starting each time at the Fetch stage.

In Figure 2.16, note that the memory accesses, Fetch⟶Memory⟶Decode and Execute DMem⟶Memory⟶Execute DMem, are each shown with two arrows, one going to memory and another returning from memory. This is because, as discussed in Section 1.5.4 memory-access is always a split-phase operation.

2.10 Continued Evolution of the RISC-V ISA (with your contribution?)

The RISC-V ISA is not frozen. As shown in Figure 2.2, the ISA has consciously been designed in a modular way and to be modularly extensible. RISC-V International (RVI, https://riscv.org) runs an organized process for the continued maintenance and evolution of the ISA. Special-interest groups drawn widely from RVI membership constitute various committees under the aegis of RVI to propose, develop, and specify new extensions to the ISA (for cryptography, for image and video manipulation, for high-performance computing, for AI, and so on). There is a formal public-review and ratification process for any new proposed extensions.

RVI has the usual corporate membership tiers seen in many consortiums but, unusually, it also has inexpensive memberships for individuals (students, hobbyists, ...) and academic institutions. So please feel free to join, in order to monitor the RISC-V ecosystem closely or, even better, to actively contribute to its future,

Chapter 3

BSV: Overview of the BSV program structure

3.1 Introduction

In this chapter we describe the top-level view of a **BSV** program. The goal is to quickly develop an ability to scan **BSV** code (from Fife or Drum or any other example) and to discern the structure and relationships of the components in the code.

At the syntactic level, a **BSV** design can be organized into multiple source files, each containing a **BSV** "`package`". Packages contain top-level definitions of types, values, functions, interfaces and modules. The names of these entities are local to the package. Components defined in one package are visible in other packages using an "`import`" statement.

At the semantic level, a **BSV** design is organized into hardware "`module`s". Each module presents, to the outside world (the environment), an "`interface`" containing "`method`s", which constitute the API by which the environment interacts with the module.

A module may contain "`rule`s" which are independent processes. It can invoke instances of other modules by invoking their interface methods.

The sections of this chapter present an overview of these major components, after first showing a simple (even trivial) example **BSV** program.

3.2 A Minimal BSV program

Here is a minimal complete **BSV** program (although it is not very interesting as hardware!).

in file Exercises/Ex-03-A-Hello-World/Top.bsv

```
1  module mkTop (Empty);
2
3     rule rl_once;
4        $display ("Hello, World!");
5        $finish (0);
6     endrule
7
8  endmodule
```

It defines one module, `mkTop`, with an `Empty` interface, *i.e.,* a module with no inputs or outputs (we will discuss modules and interfaces in more detail later). The module contains one rule, `rl_once`, whose rule body contains two statements, a `$display()` statement/expression that prints the message and a `$finish(0)` statement/expression which terminates the simulation immediately, returning an exit value of 0 to the operating system. Rules can normally execute repeatedly forever, but here the `$finish()` terminates it after one execution (rules are discussed in more detail in Chapters 13 and 18).

NOTE:

> `$display()` is only meaningful in simulation, where the simulator can print out characters to the terminal in which the simulation is running. Actual hardware may or may not have any terminal at all. Some hardware may have a UART (Universal Asynchronous Receiver/Transmitter) that can send characters to a terminal. For this, the CPU must run a RISC-V program that sends the characters, one by one, to the terminal.
>
> `$finish()` is only meaningful in simulation, where the simulation is a program running under some operating system (Linux, Windows, MacOS, ...) and it is meaningful to "terminate" the simulation and return to a command-line with an exit value. Actual hardware has no concept of "terminating", it just runs forever as long as it is supplied power and a clock signal.

We can simulate in **Bluesim** or in Verilog simulation; in both cases it just prints "Hello World!" and exits. For **Bluesim**, we can compile and link it using the free and open-source *bsc* compiler, and then run the executable:

```
1  $ bsc -sim  -g mkTop  Top.bsv                          # compile
2  $ bsc -sim  -e mkTop  -o exe_HW_bsim                   # link
3  $ ./exe_HW_bsim                                        # run
4  Hello, World!                                          # output
```

We can compile it to Verilog, then compile the Verilog and link it for Verilog simulation using the free and open-source *verilator* compiler, and then run the executable:

```
1  $ bsc -verilog  -g mkTop  Top.bsv                      # compile -> mkTop.v
2  $ bsc -vsim verilator  -e mkTop  -o exe_HW_vsim        # link
3  $ ./exe_HW_vsim                                        # run
4  Hello, World!                                          # output
5  - mkTop.v:41: Verilog $finish
```

In both the **Bluesim** and **Verilator** cases, the "`-g`" flag tells *bsc* the name of the top-level module for which to generate code, "`-e`" tells it which is the top-level module for simulation, and "`-o`" tells it the desired name for the final executable.

In **Bluesim** compile-link, "`-sim`" tells *bsim* that our target is **Bluesim**.

In Verilog compile-link, "`-verilog`" tells *bsim* to generate Verilog, and it produces the Verilog file `mkTop.v`. The "`-vsim`" flag tells *bsc* to use the **Verilator** compiler to compile the generated Verilog into a Verilog simulation executable.

EXERCISE: Please see Ex-03-A-Hello-World in Appendix E, page E-1. □

3.2.1 More about `$display()`

The `$display()` function in **BSV** is similar to `$display()` in Verilog and SystemVerilog, and to `printf()` in C/C++. The first argument is a string containing formatting directives for the remaining arguments. Example:

```
1     Bit #(4) a = 'b_1010;
2     Bit #(4) b = 'b_0110;
3
4     $display ("  %04b %04b => %d", a, b, a == b);
```

The `a` and `b` arguments to `$display()` are printed in binary format using 4-characters, prefixing it, if necessary, with the "0" character.

The `a==b` argument is an expression whose value is of type `Bool` (Section 4.7). Here, we are printing its bit-representation with the "`%d`" format (decimal integer), so it will print "0" for `False` and "1" for `True`.

BSV's `$display()` goes beyond SystemVerilog's `$display()` in that arguments can can also be of type `Fmt` which are computed formatted objects created by the `fshow()` function which is predefined on all the standard types including `Bool`. In our example, we can have it print the *words* `False` and `True` instead of "0" and "1" by using `fshow()`; in this case we omit the "`%d`" from the format string:

```
...
$display ("  %04b %04b => ", a, b, fshow (a == b));
...
```

The use of `Fmt` and `fshow()` is discussed in more detail in Section 11.3.

3.3 Packages and files

Like many programming languages, **BSV** has facilities to organize large programs into separately compilable, purposeful, reusable parts. A **BSV** program may be organized into one or more *files* or *packages*. This is illustrated in Figure 3.1.

A package can contain a number of top-level statements (more detail about this in Section 3.3.1). One kind of top-level statment is the "`import`" statement, illustrated in the diagram. This allows an importing package p_1 to use all the identifiers exported by a package p_2 (in addition to indentifiers defined in p_1 itself).

There is a one-to-one correspondence between files and packages. In each import statements in the diagram we provide a package name p; the *bsc* compiler uses the imported package name p to find a file called p`.bsv` containing the definition of package p.

3.3.1 What's in a Package?

Figure 3.2 illustrates the typical structure and contents of a package/file.

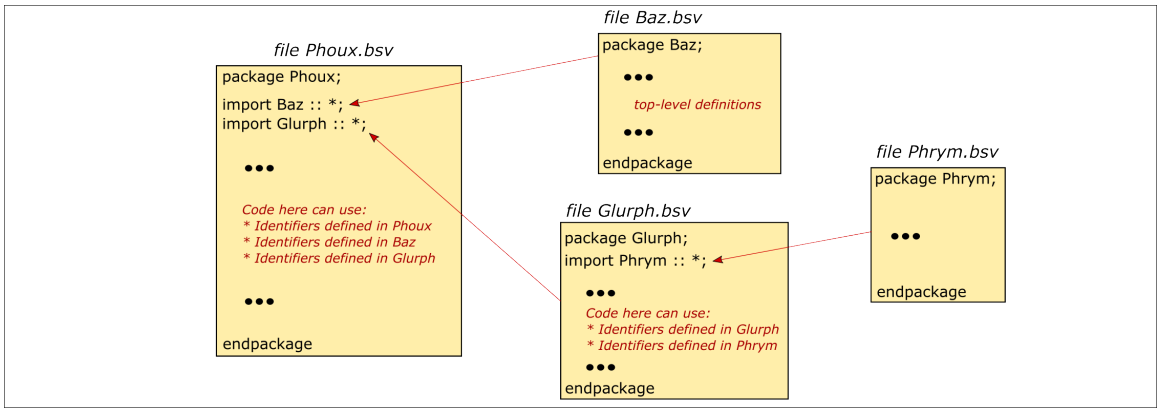


Figure 3.1: File-level view of a **BSV** program

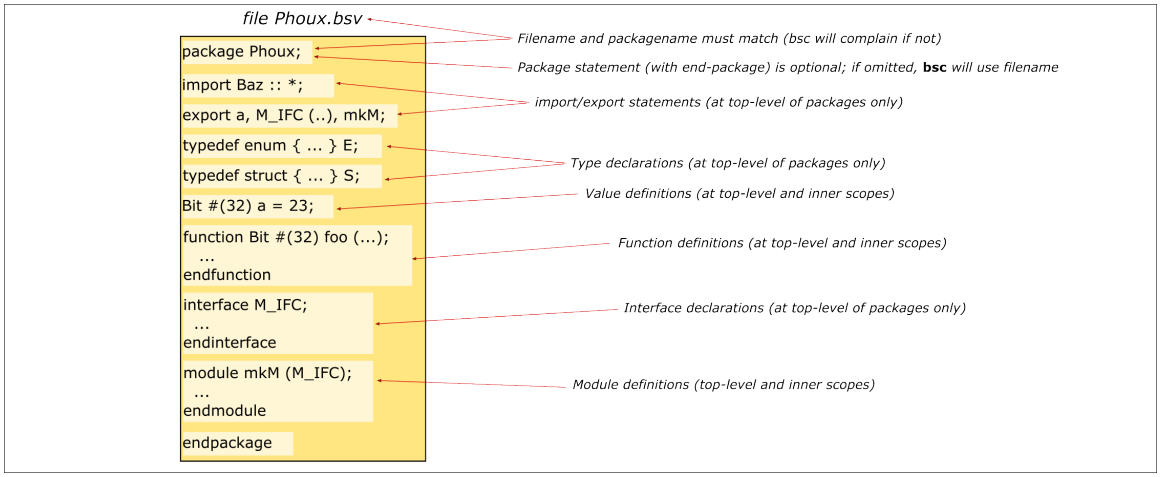


Figure 3.2: What's in a **BSV** package/file?

The `package`–`endpackage` keywords bracketing the text—the first and last items in the file—are optional; if omitted, the *bsc* compiler will implicitly create a package name using the file name. We recommend always to use `package`–`endpackage`, except perhaps for small, experimental, one-off programs to test some small concept.

At the top-level of a package we find import/export statements, type declarations, value definitions, function definitions interface declarations and module definitions.

Import/export statements and type and interface declarations are only allowed at the top-level of a package and not inside any nested scopes. An interface declaration is just a kind of type declaration; its declared interface identifier is used just like a type.

Value, function and module definitions appear at package top-level, but can also be inside nested scopes (inside function and module definitions, or other local scopes like `begin`–`end` and `action`–`endaction` blocks). Function and module definitions are, in fact, just value definitions of functional and module type, respectively.

The top-to-bottom order of entities in a package is not important, just that if an entity x is defined in the package and used by another entity y in the same package, then x should be given before y. We typically place import/export statements at the top of the file.

Each package/file is separately compiled by the *bsc* compiler, which saves information about the compilation so that it won't be recompiled unless the source file has subsequently been modified.

3.3.2 Extending our example minimal BSV program

Let us extend our minimal **BSV** program into one that is split into two packages/files. We create a new file `DUT.bsv` that just defines some top-level scalar constants (strings, bit-vectors).

in file Ex-03-B-Top-and-DUT/DUT.bsv

```
1  package DUT;
2
3  String title   = "The C Programming Language";
4  String authors = "Kernighan and Ritchie";
5
6  Bit #(11) year = 1978;
7
8  Bit #(4)  month = 2;
9
10 Bit #(5)  day = 22;
11
12 endpackage
```

The first and last lines specify that this is a package called `DUT`. This was not strictly necessary since *bsc* will use the package name `DUT` anyway from the filename `DUT.bsv`, but we prefer always to have package names in our files.

In each line we have a *type*, a *variable*/*identifier*/*name*, and a value; it declares the variable to have that type and that value.

Next, we extend our `mkTop` module from the previous example:

in file Ex-03-B-Top-and-DUT/Top.bsv

```
package Top;

import DUT :: *;

module mkTop (Empty);

   rule rl_once;
      $display ("Hello, World!");
      $display ("  (From the book: %s", title);
      $display ("   by:            %s", authors);
      $display ("   which was first published on: %4d-%02d-%02d)",
                year, month, day);
      $finish (0);
   endrule

endmodule

endpackage
```

We have explicitly named the package `Top`. We `import` the `DUT` package so we can use the variables/identifiers/names defined in that package. In the rule, we have added two more `$display()` statements to print values from the `DUT` package.

We can compile and link it for **Bluesim** using the *bsc* compiler, and then run the executable:[1]

```
$ bsc  -u  -sim  -g mkTop  Top.bsv                    # compile
$ bsc  -sim  -e mkTop  -o exe_HW_bsim                 # link
$ ./exe_HW_bsim                                       # run
Hello, World!                                         # output
  (From the book: The C Programming Language
   by:            Kernighan and Ritchie
   which was first published on: 1978-02-22)
```

We can compile it to Verilog, then compile the Verilog and link it for Verilog simulation using the *verilator* compiler, and then run the executable:

```
$ bsc -u  -verilog  -g mkTop  Top.bsv                 # compile -> mkTop.v
$ bsc -vsim verilator  -e mkTop  -o exe_HW_vsim       # link
$ ./exe_HW_vsim                                       # run
Hello, World!                                         # output
  (From the book: The C Programming Language
   by:            Kernighan and Ritchie
   which was first published on: 1978-02-22)
- mkTop.v:47: Verilog $finish
```

In both, for the compile step, we added the "`-u`" flag, which tells *bsc* to automatically compile any imported files. This is why we never had to mention the `DUT.bsv` source file in the above commands.

EXERCISE: Please see Ex-03-B-Top-and-DUT in Appendix E, page E-3. □

---

[1] Note: Bluesim seems to print "`<space>2`" for "`02`".

NOTE: From this point on we are not going to show the compile-and-link steps for each example. Please consult the *Bluespec Compiler (BSC) User Guide* [24] for full details on how to use the *bsc* compiler.

3.3.3 Visibility of names, exports and imports

The `package`, `import` and `export` statements work just like in SystemVerilog (in fact, using the same syntax). The `export` and `import` statements control visibility of names across packages. This is illustrated in Figure 3.3.

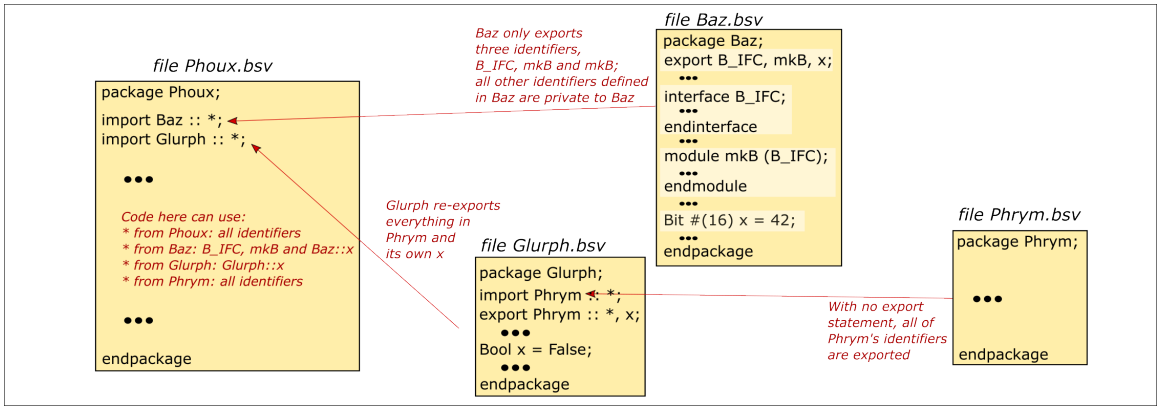


Figure 3.3: Namespace control with imports and exports

A package `P` that defines a name x can make it visible outside using an `export` statement. As a shorthand, if there is no `export` statement, then *all* the names defined in a package are made visible outside. For convenient readability, if there are many explicitly exported names, they can be provided using multiple `export` statements.

A package `Glurph` can also *re-export* names it has imported from some other package `Phrym`.

If a package `Phoux` needs to use a name `x` defined in some other package `Baz`, it must explicitly "`import`" the package with the syntax:

```
import Baz :: *;
```

which makes all the names exported by `Baz` available for use in `Phoux`.

NOTE: In SystemVerilog, in `import` statement, instead of "`*`" one can list just the names actually needed, one does not have to import all the names exported by another package. This selective import is not currently supported in **BSV**.

3.3.4 Resolving ambiguous imports

If a package `Phoux` imports two other packages `Baz` and `Glurph`, and both those packages define an identifier `x`, then the identifier `x` is ambiguous in `Phoux`. This can always be resolved by replacing any use of `x` in `Phoux` by a so-called *fully qualified name*, `Baz::x` or `Glurph::x` to identify exactly which `x` is intended.

3.3.5 Exporting types abstractly

When exporting a struct type `S` or enum type `E` using `export`, there is a difference between these two ways of exporting:

```
export S (..);      vs.      export S;
export E (..);      vs.      export E;
```

In the versions on the left the field names of the struct and the labels of the enum are also made visible; on the right, they are not. The latter case is useful for defining a so-called "*abtract data type*", *i.e.,* a type that is known outside the package but whose internal details are hidden.

Since an interface is just like a struct type, if we explicitly export it we typically use:

```
export M_IFC (..);
```

since we normally want all the interface methods in the interface to be visible.

3.4 Interface and Module Declarations

3.4.1 What's in an interface declaration?

A **BSV** interface declaration can appear as a top-level declaration in a package. It is a syntactic specification of the "API" (Application Programming Interface) of one or more **BSV** modules, *i.e.,* it names the *methods* offered by a module, and their arguments and result types. Figure 3.4 shows examples of what one might find in an interface declaration (a top-level declaration inside a package).

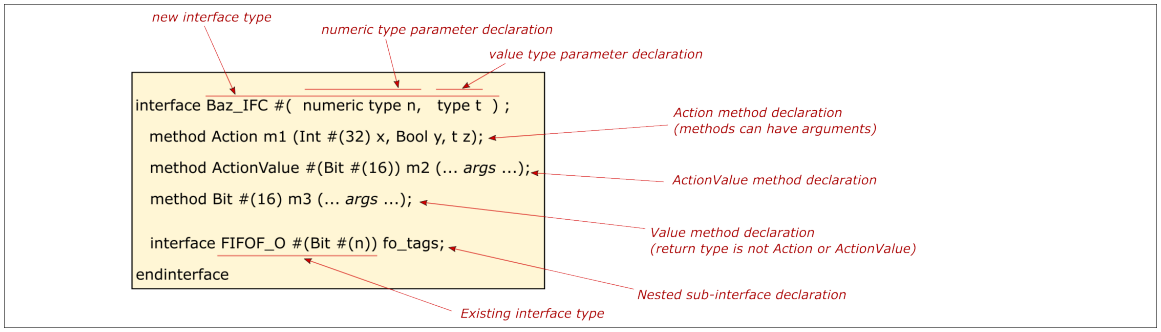


Figure 3.4: Typical components in an interface declaration

The example declaration introduces a new interface type, `Baz_IFC`. Interface types may or may not have type parameters; the example shows two. Each type parameter can be of "numeric" type or ordinary "value" type. Numeric types are typically used for "sizes"; for example, the type "`Bit#(16)`" applies the type constructor "`Bit`" to the numeric type "`16`" to describe the type of bit-vectors of width 16 bits.

In places where we *use* this newly declared interface type, it may look something like this: `Baz_IFC#(3,Bool)`.

The order in which the method/sub-interface declarations are given is not important.

Some syntax notes for Figure 3.4:

- Syntax keywords: `interface`, `endinterface`, `method`, `numeric`, `type`
- Type names (begin with an uppercase letter):
  - `Action`, `ActionValue`, `Bit`, `Int`, `Bool` (all standard types in the *bsc* library)
  - `FIFO_O` (defined in an additional **BSV** library, and discussed in Section 7.5.4)
- Type variables (begin with a lower case letter): `n`, `t`
- Ordinary variables (begin with a lower case letter): `m1`, `x`, `y`, `z`, `m2`, `m3`, `fo_tags`

3.4.2 What's in a module declaration?

A module declaration specifies one of possibly many *implementations* of a module interface. If we think of an interface type as an API, then a module declaration describes an object (an actual circuit in hardware) that offers that interface. Figure 3.5 shows the typical contents of a module declaration.

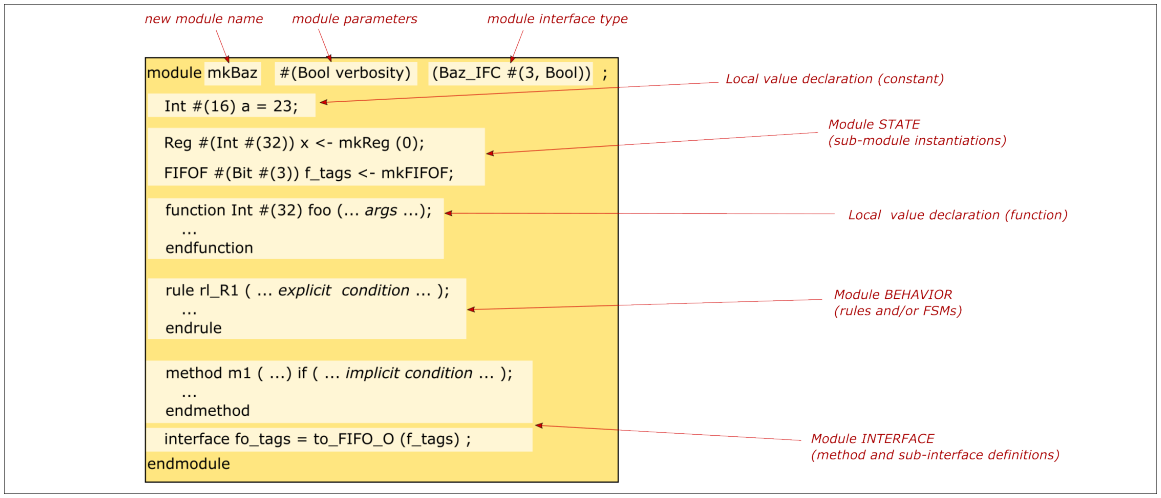


Figure 3.5: Typical components in a module declaration

The order in which the components inside the module are given is not important except:

- If an item i_2 uses an item i_1 then i_1's definition should precede i_2.
- By convention, sub-module instantiations are given first (the STATE section) followed by rules (the BEHAVIOR section).
- Method and sub-interface definitions must be the last items in the module (the INTERFACE) section.

Some syntax notes for Figure 3.5:

- Syntax keywords: `module`, `endmodule`, `function`, `endfunction`, `rule`, `endrule`, `method`, `endmethod`,`interface`
- Type names (begin with an uppercase letter): `Bool`, `Baz_IFC`, `Int`, `Reg`, `FIFOF`

- Ordinary variables (begin with a lower case letter): `mkBaz`, `verbosity`, `a`, `x`, `mkReg`, `f_tags`, `mkFIFOF`, `foo`, `rl_R1`, `m1`, `fo_tags`

3.4.3 Extending our example minimal BSV program further

Let us extend our minimal **BSV** program so that the DUT contains a module with an interface.

in file Ex-03-C-Module-and-Interface/DUT.bsv

```
1  package DUT;
2
3  export DUT_IFC (..), mkDUT;
4
5  String title   = "The C Programming Language";
6  String authors = "Kernighan and Ritchie";
7
8  Bit #(11) year = 1978;
9
10 Bit #(4)  month = 2;
11
12 Bit #(5)  day = 22;
13
14 interface DUT_IFC;
15    method Tuple2 #(String, String)                m_title_authors;
16    method Tuple3 #(Bit #(11), Bit #(4), Bit #(5)) m_date;
17 endinterface
18
19 module mkDUT (DUT_IFC);
20    method m_title_authors = tuple2 (title, authors);
21    method m_date          = tuple3 (year, month, day);
22 endmodule
23
24 endpackage
```

The "`interface`" definition contains 2 methods, `m_title_authors` and `m_date`. The first returns a "2-tuple" (a pair) of values, both of type `String`. The second returns a 3-tuple (a triple) of bit-vector values

The "`export`" statement makes `DUT_IFC` and its methods, and `mkDUT` visible outside the module. The variables `title`, `authors`, `year`, `month` and `day` are no longer visible outside the module.

The module `mkDUT` simply defines the two methods in its interface.

Next, we modify our `mkTop` module from the previous example:

in file Ex-03-C-Module-and-Interface/Top.bsv

```
1  package Top;
2
3  import DUT :: *;
4
5  module mkTop (Empty);
6
7     DUT_IFC dut <- mkDUT;
8
9     rule rl_once;
10       match { .title, .authors }   = dut.m_title_authors;
```

```
11        match { .year, .month, .day} = dut.m_date;
12
13        $display ("Hello, World!");
14        $display ("  (From the book: %s", title);
15        $display ("   by:            %s", authors);
16        $display ("   which was first published on: %4d-%02d-%02d)",
17                  year, month, day);
18        $finish (0);
19     endrule
20
21  endmodule
22
23  endpackage
```

The module `mkTop` *instantiates* the module `mkDUT` and binds the name `dut` to its interface. Module instantiation is described in more detail in Section 7.2.4

In the rule, we invoke `dut`'s methods and, using a "pattern-matching" statement, bind the names `title`, `authors`, ... *etc.* to the tuple components. The `match` statement is described in more detail in Section 5.3

We can compile and link it for **Bluesim** and **Verilator** as before; running it produces the same outputs as before.

EXERCISE: Please see Ex-03-C-Module-and-Interface in Appendix E, page E-3. □

3.5 Rules and Interface Definitions

3.5.1 What's in a rule?

A "`rule`" is the fundamental behavioral construct in **BSV**[2]. Zero or more rules may appear at the top-level of a **BSV** module. Figure 3.6 illustrates the typical components of a rule.

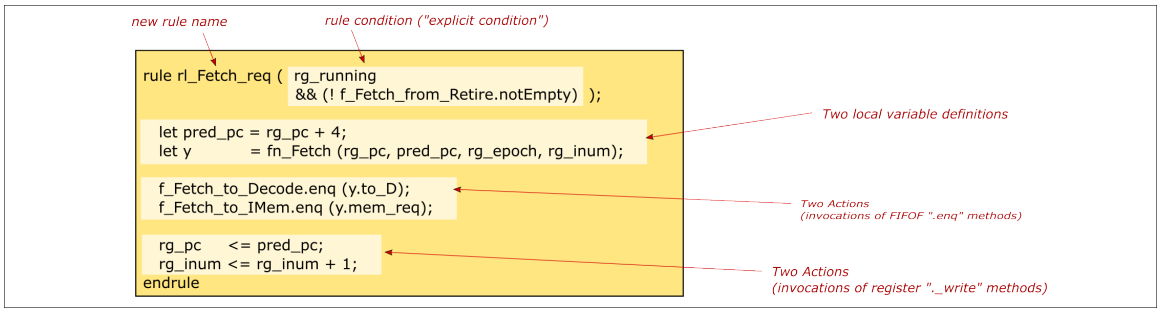


Figure 3.6: Typical components of a rule

A rule condition is also known as its "explicit condition" and is always an expression of type Bool (and, therefore, by **BSV**'s strong type-checking rules, it cannot have a side- effect, *i.e.*, it cannot contain an Action).

[2]Whereas "`always @(posedge CLK)`" is the fundamental behavioral construct in Verilog and SystemVerilog.

The rule body as a whole is an expression of type `Action`, and may contain many sub-actions, also of type `Action` (`Action` is a recursively defined type).

BSV rules are discussed in more detail in Chapters 13 and 18. Rules are used in the Fife CPU (Chapter 16) and in a version of the Drum CPU (Chapter 14).

Some syntax notes for Figure 3.6:

- Syntax keywords: `rule`, `endrule`, `let`
- All the identifiers in the example are ordinary variables, beginning with a lower case letter.

3.5.2 What's in an interface definition?

Section 3.4.1 discussed interface type *declarations*. Here we discuss interface *definitions*. The former just specifies the names of the interface methods and their argument and result types (like a C "function prototype"). The latter specifies how the methods are actually implemented in a particular module (like a full C function).

Interface definitions occur at the top-level of a module body, at the end of the module body (just before the `endmodule` keyword). Interface definitions must provide a definition for each method and sub-interface mentioned in the interface type declaration (the *bsc* compiler will issue a warning if any of them are left undefined).

Figure 3.7 illustrates typical components in an interface definition. All method definitions

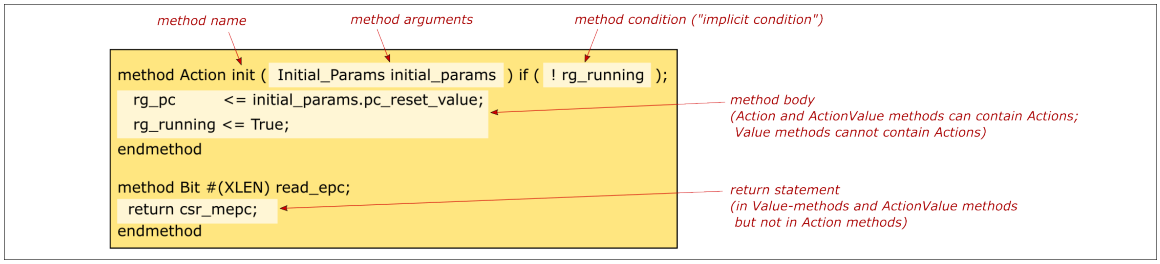


Figure 3.7: Typical components of an interface definition

have an *implicit condition* that is always an expression of type `Bool`, and using identifiers defined earlier inside the module. If the implicit condition phrase is missing (as in the second method in the diagram), it is taken to be constantly true.

Action and ActionValue methods can contain Actions (*e.g.,* method `init`). Value methods cannot contain Actions (*e.g.,* method `read_epc`).

ActionValue and Value-methods must have a `return` statement specifying the returned value (*e.g.,* method `read_epc`).

Some syntax notes for Figure 3.7:

- Syntax keywords: `method`, `endmethod`, `if`, `return`
- Type names (begin with an uppercase letter): `Action`, `Bit`
- All the remaining identifiers in the example are ordinary variables, beginning with a lower case letter.

3.6 Static Elaboration and Hardware Module Structure

In developing a piece of software we think of two distinct phases, *static* and *dynamic*. The former is what the compiler does: parsing, type-checking, analysis and transformation, optimization and code-generation. The latter is what happens when we actually run the object code produced by the compiler: allocate a stack-frame for the top-level function, start executing it, and allocate and de-allocate stack frames as we call and return functions. Even in so-called interpreted languages (such a Python, Tcl, Perl, Javascript) these two phases exist, although they may be performed adjacently one after the other and may not be distinguishable to the user.

In **BSV** (and in Verilog, SystemVerilog and VHDL), there are *three* distinct phases: static (compilation), *static-elaboration* and dynamic. The compilation phase is just like in a software language compiler: parsing, type-checking, analysis and transformation, optimization and code-generation.

The new phase is static elaboration. We can think of the input to this phase as a collection of (parsed, type-checked) module definitions. One of these is designated as the *top-level module*, M_{top}. The static elaboration phase creates a hierarchy (a tree-structure) of module *instances*, starting with an instance MI_{top} of M_{top} at the root of the hierarchy. Effectively, it *unfolds* the module definitions into an actual hardware structure.

The tool creates an instance MI_{top} of the M_{top} module definition. Inside M_{top}, the code may specify instantiations of sub-modules M_1, M_2, ... The tool creates the instances MI_1, MI_2, ... and connects them into MI_{top}. This process is followed recursively for sub-modules of M_1, M_2, ... until the tool reaches "leaf" modules that do not instantiate any sub-modules. This is illustrated in Figure 3.8

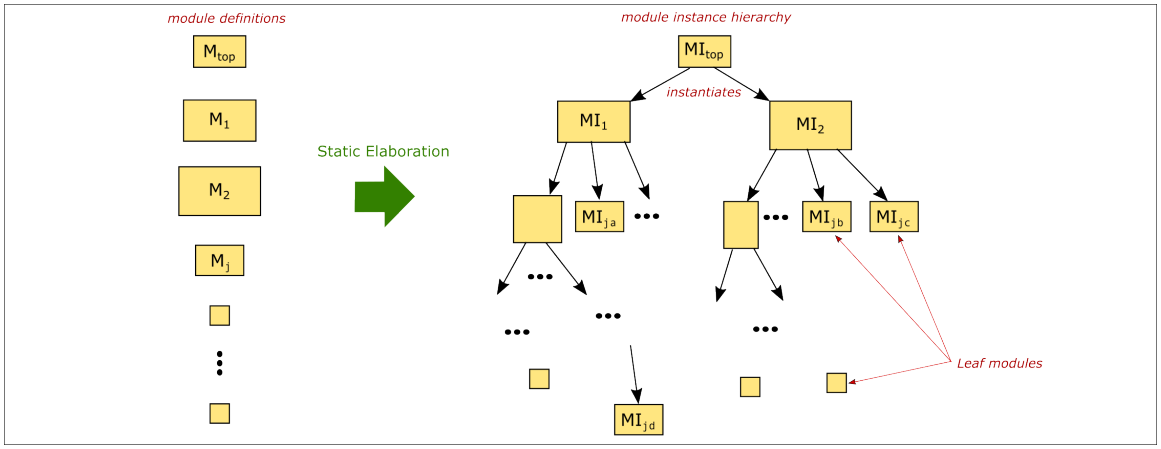


Figure 3.8: Static elaboration

Note, as illustrated in the diagram, a module definition M_j may be instantiated more than once, MI_{ja}, MI_{jb}, MI_{jc}, ... in various places in the hierarchy. In particular, the `mkReg` (register) and `mkFIFOF` (FIFO) modules may be instantiated dozens of times, in dozens of modules.

In simulation (**Bluesim** or Verilog simulation), this static elaboration is performed once, at the start of the simulation.

When synthesizing to FPGA or ASIC, this static elaboration is performed by the synthesis tool once, at the start of its operations.

3.6.1 Module interaction *via* methods

Figure 3.9 illustrates the fundamental *behavioral* structures in hardware produced from **BSV**. Modules contain rules, and rules interact with other modules by invoking the methods in their interfaces.

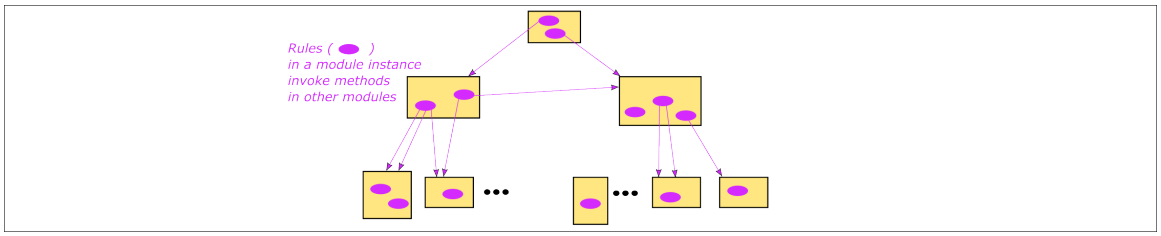


Figure 3.9: Module interaction *via* methods

At this level of abstraction, this looks just like objects and interaction between objects in an object-oriented programming language, and this is reasonable as an approximate mental model of what happens in a **BSV** program. Some differences in **BSV** compared to traditional object-oriented programming languages:

- Objects cannot be allocated dynamically (modules cannot be instantiated dynamically). All module instantiation is done once, during static elaboration.
- Rules in **BSV** modules are (potentially) infinite processes. A rule can "fire" repeatedly whenever its explicit condition and the implicit conditions of the methods it invokes are true.
- All methods in **BSV** have implicit conditions that dictate when they can be invoked.
- Each rule "firing" is *atomic*—all its state updates are performed together, conceptually at the same instant, and semantically either before or after all other rule firings, *i.e.,* there is never any interleaving of actions in one rule with actions in another rule.

  Atomicity is the most powerful principle known in Computer Science for reasoning about correctness in the presence of concurrency (such as rules in a **BSV** program or multithreading in a concurrent software program). It is one of the key features that distinguishes **BSV** from Verilog, SystemVerilog, and VHDL (indeed possibly all other hardware design languages).

3.7 Conclusion

This chapter has given a top-level view of **BSV** programs, starting with a collection of files. With this knowledge, it should possible to peruse a **BSV** program in multiple files in multiple directories (such as Drum or Fife) and start understanding its major structural components

and how they are put together, *i.e.,* this chapter gives you the top-level structural "vocabulary" of **BSV** programs. It does not yet describe *how* the resulting hardware works; that will begin in subsequent chapters.

Chapter 4

BSV: Combinational circuits for the RISC-V Stage functions

4.1 Introduction

It is useful to start with the Decode stage of Figure 2.16 because it involves only bit-vectors, operations on bit-vectors, conditionals to classify instructions into classes, and `enum` types to name and encode instruction classes.

The inputs to the Decode stage as depicted in the diagram are:

- A 32-bit piece of data—a RISC-V instruction—that has become available by reading it from memory at the PC address.[1]
- Any additional information passed on from the Fetch stage.

The outputs of the Decode stage have information needed by the next stage (Register-Read and Dispatch). For a RISC-V instruction, useful information includes:

- Was the Fetch itself successful, or did it encounter a memory error; if so, what kind of memory error?
- Is it a legal 32-bit instruction?
- If legal, what is its broad classification: Control (Branch or Jump)? Integer Arithmetic or Logic? Memory Access? This will help in choosing the next stage to which we must dispatch to execute the instruction.
- Does it have zero, one or two input registers? If so, which ones? This will help the next stage in reading registers.
- Does it have zero or one output registers? If so, which one? This will help the final Register Write stage in writing back a value to a register.

To compute these values, we need to examine "slices" of the 32-bit instruction ("bit vector"), such as the 7-bit "opcode" slice, the 5-bit "rs1", "rs2" and "rd" slices, and so on. We need to be able to compare these slices to constants (*e.g.,* "Is the opcode a BRANCH opcode?"). We need to do things conditionally, *e.g.,* if it is a BRANCH instruction, then it has an rs1 and rs2 slice but no rd slice, but if it is a JAL instruction it has an rd slice but no rs1 or rs2.

[1] When implementing the so-called "C" RISC-V ISA extension ("compressed instructions"), instructions can also be 16 bits, but we ignore that for RV32I.

Finally, as in any good programming language, we need to package all this functionality inside "functions" with clearly specified input(s) and output(s). In the next several sections we will learn the **BSV** concepts needed to code these ideas.

4.2 Hexadecimal and Binary Notation for literal integers

BSV uses the same notation as Verilog and SystemVerilog for hexadecimal and binary literal integers. Some examples:

```
3'b010           // Binary literal, 3 bits wide
7'b_110_0011     // Binary literal, 7 bits wide
5'h3             // Hex literal, 5 bits wide
32'h3            // Hex literal, 32 bits wide
32'h_efff_0f17   // Hex literal, 32 bits wide (an AUIPC instruction)
'h23             // Hex literal, context determines width
```

As these examples show, a hexadecimal or binary integer literal is introduced by an optional bit-width, then a "tick" (single-quote) character, and then the binary or hexadecimal digits for the number. The character "_" may be used freely to space out groups of digits to improve readability for humans (the compiler ignores these spacers).

The last line shows that we can omit the size prefix, in which case the size will be inferred by the compiler from the context. For example, if we had:

```
   Bit #(32) pc_val = 'h_8000_0000;
```

then the literal is inferred to be 32 bits wide, and this will be extended, if necessary to fit the context where it is used.

4.3 Syntax of Identifiers

The syntax of an identifier (name) in **BSV** follows the same conventions as in many programming languages: any sequence of alphabets, digits and underscore characters, with the first letter always being alphabetic.

BSV follows the Haskell system where an identifier has a different role depending on whether its first letter is uppercase or lowercase. An uppercase first letter names a *constant*, either a value constant or a type constant. A lowercase first letter names a *variable*, either a value variable or a type variables.

The identifiers `True` and `False` are predefined value constants (they begin with an uppercase letter) representing standard boolean values. In the enum type-definition in Section 4.13, the identifiers `OPCLASS_SYSTEM`, `OPCLASS_CONTROL`, `OPCLASS_INT`, `OPCLASS_MEM` and `OPCLASS_FENCE` are also value constants.

The identifiers `Bool` and `Bit` are predefined type constants. In the enum type-definition in Section 4.13, the identifier `OpClass` is a type constant.[2]

[2]The identifiers `Bits`, `Eq`, and `FShow` are all predefined "typeclass" constants. Typeclasses are an somewhat advanced topic which we will not be discussing much in this book beyond the brief mention in Section 20.5.

Other identifiers in the following sections, like `pc_val`, `x`, `y`, `a`, `b`, `opcode`, and `funct3` are all ordinary value variables (begin with a lowercase letter). Even function names, such as `instr_funct3` and `is_legal_BRANCH` are value variables.

Type variables are only used in so-called *polymorphic* types and functions, where a single function is defined to work over a family of types. For example we can write a function to exchange the order of bits in a bit-vector of any width; its argument and result types would be given as `Bit#(n)`, where `n` is a type variable.

4.4 Syntax of comments

Comments in **BSV** have the same syntactic conventions as in Verilog, SystemVerilog and C/C++:

- A pair of forward-slashes ("`//`") begins a comment (text ignored by *bsc*) that spans to the end of the current line. There are many examples of this in the code fragments already shown above.
- A region of text, possibly spanning multiple lines, is a comment (ignored by *bsc*) if preceded by "`/*`" and followed by "`*/`". This is often used to "comment-out" regions of text during debugging or trying out alternatives, and for long documentation notes.

4.5 Introduction to Types

Like all *safe* modern programming languages, **BSV** has an expressive type system and is strongy typechecked.

By *expressive* type system we mean that it has a rich vocabulary for organizing values: booleans, bit-vectors, signed integers, enums, structs, vectors, functions, actions, rules, modules and more. By *strongly typechecked* we mean that there are strict compile-time rules about what types of arguments are allowed for each operator or function, and what type of result they produce. These rules disallow misinterpretatio of values, mistakes like taking the square-root of an ASCII character, or extracting the struct field from something that does not represent a struct. Types are *language-level abstractions*: ultimately both software and hardware compute with raw bits, but a language's type-system and type-checking ensure that bits are organized and used in intended ways.

BSV's type-checking rules go beyond most programming languages by also *checking many size constraints*. For example, a **BSV** function that expects a bit-vector of width exactly 32 bits cannot be given a bit-vector of any other size. In many other languages, they will silently extend a narrower argument or truncate a wider argument to the expected size; that behavior is very dangerous in hardware design, where we work with scalar values of many, many different precisely-sized bit-widths, unlike software languages where we typically use a bit-width that is generously large enough to accommodate our scalar values (8, 16, 32, 64 bits).

Figure 4.1 illustrates the type hierarchy in **BSV**. The lower two layers are quite conventional: Types represent sets of Values. The upper-layer (Kinds) expresses the idea that within a

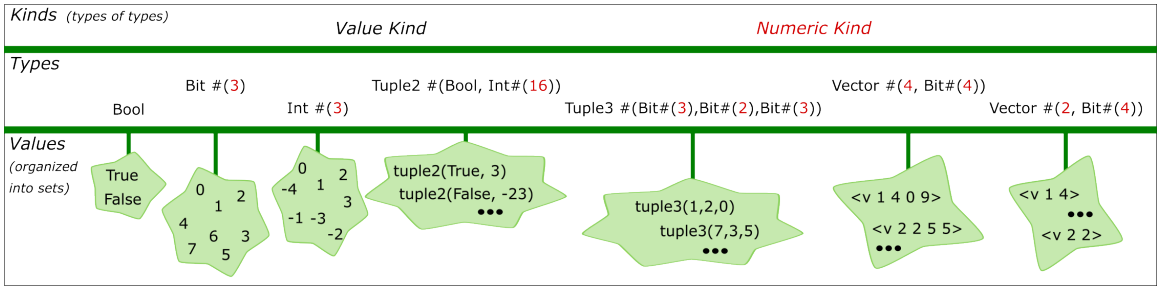


Figure 4.1: The hierarchy of Values, Types and Kinds

type expression (middle layer), certain components of the type-expression (shown in red) are of *Numeric Kind*, representing some "size" feature of the type.

NOTE:

About numeric kinds: Even though we use numeric notation like "3" and "16" both at the value level and at the type level, these two uses are completely distinct. They occur in distinct contexts—value expressions *vs.* type expressions—and so there is no ambiguity.

Numeric types are all known to *bsc* at compile time whereas, of course, numeric values are computed and used dynamically during execution.

There is a **BSV** pseudo-function `valueOf(`$t_{numeric}$`)` which, given a numeric type $t_{numeric}$, produces the numeric *value* corresponding to $t_{numeric}$. This is feasible because $t_{numeric}$ is statically known to *bsc*. It does not make sense to have a function in the opposite direction, from numeric values to numeric types, because one cannot compute something static from a dynamic value. `valueOf(`$t_{numeric}$`)` is a pseudo-function and not a normal function because its "argument" is a type, not a value.

We can of course perform arbitrary arithmetic on numeric values because they are dynamic.

We can perform only limited forms of arithmetic on numeric types because they have to be evaluted statically by *bsc*. Consider a generic function that takes two arguments of type `Bit#(m)` and `Bit#(n)` and returns the concatenation of these bit-vectors: its output type is `Bit#(m+n)`. By limiting the available arithmetic *bsc* can resolve it statically. This is a somewhat advanced topic which we postpone until Section 20.5.

Types can nested to arbitrary depth:

type ::= *type-constructor* `#(` *type*, ..., *type* `)`

A *type-constructor* always begins with an upper-case letter (is a type constant).

For each *type-constructor*, each *type* argument (parameter) is fixed to be either of value kind or numeric kind.

For example,

- In `Bit #(n)`, `n` is always of numeric kind.
- In `Vector #(n,t)`, `n` is always of numeric kind, `t` is always of value kind.

- In `Tuple3 #(t1,t2,t3)`, all three parameters are always of value kind.

Note that `Bit#(3)` and `Int#(3)`, although both represented in hardware with 3 bits, are distinct types. The former can represent values in the range 0..7; the latter in the range -4..3. `Bit#(3)` is a distinct type from `Bit#(4)`. 2-tuples (pairs of values of heterogeneous types) are distinct from 3-tuples. 4-element vectors are a distinct type from 2-element vectors.

4.6 Bit vectors, and declaring identifiers

The simplest and lowest-level type in **BSV** is the bit-vector (a vector made up of a particular number of bits). It is "lowest-level" in that it is the only type that directly maps into hardware and can be observed directly in hardware with a waveform viewer or an oscilloscope. Later we will see that in **BSV** one can define more abstract types such as integers, booleans, vectors and arrays, lists, structs (records), tagged unions (algebraic types), trees, and so on. These are all language-level abstractions, *i.e.,* a language-level view of bits; ultimately, all values are represented in hardware as bit-vectors.

The **BSV** statement:

```
1   Bit #(32) pc_val = 32'h_8000_0000;
```

declares the identifier `pc_val` to have the type `Bit#(32)`, *i.e.,* a bit-vector of 32 bits, and initializes it to have the value shown in hexadecimal on the right-hand side. The general syntax is the same as in Verilog, SystemVerilog and C:

type identifier = *initial_value*;

The BSC type `Bit#(32)` is roughly equivalent to the C type `uint32_t`. Unlike C, where only a few sizes are available, all multiples of 8 bits— (`uint8_t`, `uint16_t`, `uint32_t` and `uint64_t`)—bit-vectors in **BSV** can have any size (`Bit#(3)`, `Bit#(51)`, `Bit#(512)`, ...).

The initial value's size can be left unspecified:

```
1   Bit #(32) pc_val = 'h_1000;
```

As explained in Section 4.2, the literal constant `'h_1000` is of unspecified size (as opposed to `32'h_1000` which has precisely 32 bits). Technically, the literal constant has **BSV** type `Integer`, for true, unbounded, mathematical integers (see also Section 4.8). *bsc* implicitly applies the `fromInteger` function to this constant, which converts it from `Integer` to the type required by the context, here `Bit#(32)`. If the constant is too large for the context, *bsc* will not silently truncate it; it will give an error-message instead.

4.6.1 "`let`" bindings—having *bsc* infer the identifier's type

In an identifier binding, the *type* on the left-hand side can usually be replaced by the keyword "`let`", indicating that we would like *bsc* compiler to infer the type for the identifier being defined. For example:

```
1   Bit #(32) pc_val  = 32'h_8000_0000;
2   ...
3   let       next_pc = pc_val + 4;
```

Here, *bsc* can infer that the type of the expression "`pc_val+4`" is `Bit#(32)`, so this is equivalent to writing the type explicitly:

```
1   ...
2   Bit #(32) next_pc = pc_val + 4;
```

`let`-bindings should be used judiciously, only where the type is also obvious to the human reader.

4.6.2 Don't-care values

If the identifier will be assigned later, you can indicate that you don't care about its initial value, like this:

type identifier = `?`;

"Don't care" values are useful for several reasons. First, this conveys to the human reader that the value here is irrelevant.

Second, it can result in more efficient circuitry. If we had said "0", for example, the *bsc* compiler has to create circuitry ensuring that field's value is 0. By saying "`?`", the *bsc* compiler is allowed to pick any value that minimizes circuitry.

Third, in places where it does not result in additional hardware, the *bsc* compiler usually injects the specific value `'h_AAAA_AAAA` (of suitable bit-width). While debugging, observing such a value in some computation is often a clue that something is wrong.

NOTE:

Verilog, SystemVerilog and VHDL have a concept of "X" values. Each bit of a register or wire carries an "X" value until it has been assigned a specific binary value (0 or 1). However, note that this is *only in simulation*, where the simulator can and does model 3-valued logic (0, 1 and X) for each bit, and is able to propagate X values through operators, registers, *etc.* Hardware only implements 2-valued logic—every bit is either 0 or 1. Thus, this is an artefact that is only useful during debugging in simulation and static analysis.

BSV only has 2-valued logic; there is no concept of an "X" value. A BSV "`?`" expression has some specific, but potentially unpredictable, binary value.

EXERCISE: Please see Ex-04-A-Bit-Vectors in Appendix E, page E-4. □

4.6.3 Built-in Operators on Bit Vectors

The bits in a **BSV** bit-vector of size n are indexed from $n-1$ (most-significant bit) to 0 (least-significant bit). You can extract a *slice* of a bit-vector using usual Verilog notation:

```
1  Bit #(12) page_offset = pc_val [11:0];
2  Bit #(1)  pc_lsb      = pc_val [0];
3  Bit #(1)  pc_msb      = pc_val [31];
```

In the first line, we extract 12 bits of `pc_val` to get a bit-vector of size 12.[3] In the remaining lines, we extract one bit from `pc_val`.

BSV is *strongly typed* with respect to sizes, *i.e.,* it is very strict about matching sizes. For example, this statement:

```
1  Bit #(12) page_offset = pc_val [10:0];
```

will be reported as a type-error by the *bsc* compiler because the slice-expression on the right-hand side has type `Bit#(11)` which does not match the declared type `Bit#(12)`.

EXERCISE: Please see Ex-04-B-Bit-Vectors-Slicing in Appendix E, page E-4. □

BSV bit-vectors can be compared for equality and inequality. **BSV** bit-vectors are synonymous with unsigned integers, and so a number of other operations are also available on bit-vectors. Examples:

```
1  Bit #(12) x, a, b, c, d, e, f;
2
3  // Comparison ops: result type is Bool
4  if (a == b) ...;        // equality
5  if (a != b) ...;        // not-equal to
6  if (a < b) ...;         // less-than
7  if (a <= b) ...;        // less-than-or-equal-to
8  if (a > b) ...;         // greater-than
9  if (a >= b) ...;        // greater-than-or-equal-to
10
11 // Arithmetic ops: result type is Bit #(12)
12 x = a + b - c * d;      // add, subtract, multiply
13
14 // Bitwise logic ops: result type is Bit #(12)
15 //   AND  OR   unary INVERT   XOR  XNOR  XNOR
16 x = a &  b |     (~ c)         ^   d ^~ e ~^ f;
17
18 // Shifts
19 x = (a << 3) & (b >> 14);    // left- and right-shift
```

[3] In **BSV** we always write the msb (most-significant bit) to the left and the least-significant bit (lsb) to the right, and bit indexes always ascend from lsb to msb. Verilog and SystemVerilog have more variations on bit indexing and slicing.

Please see the *BSV Language Reference Guide* [2], Section 10.3, "Unary and binary operators" for a full list of available unary and binary operators. Unlike Haskell, in **BSV** you cannot define new unary or binary infix operators.

In such expressions, as usual bit-vector sizes must match exactly, else we'll get a type error, *e.g.,* we cannot compare a `Bit#(12)` value with `Bit#(11)` value. Unlike C and Verilog, **BSV** does not implicitly extend or truncate bit-vectors to match sizes.

Two functions are available to zero-extend and truncate bit-vectors.

```
1  Bit #(12) a;
2  Bit #(10) b;
3  b = a;                          // Type error: mismatched sizes
4  a = b;                          // Type error: mismatched sizes
5  b = truncate (a);               // Ok; truncates a to Bit #(10), then assigns
6  a = zeroExtend (b);             // Ok; extends b to Bit #(12), then assigns
7  if (a == zeroExtend (b)) ...    // Ok
8  if (truncate (a) < b) ...       // Ok
```

The functions `truncate()` and `zeroExtend()` are *polymorphic* in that they will truncate/extend by the appropriate amount as demanded by the context.

4.7 The `Bool` type and boolean values

In **BSV**, `Bool` is the type of a boolean value, written `True` and `False`. It has the usual boolean operators `&&` (boolean/logical AND), `||` (boolean/logical OR) and `!` (boolean/logical NOT).

4.7.1 Caution: `Bool`, `Bit#(1)` and `Int#(1)` are distinct types

BSV is unlike languages like C and Python which are very loose about what can be used as a boolean value. For example in C, any numeric value or pointer can be used in the condition of an if-then-else statement, where any non-zero numeric value, or non-NULL pointer is considered "True".

In **BSV**, `Bool` and `Bit#(1)` are *distinct* types, *i.e., bsc*'s type-checking will complain if one is used where the other is expected. This is because not all `Bit#(1)` values are meaningful as boolean values.

The boolean/logical operators mentioned above (such as `&&`) operate on `Bool` types and are distinct from the bit-wise logic operators mentioned earlier (such as `&`), which operate on `Bit#(n)` types.

Note that bitwise comparison operators, such as in the example `if (a <= b) ...` shown in Section 4.6 above, take `Bit#(n)` arguments and produce `Bool` results.

4.8 Integer types

These are the two main integer types that we use in **BSV**:

```
   Bit #(n)        // bit-vectors (unsigned integers), represented with n bits
   Int #(n)        // signed integers, represented with n bits
```

The most common type used in processor design (and possibly in all hardware design) is `Bit#(n)`. In **BSV**, arithmetic and logical operations on `Bit#(n)` types treat the values as *unsigned* integers.

`Int#(n)` is used whenever we need to represent negative numbers and perform signed operations. These are represented in bits in hardware in "2's complement" representation (see https://en.wikipedia.org/wiki/Two%27s_complement).

Fixed-width integer types all "wrap-around". For example, if we add 1 to a `Bit#(3)` value `3'b_111`, the result will be `3'b_000`; if we subtract 1 from a `Bit#(3)` value `3'b_000`, the result will be `3'b_111`.

Note: for completeness we mention that there are two more basic integer types available in **BSV**, but these are less frequently used (by this author).

```
   UInt #(n)       // unsigned integers, represented with n bits
```

`UInt#(n)` can be used to represent unsigned integers, n bits wide. However, this author mostly uses `Bit#(n)` for unsigned integers, and rarely uses `UInt#(n)`, since essentially the same operators (`+`, `-`, `&`, `|`, shifts, ..., see Section 4.6.3) are defined on both types.

```
   Integer         // Mathematical integers (unbounded, no bit-width limit)
```

`Integer` is used for true mathematical integers, *i.e.,* they are unbounded, ranging from minus infinity to plus infinity (though in practice limited by the amount of memory in your computer!). Being unbounded, they cannot be represented in any fixed-size hardware. `Integer` is used only for compile-time integers, such as a verbosity level for debugging, or the size of a vector of interfaces.

Observe that all four type names begin with an uppercase letter, as discussed in Section 4.3.

4.9 Functions

Expressions and statements can be packaged into *functions*. The syntax of **BSV** function declarations is the same as in Verilog, SystemVerilog and C/C++, specifying types for the result and for each argument. We modify our example in Section 3.2.1:

```
function Action print_BV_BV_Bool (String op, Bit #(4) a, Bit #(4) b, Bool result);
   $display ("  %s: %04b %04b => %d or ", op, a, b, result, fshow (result));
endfunction

module mkTop (Empty);

   rule rl_once;
      Bit #(4) a = 'b_1010;
      Bit #(4) b = 'b_0110;
```

```
10
11      print_BV_BV_Bool ("==", a, b, a == b);
12
13      $finish (0);
14    endrule
15
16  endmodule
```

The result type `Action` is used for functions that are pure side effects, *i.e.,* they perform some action(s) and do not return any result. This is discussed in more detail in Section 4.11.

EXERCISE: Please see Ex-04-C-Bit-Vectors-Operations in Appendix E, page E-5. □

EXERCISE: Please see Ex-04-D-Decode-Imm in Appendix E, page E-5. □

4.9.1 More about BSV functions

4.9.1.1 BSV functions describe both structure and computation

BSV functions play two roles: circuit *structural descriptions* and circuit *functionality* or *computation*. As an example of the former, a **BSV** function may take an `Integer` argument n and produce a module containing a vector of n registers. This function purely concerns the structure of the desired circuit. n is not a run-time value carried on wires, through gates or in registers.

As an example of the latter, a **BSV** function may be used to add the values from two input registers, with its result placed in an output register. The values flowing through this function are run-time values that flow through wires and gates and registers.

4.9.1.2 BSV functions are inlined and have zero incremental hardware cost

BSV functions have *zero* incremental hardware cost, and so the programmer should feel free to use them liberally wherever they are useful and improve readability.

In software languages, functions have a cost because they are invoked at run-time. The caller must allocate and link a stack frame, save some registers, load arguments into registers, and jump to the function (the callee). The callee may save more registers, perform its work, restore some registers and jump back to the caller. The caller may save the function result, restore more registers, and deallocate the stack frame, before resuming. All this is "function-call overhead" to do the real work inside the callee.

In **BSV**, functions are always completely "inlined" during static elaboration (see Section3.6), so they behave exactly as if the work in the callee's body was written *in situ* wherever invoked; there is zero function-call overhead.

4.9.1.3 BSV functions can be recursive

Even though **BSV** functions are always inlined, they can be recursive, when they are used for circuit structural description. For example, the most succinct and clear way to express a Butterfly or Radix Sorting Network[4] or a balanced AND-OR tree multiplexer in **BSV** is to follow the naturally recursive definition in the specifications of these algorithms. Here, recursion is used purely for circuit structure description; the recursion is not being used to compute any run-time value.

4.9.2 BSV function argument and result types are not just for data

It is not surprising that **BSV** function and argument types can be scalars (bit-vectors, integers, booleans, enums) and also (recursively) structs and vectors. All these are data types that can flow on wires, through gates, and live in registers and memory.

Unlike Verilog and SystemVerilog, **BSV** function argument and result types can also be modules, interfaces, rules and actions, or any user-defined data type, or even another function. The technical term for this is that **BSV** functions are fully "higher-order".

For example, in a CPU design, a **BSV** function may produce, as its output a module that *is the Fetch module*. An argument to this function may be an integer representing how far it is allowed to look ahead in the instruction stream. So, applying this function to different integers produces different modules with different "prefetch depth". Another argument to this function may itself be a module that is the branch-predictor; by applying to different arguments we can get different Fetch units with different branch-predictors.

All these are examples of powerful parameterization. None of this would be surprising in a software programming language, but this capability is missing in Verilog and SystemVerilog, and in fact in most HDLs.

4.10 Example: recognizing legal RISC-V BRANCH instructions

The RISC-V ISA has a family of six conditional-branch instructions. Figure 4.2 is an excerpt from the Unprivileged ISA specification document [27]. The first line just gives us

31 27	26 25	24 20	19 15	14 12	11 7	6 0	
imm[12|10:5]		rs2	rs1	funct3	imm[4:1|11]	opcode	B-type
imm[12|10:5]		rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]		rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]		rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]		rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]		rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]		rs2	rs1	111	imm[4:1|11]	1100011	BGEU

Figure 4.2: RISC-V conditional BRANCH instructions

[4] See diagram in "Cooley-Tukey FFT algorithm" here: https://en.wikipedia.org/wiki/Cooley%E2%80%93Tukey_FFT_algorithm

the names of the various slices of a 32-bit BRANCH-type instruction, and the subsequent lines describe the six instructions. Note that they only differ in the `funct3` slice, where they use only six of the possible eight 3-bit codes.

Assuming `instr` is a 32-bit instruction, we can write **BSV** code to compute whether `instr` is or is not a legal BRANCH instruction:

```
1  Bit #(32) instr         = ...;
2  Bit #(7)  opcode_BRANCH = 7'b_110_0011;
3
4  Bit #(7) opcode = instr [6:0];
5  Bit #(3) funct3 = instr [14:12];
6  Bool legal = (opcode == opcode_BRANCH)
7                && (funct3 != 3'b010)
8                && (funct3 != 3'b011));
```

Line 1 defines `opcode_BRANCH` as a 7-bit constant whose binary value is 1100011. The `'7b` prefix indicates that the number should be read as a binary, not decimal, number. The "_" underscore characters are present merely for our (human) readability, and have no semantic significance. Lines 3-4 extract relevant slices, and finally lines 5-7 define the desired legality condition.

Figure 4.3 shows the hardware circuit described by the code. Some observations:

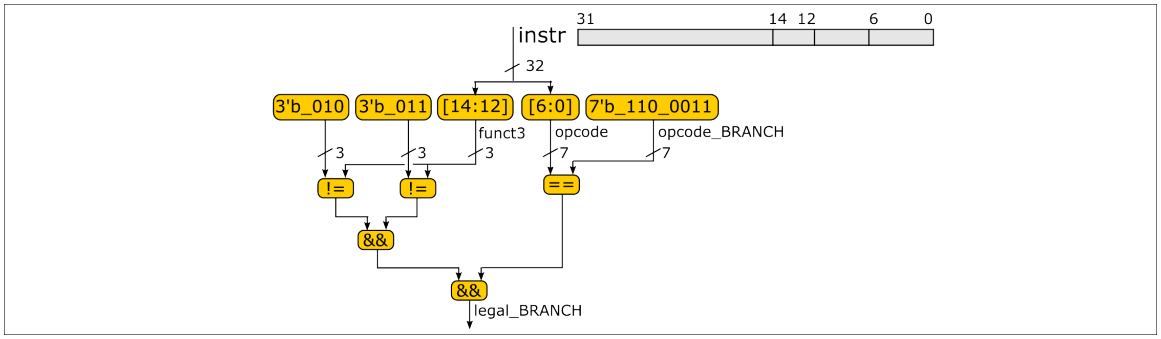


Figure 4.3: Testing for a legal BRANCH instruction

- Lines with arrow-heads in the figure represent bundles of one or more wires, also called "buses". For buses that have more than one wire, we show a small diagonal cross-hatch labeled with the number of wires (such as "3" or "7"). A bundle of n wires carries a value of type `Bit #(`n`)`.
- Names/identifiers in **BSV** code that are bound to values are simply names for buses (in most software programming languages names represent memory locations; this is *not* the case in **BSV**).

4.10.1 Combinational circuits and primitives

Figure 4.3 is an example of a so-called *combinational* circuit. In general, a combinational circuit is any interconnection of combinational primitive "operators" that *does not contain cycles* (*i.e.,* a bus connecting back to an earlier part of the circuit). Examples of combinational primitive operators in **BSV** include comparisons (like `==` and `!=`), boolean operations

(like `&&`), bit-slicing (`[n1:n2]`) truncation and extension, arithmetic (like `+`, `-`, `*`), shifts (`<<` and `>>`), and multiplexers (discussed in Section 4.14, later).

In **BSV**, and Verilog/SystemVerilog RTL, we consider such operators as "primitive". In fact, such operators must themselves be implemented using lower-level circuit primitives such as AND, OR, and NOT gates which, in turn, must be implemented with even lower-level circuit structures such as transistors. We do not concern ourselves with such lower-level implementation because nowadays this is performed for us automatically by excellent so-called "synthesis" tools.

4.10.1.1 Combinational circuits have no side-effects (are "pure")

There is no "storage" in a combinational circuit, nor any concept of "updating" any storage (no "side-effects"). When a 32-bit value is presented at the input (top) of the circuit in Figure 4.3, conceptually we "instantly" see the 1-bit result at the output (bottom) of the circuit, *i.e.,* a combinational circuit is conceptually a pure, instantaneous, mathematical function from intputs to outputs. If we change the 32-bit value presented at the input, conceptually the output changes instantaneously in response.

NOTE: Circuits are physical artefacts and must follow the laws of physics. Electrical signals will take some finite time to propagate from inputs to outputs through wires and silicon. This propagation delay will place a limit on the "clock speed" at which we are able to run a digital circuit. We ignore this for the moment, and discuss this in detail later.

4.10.2 Encapsulating the code in functions

The code fragments shown above can be encapsulated into **BSV** functions:

src_Common/Instr_Bits.bsv: line 30 ...

```
function Bit #(7) instr_opcode (Bit #(32) instr);
   return instr [6:0];
endfunction
```

src_Common/Instr_Bits.bsv: line 34 ...

```
function Bit #(3) instr_funct3 (Bit #(32) instr);
   return instr [14:12];
endfunction
```

src_Common/Instr_Bits.bsv: line 153 ...

```
function Bool is_legal_BRANCH (Bit #(32) instr);
   let funct3 = instr_funct3 (instr);
   return ((instr_opcode (instr) == opcode_BRANCH)
           && (funct3 != 3'b010)
           && (funct3 != 3'b011));
endfunction
```

Functions are invoked using the "application" syntax commonly used in most programming languages:

```
   Bit #(32) x, y;

   Bool result_x = is_legal_BRANCH (x);
   Bool result_y = is_legal_BRANCH (y);
```

BSV function declaration application syntax are essentially the same as in SystemVerilog.

EXERCISE: Please see Ex-04-E-is-legal-XXX in Appendix E, page E-5. □

4.11 Pure functions vs. functions with side-effects (`Action`, `ActionValue`)

BSV has a system of data types and type-checking similar to Haskell in that it systematically distinguishes expressions which are "pure" (guaranteed not to have any side-effects) from expressions that may have side-effects.[5]

The detailed reason for this distinction need not detain us now—suffice it to say here briefly that **BSV**'s semantics are fundamentally based on a concept of "rules"; that rules are condition-action pairs; that conditions *must not* have side effects (change the state of any hardware); and that the compiler needs to guarantee this, *i.e.,* that conditions are pure boolean expressions. These points will be discussed in more detail in Chapter 13.

One useful side-effect during debugging is the `$display()` statement. Why is `$display()` considered a side-effect? A pure expression does not modify any state, and therefore it can be optimized away by the compiler (evaluated zero times) if its result is never used. The compiler can also duplicate a pure expression (so it is evaluated more than once) for reasons such as cost (cheaper to recompute a value at some location in the code than to communicate the value that was computed elsewhere). Neither of these properties is true with `$display()`! Thus, the compiler needs to know about the purity or otherwise of every expression.

Keeping track of the purity or otherwise of an expression is not a local property. An expression may invoke a function which, in turn, invokes another function, and so on. The expression is pure only if there are no side effects anywhere in such a call chain (those functions may be defined in separate files, in libraries, and so on). **BSV** used a *monadic* type sytem (the same as in Haskell) to systematically track purity/impurity of expressions. Consider two function declarations:

```
1  function Bool f1 (...); ...
2
3  function ActionValue #(Bool) f2 (...); ...
```

Both of these functions return a boolean value. But an application `f1(x)` is *guaranteed* to be pure (by **BSV**'s type system), whereas an application `f2(x)` is assumed possibly to have a side-effect. These functions also have different syntax for how they are invoked:

```
1  let result1 = f1 (x);
2
3  let result2 <- f2 (x);
```

[5] For Haskell enthusiasts: `Action` and `ActionValue` are in the action monad.

The "`<-`" syntax is/can only be used to invoke functions with `ActionValue` type, and is also a good visual cue to indicate that the invocation is *performing some action* (side-effect) and also returns a value.

BSV also has an `Action` type, which is just a convenience for the special case of an expression that has a side effect and does not return any interesting value. You can think of `Action` as a synonym for `ActionValue #(void)`, where you can think of `void` as "uninteresting value". For example, you can think of `$display()` as a built-in function of type `Action`.

4.11.1 Combinational circuits = "doesn't have `Action` or `ActionValue` type"

A pleasing consequence of **BSV**'s type system is that we can identify precisely which expressions become combinational circuits. If the type of the expression is *not* `ActionValue#(`t`)` or `Action`, then it *must* be a combinational circuit, and *vice versa*. We also refer to such types as "value types".

4.11.2 Using `ActionValue` on pure functions for `$display` debugging

Sometimes when we write a complex pure function whose result type is t, we may deliberately write its result type as `ActionValue#(`t`)` so that we can insert `$display` statements for debugging inside the function body. If we merely inserted `$display` statements without changing the function type, the compiler will complain that the function does not type-check correctly.

We use this "trick" frequently in Drum and Fife source codes, for tracing and debugging. We will see many examples in the coming chapters.

4.12 `StmtFSM`: a useful facility for testbenches

Our previous exercises contained a single "rule"; although rules can execute repeatedly forever, our rules executed just once and terminated simulation because they contained a `$finish()` statement.

Suppose we want to to write a testbench that sequentially executes N different tests, one after the other (on successive clocks). A simple idiom for this is:

```
import StmtFSM ::  *
...
module mkTop (Empty);
  mkAutoFSM (
    seq
      action_1
      ...
      action_N
    endseq);
endmodule
```

`StmtFSM` and `mkAutoFSM` will be discussed in more detail in Chapter 10. For now all we need to know that this is like a sequential program that performs a sequence of actions.

EXERCISE: Please see Ex-04-F-FSM-Testbench in Appendix E, page E-6. □

4.13 User-defined types: `enum` types

In Figure 2.16, the Register-Read and Dispatch stage needs to know which of the four alternative downstream paths should be selected for executing the insruction. In particular, we need to know whether the incoming instruction is a system instruction, a control instruction (branch or jump), an integer arithmetic/logic instruction, or a memory-accessing instruction. We could think of coding these "classes" using numbers (0 for system, 1 for control, 2 for integer, 3 for Mem), but it is more readable, and cleaner, to use an "enum" type (similar to enum types in SystemVerilog and C):

src_Common/Inter_Stage.bsv: line 43 ...

```
1 typedef enum {OPCLASS_SYSTEM,     // EBREAK, ECALL, CSRRxx
2               OPCLASS_CONTROL,    // BRANCH, JAL, JALR
3               OPCLASS_INT,
4               OPCLASS_MEM,        // LOAD, STORE, AMO
5               OPCLASS_FENCE}      // FENCE
6 OpClass
7 deriving (Bits, Eq, FShow);
```

This defines a type `OpClass` containing five constants (the last two (`OPCLASS_MEM` and `OPCLASS_FENCE`) both indicate the memory-access path.

4.13.1 `deriving (Bits)`

Because we said "`deriving(Bits)`", the *bsc* compiler will automatically represent them with the obvious codes 0, 1, 2, 3 and 4 in a minimal bit-width (`Bit#(3)`).

For a type that will only be used at compile time and never for a run-time value, there is no need to have a hardware reprsentation in bits. If we *do* want a custom representation in bits, **BSV** has a mechanism by which we can specify this, but we don't discuss this further here because it involves a more advanced **BSV** capability called "typeclasses" and "typeclass instances".

4.13.2 `deriving (Eq)`

Because we said "`deriving(Eq)`", the *bsc* compiler will automatically define the "equality" (and "inequality") functions for values of this new type, in the natural and obvious way (here, just compare the bit representations for equality). For other definitions of equality/inequality, we would *not* say "`deriving(Eq)`", and we would instead define equality/inequality explicitly (see "typeclass instances", later).

4.13.3 deriving (FShow)

Given a value *opclass* of type `OpClass`, if we directly print it, *e.g.,*:

```
1   opclass = OPCLASS_MEM;
2   $display ("opclass = %d", opclass);
```

the output will be "3", which is the numeric code the compiler assigns to the constant `OPCLASS_MEM` given its position in the list of labels in the `enum` definition shown in Section 4.13 (starting at zero).

Because we said "`deriving(FShow)`" in the `enum` declararion, the *bsc* compiler will automatically define an "`fshow()`" function for this type: if we print as follows:

```
1   opclass = OPCLASS_MEM;
2   $display ("opclass = ", fshow (opclass));
```

the output will be "OPCLASS_MEM", *i.e.,* the symbolic name.

4.14 if-then-else statements and hardware multiplexers

In most programming languages, "if-then-else"‘ is a so-called "control" construct: depending on the boolean condition, either the then-arm or the else-arm is executed (*but not both*!).

In **BSV** an "if-then-else‘ represents a hardware *multiplexer*. The then-arm and else-arm each represent hardware that computes some value. The if-then-else construct simply selects the output of one of the two arms and passes it on as its output. Stated another way, the if-then-else "multiplexes" the two arm-outputs into a single output. In programming-language terms, *both* arms of the conditional are always "executed"—each arm represents an actual piece of hardware that is continuously computing its output.

The data type of the condition in an if-then-else must always exactly be `Bool` (not `Bit#(1)`, not an integer, *etc.*). The types of the two arms of the conditional must be exactly the same, and this is also the type of the output of the output of the if-then-else.

For example, here is a function that distinguishes CONTROL instructions from integer instructions, returning an `OpClass` (Section 4.13):

```
1  function OpClass instr_opclass (Bit #(32) instr);
2     OpClass result;
3     if (is_legal_BRANCH (instr)
4        || is_legal_JAL (instr)
5        || is_legal_JALR (instr))
6        result = OPCLASS_CONTROL;
7     else
8        result = OPCLASS_INT;
9     return result;
10 endfunction
```

This can also be written using so-called "conditional expressions" (using the same syntax as in SystemVerilog and C):

```
function OpClass instr_opclass (Bit #(32) instr);
   return ((is_legal_BRANCH (instr)
            || is_legal_JAL (instr)
            || is_legal_JALR (instr))
           ? OPCLASS_CONTROL
           : OPCLASS_INT);
endfunction
```

It's a matter of taste and style whether one uses if-then-else expressions or C-style conditional expressions. It may also depend on the size of the sub-epressions. The primary goal should be readability.

Both these code fragments represent the same hardware, shown in Figure 4.4. The 32-bit

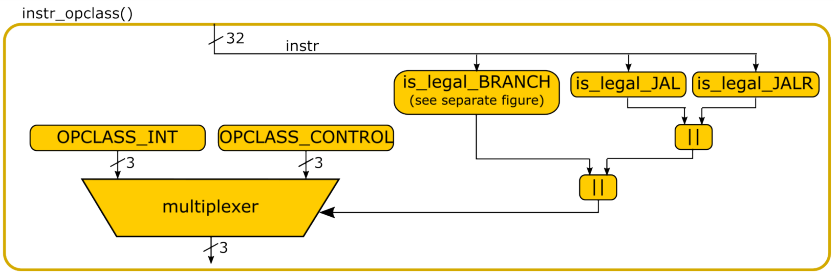


Figure 4.4: If-then-else is a multiplexer

`instr` argument is fed into the circuits for `is_legal_BRANCH()` (hardware schematic in Figure 4.3), `is_legal_JAL()` and `is_legal_JALR()` which are OR'd to produce a `Bool` output which, in turn, is used to select one of two 3-bit constant values, producing a final 3-bit result. The multiplexer, also called a "MUX" for short, is a primitive combinational circuit.

If-then-elses and conditional expressions can of course be nested:

```
function OpClass instr_opclass (Bit #(32) instr);
   OpClass result;
   if (is_legal_BRANCH (instr)
       || is_legal_JAL (instr)
       || is_legal_JALR (instr))
      result = OPCLASS_CONTROL;
   else if (is_legal_OP (instr)
            || is_legal_OP_IMM (instr)
            || is_legal_LUI (instr)
            || is_legal_AUIPC (instr))
      result = OPCLASS_INT;
   else if (is_legal_LOAD (instr)
            || is_legal_STORE (instr))
      result = OPCLASS_MEM;
   else if (is_legal_ECALL (instr)
            || is_legal_EBREAK (instr)
            || is_legal_MRET (instr)
            || is_legal_CSRRxx (instr))
      result = OPCLASS_SYSTEM;
   return result;
endfunction
```

This represents a cascade of multiplexers in hardware, as shown in Figure 4.5

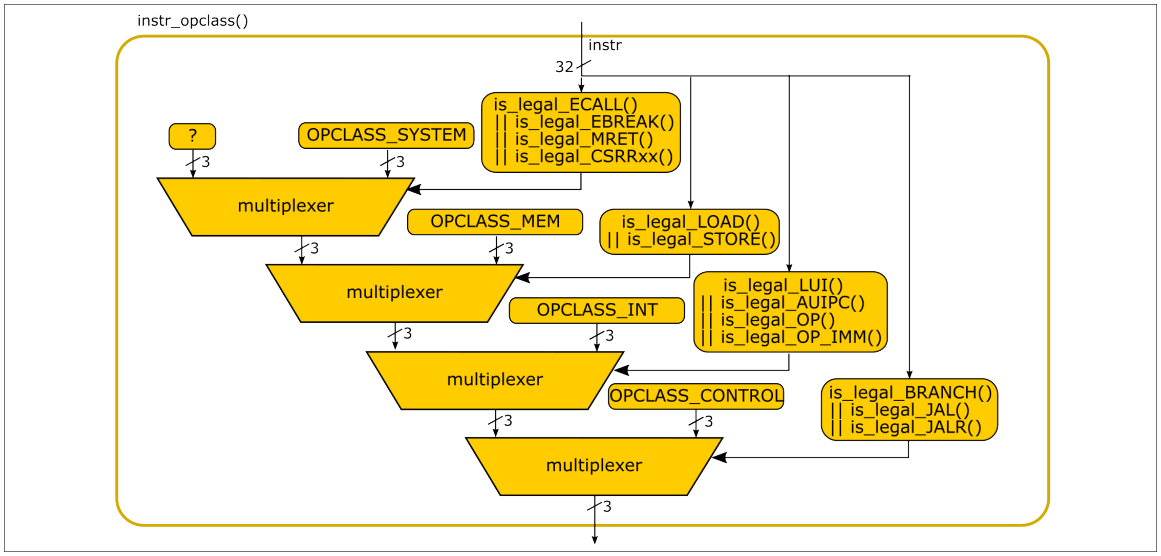


Figure 4.5: Nested if-then-elses become cascaded multiplexers

4.14.1 Parallel multiplexers and MUX synthesis

The circuit in Figure 4.5 has a serial structure—the `OPCLASS_CONTROL` branch has priority, and only if its condition is False can one of the other results flow through. Also observe that the longest path length increases *linearly* with number of classes—here, `OPCLASS_SYSTEM` flows through all four multiplexers.

But we know from RISC-V instruction encodings that the `OPCLASS_CONTROL`, `OPCLASS_INT` and `OPCLASS_MEM` conditions are *mutually exclusive*; no instruction simultaneously falls into more than one such class. In such situations (mutually exclusive conditions) it is possible to create more a efficient circuit called a *parallel MUX*. The structure is illustrated in Figure 4.6.

This kind of MUX is also called an AND-OR MUX or "parallel" MUX because of its structure. It relies for correct operation on *precisely one* of the bitwise-OR arguments being True. Here, we are assured of this because of the mutual exclusivity of the conditions.

While the circuit-depth in the cascade-of-multiplexers is *linear* in the number of if-then-else arms, for the AND-OR MUX it is *logarithmic* (smaller circuit delay $\Longrightarrow$ possibly higher clock speed). Downstream RTL-to-gates synthesis tools could perform this transformation automatically.

4.15 Case-expressions

In the special case where a nested if-then-else is simply testing a value against a series of alternative constant values, we can use a `case` expression. For example:

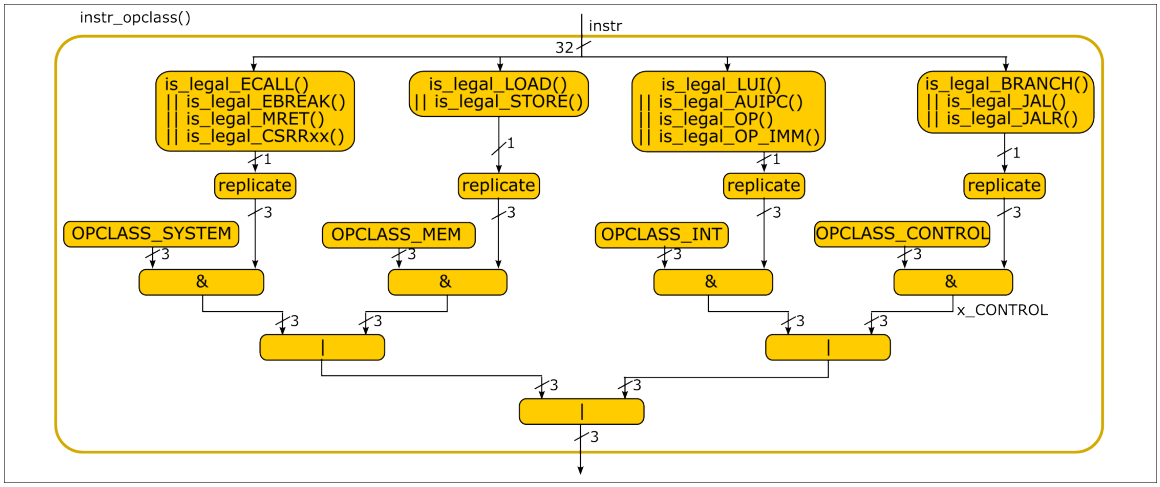


Figure 4.6: Nested if-then-elses using an AND-OR MUX (for mutually exclusive conditions)
.

from src_Common/Fn_EX_Control.bsv

```
1 Bool branch_taken = case (instr_funct3 (instr))
2                        funct3_BEQ:  (rs1_val == rs2_val);
3                        funct3_BNE:  (rs1_val != rs2_val);
4                        funct3_BLT:  signedLT (rs1_val, rs2_val);
5                        funct3_BGE:  signedGE (rs1_val, rs2_val);
6                        funct3_BLTU: (rs1_val < rs2_val);
7                        funct3_BGEU: (rs1_val >= rs2_val);
8                    endcase;
```

NOTE: Note that **BSV** is like Verilog/SystemVerilog in that exactly one arm of the case expression is executed. This is unlike C/C++, where a case arm "falls through" to the next case arm, unless one has a `break`, `return` or `goto` statement.

In this example each case-arm is a pure value, and we call the whole construct a `case` expression. Often each case-arm is an `Action` (such as a register assignment) in which case we sometimes call it a `case` statement. They are really the same thing, differing only in the data types under consideration.

EXERCISE: Please see Ex-04-G-Enums-Muxes in Appendix E, page E-7. □

4.16 Sharing code for RV32 and RV64 *via* parameterization

The RISC-V ISA is actually two ISAs—a 32-bit ISA called RV32 and a 64-bit ISA called RV64. These are not randomly different; they have been carefully engineered to overlap as much as possible:

- RV64I's architectural state is the same in RV32I except that the PC and the 32 general-purpose registers are 64 bits wide instead of 32 bits wide.

- RV64 adds new LOAD and STORE instructions to move 64-bit values between registers and memory.
- Most of the RV32 instructions are exactly the same in RV64, except that they operate on 64-bit values.
- Three R32 instructions are slightly different in RV64–the shift instructions SLLI, SRLI and SRAI have 5-bit shift-amounts in RV32 (allowing up to 32-bit shifts), whereas they have 6-bit shift-amounts in RV64 (allowing up to 64-bit shifts).
- RV64 adds several new instructions that compute on 32-bit values within the 64-bit registers (ADDIW, SLLIW, ..., LWU).

(See Section 2.8 or the RISC-V ISA Privileged Spec for a listing of the RV64I instructions.)

Because of this large overlap we can share much **BSV** code between RV32 and RV64, *i.e.,* we would like to parameterize our **BSV** code so that it can be re-used between RV32 and RV64 implementations.

4.16.1 Using Type Synonyms and Macros to Parameterize for RV32 and RV64

In Section 4.5 we introduced the concept of types of *numeric kind*. For example, the "32" in these examples is a *type* (not a *value*) of numeric kind:

```
1  Bit #(32) instr;
2  Bit #(32) pc_val;
```

The first declaration works for both RV32 and RV64, since instructions are 32 bits wide in both. However, the second declaration only works in RV32, since the program counter is 64 bits wide in RV64 (type `Bit#(64)`).

4.16.1.1 Type synonyms

In **BSV** we can define a *type synonym*, a new symbolic name for an existing type. Example, for RV32:

```
1  typedef 32 XLEN;      // new name for numeric type 32
2
3  Bit #(XLEN) pc_val;
4  Bit #(XLEN) rs1_val;  // Value read from register rs1 in register file
5  Bit #(XLEN) rs2_val;  // Value read from register rs2 in register file
6  Bit #(XLEN) rd_val;   // Value written to register rd in register file
```

By changing the single definition in line 1 to:

```
1  typedef 64 XLEN;      // new name for numeric type 64
```

the remaining code will work for RV64 as well.

4.16.1.2 Macros, preprocessing, and conditional compilation

Instead of editing the line defining `XLEN`, we can automate that choice by using macros and the preprocessor during compilation.

Just like in Verilog, SystemVerilog and C/C++, the *bsc* compiler runs **BSV** source code through a "preprocessor" before compilation, which can perform simple text ("macro") substitutions. Using this facility, we can pass an argument to the compiler that has the effect of configuring the source code for RV32 or RV64:

src_Common/Arch.bsv: line 13 ...

```
1  `ifdef RV32
2
3  typedef 32 XLEN;
4
5  `elsif RV64
6
7  typedef 64 XLEN;
8
9  `endif
10
11 Integer xlen = valueOf (XLEN);
```

The last line defines `xlen`, an value-level variable whose integer value is the same as that expressed by the numeric type `XLEN` (`valueOf` was discussed in Section 4.5).

As in Verilog and SystemVerilog, preprocessor directives begin with a ` character (back-tick) (analogous to `#ifdef` in the C/C++ preprocessor).

When we invoke the *bsc* compiler, we can pass it command line arguments `-DRV32` or `-DRV64`; the preprocessor will then select the appropriate `typedef` line. Thus, we can write common code that will work for both RV32 and RV64. The integer value `xlen` will have the numeric value 32 or 64, corresponding to `XLEN`.

Preprocessor macros allow us to conditionally compile different source text based on the macro definitions we supply to the compiler. We can also compile alternative code based on the value of `xlen`. Consider this function:

src_Common/Instr_Bits.bsv: line 305 ...

```
1  function Bool is_legal_OP_IMM (Bit #(32) instr);
2     let funct3 = instr_funct3 (instr);
3     let funct7 = instr_funct7 (instr);
4     Bool is_legal_SLLI = (((xlen == 32) && (funct7 == 7'b000_0000))
5                           || ((xlen == 64) && (funct7 [6:1] == 6'b0)));
6     Bool is_legal_SRxI = ((   (xlen == 32) && ((funct7 == 7'b010_0000)
7                                                || (funct7 == 7'b000_0000)))
8                           || ((xlen == 64) && ((funct7 [6:1] == 6'b01_0000)
9                                                || (funct7 [6:1] == 6'b00_0000))));
10    return ((instr_opcode (instr) == opcode_OP_IMM)
11            && ((funct3 == funct3_SLLI)
12               ? is_legal_SLLI
13                : ((funct3 == funct3_SRxI)
14                   ? is_legal_SRxI
15                   : True)));
16 endfunction
```

This function checks that `instr[25]` (*i.e.,* `funct7[0]`) is zero in RV32I, but can be zero or 1 in RV64I. Recall, from Figures 2.6 and 2.15, that the `SLLI`, `SRLI` and `SRAI` instructions have a 5-bit shift-amount field ("shamt") in RV32I and a 6-bit shift-amount field in RV64I. Although written using the *value* `xlen`, because it is statically known to *bsc* it will be reduced by *bsc* statically to just one of the two alternatives.

Whenever possible, it is preferable to use the `if(xlen==...)` form instead of the `` `ifdef `` form for conditional compilation because (a) the code is more readable; (b) there is zero run-time (hardware) cost because it will be statically reduced by the compiler; and (c) as we know from experience in many languages, preprocessor macros can be quite dodgy (scoping, inadvertant variable capture, inadvertant surprises due to associativity of infix operators, and so on).

Note that for the `if(xlen==...)` form, both arms of the conditional must type-check correctly, whether `xlen` is 32 or 64. There are ways to achieve this with judicious use of bit-slicing, `extend()` and `truncate()`; we will point them out as we encounter them. If the two arms cannot both type-check whether `xlen` is 32 or 64, we may have to resort to the `` `ifdef `` form.

Chapter 5

BSV: Struct types and tuples; and RISC-V: Memory requests & responses

5.1 RISC-V: structs communicated between stages

In Figure 5.1, various kinds of information need to be communicated between the stages—program counter values, instructions, values read from registers, values to be written back to registers, and so on. `struct` data types (short for "structures") are suitable for bundling

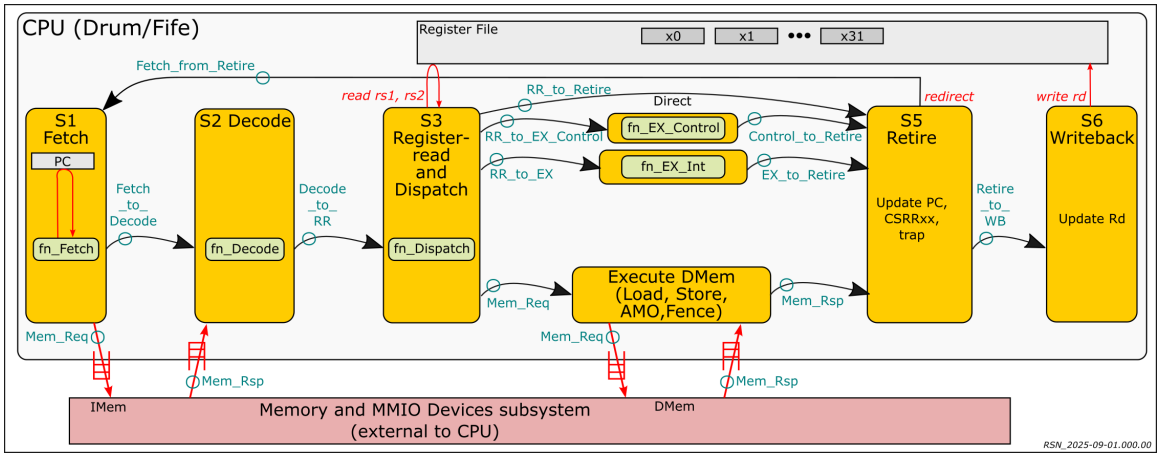


Figure 5.1: Simple interpretation of RISC-V instructions
(Same as Fig. 2.16, with arrows annotated with `struct` types)

together heterogeneous collections of values.[1] Each component of a struct is called a "field" or a "member" of the struct. Figure 5.1 annotates Figure 2.16 with struct types communicated on each of the black arrows between stages, and each of the red arrows to and from memory. In this and the next few chapters we will flesh out the details of all these struct types. We will use exactly the same struct types for **Fife** and **Drum**, *i.e.*, whether the implementation is pipelined or not. All these `struct` declarations can be found in the file: `src_Common/Inter_Stage.bsv`.

[1] Structs in **BSV** are just like structs in C/C++ and SystemVerilog. They are also known as "records" in some programming languages.

5.2 BSV: `struct` types

In Figure 5.1, consider the red arrow from the Fetch stage to the IMem port of memory, or the red arrow from the Execute DMem stage to the DMem port of memory. We want to communicate several values, including:

- The type of memory request: LOAD, STORE, *etc.*
- The size of the memory request: 1, 2 or 4 bytes.
- The address in memory that is to be accessed.
- For a STORE, the data to be written.
- ... (possibly additional fields for debugging) ...

This heterogeneous collection of values (*i.e.,* collection of values of different types) is most conveniently expressed as a `struct` type:

src_Common/Mem_Req_Rsp.bsv: line 50 ...

```
typedef struct {Mem_Req_Type  req_type;
                Mem_Req_Size  size;
                Bit #(64)     addr;
                Bit #(64)     data;     // CPU => mem data
                ...
} Mem_Req
deriving (Bits, FShow);
```

Here, we are defining a new type `Mem_Req` containing fields like `req_type`, `size`, `addr` and `data`, each having a specific type. We will discuss `Mem_Req_Type` and `Mem_Req_Size` soon (in Sections 5.2.6 and 5.2.7, respectively); for now, just think of them as enum types (small bit-width scalars).

The order in which the fields are declared has no semantic significance; the order can be chosen freely for best readability.[2]

5.2.1 Creating (constructing) struct values

We can create a new value of type `Mem_Req` with syntax like this:

```
   Mem_Req x = Mem_Req {req_type: ... value of field ... ,
                        size:     ... value of field ... ,
                        addr:     ... value of field ... ,
                        data:     ... value of field ... ,
                        ...   };
```

The identifier `Mem_Req` is used in two ways. On the left, as usual, it is the *type* of the identifier that we are declaring (`x`). On the right, it is a *constructor*, *i.e.,* it takes the field values and constructs a struct value. The entire right-hand side is sometimes called a "struct expression", *i.e.,* it is an expression which, when evaluated, produces a struct value.

[2] The order of fields will affect the bit-vector representation of the struct (see Section 5.2.4) but that is an implementation detail, not a semantic property.

NOTE: In C, we use the term "value" only for "small" items that are ≤ 64 bits in size (integers, floating point values, and pointers). This is an unfortunate legacy view perhaps tied to implementation detail—things that can be passed in registers—which should not be relevant in the semantics of a high-level language. In more modern languages (including C++) we can have values of any size, including structs, arrays, arrays of structs, and so on.

The repetition of the word `Mem_Req` on both the left-hand and right-hand sides of the "`=`" seems verbose. The *bsc* compiler's type-analysis is easily able to infer the type from the right-hand side, so we can just use the keyword "`let`" (`let`-bindings are discussed in Section 4.6.1):

```
1  let  x   =  Mem_Req {req_type: ... value of field ... ,
2                       size:     ... value of field ... ,
3                       addr:     ... value of field ... ,
4                       data:     ... value of field ... ,
5                       ...   };
```

The order in which the field values are given does not matter. In particular, the order may be different from the order in the `typedef` declaration. The *bsc* compiler will put the fields into the correct offsets in the struct value.

Not all field values need be given in a struct expression. The *bsc* compiler will issue a warning for each unspecified field, and insert an "unspecified" (and unpredictable) value there. You can indicate that a field is intentionally left unspecified (and suppress the compiler warning) using "`?`", **BSV**'s notation for a "don't care" value (these are discussed in Section 4.6.2).

```
1  let  x   =  Mem_Req {req_type: MEM_REQ_LOAD,
2                       size:     ... value of field ... ,
3                       addr:     ... value of field ... ,
4                       data:     ?,
5                       ...   };
```

In our example, when the `req_type` field is `MEM_REQ_LOAD` the `data` field is meaningless, and we indicate this explicitly with "`?`".

5.2.2 Selecting struct fields

Struct fields can be selected using the usual "dot" notation common to SystemVerilog and C/C++:

```
1  x.req_type
2  x.size
```

5.2.3 Updating struct fields using assignment

Struct fields can be updated with assignment using the usual "dot" notation common to SystemVerilog and C/C++:

```
1  x.req_type = MEM_REQ_STORE;
2  x.data     = ... new value ... ;
```

5.2.4 Representation of a struct in bits

Because we said "`deriving(Bits)`" in the struct type declaration, the *bsc* compiler will automatically work out a representation in bits for `Mem_Req` struct values, using the straightforward method of simply concatenating the bit-vector representation of each field into a wider bit-vector for the whole struct. The total bit-size of a `Mem_Req` struct value is simply the sum of the bit-sizes of the individual fields.

If we had *not* said "`deriving(Bits)`", the *bsc* compiler will not define any bit-representation for this type. As mentioned in Section 4.13.1, (a) we do not need any hardware representation in bits for types that are only used at statically (at compile time) and (b) if we *do* need a bit-representation, **BSV** has a mechanism for specifying this explicitly ("typeclass instances").

	In C/C++, compilers will often "pad" out fields (insert unused bytes between fields) so that fields are aligned on byte and word boundaries, for more efficient access in byte-structured memories. Thus, a struct's size in C/C++ may be larger than the sum of the field sizes, and may even vary depending on the compiler's target architecture or optimization flags.
NOTE:	In hardware design, struct values reside on wires, in registers, in FIFOs, *etc.* which have no "byte-structured" bias, and so we do not play any such "padding" games in **BSV**.
	SystemVerilog makes a distinction beteen "packed" and "unpacked" values; the former may have padding (for efficient simulation) whereas the latter has no padding (for efficient hardware).

5.2.5 Printing a struct for logging/debugging

We can print a struct value directly, *e.g.,*

```
1    Mem_Req mem_req;
2    ...
3    $display ("mem_req is: ", mem_req);
```

It will simply print a hexadecimal representation of the entire bit-vector representing `mem_req`. This can be quite difficult to read, for two reasons. First, the bit-vector representing a struct can be quite large (sometimes hundreds of bits!). Second, because bits are packed, fields may not align with hexadecimal digit boundaries, and so one cannot correlate hexadecimal digits directly with fields. A similar problem arises if we view a struct-valued signal in a waveform viewer, which does not know anything about **BSV** types and just shows the hexadecimal value.

For this reason, experienced **BSV** coders sometimes (perhaps temporarily for debugging) rearrange the order and sizes of fields in a struct so that they are well-aligned with hexadecimal digit boundaries.

Because we said "`deriving(FShow)`" in the type declaration, the *bsc* compiler will automatically define an "`fshow()`" function for this type. This gives us a way to print a struct in a more readable format:

```
    Mem_Req mem_req;
    ...
    $display ("mem_req is: ", fshow (mem_req));
```

(Note: we described `deriving (FShow)` for enum types in Section 4.13.3).

Now, the printout will look something like this:

```
Mem_Req { req_type: 'h1e, size: MEM_4B, addr: 'h0000000080000000,
          data: 'h0000000000000000, ...}
```

i.e., the struct value display is formatted to show the struct type and the individual fields.

5.2.6 Memory request type

A memory-request in RV32I is either a FETCH, a LOAD, a STORE, a FENCE, or a FENCE.I.[3] We could define an enum type for this:

```
typedef enum {MEM_REQ_FETCH,
              MEM_REQ_LOAD,
              MEM_REQ_STORE,
              MEM_REQ_FENCE,
              MEM_REQ_FENCE_I} Mem_Req_Type
deriving (Bits, FShow, Eq);
```

which will use a 3-bit encoding (0 for FETCH, 1 for LOAD, 2 for STORE, 3 for FENCE and 4 for FENCE_I). However, we look to the future where we might extend **Fife** and **Drum** to implement the "A" extension (for AMO or "Atomic Memory Operations"). The RISC-V ISA Unprivileged Spec document, Chapter 8, describes additional memory operations LR (Load-Reserved), SC (Store-Conditional), AMOSWAP, AMOADD, AMOXOR, AMOAND, AMOOR, AMOMIN, AMOMAX, AMOMINU, and AMOMAXU, each of which comes in a 32-bit version (in RV32 and RV64) and a 64-bit version (in RV64). These operations are coded with 5 bits in the instruction (`instr[31:27]`, which we call "`funct5`").

Accordingly, we use a 5-bit encoding for the memory request type. When we implement the A extension, we will directly use the 5-bit codes from the instruction. For LOAD, STORE and FENCE we use 5-bit codes that are unused in the A extension:

src_Common/Instr_Bits.bsv: line 229 ...

```
Bit #(5) funct5_FETCH   = 5'b_00110;
Bit #(5) funct5_LOAD    = 5'b_11110;
Bit #(5) funct5_STORE   = 5'b_11111;
Bit #(5) funct5_FENCE   = 5'b_11101;
Bit #(5) funct5_FENCE_I = 5'b_11001;
```

[3] A FETCH is just a LOAD, but it is useful to distinguish this for debugging.

5.2.7 Memory request size

An IMem request (FETCH) is for one 32-bit instruction (4 bytes).[4]

An RV32I DMem request may be for 1, 2 or 4 bytes. Looking to the future, accesses may also be for 8 bytes (doublewords in RV64I, and/or double-precision floating point values in the "D" ISA extension for RV32 and RV64). We express these request-size options using an enum type:

src_Common/Mem_Req_Rsp.bsv: line 47 ...

```
typedef enum {MEM_1B, MEM_2B, MEM_4B, MEM_8B} Mem_Req_Size
deriving (Bits, FShow, Eq);
```

Similarly, we have also declared the `addr` and `data` fields in the `Mem_Req` struct to be 64 bits, not 32. For STORE requests of 1 and 2 bytes (*i.e.,* smaller than the `data` field) we assume the data is passed in the least-significant bytes of the `data` field.

5.3 BSV: Tuples and the `match` statement

Tuples are a special kind of pre-defined *immutable* structs in **BSV**.[5] For example, when a function needs to return a pair of values, instead of declaring a struct type with two fields for the result, it is often more convenient to just use the built-in `Tuple2` type. Example:

in src_Common/CSRs.bsv

```
   function ActionValue #(Tuple2 #(Bool, Bit #(XLEN)))
            fav_csr_read (Bit #(12) csr_addr);
      ...
         return tuple2 (exception, y);
   endfunction
```

The function returns a pair of values which have types `Bool` and `Bit#(XLEN)`, respectively. The overall type of the result is `Tuple2 #(Bool, Bit #(XLEN))`.

The expression in the `return` statement shows how one can construct a 2-tuple value: simply apply the predefined function `tuple2` to the two component values.

Tuples are special in **BSV** because, unlike structs, tuples are immutable—there is no assignment statement to update a field of a tuple.

Tuples are also special in **BSV** because there is no ".field" notation for component selection. To select an individual field of a tuple, one can use one of the built-in selector functions:

```
   let xy <- fav_csr_read (...);
   let exc = tpl_1 (xy);    // exc has type: Bool
   let v   = tpl_2 (xy);    // v   has type: Bit #(XLEN)
```

Alternatively, to access multiple fields of a tuple at once, one can use a "match" statement. Example (from `src_Drum/CPU_FSM.bsv`):

[4] When implementing the **C** RISC-V ISA extension (compressed instructions), instructions can also be 16 bits (2 bytes). When implementing more sophisticated Fetch units, we may actually fetch chunks larger than 4 bytes, such as a full cache line.

[5] Python has a similar concept of immutable tuples, and pattern-matching to access tuple components.

```
1    match { .exc, .v } <- fav_csr_read (csr_addr);
```

The `match` keyword is followed by a *pattern* that should match the tuple, *i.e.,* here it should be a list of two variable names surrounded by braces with each variable preceded by a "`.`". The leading "`.`" is important: "`x`" would represent the value of an existing variable to be matched with the corresponding value in the tuple, whereas "`.x`" *introduces a new variable* `x` to be bound to the corresponding value in the tuple; the latter is what we want. We refer the reader to the **BSV** Language Reference Guide [25] for many more capabilities of pattern-matching, including fields to be ignored, pattern-matching on structs and tagged unions, pattern-matching `case` expressions, and more.

BSV defines tuples of up to 8 components (`Tuple_3`, `Tuple_4`, ...) but we recommend not using tuples when you have more than, say, 2 or 3 components; define a struct instead, with readable and meaningful field names.

5.4 RISC-V: Memory Requests and Responses; IMem and DMem

In Figure 5.1, the `Mem_Req` and `Mem_Rsp` structs are used in two places. The Fetch stage issues a memory request, and the corresponding memory response is received by the Decode stage. Similarly, the "Execute Memory Ops" stage issues a memory request and consumes a memory response. We use the shorthand term "IMem" for the first context (for Instruction Memory) and "DMem" for the latter context (for Data Memory).

5.4.1 Separation of IMem and DMem (Harvard Architecture)

The separation of memory channels for instructions and data (loads/stores) is quite standard in modern CPU architectures, and is informally called a "Harvard Architecture". The term refers to the architecture of the Harvard Mark I computer, designed and built by Harvard University and IBM in the 1940s (the term itself was coined much later). It sometimes refers just to separate, concurrent paths to memory for instructions and data, and sometimes also to physically separate memories for instructions and data (more discussion in Wikipedia: https://en.wikipedia.org/wiki/Harvard_architecture).

Modern software is typically not "self-modifying", *i.e.,* instructions and data are placed in different areas of memory, and load/store instructions never write into the instruction area, *i.e.,* programs never over-write instructions in memory. This allows separate hardware for memory access for instructions *vs.* memory access for data, which can run concurrently, *i.e.,* we may fetch an instruction at the same time as we are accessing data memory for a previous load/store instruction (we will see this in **Fife**). We can also tune and optimize each memory path separately for their different dynamic behavioral patterns. In some systems we can also *protect* the instruction memory area, *i.e.,* enforce in hardware the policy of not over-writing instructions.

This view of strict separation of IMem and DMem has to be tempered somewhat when considering languages like JavaScript, Python *etc.* that employ so-called "JIT" compiling ("Just-In-Time"). The run-time systems of such languages generate instructions on-the-fly, *i.e.,*, LOAD/STORE instructions produce *data* through the DMem channel that will (soon)

be fetched as *instructions*. But even in these systems, there is a strict protocol of *phases*. During a code-generation phase, the data produced is considered as ordinary data. Then there is a deliberately executed phase-change, where the virtual memory protections of the data-pages just written are changed so that they are now viewed as read-only instruction pages, after which these new instructions can be fetched.

5.4.2 Address Alignment

Although nowadays we think of all computer memories in units of 8-bit bytes and being byte-addressed,[6] in practice in hardware, it is usually simpler if memory-requests are *aligned* to an address according to the request size. Specifically, the address for a 2-byte request should be even, *i.e.,* the least significant bit of the address, `addr[0]`, should be zero. The address for a 4-byte request should have zero in the two least significant bits (`addr[1:0]`) and the address for an 8-byte request should have zero in the three least significant bits (`addr[2:0]`).

We can see why address-alignment is desirable. Memory implementations (chips) are usually architected to retrieve multiple bytes at a time (*e.g.,* 64 bytes) so that all those bytes can share addressing and control circuitry. With such an organization, a misaligned access request may straddle the boundaries of such "naturally sized" units and so may require two consecutive reads/writes. Caches are usually organized to hold multi-byte *cache lines* (*e.g.,*, 32 bytes) in order to share the addressing and miss-handling circuitry, and to move data efficiently in and out of the cache. Again, a misaligned access request may straddle a cache-line boundary, and may require two consecutive accesses, which may hit or miss independently. Virtual memory systems are usually organized in *pages*, units of typically 4K-8K bytes, in order to share virtual-memory handling circuitry, and to move data efficiently between main memory and disks. Again, a misaligned access may straddle a page boundary, and may require two consecutive accesses, which may hit or page-fault independently and differently. In short, misaligned accesses add significant complexity to memory-system hardware design.

We can organize our software so that misaligned accesses are exceedingly rare. Most software is produced by compilers, and the compiler ensures that instructions and data are placed in memory at aligned addresses, possibly by padding gaps between "adjacent" smaller-sized data (such as a pair of 1-byte-sized fields in a struct). This padding may waste a few bytes of memory, but pays back in greater speed and reduced complexity.

Although misaligned accesses are rare, we cannot always guarantee their absence in software, since software can calculate an arbitrary address before performing a memory access. It seems wasteful to have to pay for extra hardware complexity (with attendant loss in overall performance) for such rare cases. In many computer systems, therefore, these rare misaligned accesses are relegated to software handling:

- The memory system simply refuses to handle a misaligned access, and returns a "misaligned" error instead.
- The CPU, receiving such an error response, undergoes a "trap" which directs it to piece of software called an trap-handler (or exception handler). The trap-handler (in software)

[6] Some early computers, until about the late 1970s, had other memory granularities—multiples of 6, 7, 9 bits, *etc.* Those were the days of bespoke memories for each computer design. Mass-production of memory chips resulted in standardization to 8-bit bytes.

performs the memory access in multiple smaller pieces (in the worst case, of size 1 byte), each of which is aligned. In other words, the trap-handler "completes" the original memory access before resuming the main-line code that attempted it.

The RISC-V ISA specification does not forbid misaligned accesses nor prescribe how they should be handled. Some implementations will handle it in hardware, and other implementations will return a "misaligned" error and rely on a trap handler to complete the access. Some implementations may only run software where it has been proven to not generate misaligned memory requests, and therefore may not even contain a trap-handler for misaligned accesses.

EXERCISE: Please see Ex-05-A-Structs in Appendix E, page E-8. □

5.4.3 Memory Responses

The response from memory for any request may be to report success, an alignment error, or some other error. Examples of "other errors" are:

- Absence of memory at the given address. For example, although RV32I addresses are 32-bits, which can address 4GiB of memory, we may provision our system with something smaller, say 1 GiB.
- An unsupported operation. *E.g.,* an attempt to write into a read-only memory (ROM).
- Corruption of data, due to electrical glitches, environmental electromagnetic pulses, *etc.*. These errors are usually detected with some kind of error-detecting code, such as parity bits.

These different memory response-types can be encoded in an enum type:

src_Common/Mem_Req_Rsp.bsv: line 66 ...

```
typedef enum {MEM_RSP_OK,
              MEM_RSP_MISALIGNED,
              MEM_RSP_ERR,
              ...
} Mem_Rsp_Type
deriving (Bits, FShow, Eq);
```

A memory-response contains the response-type. For a LOAD request with an OK response, it also contains the data that was read from memory. This can be expressed in a struct:

src_Common/Mem_Req_Rsp.bsv: line 75 ...

```
typedef struct {Mem_Rsp_Type  rsp_type;
                Bit #(64)     data;      // mem => CPU data or DEFERRED

                Mem_Req_Type  req_type;  // for DEFERRED
                Mem_Req_Size  size;      // for DEFERRED
                Bit #(64)     addr;      // is also tval if not OK
                ...

} Mem_Rsp
deriving (Bits, FShow);
```

For LOAD requests of 1 and 2 bytes (*i.e.,* smaller than the `data` field) we assume the data is returned in the least-significant bytes of the `data` field.

EXERCISE: Please see Ex-05-B-Mem-Req-Rsp in Appendix E, page E-9. □

Chapter 6

RISC-V: Core functions for ISA execution (used in Drum and Fife)

6.1 Introduction

In this chapter we discuss the core functions of Figure 6.1: `fn_Fetch`, `fn_Decode`, `fn_Dispatch`, `fn_EX_Control` and `fn_EX_Int`.

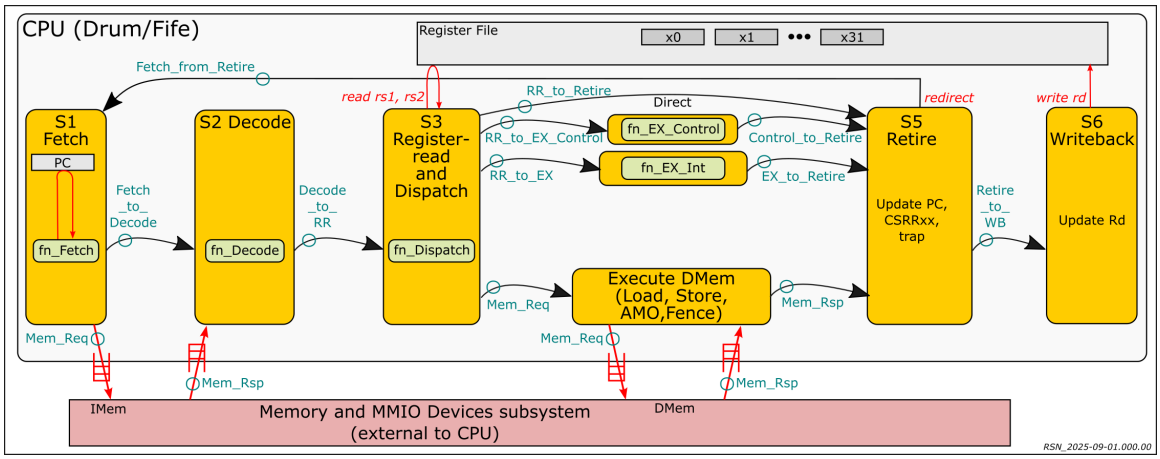


Figure 6.1: Simple interpretation of RISC-V instructions

6.2 The function `fn_Fetch`

The Fetch function *per se* is fairly simple, even trivial. Its input is the current value of the program counter (PC), which is used as the address in a memory-request to IMem. It has two outputs,

- A *memory request* to memory, to read an instruction.
- Some additional information "`Fetch_to_Decode`" passed on to the Decode stage.

We discussed the `Mem_Req` struct in Section 5.2.

The "`Fetch_to_Decode`" struct has only one interesting field, the PC:

src_Common/Inter_Stage.bsv: line 30 ...

```
typedef struct {
   Bit #(XLEN)  pc;
   ...
} Fetch_to_Decode
deriving (Bits, FShow);
```

NOTE:

> In the source code you will see that this struct defines some additional fields: `predicted_pc`, `epoch` and `halt_sentinel`. Please ignore these for now; the first two are not used in **Drum**, only in **Fife**, and the latter is used only if we have hardware debugger support.
>
> In the source code you will also see an additional field: `xtra`, of type `Fetch_to_Decode_Xtra`. This, too, can be ignored for now. For many a struct of type X we carry some additional information in a field of type X`_Xtra`, that is *only* used for debugging and otherwise are not necessary in a production implementation. To avoid visual clutter in the production code, we choose to define these in a separate file that is inserted into this file (see `` `include "Inter_Stage_Xtra.bsvi" ``) near the top of this file). This is just a stylistic choice; all the fields could have been defined directly within the `Fetch_to_Decode` struct. Also, in a production design, we can easily remove these additional fields to eliminate the wires and registers that they occupy.
>
> In this case, the type `Fetch_to_Decode_Xtra` is a struct with one field, `inum`, that holds the "instruction number", which is a sequence number counting every instruction fetched. In **Drum** this number is exactly the number of insructions executed. In **Fife** this number will be larger because some instructions are fetched speculatively and discarded (note executed).

To pass both results of `fn_Fetch`, we simply use a *nested* struct, *i.e.,* a struct containing the two component structs:

src_Common/Fn_Fetch.bsv: line 25 ...

```
typedef struct {
   Fetch_to_Decode  to_D;
   Mem_Req          mem_req;
} Result_F
deriving (Bits, FShow);
```

Finally, as mentioned earlier, the function `fn_Fetch` is almost trivial: it merely fills in the two result structs based on argument values and returns them:

src_Common/Fn_Fetch.bsv: line 33 ...

```
function ActionValue #(Result_F)
         fn_Fetch (Bit #(XLEN)  pc,
                   ...
   actionvalue
      Result_F y = ?;
      // Info to next stage
      y.to_D = Fetch_to_Decode {pc:            pc,
                                ...
                               };

      // Request to IMem
      y.mem_req = Mem_Req {req_type: funct5_FETCH,
                           size:     MEM_4B,
                           addr:     zeroExtend (pc),
                           ...
```

```
16                              };
17      return y;
18   endactionvalue
19 endfunction
```

As the comment at the top of the code excerpt says, `fn_Fetch` is actually a pure function (no side-effects), and we could have written it as follows:

```
function Result_F
         fn_Fetch (Bit #(XLEN)  pc,
                   ...
                  );
      Result_F y = ?;
      ...
      return y;
endfunction
```

i.e., the return-type is `Result_F` instead of `ActionValue#(Result_F)`, and we simply drop the "`actionvalue`—`endactionvalue`" bracket keywords. Recall Section 4.11 where we discussed pure *vs.* side-effecting functions, and their distinction in the type-system through the absence/presence of the `ActionValue` type constructor. Even though this is a pure function, by couching it as an `ActionValue` type, we preserve the option of inserting a `$display` (which *is* a side-effect) for debugging purposes, should we need it in the future.

In line 5 we declare variable `y` of type `Result_F`, with unspecified initial value (recall Section 4.6.2 where we discussed using "`?`" to indicate an unspecified or don't-care value). The two fields of `y` are then updated in lines 8 and 13, respectively, and `y` is finally returned as the result of the function.

EXERCISE: Please see Ex-06-A-Fetch in Appendix E, page E-12. □

6.3 The `fn_Decode` function

The core function for the Decode stage is `fn_Decode`. Its arguments are a struct of type `Fetch_to_Decode` from the Fetch stage (discussed in Section 6.2) and a `Mem_Rsp` memory response struct from memory (discussed in Section 5.4.3). Its result is a struct `Decode_to_RR`:

src_Common/Inter_Stage.bsv: line 52 ...

```
1  typedef struct {
2     Bit #(XLEN)  pc;
3     ...
4     // If exception
5     Bool         exception;  // Fetch exception/ decode illegal instr
6     Bit #(4)     cause;
7     Bit #(XLEN)  tval;
8
9     // If not exception
10    Bit #(XLEN)  fallthru_pc;
11    Bit #(32)    instr;
12    OpClass      opclass;
```

```
13     Bool         has_rs1;
14     Bool         has_rs2;
15     Bool         has_rd;
16     Bool         writes_mem;   // All mem ops other than LOAD
17     Bit #(XLEN)  imm;          // Canonical (bit-swizzled)
18     ...
19  } Decode_to_RR
20  deriving (Bits, FShow);
```

The field `pc` carries the PC for this instruction. This will be needed by BRANCH, JAL, JALR and AUIPC instructions to compute addresses that are offset from the current PC. It will also be needed for any traps (exceptions) that may occur in the execution of this instruction, since trap-handling saves the PC in the `mepc` CSR.

The boolean `exception` flag indicates:

- whether an error was encountered in the Fetch-to-Memory-to-Decode path, or
- whether the Decode stage's analysis indicates that the instruction is not legal (unrecognized 32-bit code).

When the exception flag is true, `cause` and `tval` fields provide more detail.

If there is no exception (no Fetch memory error, and instruction is legal), other fields provide more analytical detail for subsequent stages:

- The `fallthru_pc` is the address of the next instruction following this one in memory. For RV32I and RV64I, this will always be PC+4, since all instructions are 4-bytes long.[1] For most instructions, `fallthru_pc` is indeed the unique next PC. For conditional BRANCH instructions, this is the next PC if the BRANCH is not taken. For JAL and JALR instructions (unconditional jumps), this is the PC saved by the instruction in the `ra` register ("return address").
- The instruction itself, `instr`. This will be needed for opcode details, the rs1, rs2 and rd register indexes, immediate values, *etc.*

The next several fields are derived by analyzing the instruction. They could be re-derived wherever needed by re-analyzing the instruction, but we perform that work just once in the Decode stage and communicate the results forward.

- The `opclass` field indicates which Execute-stage path in Figure 6.1 will process this instruction.
- The `has_rs1` and `has_rs2` fields indicate whether this instruction reads any rs1 and/or rs2 registers; this will be control reading from the register file.
- The `has_rd` field indicates whether instruction writes an rd register. This will control writing to the register file.

[1] If we implement the "C" RISC-V ISA extension (compressed instructions), the fall-through PC could be PC+2.

- The `writes_mem` field indicates whether or not this instruction writes to memory. This is useful in **Fife** in the Retire stage to know whether or not there may be a pending write to memory that needs to be committed.

- The `imm` field contains any "immediate" value in the instruction. Refer to the top of each page of "Table 24.2 Instruction listing for RISC-V" in the Unprivileged Spec [27], which shows that the I-, S-, B-, U- and J-type instructions have immediate values of different sizes and encode them in different ways ("bit-swizzled"). We untangle these once in the Decode stage, and pass on the normalized results in the `imm` field.

The code for `fn_Decode` is mostly a big if-then-else that analyses the incoming instruction and produces some summary information:

src_Common/Fn_Decode.bsv: line 27 ...

```
function ActionValue #(Decode_to_RR)
         fn_Decode (Fetch_to_Decode  x_F_to_D,
                    Mem_Rsp          rsp_IMem,
                    ...
   actionvalue
      Bit #(32) instr = truncate (rsp_IMem.data);
      Bit #(5)  rd    = instr_rd (instr);

      let fallthru_pc = x_F_to_D.pc + 4;

      // Baseline info to next stage
      let y = Decode_to_RR {pc:            x_F_to_D.pc,
                            ...
                            // exception
                            exception:     False,
                            cause:         ?,
                            tval:          0,

                            // not-exception
                            fallthru_pc:   fallthru_pc,
                            instr:         instr,
                            opclass:       ?,
                            has_rs1:       False,
                            has_rs2:       False,
                            has_rd:        False,
                            writes_mem:    False,
                            imm:           0,

                            xtra: Decode_to_RR_Xtra {
                               inum: x_F_to_D.xtra.inum
                            }};

      Bool non_zero_rd = (rd != 0);

      if (rsp_IMem.rsp_type == MEM_RSP_MISALIGNED) begin
         y.exception = True;
         y.cause     = cause_INSTRUCTION_ADDRESS_MISALIGNED;
         y.tval      = truncate (rsp_IMem.addr);
      end
      else if (rsp_IMem.rsp_type == MEM_RSP_ERR) begin
         y.exception = True;
         y.cause     = cause_INSTRUCTION_ACCESS_FAULT;
         y.tval      = truncate (rsp_IMem.addr);
```

```
44      end
45      ...
46      else if (is_legal_LUI (instr) || is_legal_AUIPC (instr)) begin
47         y.opclass = OPCLASS_INT;
48         y.has_rd  = non_zero_rd;
49         y.imm     = signExtend ({ instr_imm_U (instr), 12'h000 });
50      end
51      else if (is_legal_BRANCH (instr)) begin
52         y.opclass = OPCLASS_CONTROL;
53         y.has_rs1 = True;
54         y.has_rs2 = True;
55         y.imm     = signExtend (instr_imm_B (instr));
56      end
57      else if (is_legal_JAL (instr)) begin
58         y.opclass = OPCLASS_CONTROL;
59         y.has_rd  = non_zero_rd;
60         y.imm     = signExtend (instr_imm_J (instr));
61      end
62      else if (is_legal_JALR (instr)) begin
63         y.opclass = OPCLASS_CONTROL;
64         y.has_rs1 = True;
65         y.has_rd  = non_zero_rd;
66         y.imm     = signExtend (instr_imm_I (instr));
67      end
68      else if (is_legal_LOAD (instr)) begin
69         y.opclass = OPCLASS_MEM;
70         y.has_rs1 = True;
71         y.has_rd  = non_zero_rd;
72         y.imm     = signExtend (instr_imm_I (instr));
73      end
74      else if (is_legal_STORE (instr)) begin
75         y.opclass    = OPCLASS_MEM;
76         y.has_rs1    = True;
77         y.has_rs2    = True;
78         y.writes_mem = True;
79         y.imm        = signExtend (instr_imm_S (instr));
80      end
81      else if (is_legal_OP_IMM (instr)) begin
82         y.opclass = OPCLASS_INT;
83         y.has_rs1 = True;
84         y.has_rd  = non_zero_rd;
85         y.imm     = signExtend (instr_imm_I (instr));
86      end
87      else if (is_legal_OP (instr)) begin
88         y.opclass = OPCLASS_INT;
89         y.has_rs1 = True;
90         y.has_rs2 = True;
91         y.has_rd  = non_zero_rd;
92      end
93      else if (is_legal_ECALL (instr)
94               || is_legal_EBREAK (instr)
95               || is_legal_MRET (instr)) begin
96         y.opclass = OPCLASS_SYSTEM;
97      end
98      else if (is_legal_CSRRxx (instr)) begin
99         y.opclass = OPCLASS_SYSTEM;
100        y.has_rs1 = (instr_funct3 (instr) [2] == 0);
101        y.has_rd  = non_zero_rd;
102     end
```

```
103         else if (is_legal_FENCE (instr)) begin
104            y.opclass = OPCLASS_FENCE;
105         end
106         else if (is_legal_FENCE_I (instr)) begin
107            y.opclass = OPCLASS_FENCE;
108         end
109         else begin
110            y.exception = True;
111            y.cause     = cause_ILLEGAL_INSTRUCTION;
112            y.tval      = truncate (instr);
113         end
114
115         return y;
116      endactionvalue
117   endfunction
```

In line 6, we extract the instruction from the `Mem_Rsp` memory response `data` from the Fetch operation. The `truncate` operation is used to shrink the bit-vector width of `rsp_IMem.data` (64 bits) to the bit-vector width of `instr` (32 bits). (Section 5.2 discussed why we declared `rsp_IMem.data` to be 64-bits wide). The `truncate` operation is polymorphic, accepting arguments of any bit-width that is at least as wide as the required output. Note, `truncate` keeps least-significant bits and drops most-significant bits.

In line 7 we extract the `rd` ("destination register") field from the instruction. In line 9 we compute the fall-through PC, PC+4 (with the caveat that if extend the implementation to support the "C" RISC-V ISA extension ("Compressed" instructions), it may be PC+2, which information can be gleaned from the instruction encoding).

In lines 11-31 we create a baseline `Decode_to_RR` value which we will selectively modify in the if-then-else statements that follow.

In lines 35-44 we first handle the sitations where the Fetch operation to memory itself returned an error. We mark the `exception` field True and fill in the appropriate `cause` and `tval` (these will be placed in MCAUSE and MTVAL CSRs in the Retire stage).

The rest of the code is a series of if-then-else clauses. Each clause identifies one class of instruction and updates the `opclass` field correspondingly. The repertoire of instructions that we consider are the forty instructions listed in the "RV32I Base Instruction Set" table of "Table 24.2: Instruction listing for RISC-V" of the Unprivileged Spec [27].

Each if-then-else clause also fills in the `has_rs1`, `has_rs2` and `has_rd` fields, as appropriate, for each class of instruction. Note that the `has_rd` field is set to False if `rd` is zero (recall that general-purpose register `x0` ignores writes and always reads as 0).

We also decode each kind of "immediate" and fill in the `imm` field in the struct. Recall from Section 2.5, including Figures 2.4 and 2.5, that different classes of instructions encode immediate values in different ways, and the immediate values can have different bit-widths. We use the functions `instr_imm_I()`, `instr_imm_S()`, `instr_imm_B()`, `instr_imm_U()` and `instr_imm_J()` to extract the and rearrange the immediate bits appropriately for each class of instruction (see also Exercise Ex-04-D-Decode-Imm on page E-5). Then, here in `fn_Decode`, we zero- or sign-extend each immediate as appropriate so that, from this point onwards, each immediate can be treated as an ordinary `Bit#(XLEN)` value.

The final "else" clause is selected if the instruction does not match any of the forty RV32I

instructions. In this case we set the `exception` field, and set the `cause` field to indicate an illegal instruction.

Observe that the entire `fn_Decode()` function is just a (large) combinational circuit—it is an acyclic composition of smaller combinational circuits, many of which we've seen earlier. The whole `fn_Decode()` function can be visualized as a box with incoming wires corresponding to the `Fetch_to_Decode` struct and the `Mem_Rsp` struct, outgoing wires corresponding to the `Decode_to_RR` struct, and filled with logic gates that compute each output wire as a function of the input wires.

EXERCISE: Please see Ex-06-B-Decode in Appendix E, page E-14. □

6.4 The `fn_Dispatch` function after reading input registers

In the Register-Read and Dispatch stage ("RR"), we first read the `rs1` and `rs2` values from the Register File. We will cover register files in Section 7.4, and register-reads when we discuss the **Drum** CPU itself, in Chapter 10.

With the `rs1` and `rs2` values in hand, we use function `fn_Dispatch` to determine the information to be passed to the various alternative stages ("flows") that follow. Its argument is a `Decode_to_RR` struct from the Decode stage, which was discussed in Section 6.3. Before we look at the result type of `fn_Dispatch`, let us discuss the four possible subsequent flows:

- "Direct": Some information is sent directly from RR to Retire for *every* instruction. Crucially, RR sends a *tag* that indicates which of the four flows is being followed for the current instruction.

  Additional direct information includes information from RR: PC, whether an exception has already been seen in the Fetch or Decode stages, `has_rd`, `writes_mem`, the instruction, the fall-through PC and the `rs1` value (needed for CSRRx instructions).

  The direct flow is also used for all SYSTEM instructions, including CSRRxx, ECALL, EBREAK, and MRET.

- "Control": If the instruction is a BRANCH, JAL or JALR, we produce information for the Execute Control flow.

- "Integer": If the instruction is a LUI, AUIPC or integer arithmetic or logic (IALU) operation, we produce information for the Execute Integer flow.

- "DMem": If the instruction is a LOAD, STORE or FENCE, we produce information for the Execute Memory flow.

The following enum type declaration defines four constants to identify the flow for this instruction:

src_Common/Inter_Stage.bsv: line 82 ...

```
1 typedef enum {EXEC_TAG_DIRECT,
2               EXEC_TAG_CONTROL,
3               EXEC_TAG_INT,
```

```
                EXEC_TAG_DMEM
} Exec_Tag
deriving (Bits, Eq, FShow);
```

The result type of `fn_Dispatch` is `Result_Dispatch`. It is just a nested struct that contains four different struct types for the four flows (the struct types can be seen on arc labels in Figure 6.1).

src_Common/Fn_Dispatch.bsv: line 29 ...

```
typedef struct {
   RR_to_Retire      to_Retire;
   RR_to_EX_Control  to_EX_Control;
   RR_to_EX          to_EX;
   Mem_Req           to_EX_DMem;
} Result_Dispatch
deriving (Bits, FShow);
```

The first component is the information sent directly to Retire:

src_Common/Inter_Stage.bsv: line 88 ...

```
typedef struct {
   Exec_Tag     exec_tag;    // ``flow'' for this instr

   Bit #(XLEN)  pc;
   ...

   // If exception
   Bool         exception;   // Fetch exception, decode illegal instr
   Bit #(4)     cause;
   Bit #(XLEN)  tval;

   // If not exception
   Bit #(XLEN)  fallthru_pc;
   Bit #(32)    instr;
   Bit #(XLEN)  rs1_val;     // For CSRRXX instrs
   Bool         has_rd;      // From RR
   Bool         writes_mem;  // From RR
   ...
} RR_to_Retire
deriving (Bits, FShow);
```

The `exec_tag` informs Retire about the flow for this instruction. The `exception` and `cause` fields from `Decode_to_RR` are carried through, as-is, since it is the Retire stage that handles all exceptions. Note, in addition to passing these exceptions in RR to Retire, the Control or Execute stages can also produce exceptions. The `pc` field is needed in case Retire needs to handle a trap or interrupt, which saves the `pc` before handling it.

The `has_rd` field is carried through to control whether Retire tries to write a value back to the register file nor not. The `fallthru_pc` is used for most instructions that complete successfully (without raising an exception).

The second component of `Result_Dispatch` is the information sent to Execute Control (we will see how these fields are used in Section 6.5 when we discuss `fn_EX_Control`):

src_Common/Inter_Stage.bsv: line 119 ...

```
typedef struct {
   Bit #(XLEN)  pc;
```

```
   Bit #(XLEN)  fallthru_pc;
   Bit #(32)    instr;
   Bit #(XLEN)  rs1_val;
   Bit #(XLEN)  rs2_val;
   Bit #(XLEN)  imm;
   ...
} RR_to_EX_Control
deriving (Bits, FShow);
```

The third component of `Result_Dispatch` is the information sent to Execute Int (we will see how these fields are used in Section 6.6 when we discuss `fn_EX_Int`):

src_Common/Inter_Stage.bsv: line 152 ...

```
typedef struct {
   Bit #(32)    instr;
   Bit #(XLEN)  rs1_val;
   Bit #(XLEN)  rs2_val;
   Bit #(XLEN)  imm;
   ...
} RR_to_EX
deriving (Bits, FShow);
```

We choose to call it `RR_to_EX` instead of `RR_to_EX_Int` because the same struct type is likely also to be used in future for the other optional execution pipes between the Dispatch and Retire stages: Integer Multiply Divide (**M** ISA extension), floating point (**F,D** ISA extensions) and Custom (non-standard ISA extensions).

The third component of `Result_Dispatch` is the information sent to Execute DMem, and is the same `Mem_Req` struct we have already discussed in Section 5.2, and which is also an output component of `fn_Fetch`.

The code for `fn_Dispatch` is shown below. Its arguments are the `Decode_to_RR` struct from the Decode stage, and values from the source registers `rs1` and `rs2`.

src_Common/Fn_Dispatch.bsv: line 39 ...

```
function ActionValue #(Result_Dispatch)
         fn_Dispatch (Decode_to_RR         x,
                      Bit #(XLEN)          rs1_val,
                      Bit #(XLEN)          rs2_val,
                      ...
   actionvalue
      // Compute tag to control merging at Retire
      Exec_Tag exec_tag = EXEC_TAG_DIRECT;    // exceptions and OPCLASS_SYSTEM
      if (! x.exception) begin
         if      (x.opclass == OPCLASS_CONTROL) exec_tag = EXEC_TAG_CONTROL;
         else if (x.opclass == OPCLASS_INT)     exec_tag = EXEC_TAG_INT;
         else if (x.opclass == OPCLASS_MEM)     exec_tag = EXEC_TAG_DMEM;
         else if (x.opclass == OPCLASS_FENCE)   exec_tag = EXEC_TAG_DMEM;
      end

      let to_Retire = RR_to_Retire {exec_tag:      exec_tag,

                                    pc:            x.pc,
                                    ...

                                    exception:     x.exception,
                                    cause:         x.cause,
```

```
23                                  tval:         x.tval,
24
25                                  fallthru_pc:  x.fallthru_pc,
26                                  instr:        x.instr,
27                                  rs1_val:      rs1_val,
28                                  has_rd:       x.has_rd,
29                                  writes_mem:   x.writes_mem,
30                                  ...
31                                  };
32       // ----------------
33       // Info for EX_Control
34       let to_EX_Control = RR_to_EX_Control {pc:           x.pc,
35                                             fallthru_pc:  x.fallthru_pc,
36                                             instr:        x.instr,
37                                             rs1_val:      rs1_val,
38                                             rs2_val:      rs2_val,
39                                             imm:          x.imm,
40                                             ...
41                                             };
42       // ----------------
43       // Info for Execute Int pipe
44       let to_EX  = RR_to_EX {instr:   x.instr,
45                              rs1_val: rs1_val,
46                              rs2_val: rs2_val,
47                              imm:     x.imm,
48                              ...
49                              };
50       // ----------------
51       // Info for Execute DMem pipe
52       Bit #(XLEN)  eaddr    = rs1_val + x.imm;
53       Mem_Req_Size mrq_size = unpack (x.instr [13:12]);  // B, H, W or D
54       Mem_Req_Type mrq_type = (is_LOAD (x.instr) ? funct5_LOAD
55                                : (is_STORE (x.instr) ? funct5_STORE
56                                   : (is_FENCE (x.instr) ? funct5_FENCE
57                                      : (is_FENCE_I (x.instr) ? funct5_FENCE_I
58                                         : funct5_INVAL))));
59
60       let to_EX_DMem = Mem_Req {req_type: mrq_type,
61                                 size:     mrq_size,
62                                 addr:     zeroExtend (eaddr),
63                                 data:     zeroExtend (rs2_val),
64                                 epoch:    x.epoch,
65                                 ...
66                                 };
67       // ----------------
68       // Construct and return final result
69       let result = Result_Dispatch {to_Retire:     to_Retire,
70                                     to_EX_Control: to_EX_Control,
71                                     to_EX:         to_EX,
72                                     to_EX_DMem:    to_EX_DMem};
73       return result;
74    endactionvalue
75 endfunction
```

Lines 7-14 compute `exec_tag`, *i.e.,* the flow to be followed.

Lines 16-31, 34-41, 44-49 and 51-66 construct the flow-specific struct values. The latter three are meaningful only if `exec_tag` indicates Control, Integer of DMem, respectively,

but there is no harm in constructing them, even if they may contain bogus data; they will only be used when their flows are chosen and ignored otherwise.

Lines 68-73 construct and return the final result with the four component structs.

EXERCISE: Please see Ex-06-C-Dispatch in Appendix E, page E-15. □

6.5 The `fn_EX_Control` function

This function is a simple one-input one-output function. The input type `RR_to_EX_Control` was described in Section 6.4, where it was an output type. The output type of `fn_EX_Control` is shown below:

src_Common/Inter_Stage.bsv: line 132 ...

```
typedef struct {
   // If exception
   Bool         exception;  // Misaligned BRANCH/JAL/JALR target
   Bit #(4)     cause;
   Bit #(XLEN)  tval;

   // If not exception
   Bit #(XLEN)  next_pc;
   Bit #(XLEN)  data;       // Return-PC for JAL/JALR
   ...
} EX_Control_to_Retire
deriving (Bits, FShow);
```

Here is the Execute Control function:

src_Common/Fn_EX_Control.bsv: line 31 ...

```
function ActionValue #(EX_Control_to_Retire)
         fn_EX_Control (RR_to_EX_Control  x,
                        ...
   actionvalue
      let instr   = x.instr;
      let rs1_val = x.rs1_val;
      let rs2_val = x.rs2_val;
      ...
      Bit #(XLEN)  next_pc   = ?;
      Bool         exception = False;    // Misaligned target_pc

      if (is_BRANCH (instr)) begin
         Bool branch_taken = case (instr_funct3 (instr))
                                funct3_BEQ:  (rs1_val == rs2_val);
                                funct3_BNE:  (rs1_val != rs2_val);
                                funct3_BLT:  signedLT (rs1_val, rs2_val);
                                funct3_BGE:  signedGE (rs1_val, rs2_val);
                                funct3_BLTU: (rs1_val < rs2_val);
                                funct3_BGEU: (rs1_val >= rs2_val);
                             endcase;
         let target_pc = x.pc + x.imm;
         next_pc = (branch_taken ? target_pc : x.fallthru_pc);
         exception = (branch_taken && (target_pc [1:0] != 0));
         ...
      end
```

```
26        else if (is_JAL (instr)) begin
27           next_pc = x.pc + x.imm;
28           exception = (next_pc [1:0] != 0);
29           ...
30        end
31        else if (is_JALR (instr)) begin
32           // zero out LSB in target PC
33           next_pc = ((rs1_val + x.imm) & ~1);
34           exception = (next_pc [1:0] != 0);
35           ...
36        end
37        ...
38        let y = EX_Control_to_Retire {exception:  exception,
39                                      cause:      cause_INSTRUCTION_ADDRESS_MISALIGNED,
40                                      tval:       next_pc,
41                                      next_pc:    next_pc,
42                                      data:       x.fallthru_pc,
43                                      ...
44                                      };
45        return y;
46     endactionvalue
47  endfunction
```

The first "`if`" clause handles BRANCH (conditional branch) instructions. The `case` expression computes the boolean value `branch_taken`, *i.e.,* the decision whether to take the branch or not. A case-clause is chosen based on the 3-bit `funct3` field of the instruction that identifies the specific condition to be tested.[2] Note that for BLT and BGE, we use BSV library functions `signedLT` and `signedGE` that interpret `rs1_val` and `rs2_val` as 2's-complement signed integers.

Line 21 computes the target PC should the the branch be taken. The next line computes the next PC, which is either the target PC or the fall-through PC depending on whether or not the branch is taken. Finally, the next line checks, if the branch is taken, that the target PC is a properly aligned address (if not, we must raise an exception).

The next "`else if`" clause handles JAL (Jump and Link) instructions and the final "`else if`" clause handles the JALR (Jump and Link Register) instructions. They are both straightforward, unconditional calculations of a next PC, along with an alignment-check that the next PC is suitably aligned. As per the Unprivileged ISA specification document, JALR also zeroes the least-significant bit of target PC.

The final section constructs the `EX_Control_to_Retire` struct result and returns it.

EXERCISE: Please see Ex-06-D-Ex-Control in Appendix E, page E-16. □

6.6 The `fn_EX_Int` function

This function is a simple one-input one-output function. The input type `RR_to_EX` was described in Section 6.4, where it was an output type. The output type of `fn_EX_Int` is

[2] Unlike C/C++, where a case-clause "falls through" to the next case-clause unless you have a `break` statement, in BSV only one case-clause is executed, there is no fall-through.

shown below:

src_Common/Inter_Stage.bsv: line 164 ...

```
typedef struct {
   // If exception
   Bool         exception;
   Bit #(4)     cause;
   Bit #(XLEN)  tval;

   // If not exception
   Bit #(XLEN)  data;
   ...
} EX_to_Retire
deriving (Bits, FShow);
```

We choose to call it `EX_to_Retire` instead of `EX_Int_to_Retire` because the same struct type is likely also to be used in future for other optional execution pipes Integer Multiply Divide (**M** ISA extension), floating point (**F,D** ISA extensions) and Custom (non-standard ISA extensions).

Here is the Execute Integer function:

src_Common/Fn_EX_Int.bsv: line 30 ...

```
function ActionValue #(EX_to_Retire)
         fn_EX_Int (RR_to_EX  x,
                    ...
   actionvalue
      let instr = x.instr;

      let y = EX_to_Retire {exception:  False,
                            cause:      ?,
                            tval:       ?,
                            data:       ?,
                            ...
                            };

      if (is_LUI (instr)) begin
         y.data = x.imm;
         ...
      end
      else if (is_AUIPC (instr)) begin
         y.data = x.xtra.pc + x.imm;
         ...
      end
      else begin
         let result <- fn_IALU (instr, x.rs1_val, x.rs2_val, x.imm,
                                ...
         y.data = result;
         ...
      end
      return y;
   endactionvalue
endfunction
```

The first "`if`" handles LUI instructions (Load Upper Immediate). The following "`else if`" handles AUIPC instructions (Add Upper Immediate to PC). The final "`else`" handles all the remaining Integer ops by invoking an "ALU" function:

src_Common/IALU.bsv: line 27 ...

```
function ActionValue #(Bit #(XLEN))
         fn_IALU (Bit #(32)    instr,
                  Bit #(XLEN)  v1,
                  Bit #(XLEN)  v2,
                  Bit #(32)    imm,
                  ...
   actionvalue
      Bit #(7)    opcode = instr_opcode (instr);
      Bit #(3)    funct3 = instr_funct3 (instr);
      // Signed int versions of v1, v2 and imm
      Int #(XLEN) iv1    = unpack (v1);
      Int #(XLEN) iv2    = unpack (v2);
      Int #(XLEN) i_imm  = unpack (imm);
      ...
      Bit #(XLEN) y_OP     = 0;
      if (opcode == opcode_OP) begin
         ...
         Bit #(6) shamt = (v2 [5:0] & ((xlen == 32) ? 'h1F : 'h3F));
         case (funct3)
            funct3_ADD:  y_OP = pack ((instr [30] == 1'b0)
                                      ? (iv1 + iv2)
                                      : (iv1 - iv2));
            funct3_SLL:  y_OP = v1 << shamt;
            funct3_SLT:  y_OP = ((iv1 < iv2) ? 1 : 0);
            funct3_SLTU: y_OP = ((v1  < v2)  ? 1 : 0);
            funct3_XOR:  y_OP = v1 ^ v2;
            funct3_SRL:  // Note: funct3_SRL == funct3_SRA; distinguish on instr [30]
                         y_OP = ((instr [30] == 1'b0)
                                 ? v1 >> shamt                // SRL
                                 : pack (iv1 >> shamt));      // SRA
            funct3_OR:   y_OP = v1 | v2;
            funct3_AND:  y_OP = v1 & v2;
            ...
         endcase
         ...
      end

      Bit #(XLEN) y_OP_IMM = 0;
      if (opcode == opcode_OP_IMM) begin
         ...
         Bit #(6) shamt = (imm [5:0] & ((xlen == 32) ? 'h1F : 'h3F));
         case (funct3)
            funct3_ADDI:  y_OP_IMM = pack (iv1 + i_imm);
            funct3_SLTI:  y_OP_IMM = ((iv1 < i_imm) ? 1 : 0);
            funct3_SLTIU: y_OP_IMM = ((v1  < imm)   ? 1 : 0);
            funct3_XORI:  y_OP_IMM = v1 ^ imm;
            funct3_ORI:   y_OP_IMM = v1 | imm;
            funct3_ANDI:  y_OP_IMM = v1 & imm;
            funct3_SLLI:  y_OP_IMM = v1 << shamt;
            funct3_SRLI:  // Note: funct3_SRLI == funct3_SRAI; distinguish on instr [30]
                          y_OP_IMM = ((instr [30] == 1'b0)
                                      ? (v1 >> shamt)          // SRLI
                                      : pack (iv1 >> shamt));  // SRAI
            ...
         endcase
         ...
      end

```

```
59        Bit #(XLEN) result = y_OP | y_OP_IMM;
60        ...
61        return result;
62     endactionvalue
63  endfunction
```

The reason we create a separate `fn_IALU` function, instead of inlining it into `Fn_EX_Int` is because `fn_IALU` is not very RISC-V specific, and may be useful in other contexts that have nothing to do with RISC-V, **Drum** or **Fife**.

In lines 11-13, we define signed-integer versions `iv1`, `iv2` and `i_imm` of the unsigned integer values `v1`, `v2`, and `imm`, respectively, using the standard BSV `unpack` function. There is no hardware cost to this, these declarations are simply declarations to "view" the same bits differently (as 2's-complement coded integers). The difference arises later, when we apply certain operators to these values. For example, the line computing SLT (Set Less Than (signed)) uses the signed values `iv1` and `iv2` whereas the line computing SLTU (Set Less Than Unsigned) uses the unsigned values `v1` and `v2`. Between the *bsc* compiler and the Verilog back-end, different code will be generated for the "`<`" operator to perform the correct kind of comparison.

Line 18 extracts a 5-bit (for RV32) or 6-bit (for RV64) "shift amount" from the `rs2` value for the shift operators SLL, SRL and SRA. Line 41 extracts the "shift amount" from the `imm` value for the shift operators SLLI, SRLI and SRAI.

SRL (Shift Right Logical) and SRA (Shift Right Arithmetic) differ in whether they treat the argument as a signed or unsigned value, the difference being whether the new bits shifted into the most-significant bit are zero (SRL) or replicate the most-significant bit (SRA). SRLI and SRAI exhibit a similar difference.

In lines 30 (SRA) and 53 (SRAI) we finally apply the "`pack`" operator to produce the result. This is because the expression "`(iv1 >> shamt)`" has type `Int#(XLEN)` whereas the result needs to be of type `Bit#(XLEN)`. The "`pack`" operator performs this type-change for us.

Lines 15-36 define the `y_OP` result when the opcode is `opcode_OP` *i.e.,* `7'b_011_0011`, *i.e.,* the "3-address" operators where the inputs come from `rs1` and `rs2`. `y_OP` defaults to 0 when it is not an `op_OP`.

Lines 38-57 define the `y_OP_IMM` result when the opcode is `opcode_OP_IMM` *i.e.,* `7'b_001_0011`, *i.e.,* the "2-address" operators where one input come from `rs1` and the other input comes from an immediate value in the instruction. `y_OP_IMM` defaults to 0 when it is not an `op_OP_IMM`.

Finally, line 59 combines these results using the "OR" function. We rely on the fact that exactly one of `y_OP` and `y_OP_IMM` can be relevant; the other one must be zero (and therefore has no effect through OR'ing).

EXERCISE: Please see Ex-06-E-Ex-Int in Appendix E, page E-17. □

6.7 No shared functions for Execute DMem and Retire

We don't need any separate functions for the Execute DMem stage in Figure 6.1 because `fn_Dispatch` has created a `Mem_Req` struct that we can send directly to memory. Similarly, the memory returns a `Mem_Rsp` that is sent directly to the Retire stage.

The code for the Retire stage is discussed in detail separately in Chapter 10 for **Drum** and Chapter 16 for **Fife**. Although functionally the same, the codes are structurally sufficiently different that it is not worth attempting to abstract any common functionality into a shared function.

Chapter 7

BSV: Modules and Interfaces: Registers, Register Files and FIFOs

7.1 Introduction

It is good engineering practice to organize the code for any non-trivial system, whether in hardware or software, into a well-structured composition of smaller, manageable *modules*. Each module should have a clear, independent specification so that it can be understood on its own, and so that it can transparently be substituted by another module with the same functionality but perhaps other desirable properties (*e.g.,* speed, area, power). The external specification of a module—its "interface"—should not rely on, and ideally not even mention, internal implementation details of the module. For example, each of the units shown in Figure 2.16 could be a separate module.

This chapter is mostly a **BSV** chapter; we discuss modules and interfaces in **BSV**, including a few that are provided by the **BSV** libraries and that we will use in subsequent chapters to implement our RISC-V CPUs.

7.2 Modules: state, interfaces and behavior

Modules encapsulate modifiable *state*. Examples include Registers, Register Files and FIFOs (all of which will be discussed in this chapter). Modules with state are the only entities containing values that *persist* over time, *i.e.,* a value written at one moment in time can be read at a later moment in time.

Modules also encapsulate external behavior, using *interface methods*. In this sense they are similar to "objects" in object-oriented programming languages such as C++, Java, and Python. A **BSV** module is like an object constructor; a module *instance* is like an object; its internal state is like the internal, private "members" of the object, and its interface is *a set of methods* that can be invoked like functions/procedures, and which can access the internal state.

Modules and interfaces clearly separate concerns of externally-visible functionality ("external API"; *what* a module does; a module *specification*) *vs.* internal implementation details (*how* the module does it).

BSV modules are typically organized in a *hierarchy*—a top-level module, which instantiates sub-modules which, in turn, instantiate lower-level modules, and so on. In **Drum** and **Fife**:

Top-level module
    CPU module
        CPU sub-modules
            Library modules: Registers, Register file, FIFOs, ...
    Memory system
        Memory module(s)
        MMIO device modules (*e.g.,* UART, timer, GPIO, interrupt controller)

In the next several sections we describe the concepts of **BSV** modules and interfaces. These sections may require re-reading a couple of times; the concepts become properly internalized only after seeing/using/creating several examples.

NOTE: We sometimes write **BSV** modules that do not themselves contain internal state, for stylistic and readability reasons. One example is seen in Section 7.5.6 where a module is used to encapsulate the logic of connecting two complementary interfaces.

7.2.1 Internal behavior (*rules*)

Unlike most programming languages, **BSV** modules typically also contain *internal* free-running processes called *rules* that run concurrently with the rest of the system (all other rules in the system). Rules realize the independent, concurrent, internal behavior of a module.

Rules are discussed in more detail in Chapter 13. Before that, in Chapter 9 we will discuss **BSV**'s special notation for FSMs (Finite State Machines), which are simpler to use as a first step.

7.2.2 Interface declarations

An *interface declaration* in **BSV** declares a new interface, which is a new **BSV** *type*, and looks like this:

```
interface interface-type;
    ...
    ... method and sub-interface declarations ...
    ...
endinterface
```

The interface represents the external view of a module, *i.e.,* it declares a set of *methods* that can be invoked from an external context. Each method declaration only lists its arguments and their types, and the method's overall result type. The *body* of the method is defined in the module definition of each module that offers this interface type.

Interfaces can be nested (can contain sub-interfaces which themselves have methods or sub-sub-interfaces, and so on). This is just a syntactic abstraction mechanism; ultimately, all interactions with a module are through its methods, whether at the top level of the interface type, in a sub-interface, in a sub-sub-interface, *etc.*

There can be many module definitions each of which offers the same interface, *i.e.,* these are different *implementations* of the same interface. For example, the **BSV** library contains a repertoire of FIFO modules, all of which have the same FIFO interface type. Different implementations of a particular interface type typically differ on some dimension such as performance (latency, bandwdith, MHz), silicon area/FPGA gates, power consumption, *etc.*. One chooses a particular implementation based on such practical requirements.

Sections 7.3.1, 7.4.1 and 7.5.1 show the interface declaration for **BSV** library modules: Registers, Register Files and FIFOs, respectively.

7.2.2.1 Hardware for an interface

An interface method has zero or more arguments. Its result-type falls in one of three categories, as introduced in Section 4.11:

- Type `Action`: possible side-effect
- Type `ActionValue #(`t`)`: possible side-effect that also returns a value of type t
- Type t ("value") that is neither of the above: pure (no side-effect) returning a value of type t

The types of the method arguments, and the method result type, together completely determine the specific hardware (input/output wires and buses) at the module interface corresponding to that method. This is illustrated in Figure 7.1.

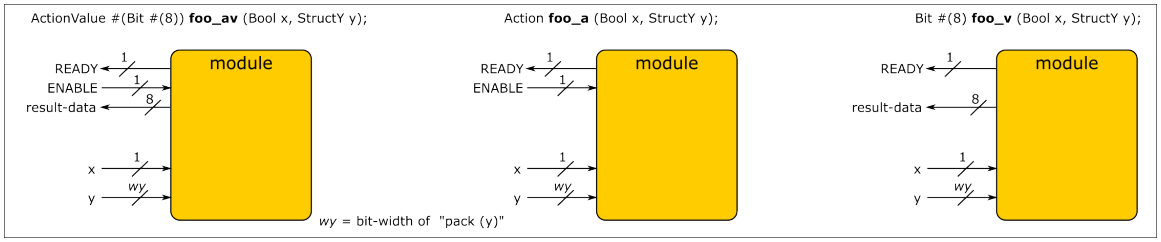


Figure 7.1: Hardware wires/buses for interface methods of `ActionValue(`t`)`, `Action` and value result types

All methods have a READY output wire corresponding to its *implicit condition*. This is discussed in more detail in Chapter 13, but, briefly, it is a boolean value indicating when it is meaninful to invoke this interface method. For example consider the "`first`" method in a FIFO that returns the value at the head of the FIFO. This method is not meaningful if the FIFO is empty. Thus, its READY method is an indication that there is data available in the FIFO.

`Action` and `ActionValue(`t`)` methods have an ENABLE input wire. The environment of the module asserts this wire (drive "1" on the wire) when it wants the module to perform the method's action (write a register, enqueue and item, dequeue an item, ...). Value methods do not have any side-effect, *i.e.,* they do not perform any action, and so they do not have any ENABLE input.

`ActionValue(`t`)` and value methods have a result-data output bus (bundle of wires). `Action` methods, since they return no result, have not result-data output bus.

For all three categories of methods, arguments are treated in the same way: each argument of type t has an input data bus of the appropriate width to carry values of type t.

7.2.3 Module definitions

A *module definition* in **BSV** describes a module with a particular interface type:

```
(* synthesize *)
module module-name ( interface-type );
    ... instantiation of module state (registers, FIFOs, other sub-modules) ...
    ... behavior (rules and FSMs) ...
    ... interface (API) method implementations ...
endmodule
```

The `(* synthesize *)` attribute at the top is optional. With this attribute, the *bsc* compiler creates a separate Verilog module for this **BSV** module, when compiling to Verilog. Without the attribute, the compiler will "inline" it into any parent module where it is instantiated. We recommend using this attribute on all modules.[1]

A module definition defines a module *constructor*. The constructor can be invoked multiple times to obtain multiple *instances* of the module.

NOTE:

A notable difference between **BSV** and other HDLs (Verilog, SystemVerilog and VHDL) is that even a lowly register is not special; it is just another module, with an interface containing "`_read()`" and "`_write()`" methods.

In fact **BSV** treats *all* "state elements" (components that store persistent values) uniformly as modules with interfaces ("It's modules all the way down").

7.2.4 Module instantiation and method invocation

A module is *instantiated* using this syntax:

```
interface-type x <- constructor (constructor-arg,...,constructor-arg);
```

This creates a new instance of the module and binds the offered interface to the identifier *x*. Some constructors have no arguments, in which case even the parentheses surrounding the arguments can be omitted.

Subsequently, methods of the module can be invoked using the syntax

```
x.method-name (method-arg,...,method-arg);
```

Some methods have no arguments, in which case even the parentheses surrounding the arguments can be omitted.

7.3 BSV Library Modules: Registers

A register is the simplest storage element in digital hardware, a single memory cell containing a single value (represented as a bit-vector). We can (over-)write with a new value, and we can read out the value stored by the most recent write.

[1] But see also Section 7.6.1 which desribes some modules where this attribute is ignored by the compiler.

7.3.1 `Reg#(t)`, the register interface from the BSV library

The standard register interface type in **BSV** has two methods:

```
1 interface Reg #(t);
2    method t _read();
3    method Action _write (t x);
4 endinterface
```

Here, "`t`" is the type of value stored in the register (discussed in more detail below).

The `_read()` method (with no arguments) just returns the value stored in the register, of type `t`. The `_write()` method takes one argument, a value of type `t`, and stores it in the register, over-writing any previous value and holding the new value until over-written by the next `_write()`.

We will explain the `Action` type of the `_write` method in more detail later. For now, just think of it as the type of any method that is a pure side-effect, *i.e.,* the method modifies some internal state of the module, and does not return any value.

7.3.2 Registers are strongly typed

Unlike Verilog, SystemVerilog and VHDL, **BSV** registers are "strongly typed". Each register instance can only hold values of one particular type, specified at the place where the register is instantiated.

Further, the register-contents type need not be `Bits#()`; it can more generally be *any* **BSV** type that has a representation in bits. Thus, the type of a value in a register can be an enum, a struct, a nested struct, *etc.*, if we have used a `deriving(Bits)` declaration (or its explicit analog) to ensure that it has a representation in bits.

Any attempt to read or write a value into a register that does not match the declared type will provoke a compile-time type-checking error from the *bsc* compiler.

7.3.3 `mkReg(`v`)` and `mkRegU`: register module (constructor) from the BSV library

A standard **BSV** library register module is `mkReg`. It is used to instantiate a new register, with a specified reset value, using a statement like this:

```
1    Reg #(Bit #(XLEN)) rg_pc <- mkReg (0);
```

Here we declare a new identifier `pc` with interface type `Reg#(Bit#(XLEN))` (the register interface type) and bind it to the interface offered by a newly instantiated register. The "0" argument to `mkReg()` specifies the reset-value of the register, *i.e.,* the value held in the register immediately after the hardware has been reset (you can initialize it to any value in the register's type)..

An alternative register constructor provided by the **BSV** library is `mkRegU`, where the "U" indicates that it is uninitialized, *i.e.,* has no specified reset value:

```
1   Reg #(Bit #(XLEN)) rg_pc <- mkRegU;
```

`mkRegU` instantiates a register with an unspecified (unpredictable) reset value, and hence does not need an argument. Unitialized registers are slightly cheaper in hardware (no reset wires, no initialization value wires, no multiplexer to choose between initialization and other update values). They are often used in situations where we know that the module logic will never read the register before the first write.

7.3.4 Syntactic shorthands for register access

Registers are so ubuiquitous in digital design that **BSV** provides some special syntactic shorthands for reading and writing registers.

Just mentioning a register in an expression can be used as a shorthand for invoking its `_read` method. Thus, the expression:

```
rg_pc + 4
```

is shorthand for:

```
rg_pc._read + 4
```

To invoke the `_write` method on a register, one can use a conventional assignment statement. Thus, the expression:

```
rg_pc._write (v)
```

can be written like this:[2]

```
rg_pc <= v
```

A statement like this:

```
rg_pc <= rg_pc + 4
```

contains both shorthands:

```
rg_pc._write (rg_pc._read + 4)
```

NOTE: The use of the "`rg_`" prefix in the above examples is just our own syntactic convention, and not required in **BSV** syntax, where any legal identifier can be bound to a register interface. We will be mixing identifiers bound to ordinary values and identifiers bound to register interfaces in various expressions. The `rg_` prefix reminds us that there is an implicit "`_read`" on the latter.

[2] Rather than use "`=`" or "`:=`" common in software programming languages, we use "`<=`", which is the Verilog/SystemVerilog notation for "delayed assignment".

7.4 BSV Library Modules: Register files

A register file is an array of registers with a common pair of methods to read or write a particular register identified by an index, which is an argument to the methods for reading and writing.

7.4.1 The register file interface `RegFile#(index_t,data_t)`

The standard register file interface type in the **BSV** library is:

```
1 interface RegFile #(type index_t, type data_t);
2    method Action upd (index_t addr, data_t d);
3    method data_t sub (index_t addr);
4 endinterface: RegFile
```

Here, "`index_t`" is the type for the index, which we use to identify one of the registers in the register file. For RISC-V, since we have 32 registers, we will use `Bit#(5)` as the index type.

"`data_t`" is the type of value stored in each of the registers. For RISC-V, this will be `Bit#(XLEN)`.

The *rf*`.upd(j,v)` method allows us to store the value v in the j'th register of register file *rf*. The *rf*`.sub(j)` method returns the current value v in the j'th register of register file *rf*.

NOTE: The index type `index_t` can be any type that has a representation in bits, *i.e.,* for which we have used the `deriving(Bits)` annotation in the type declaration (or for which we have provided a so-called `Bits` instance explicitly).

BSV Register files, like **BSV** registers, are strongly typed. At time of instantiation of a register file *rf*, we specify its `index_t` and `data_t` types. In subsequent uses of *rf*, the provided index and data value, and returned data value, must have exactly those types (else the *bsc* compiler will raise a compile-time type-error.).

7.4.2 `mkRegFileFull`, a register file module (constructor)

The **BSV** library contains a couple of register file modules (constructors). For RISC-V we can use `mkRegFileFull`:

```
RegFile #(Bit #(5), Bit #(XLEN)) gprs <- mkRegFileFull;
```

Here we declare a new identifier `gprs` with interface type `RegFile#(Bit#(5),Bit#(XLEN))` (the register file interface type) and bind it to the interface offered by a newly instantiated register file. The number of registers in the register file is known from the full range of the index type `Bit#(5)`, *i.e.,* it will have 32 registers, indexed from 0 to 31. Each register is `XLEN` bits wide.

7.5 BSV Library Modules: FIFOs

FIFOs (First-in-First-out) elements are *ordered queues* of values and are broadly useful in many hardware designs (arguably as useful as registers). We can enqueue a new value into a FIFO at the tail (back) of the queue, and dequeue a value from the head (front) of the queue. Most **BSV** FIFOs are automatically "flow-controlled", *i.e.,* it is impossible to enqueue into a full FIFO and to dequeue from an empty FIFO.

7.5.1 `FIFOF#(t)`, the FIFO interface type

A standard FIFO interface type in the **BSV** library is:[3]

```
1 interface FIFOF #(t);
2    method Bool notEmpty();
3    method Bool notFull();
4    method t first();
5    method Action deq();
6    method Action enq (t x);
7    method Action clear();
8 endinterface
```

Here, "`t`" is the type of values stored in the FIFO (discussed in more detail below).

The `f.notEmpty()` and `f.notFull()` are simple predicates to test if a FIFOF `f` is empty or full, respectively.

The `f.first()` and `f.deq()` methods are used to access the head of the queue. They are only available if the FIFO is not empty. The `first()` method returns the value at the head of the queue. This is non-destructive, *i.e.,* it does not modify the FIFO. The `f.deq()` method modifies the FIFO: it discards the value at head of the queue and advances the queue.

The `f.enq(x)` method is used to access the tail of the queue, and is only available if the FIFO is not full. It modifies the FIFO by appending the argument `x` to the tail of the queue.

The `clear` method is used to empty the queue immediately (discard all its contents).

Notice that the `FIFOF#(t)` interface does not indicate the *capacity* of the FIFO, *i.e.,* the number of elements it can hold from head to tail. This is deliberate; we may choose different capacities for each FIFO instance as required by its use context. We also want flexibly and transparently to be able to substitute a FIFO with another that has greater or less capacity.

7.5.1.1 `pop`: a useful function combining `first` and `deq`

The reason that the `first` and `deq` methods are separated is because there are many situations where we wish to examine the head of a FIFO and only consume the value if it matches some condition of interest. But there are also many situations where we don't need this generality, and we invoke both methods together in the same action. For convenience we define a function for this:

[3] The **BSV** library also defines the `FIFO#(t)` interface which is the same as the `FIFOF#(t)` except that it omits the `notEmpty` and `notFull` methods. We prefer the latter, which provides more capability.

```
function ActionValue #(t) pop (FIFOF #(t) fifo);
   actionvalue
      let x = fifo.first;
      fifo.deq;
      return x;
   endactionvalue
endfunction
```

This must be an `ActionValue` function because it performs a side-effect (`deq`) and returns a value (from `first`). This is then invoked with the usual syntax for invoking ActionValues:

```
   let y <- pop (fifo);
```

7.5.2 FIFO modules (constructors): `mkFIFOF` and more

The **BSV** library contains many different FIFO modules (constructors): single-element FIFOs, FIFOs of a specified depth (queue length), FIFOs with and without automatic flow-control, *etc.* The most commonly used is `mkFIFOF`:

```
   FIFOF #(Mem_Req) f_to_IMem   <- mkFIFOF;
   FIFOF #(Mem_Rsp) f_from_IMem <- mkFIFOF;
```

Here we declare a new identifier `f_to_IMem` with interface type `FIFOF#(Mem_Req)` and bind it to the interface offered by a newly instantiated FIFO. Similarly, we declare a new identifier `f_from_IMem` with interface type `FIFOF#(Mem_Rsp)` and bind it to the interface offered by a newly instantiated FIFO. Due to **BSV**'s strong-typing, the first FIFO can only hold items of type `Mem_Req` and the second FIFO can only hold items of type `Mem_Rsp`.

`mkFIFOF` is used as a single-element FIFO. It will support sustained simultaneous enqueueing and dequeuing, and is therefore useful as an intermediate buffer between pipeline stages.

7.5.2.1 `mkPipelineFIFOF` and `mkBypassFIFOF`

`mkPipelineFIFOF` and `mkBypassFIFOF` are two FIFO module constructors from the **BSV** library that are used heavily in **Fife** code shown in Chapter 16.

To first approximation, they can be regarded as single-element FIFOs that can be used instead of `mkFIFOF`. They have subtle *performance* differences from `mkFIFOF`, which is why we use them in **Fife**. We will discuss these performance properties in detail in Sec 18.5.

7.5.2.2 `mkSizedFIFOF`: explicitly depth-sized FIFOs

Sometimes we need deeper FIFOs (higher capacity), in order to balance variations in data flow rates. For this, we can use the following constructor:

```
   FIFOF #(RR_to_Retire)  f_RR_to_Retire <- mkSizedFIFOF (8);
```

This instantiates a FIFO whose queue capacity is 8.

NOTE: Module constructor arguments can play different roles. In `mkReg(0)` the argument (0) becomes a dynamic value, the value held in the register after reset. In `mkSizedFIFO(8)` the argument (8) has a static role describing *structure*, *i.e.,* the size of the FIFO.

7.5.2.3 Initial state of FIFOs after reset

All standard **BSV** FIFO modules are initially empty (containing zero items) on reset, *i.e.,* at the start of time for the circuit.

7.5.3 FIFOs are strongly typed

Each **BSV** FIFO instance can only hold values of one particular type.

Further, the FIFO-contents type need not be `Bit(n)`; it can more generally be *any* **BSV** type that has a representation in bits. Thus, the type of values in a FIFO can be an enum, a struct, a nested struct, *etc.*, if we have used a `deriving(Bits)` declaration (or its explicit analog) to ensure that it has a representation in bits.

7.5.4 Semi-FIFO interfaces for each end of a FIFO

FIFOs are often used to connect two separate modules together, for one module to communicate values to the next one. For example, in a CPU, the Fetch stage communicates memory requests to memory. In this situation, one module only interacts with the "enqueue" side, and the other module only interacts with the "'dequeue" side. The following "Semi-FIFO" interfaces interfaces for each "end" of a FIFO queue are useful for this purpose. The enqueue (upstream) side:

```
1 interface FIFOF_I #(t);
2    method Bool notFull();
3    method Action enq (t x);
4 endinterface
```

The dequeue (downstream) side:

```
1 interface FIFOF_O #(t);
2    method Bool notEmpty();
3    method t first();
4    method Action deq();
5 endinterface
```

There is no extra hardware implied here; these are simply limited "views", or abstractions, of an existing FIFO interface.

It is convenient to define a `pop_o()` function to combine the `first` and `deq` methods of a `FIFOF_O`, just like we defined the `pop()` function on `FIFOF`s in Section 7.5.1.1.

```
function ActionValue #(t) pop_O (FIFOF_O #(t) fo);
   actionvalue
      let x = fo.first;
      fo.deq;
      return x;
   endactionvalue
endfunction
```

This is then invoked with the usual syntax for invoking ActionValues:

```
   let y <- pop_O (fo);
```

7.5.5 Interface-transformer functions

The idea of "viewing" the output-side of a `FIFOF` interface as a `FIFOF_O` interface can be expressed in a **BSV** function:

```
function FIFOF_O #(t) to_FIFOF_O (FIFOF #(t) f);
   interface FIFOF_O #(Mem_Req) fo_IMem_req;
      method Bool notEmpty();
         return f.notEmpty;
      endmethod

      method t first();
         return f.first;
      endmethod

      method Action deq();
         f.deq;
      endmethod
   endinterface
endfunction
```

EXERCISE: Please see Ex-07-A-Interface-Transformers in Appendix E, page E-18. □

7.5.6 Connecting FIFOs

We will frequently want to connect the output of one FIFO to the input of another FIFO. For example, in **Fife**, the interface of the Fetch stage includes this semi-FIFO sub-interface to commicate `Fetch_to_Decode` values to the Decode unit:

```
interface Fetch_IFC;
   ...
   interface FIFOF_O #(Fetch_to_Decode)  fo_Fetch_to_Decode;
   ...
endinterface
```

The interface of the Decode stage has this corresponding sub-interface to receive those values:

```
interface Decode_IFC;
   ...
   interface FIFOF_I #(Fetch_to_Decode)  fi_Fetch_to_Decode;
   ...
endinterface
```

The CPU module, at the next level up, instantiates the Fetch and Decode stages. Then, they can be connected with a simple `mkConnection` one-liner:

```
module mkCPU (CPU_IFC);
   ...
   // Instantiate Fetch and Decode stages
   Fetch_IFC   stage_F  <- mkFetch;
   Decode_IFC  stage_D  <- mkDecode;
   ...
   // Connect the Fetch_to_Decode flow
   mkConnection (stage_F.fo_Fetch_to_Decode, stage_D.fi_Fetch_to_Decode);
   ...
endmodule
```

There is actually no magic in this! First, `mkConnection` is just another **BSV** module which happens to have an "`Empty`" interface with no interface methods, so the `mkConnection` is actually shorthand for:

```
   Empty tmp <- mkConnection (stage_F.fo_Fetch_to_Decode, stage_D.fi_Fetch_to_Decode);
```

The shorthand omits the "`Empty tmp <-`" left-hand side).

The module `mkConnection` is such a useful construct that it is provided in the *bsc* library. It can be implemented in **BSV** itself, and is very simple:

```
module mkConnection #(FIFOF_O #(Fetch_to_Decode) f,    // module argument
                      FIFOF_I #(Fetch_to_Decode) d)    // module argument
                     (Empty);                          // module interface
   rule rl_connect;
       let x = f.first;
       f.deq;
       d.enq (x);
   endrule
endmodule
```

`mkConnection` is a module with two arguments `f` and `d` and producing an empty interface. In **BSV**, Verilog and SystemVerilog syntax, a module's arguments are provided in `#(...)` and its interface follows in `(...)`.

The module contains a *rule* (Chapter 13), which is an infinite process. It binds `x` to `f.first`, the head of the `f` queue, and discards it from the queue (`f.deq`). It enqueues the value `x` into `d`. Being an infinite process, it repeats this every time this is possible.

Because of the automatic flow-control in **BSV** FIFOs, this rule will only execute when `f` is non-empty (contains an item, available to dequeue) and `d` is not full (has space, available to enqueue).

NOTE:

Advanced BSV topic: What if we want to connect the two semi-FIFOF interfaces with the arguments in the opposite order, *i.e.,* a `FIFOF_I` interface to a `FIFOF_O` interface? We could write a corresponding `mkConnection_I_to_O` module. What if we want to connect an ARM AXI4 M interface to an ARM AXI4 S interface? We could write a corresponding `mkConnection_AXI4_M_to_S` module.

When there are many different kinds of connection, inventing new module names `mkConnecton_X_to_Y` for each pair of interface types `X` and `Y` becomes tedious.

BSV contains a mechanism called "Typeclasses" and "Typeclass instances" that allows us to reuse the name `mkConnection` for the connection module for every such pair of interface types.

In Programming Language design this issue and solutions are called "overloading".

7.5.7 Connecting separately compiled pipeline stages

When we have a FIFO-like flow between two separately compiled stages A and B, where should the FIFO module go? In the module for stage A? Or in the module for stage B? Or in the parent module of Stages A and B?

A nice way to connect them is illustrated in Figure 7.2.

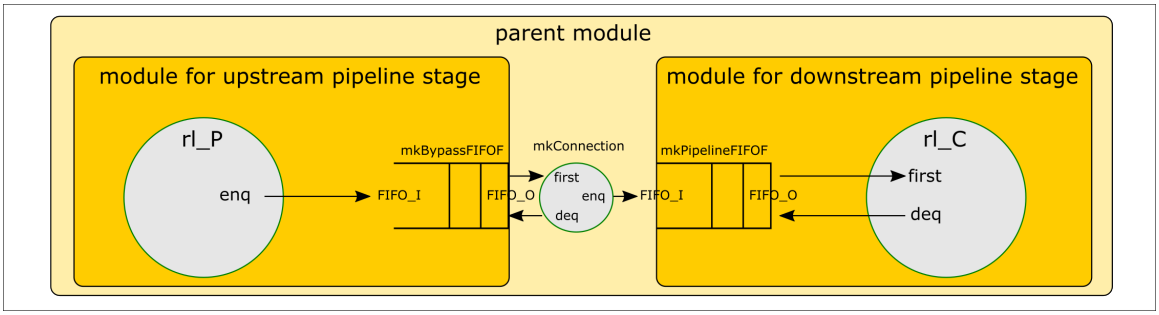


Figure 7.2: How we connect **Fife** stages

In the upstream (producer) stage we instantiate a `mkPipelineFIFOF`. The rules in the stage enqueue outgoing information into the `FIFO_I` side of the FIFO using the `enq` method, and we lift the `FIFO_O` side to the stage-module interface.

Conversely, in the downstream (consumer) stage we instantiate a `mkBypassFIFOF`. The rules in the stage consume incoming information from the `FIFO_O` side of the FIFO using the `first` and `deq` methods, and we lift the `FIFO_I` side to the stage-module interface.

Finally, in the parent module, we use `mkConnection` to connect the exposed `FIFO_O` and `FIFO_I` ends of the FIFOs.

The benefits of this technique are discussed in Section 18.5. Suffice it to say, here, that:

- Despite there being two FIFOs, data can traverse from producer to consumer in 1 tick, as desired.
- The structure allows the producer and consumer to be compiled independently by *bsc*, with no "rule-scheduling" constraints leaking across stage boundaries.

- There are no combinational paths crossing the stage boundary (through the two FIFOs).
- The structure allows us to reason about (and prove) correctness of each stage completely independently of other stages.

All of these are pleasing "modularity" properties of the design.

7.6 Polymorphic and Monomorphic Types

The previous sections showed several *polymorphic* types:

- `Reg #(t)`
- `RegFile #(index_type, content_type)`
- `GPRs_IFC #(reg_width)`
- `FIFOF #(t)`
- `FIFOF_I #(t)`
- `FIFOF_O #(t)`

In each case, there is a *type constructor* (`Reg`, `RegFile`, ..., `FIFOF_O`) applied to some arguments (`t`, `ix_width`, `reg_width`). The general syntax is:

type-constructor `#(`*arg*, ..., *arg*`)`

(When a type constructor has no arguments, we can omit "`#()`".)

When the argument of a type-constructor is an identifiers beginning with a lower-case letter, this indicates that this is a *type variable*, *i.e.,* it is place-holders for some specific type that will be instantiated later/elsewhere.

A polymorphic type (a type containing one or more type-variables) represents all possible types one can obtain by instantiating the type variables with specific, concrete types. A type with no type-variables is also called *monomorphic*.

Note that it is not just library types (`Reg`, `RegFile`, `FIFOF`) that may be polymorphic. In the previous sections, we defined new types `GPRs_IFC #(reg_width)`, `FIFOF_I #(t)` and `FIFOF_O #(t)` that are also polymorphic.

EXERCISE: Please see Ex-07-B-Polymorphic-Types in Appendix E, page E-18. □

7.6.1 Polymorphic Modules and Synthesizability into Verilog

In Section 7.2.3 we mentioned without discussion that the "`(* synthesize *)`" attribute preceding a `module` definition controls whether the *bsc* compiler generates a Verilog module for this **BSV** module or whether it is inlined into its parent module wherever it is instantiated.

Not all **BSV** modules can be compiled one-to-one into Verilog modules. Broadly speaking, polymorphic modules cannot be separately compiled into Verilog modules. The reason is that polymorphism in **BSV** is very powerful and, beyond the expressive power of Verilog.

This does not mean that we cannot use polymorphic modules in a **BSV** design; of course we can! It just means that, at each instance of the module where we have instantiated it and fully specified the types ("monomorphized" the types), the *bsc* compiler at that place has enough information to generate Verilog for that instance.

For example, consider the polymorphic `mkRISCV_GPRs` module Section 8.2. We can directly instantiate this module in a CPU module:

```
module mkCPU;
   ...
   GPRs_IFC #(XLEN) gprs <- mkGPRs;
   ...
endmoduile
```

At this instantiation position, the *bsc* compiler knows the concrete value (`XLEN`) of the type-variable `xlen`, and so can generate Verilog code for this `mkRISCV_GPRs` instance. In the final Verilog code, there will not be any separately identifiable Verilog code for the register file, it would just be part of the `mkCPU` module Verilog.

If we really wanted to generate a separately identifiable Verilog module for the register file, we can write a monomorphic wrapper module:

```
(* synthesize *)
module mkGPRs_V (GPRs_IFC #(XLEN));
   GPRs_IFC #(XLEN) ifc <- mkGPRs;
   return ifc;
endmoduile
```

This module `mkRISCV_GPRs_V` is monomorphic because the type-variable `xlen` has been instantiated to the concrete type `XLEN`, and so the "`(* synthesize *)`" attribute will be honored by the *bsc* compiler to produce a corresponding Verilog module.

We can then replace our earlier instantiation of `mkRISCV_GPRs` in the CPU module with this monomorphic module:

```
module mkCPU;
   ...
   GPRs_IFC #(XLEN) gprs <- mkGPRs_V;
   ...
endmoduile
```

EXERCISE: Please see Ex-07-C-Synthesizable-Modules in Appendix E, page E-19. □

Chapter 8

RISC-V: Modules for GPRs and CSRs

8.1 Introduction

In this chapter we discuss modules for the RISC-V GPRs (General Purpose Registers) and CSRs (Control and Status Registers).

8.2 A register file for GPRs, with special treatment of register 0 (x0)

In RISC-V, register `x0` (index 0) is defined as "always zero". Any value written to `x0` is ignored/discarded, and any read from `x0` always returns 0. So, presumably, we do not need an actual register to hold this value, just some circuitry to ensure that we always get 0 when we try to "read" from `x0`.

EXERCISE: Please see Ex-08-A-GPR-x0 in Appendix E, page E-19. □

In the previous section, we used the module `mkRegFileFull` to instantiate a register file with 32 registers (inferring 32 from the full range of the index type `Bit#(5)`). Instead, we could use an alternate register file module from the BSV library that allows us to provide, as module constructor arguments, the lower and upper indexes of interest. This instantiates exactly 31 registers indexed from 1 to 31, thereby saving XLEN bits of register state in our hardware.

```
RegFile #(Bit #(5), Bit #(XLEN)) gprs <- mkRegFile (1, 31);
```

Regardless of whether we instantiated 31 or 32 registers, RISC-V instructions can (and do) use `x0` as a source or destination register, so we need circuitry to deal with attempts to read/write `x0`. One possible solution is to make a "wrapper" module `mkGPRs` around the library register file module.

EXERCISE: Please see Ex-08-B-GPR-x0-as-Rd in Appendix E, page E-19. □

Although we could have reused the `RegFile #(t1,t2)` interface, we take the opportunity to define a new interface `GPRs_IFC` that has some RISC-V specific method and argument names, for reading the rs1, rs2 values (register source 1 and 2) and writing the rd value (register destination):

src_Common/GPRs.bsv: line 24 ...

```
interface GPRs_IFC #(numeric type xlen);
   method Bit #(xlen) read_rs1 (Bit #(5) rs1);
   method Bit #(xlen) read_rs2 (Bit #(5) rs2);
   method Action      write_rd (Bit #(5) rd, Bit #(xlen) rd_val);
   ...
endinterface
```

Here is the module implementing the interface:

src_Common/GPRs.bsv: line 44 ...

```
module mkGPRs (GPRs_IFC #(xlen));
   RegFile #(Bit #(5), Bit #(xlen)) rf <- mkRegFileFull;

   // ================================================================
   // Initialization
   Reg #(Bit #(6)) rg_init_index <- mkReg (0);

   Bool initialized = (rg_init_index [5] == 1'b1);

   rule rl_init (! initialized);
      rf.upd (truncate (rg_init_index), 0);
      rg_init_index <= rg_init_index + 1;
      if (rg_init_index == 31)
         $display ("GPRs: initialized to 0");
   endrule

   // ================================================================

   method Bit #(xlen) read_rs1 (Bit #(5) rs1) if (initialized);
      return ((rs1 == 0) ? 0 : rf.sub (rs1));
   endmethod

   method Bit #(xlen) read_rs2 (Bit #(5) rs2) if (initialized);
      return ((rs2 == 0) ? 0 : rf.sub (rs2));
   endmethod

   method Action write_rd (Bit #(5) rd, Bit #(xlen) rd_val) if (initialized);
      rf.upd (rd, rd_val);
   endmethod
   ...
endmodule
```

The module instantiates a library register file `rf`. The methods simply invoke the underlying `rf` methods. The read-methods override this by returning 0 when the index is 0.

EXERCISE: Please see Ex-08-C-GPR-Register-Files in Appendix E, page E-19. □

8.2.1 Inlined *vs.* separate module `mkGPRs`

The above interface and module definitions were parameterized with "`xlen`" which is a type-variable (starting with a lower-case letter). The interface and module are thus polymorphic in the width of the data stored in the registers. Thus, this module can be instantiated in RV32 and RV64 designs, instantiating `xlen` to 32 and 64, respectively.

Being polymorphic, it will also be inlined wherever it is instantiated. If you look at the generated Verilog, there will be no sign of any `mkGPRs` module; the module code will have been integrated (inlined) into the parent module's code.

A useful trick is to write a thin, non-polymorphic wrapper for the module; being non-polymorphic, it can be compiled without inlining into a distinct Verilog module:

src_Common/GPRs.bsv: line 87 ...

```
1 (* synthesize *)
2 module mkGPRs_synth (GPRs_IFC #(XLEN));
3    let ifc <- mkGPRs;
4    return ifc;
5 endmodule
```

Here, we have instantiated the module using `XLEN` which is not a type-variable, it is defined specifically as 32 or 64 (see Sec 4.16.1.2). The `(*synthesize*)` attribute can now be respected by the *bsc* compiler and it will produce a Verilog module `mkGPRs_synth`.

8.3 A register file for RISC-V CSRs

The RISC-V CSRs (Control and Status Registers) are not "just another register file". Here are some significant differences:

- There are 32 GPRs, addressed with a 5-bit index. All of them (except one, `x0`) are used in programs. We can thus use a *dense*, packed implementation (`RegFile` from the BSV library).

  CSRs are addressed with a 12-bit index (CSR address). But in our implementation we will use just a handful of CSRs (`mtvec`, `mtepc`, `mcause`, `mtval` and a few more), with non-consecutive addresses, *i.e.,* the addresses are *sparse*; many (most) 12-bit address values are unused.

- Each GPR (except for `x0`) is "memory-like". When a value is written, it remains available for all subsequent reads until the next write. All those reads return the same value—the value most recently written.

  CSR reads can, in general, have side effects. CSR writes, in addition to writing a value, can have other side effects as well. A CSR read may not return the same value as the value most recently written.

For these reasons, we implement each CSR with a separate, ordinary register. Here are the CSRs we need for exception-handling:

src_Common/CSRs.bsv: line 117 ...

```
   Reg #(Bit #(XLEN)) csr_mtvec    <- mkReg (0);
   Reg #(Bit #(XLEN)) csr_mscratch <- mkReg (0);
   Reg #(Bit #(XLEN)) csr_mepc     <- mkReg (0);
   Reg #(Bit #(XLEN)) csr_mcause   <- mkReg (0);
   Reg #(Bit #(XLEN)) csr_mtval    <- mkReg (0);
```

Here are the definitions of standard RISC-V 12-bit addresses for these CSRs:

src_Common/CSR_Bits.bsv: line 31 ...

```
Bit #(12) csr_addr_MTVEC     = 'h305;
Bit #(12) csr_addr_MEPC      = 'h341;
Bit #(12) csr_addr_MCAUSE    = 'h342;
Bit #(12) csr_addr_MTVAL     = 'h343;
```

To write a CSR we use a case statement that selects on the CSR address and writes to the particular CSR.

src_Common/CSRs.bsv: line 153 ...

```
   function ActionValue #(Bool)
            fav_csr_write (Bit #(12) csr_addr, Bit #(XLEN) csr_val);
      actionvalue
         ...
         Bool exception = False;
         case (csr_addr)
            ...
            csr_addr_MTVEC:     csr_mtvec    <= fn_legalize_mtvec (csr_mtvec, csr_val);
            csr_addr_MSCRATCH:  csr_mscratch <= csr_val;
            csr_addr_MEPC:      csr_mepc     <= fn_legalize_mepc (csr_val);
            csr_addr_MCAUSE:    csr_mcause   <= csr_val;
            csr_addr_MTVAL:     csr_mtval    <= csr_val;

            csr_addr_MCYCLE:    if (xlen == 32)
                                   csr_mcycle [1] <= {csr_mcycle [1] [63:32],
                                                      csr_val [31:0]  };
                                else
                                   csr_mcycle [1] <= zeroExtend (csr_val);
            csr_addr_MCYCLEH:   if (xlen == 32)
                                   csr_mcycle [1] <= {csr_val [31:0],
                                                      csr_mcycle [1] [31:0] };
                                else
                                   exception = True;

            csr_addr_MINSTRET:  if (xlen == 32)
                                   csr_minstret <= {csr_minstret [63:32], csr_val [31:0]};
                                else
                                   csr_minstret <= zeroExtend (csr_val);
            csr_addr_MINSTRETH: if (xlen == 32)
                                   csr_minstret <= {csr_val [31:0], csr_minstret [31:0]};
                                else
                                   exception = True;
            // Debugger control
            csr_addr_DCSR:     csr_dcsr     <= truncate (csr_val);
            csr_addr_DPC:      csr_dpc      <= csr_val;

            default:            exception = True;
         endcase
         return exception;
```

```
40       endactionvalue
41    endfunction
```

This function returns boolean True on success, and False on failure. For the moment, failure only means that the argument CSR address was bad, *i.e.,* it referred to some unknown CSR. Failure can also occur if we try to write to a "read-only" CSR (we will see an example later, CSR TIME).

Similarly, to read a CSR we use a case statement that selects on the CSR address and reads the particular CSR.

src_Common/CSRs.bsv: line 213 ...

```
1     function ActionValue #(Tuple2 #(Bool, Bit #(XLEN)))
2              fav_csr_read (Bit #(12) csr_addr);
3        actionvalue
4           ...
5           Bool        exception = False;
6           Bit #(XLEN) y         = ?;
7           case (csr_addr)
8              ...
9              csr_addr_MTVEC:     y = csr_mtvec;
10             csr_addr_MSCRATCH:  y = csr_mscratch;
11             csr_addr_MEPC:      y = csr_mepc;
12             csr_addr_MCAUSE:    y = csr_mcause;
13             csr_addr_MTVAL:     y = csr_mtval;
14
15             csr_addr_CYCLE:     y = truncate (csr_mcycle [1]);
16             csr_addr_CYCLEH:    if (xlen == 32)
17                                    y = csr_mcycle [1] [63:32];
18                                 else
19                                    exception = True;
20
21             csr_addr_MCYCLE:    y = truncate (csr_mcycle [1]);
22             csr_addr_MCYCLEH:   if (xlen == 32)
23                                    y = csr_mcycle [1] [63:32];
24                                 else
25                                    exception = True;
26
27             csr_addr_TIME:      y = truncate (crg_csr_TIME [1]);
28             csr_addr_TIMEH:     if (xlen == 32)
29                                    y = crg_csr_TIME [1] [63:32];
30                                 else
31                                    exception = True;
32
33             csr_addr_MINSTRET:    y = truncate (csr_minstret);
34             csr_addr_MINSTRETH: if (xlen == 32)
35                                    y = csr_minstret [63:32];
36                                 else
37                                    exception = True;
38             csr_addr_INSTRET:   y = truncate (csr_minstret);
39             csr_addr_INSTRETH:  if (xlen == 32)
40                                    y = csr_minstret [63:32];
41                                 else
42                                    exception = True;
43             // Debugger control
44             csr_addr_DCSR:     y = zeroExtend (csr_dcsr);
45             csr_addr_DPC:      y = csr_dpc;
46
```

```
47              default:            exception = True;
48          endcase
49          ...
50          return tuple2 (exception, y);
51       endactionvalue
52    endfunction
```

This function returns a pair of values (a 2-tuple). The first component, like the CSR-write function, is a boolean, True on success and False on failure. The second component is the value read from the CSR.

For the CSRs module we do not export the above read and write functions directly; they are used internally inside the module. The inteface declaration looks like this:

src_Common/CSRs.bsv: line 57 ...

```
1  interface CSRs_IFC;
2     method Action init (Initial_Params initial_params);
3
4     // CSRRXX instruction execution
5     // Returns (True, ?) if exception else (False, rd_val)
6     method ActionValue #(Tuple2 #(Bool, Bit #(XLEN)))
7            mav_csrrxx (Bit #(32) instr, Bit #(XLEN) rs1_val);
8
9     // Trap actions
10    // Returns PC from MTVEC for trap handler
11    method ActionValue #(Bit #(XLEN))
12           mav_exception (Bit #(XLEN) epc,
13                          Bool        is_interrupt,
14                          Bit #(4)    cause,
15                          Bit #(XLEN) tval);
16
17    // xRET actions
18    // Returns PC from MEPC
19    method ActionValue #(Bit #(XLEN)) mav_xRET ();
20
21    // INSTRET access
22    method Action ma_incr_instret;
23    ...
24 endinterface
```

We will discuss these interface methods in more detail when we discuss trap-handling in Chapter 10. Suffice it to say, for now, that:

- the `mav_csrrx` method directly implements the CSRRxx instructions;
- the `mav_exception` method directly implements the CSR reads and writes needed when taking a trap, and
- the `read_epc` method directly reads the MEPC CSR as needed by the MRET instruction.

The former was described in Section 2.7.2, and the latter two were described in Section 2.7.

EXERCISE: Please see Ex-08-D-CSRs in Appendix E, page E-20. □

Chapter 9

BSV: Finite State Machines (FSMs) using `StmtFSM`

9.1 Introduction

So far, we have only been discussing pure combinational functions, for which there is no concept of time. Combinational functions are just pure mathematical functions, "instantaneously" transforming input values to output values. In actual combinational circuits, there is of course a finite delay through wires and gates, as dictated by physics, but in the digital abstraction of time we think of this as instantaneous.

However, a CPU, as shown in Figure 9.1 represents a *processs*, a behavior that evolves over

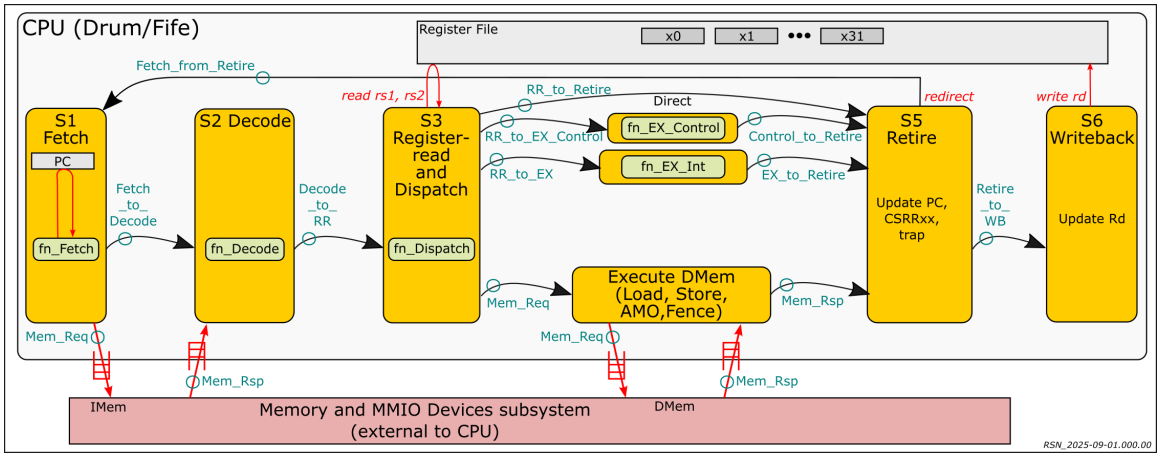


Figure 9.1: Simple interpretation of RISC-V instructions (same as Fig. 5.1)

time. For example the **Drum** CPU executes one full instruction after another, repeating forever the flow along the black arrows in the diagram. For each instruction, first it performs a Fetch operation, which sends a request to memory. Some time later, the memory sends back a response, which is then processed by the Decode stage, Register-Read-and-Dispatch stage and then one of the Execute stages. The Execute DMem stage sends a request to memory. Some time later, the memory sends back a response, which is processed in the

Retire stage. Finally, the process loops back to the Fetch stage, and repeats for the next instruction.

The simplest temporal process in hardware systems is the *FSM* (Finite State Machine). A classical notation for FSMs is the so-called "bubble-and-arrow" diagram. Figure 9.2 shows a greatly over-simplified bubble-and-arrow diagram for an FSM controlling an elevator (a lift). Each bubble represents a distinct *state* of the system, and the arrows connecting

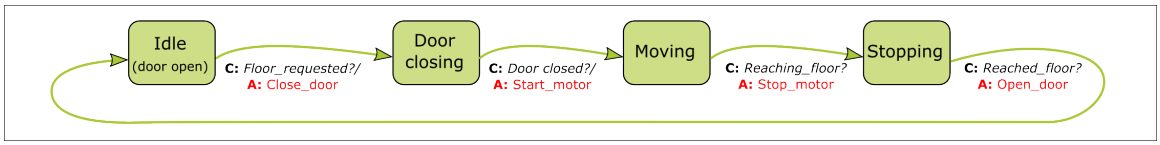


Figure 9.2: A (greatly over-simplified) FSM for controlling an elevator.

bubbles represent *transitions* between states. Transitions are typically enabled by some boolean condition on the current state, and/or availability of some particular input from the environment ("**C:**" in the diagram). A transition moves the system to another state, and may perform some action, including producing some output to the environment ("**A:**" in the diagram). In bubble-and-arrow diagrams, each arrow is often labeled with the condition that enables the transition, and by any actions performed by the transition.

Figure 9.1 can be interpreted as a bubble-and-arrow FSM diagram: each yellow rectangle (bubble) is a state, and the process transitions from state to state, thereby executing RISC-V instructions. This is exactly what the **Drum** CPU does. In Figure 9.1 the arrows are labeled by the information (a struct) that is produced in one state and consumed by another.

In this chapter we describe some special constructs for FSMs in **BSV**. In the next chapter we will discuss using these FSM constructs to code the **Drum** CPU.

NOTE: In the literature one may read about FSMs where, from some state, we could have a choice of transitions to different destination states. These are called *non-deterministic* FSMs. In our designs we are only concerned with *deterministic* FSMs where the conditions on the arrows emerging from a state are mutually exclusive, so that they always identify a unique possible next state.

9.1.1 Sequential FSMs, Concurrent FSMs, and Digital Hardware

Classical FSMs in the literature are *sequential* FSMs—every transition is from the current state to a unique, particular next state.[1]

Any digital system (including an entire computer) can theoretically be viewed as a single (possibly giant!) FSM. The current-state is the current collective state of every register bit and every memory bit in the system. State transitions depend on the current state and any external inputs. Although theoretically correct, this is an impractical, non-scalable, and not very useful way of viewing complex digital systems.

Most non-trivial digital hardware systems are better viewed as composed of multiple *concurrent, communicating FSMs*, *i.e.,* multiple classical FSMs running concurrently and independently and communicating with each other (an output from one FSM may be an input to another FSM). Figure 9.3 sketches the idea. Different **BSV** module instances are separate

[1] Even in non-deterministic FSMs, though there may be several possible next-states, exactly one next-state is non-deterministically chosen.

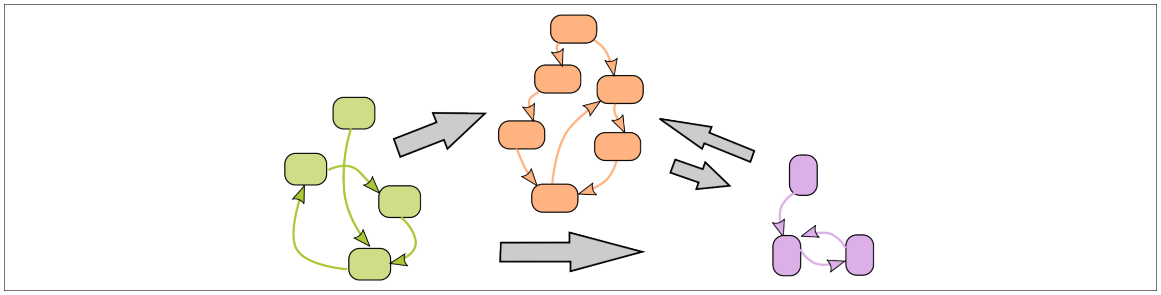


Figure 9.3: Concurrent, independent FSMs.
The grey arrows indicate communication *via* registers, FIFOs, ...

FSMs, each running their own process(es). These separate FSMs may communicate with each other *via* shared state (registers, FIFOs, register files, *etc.*). This is a more *modular* and *scalable* way to think about complex digital systems.

Drum CPU execution is a sequential FSM, but the overall system in which **Drum** is embedded can be viewed as a pair of concurrent FSMs: **Drum** and the Memory System (see Figure 12.3). Each has its own internal FSM process and behavior, and they communicate memory requests and responses back and forth.

For **Fife**, we will interpret *each* yellow box in Figure 9.1 as its own FSM. All these FSMs run concurrently (and concurrently with the memory system), and communicate various struct values (labels in Figure 9.1) through FIFOs between the FSMs.

9.1.2 Communicating between FSM states and between concurrent FSMs

In a sequential FSM, communication between states is easily accomplished using ordinary registers—one transition writes into a register, and another transition reads from the register. Because the FSM is sequential, transitions occur in some well-defined order, and so there is a well-defined order in which the register is written and read.

Communication between concurrent FSMs, on the other hand is more easily accomplished using FIFOs. Concurrent FSMs may run at different rates and so there may not be any predictable temporal ordering between when FSM A produces a datum and when another FSM B tries to consume the datum. Instead, we let FSM A *enqueue* the datum into a FIFO, and let FSM B *dequeue* the datum from the FIFO. If FSM B tried to consume the datum before it has been produced, it will simply "block", or stall, trying to read an empty FIFO, until FSM A produces the datum into the FIFO.

FIFOs between concurrent FSMs also enables *pipelining* or *streaming*. FSM A can repeatedly enqueue data into a FIFO even if FSM B is not yet ready to consume it. FSM B can repeatedly consume data already produced by FSM A. If the FIFO becomes full, FSM A will simply block when it tries to enqueue the next datum into it. If the FIFO becomes empty, FSM B will simply block until data is available. The FIFO is said to perform automatic "flow control" between producer and consumer FSMs. In this way, communication via FIFOs between concurrent FSMs is more "elastic", or "asynchronous".

In **Drum**, where Figure 9.1 represents the states (yellow bubbles) and transitions (black arrows) of a sequential FSM, we will use registers to communicate between states. We use FIFOs to communicate to/from the concurrent external FSM, the memory system.

In **Fife**, where the yellow bubbles of Figure 9.1 represent concurrent FSMs, we will use FIFOs to communicate between the FSMs.

9.2 `StmtFSM`: structured FSM processes in BSV

The fundamental behavioral/temporal primitive in **BSV**, to specify processes, is the *rule* but we will postpone a detailed discussion of rules until Chapters 13 and 18. Rules can express arbitrary process flows, both structured and unstructured. **BSV**'s `StmtFSM` is a higher-level notation that captures the idea of certain *structured* processes (rule execution flows). `StmtFSM` is ultimately implemented (by the *bsc* compiler) with rules, so it does not add any fundamental semantic power to the language—anything we write with `StmtFSM` can also be written directly with rules. Section 13.8 discusses when to use `StmtFSM` and when to use rules.

`StmtFSM` is a sub-language within **BSV** for specifying structured FSMs. The sub-language which we use in **Drum** has the following grammar (Section 9.10 describes more available features in `StmtFSM`):

```
Stmt  ::=  an Action expression
       |   a sequence (seq block) of Stmts
       |   an if-then-else of two Stmts        (conditionals)
       |   a while-loop around a Stmt
```

The basic primitive is an expression of type `Action` (which includes `action-endaction` blocks that may contain sub-actions). This specifies an FSM which just performs that action once (the `Action` expression becomes the body of a single **BSV** rule). Then, we *compose* larger FSMs from smaller FSMs by nesting them in sequences, conditionals and loops. Figure 9.4 illustrates this compositionality. Every `Stmt` has a single start state and

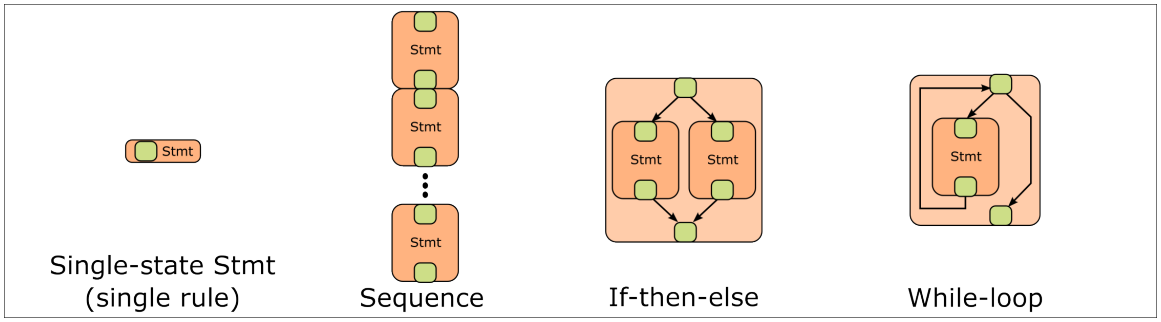


Figure 9.4: Structured FSMs, built compositionally.

single end state. Note that each composition scheme itself results in a `Stmt` with a single start and end state. This allows them to be nested recursively.

We will use these `StmtFSM` constructs to code **Drum**, which is a simple sequential process. `StmtFSM` will not be adequate to code **Fife**, which is a collection of concurrent FSMs; for that, we will use rules explicitly.

NOTE: We have already encountered examples of sequential `StmtFSM`s, with just a sequential block, in the simple testbench in Section 4.12 and in many of the testbenches in the exercises so far.

9.3 Actions and the `Action` type

The fundamental building-block for `StmtFSM` is the "action", which is a statement/expression of type `Action`. Some common examples:

```
1   rg_pc <= rg_pc + 4;            // Assignment to a register
2   f_Fetch_to_Decode.deq;         // Dequeue a fifo
3   f_Decode_to_RR.enq (v);        // Enqueue into a fifo
4   $display ("Hello, World!");    // Print something (in simulation only)
```

We discussed the `Action` (and `ActionValue`) types in Section 4.11. We used them in the return type of the `fn_Fetch` function in Section 6.2. To recap, an expression with `Action` or `ActionValue` type is one that potentially has a side effect, as in each of the above example statements.

As discussed in Section 7.3.4 the first assignment statement is syntactic shorthand for:

```
1   rg_pc._write (rg_pc._read + 4)
```

i.e., it is an invocation of the register `_write` method which, as described in Section 7.3.1 has type `Action`. Similarly, as described in Section 7.5.1, fifo `enq` and `deq` methods have return-type `Action`, so the statements `f_D_to_RR.enq (v)` and `f_D_to_RR.enq (v)` have type `Action`.

`$display()` is a built-in construct in **BSV** that also has type `Action`.

9.3.1 `Action` blocks: composing actions into larger actions

The `Action` type is recursive: it is either a primitive action (like those described above), or it is a collection of things of type `Action`, collected using an `action` block (bracketed by the **BSV** keywords `action` and `endaction`). For example the above primitive actions can be collected into a single entity which itself has type `Action`:

```
1   action
2      rg_pc <= rg_pc + 4;          // Assignment to a register
3      f_F_to_D.deq;                // Dequeue a fifo
4      f_D_to_RR.enq (v);           // Enqueue into a fifo
5      $display ("Hello, World!");  // Print something (in simulation only)
6   endaction
```

Although the actions in an `action` block must be written in some textual order, `action` *blocks do not involve any temporal sequencing.* There is no temporal ordering of the actions in an `action` block. All the actions in an `action` block (either directly in the block or, recursively in a sub-block) occur "instantly" and "simultaneously". Referring to Section 7.2.2.1, in hardware the ENABLE signals of all actions in an `action` block are asserted simultaneously. In the example above, lines 2-5 could have been written in any order with no change in meaning/behavior.

9.3.2 Binding names in `Action` blocks

It is often convenient to give a meaningful name to a sub-expression in an `Action` block. For example:

```
1   action
2      Bit #(XLEN) next_pc = rg_pc + 4;
3      rg_pc <= next_pc;
4      $display ("Next PC is %08h", next_pc);
5   endaction
```

Here, we bind the identifier `next_pc` in line 2, and then use it in lines 3 and 4. We can often replace the type in the binding with the keyword `let`, if the type is obvious from the context (the *bsc* compiler will figure it out):

```
1   action
2      let next_pc = rg_pc + 4;
3      rg_pc <= next_pc;
4      $display ("Next PC is %08h", next_pc);
5   endaction
```

The *scope* of the identifier, *i.e.,* the region of program text where it is available for use, are the remaining statements in the `Action` block (including inside any syntactically nested statements).

A binding, per se, *is not an action!* It is just a convenience, giving a name to the value of the right-hand side expression.

Bindings (whether with an explicit type or with `let`) impose some syntactic ordering on statements in the block: a binding of an identifier must precede any use of that identifier. In the previous two examples, line 2 (the binding) must precede lines 3 and 4 (the actions), but lines 3 and 4 could be written in the opposite order. Note, this is not a *temporal* sequencing; all actions in the block still occur simultaneously.

9.4 `StmtFSM`: sequences of actions

Our first construct that has temporal behavior is the `seq-endseq` block. Each item in the block is an entity of type `Action` or `Stmt`, and they are performed sequentially, one after another.

```
seq
  ... an Action or Stmt ... ;
  ...
  ... an Action or Stmt ... ;
endseq
```

The entire `seq` block itself has type `Stmt`, and so it can be nested inside other `StmtFSM` constructs.

The testbench in Section 4.12 contains an example of a `seq` block.

9.5 StmtFSM: conditionals (if-then-else)

Conditional process execution can be expressed with traditional if-then-else notation:

```
if
  ... an Action or Stmt ...
else
  ... an Action or Stmt ...
```

The entire if-then-else construct itself has type `Stmt`, and so it can be nested inside other `StmtFSM` constructs.

NOTE:

In Section 4.14 we described ordinary **BSV** if-then-else expressions, which often represent hardware multiplexers, where both arms are "evaluated" (the hardware exists for both sides) and the multiplexer merely selects one of the two outputs.

`StmtFSM` uses the same notation, but here it represents a *process*, and only one of the two arms is executed (like if-then-else in most software programming languages).

There is no ambiguity in these two uses of if-then-else notation—the context always clearly distinguishes what we mean, because there is no overlap between ordinary expressions and `StmtFSM` constructions.

9.6 StmtFSM: while-loops

Repetitive processes can be expressed with traditional while-loop notation:

```
1  while (... Bool expression ...)
2     ... expression of type Stmt ...
```

The entire while-loop construct itself has type `Stmt`, and so it can be nested inside other `StmtFSM` constructs.

9.7 StmtFSM: pausing until some condition holds

An action in a `StmtFSM` can be the `await(b)` action, which simply waits until the boolean expression in its argument evaluates to true:

```
1  await (... Bool expression ...);
```

Of course, because it is paused, the `Stmt` containing an `await` cannot itself cause the value to change since it cannot change any state that would affect the expression. The state-change thus has to be effected by some other part of the **BSV** design (a concurrent FSM, or a rule), not this particular `Stmt`.

9.8 `StmtFSM`: `mkAutoFSM`: a simple FSM module constructor

Creating an FSM using `StmtFSM` in **BSV** is a two-step process:

- Define the desired FSM behavior as an entity of type `Stmt`. Think of this as a *specification* of desired behavior.
- Instantiate a module that takes this specification and implements the behavior.

There are several `Stmt` ⟶ module constructors available in the **BSV** library. In this book we use only one of them, `mkAutoFSM`:

```
1   mkAutoFSM (... argument expression of type Stmt ...);
```

This creates an FSM with the behavior specified by the `Stmt` argument. The FSM starts running immediately as we come out of reset, starting at the first statement, and terminates the entire simulation when we fall through the last statement. Of course, it may never terminate if it contains an infinite `while` loop.

Note: "terminating" the simulation means it executes a `$finish()` action which stops the complete simulation. In hardware (where there is no concept of `$finish()`) the FSM simply goes idle.[2] There may be other FSMs and rules in the design that continue running.

9.9 `StmtFSM` in testbenches

`StmtFSM` is often used in testbenches, because we want to sequence the delivery of stimulus to the Design-Under-Test (DUT) and sequence the collection and display/checking/analysis of the resulting outputs; `StmtFSM` is a quick and simple way to write such sequences. We showed an example of a small testbench using `StmtFSM` in Section 4.12. When the DUT is pipelined, we use two concurrent FSMs, one to deliver a sequence of stimulus inputs while the other one concurrently processes the sequence of output results.

`StmtFSM` can also be used in designs; indeed, that is what we are going to describe in Chapter 10 for the **Drum** CPU. Section 13.8 discusses when to use `StmtFSM` and when to use rules.

9.10 `StmtFSM`: many more features

In this chapter we have covered all the features of `Stmt` that we need for coding **Drum**. There are many additional constructors for `Stmt` that we skip here, such as for-loops and repeat-loops. Perhaps the most interesting are fork-join concurrent blocks (`par-endpar`), which specify nested concurrent FSMs.

The type `Stmt` is a first-class type in **BSV**. You can write functions that have arguments and results of type `Stmt`. For example, you might write a function that takes a numeric argument and returns a `Stmt`; this function can be invoked more than once to produce multiple `Stmt`s that differ because of the parameter. Each of these `Stmt`s can be instantiated into its own

[2] Technically, when none of the rules implementing the FSM can fire any more.

FSM. This further emphasizes the view that `Stmt` is a specification of an FSM behavior that is then instantiated in a module to produce that behavior.

The `StmtFSM` package also defines many FSM module constructors (alternatives to `mkAutoFSM`), including modules with explicit start, stop and restart methods, modules that can be paused based on a boolean argument, and more.

We refer the reader to the FSM chapter in the *bsc Libraries Reference Guide* [26].

9.11 A historical note about "structured programming"

In software program execution, the fundamental temporal "flow" primitive (in assembly language, for example) is from one instruction to the next, or a BRANCH or JUMP that can go to any instruction. With these, we can create arbitrary control flows.

However, in most programming languages, we work at a much higher-level of abstraction. Instead of using BRANCH and JUMP willy-nilly, we use linguistic structures as if-then-else, case statements while loops, repeat loops, for loops, function calls and returns, *etc.*. Further, these structures *compose* nicely, *i.e.,* they can be nested cleanly inside each other.

Ultimately all these constructs are implemented (by a compiler) in terms of the primitives, instruction sequences/BRANCH/JUMP, and in that sense they do not add any fundamental new semantic power to programming. However, these structured constructs are useful for humans: for readability, for guiding our thinking, and for reasoning about the correctness of programs.

Early programmers, *e.g.,* using the first versions of the Fortran programming language, worked directly with statement sequences (the analog of instruction sequences) and "GOTO" statements (the analog of BRANCH/JUMP). Because GOTO could go to any statement in the program, programs often became a mess of control transfers without any structure; such programs are sometimes called "spaghetti code".

In the 1960s, Edsgar Dijkstra, a Dutch computer scientist (later to win the ACM Turing Award), recognized the importance of clear *structure* in composing programs. In 1968 he wrote a famous letter to the editor of the Communications of the ACM journal (Association for Computing Machinery) titled "Go To Statement Considered Harmful", a searing indictment of unstructured code.

The idea of constructing programs cleanly with nestable (composable) structures (if-then-else, while-loops *etc.*), so called "structured programming", grew out of that thinking; it is something we take for granted in today's programming languages.

Chapter 10

RISC-V: the Drum CPU (unpipelined; FSM-sequenced)

10.1 Introduction

In this chapter we code **Drum**'s behavior, illustrated in Figure 10.1, using BSV's `StmtFSM` construct to code the FSM.

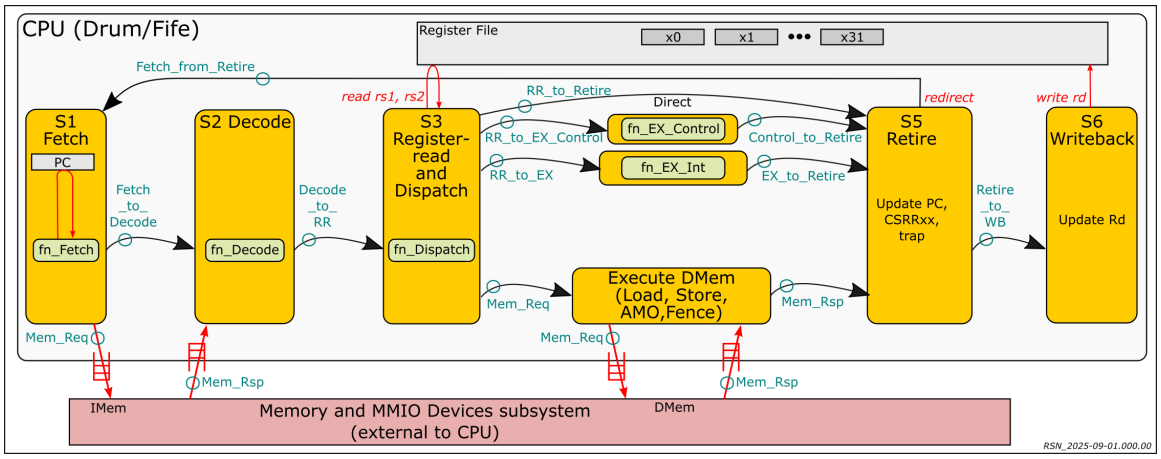


Figure 10.1: Simple interpretation of RISC-V instructions (same as Fig. 5.1)

10.2 The Drum CPU module interface

The **Drum** CPU interface is shown below.

src_Common/CPU_IFC.bsv: line 28 ...

```
1  interface CPU_IFC;
2     method Action init (Initial_Params initial_params);
3
4     // IMem
5     interface FIFOF_O #(Mem_Req) fo_IMem_req;
6     interface FIFOF_I #(Mem_Rsp) fi_IMem_rsp;
```

```
   ...
   // DMem, non-speculative
   interface FIFOF_O #(Mem_Req) fo_DMem_req;
   interface FIFOF_I #(Mem_Rsp) fi_DMem_rsp;
   ...
endinterface
```

The interface is simple:

- The `init` method carries an `Initial_Params` struct containing any initial values needed by the CPU. A typical field is the initial value of the PC, since different software systems make different assumptions about the "starting address" for code. In many RV32I example codes, the starting address is `'h_8000_0000`.
- A `FIFOF_O` interface to carry memory requests for instructions (out-bound from the CPU to the memory);
- A `FIFOF_I` interface to carry corresponding memory responses containing instructions (in-bound from memory to the CPU);
- A `FIFO_O` interface to carry memory requests from load/store instructions (out-bound from the CPU to the memory);
- A `FIFOF_I` interface to carry corresponding load/store memory responses (in-bound from memory to the CPU).

(You may wish to re-read Section 5.4.1 for the discussion on Harvard architectures, which have separate memory-access channels for instructions (Fetch, IMem) and for data (LOAD/STORE, DMem)).

Later we will see that **Fife** shares this interface, so that the **Fife** and **Drum** CPUs are easily interchangeable in any system or testbench.

10.3 The Drum CPU module

The STATE and INTERFACE sections of the **Drum** CPU module are shown below (we will discuss the elided BEHAVIOR section shortly).

src_Drum/CPU.bsv: line 87 ...

```
(* synthesize *)
module mkCPU (CPU_IFC);
   // ================================================================
   // STATE

   // Don't run until the PC (and other things) are initialized
   Reg #(CPU_RunState) rg_runstate <- mkReg (CPU_HALTED);
   ...
   // The Program Counter
   Reg #(Bit #(XLEN)) rg_pc <- mkReg (0);

   // General-Purpose Registers (GPRs)
   GPRs_IFC #(XLEN) gprs <- mkGPRs_synth;

   // Control-and-Status Registers (CSRs)
   CSRs_IFC csrs <- mkCSRs;

   // Inter-step registers
```

```
   Reg #(Fetch_to_Decode)      rg_Fetch_to_Decode      <- mkRegU;
   Reg #(Decode_to_RR)         rg_Decode_to_RR         <- mkRegU;
   Reg #(Result_Dispatch)      rg_Dispatch             <- mkRegU;
   Reg #(EX_Control_to_Retire) rg_EX_Control_to_Retire <- mkRegU;
   Reg #(EX_to_Retire)         rg_EX_to_Retire         <- mkRegU;

   // Paths to and from memory
   FIFOF #(Mem_Req) f_IMem_req  <- mkFIFOF;
   FIFOF #(Mem_Rsp) f_IMem_rsp  <- mkFIFOF;

   FIFOF #(Mem_Req) f_DMem_req  <- mkFIFOF;
   FIFOF #(Mem_Rsp) f_DMem_rsp  <- mkFIFOF;

   // Regs to set up exception handling
   Reg #(Bool)        rg_exception <- mkReg (False);
   Reg #(Bit #(XLEN)) rg_epc       <- mkRegU;
   Reg #(Bit #(4))    rg_cause     <- mkRegU;
   Reg #(Bit #(XLEN)) rg_tval      <- mkRegU;
   ...

   Bool is_running = (rg_runstate != CPU_HALTED);
   Bool is_halted  = (rg_runstate == CPU_HALTED);
```

(... BEHAVIOR elided, to be discussed shortly ...)

src_Drum/CPU.bsv: line 516 ...

```
   method Action init (Initial_Params initial_params);
      ...
      rg_pc       <= initial_params.pc_reset_value;
      rg_runstate <= CPU_RUNNING;
      csrs.init (initial_params);

      $display ("================================================================");
      $display ("%s: starting execution at PC %0h",
                cpu_name, initial_params.pc_reset_value);
      ...
   endmethod

   // IMem
   interface fo_IMem_req = to_FIFOF_O (f_IMem_req);
   interface fi_IMem_rsp = to_FIFOF_I (f_IMem_rsp);
   ...
   // DMem, non-speculative
   interface fo_DMem_req = to_FIFOF_O (f_DMem_req);
   interface fi_DMem_rsp = to_FIFOF_I (f_DMem_rsp);

   // Set TIME
   method Action set_TIME (Bit #(64) t) = csrs.set_TIME (t);
```

The STATE section first instantiates a register `rg_runstate`, initially `CPU_HALTED`, that will be set to `CPU_RUNNING` by the `init` method that initializes the PC to a specific initial value. The FSM will wait for this before starting the first instruction-fetch.

Then, `mkCPU` instantiates a register for the PC, and then the GPRs module (desribed in Section 8.2) and the CSRs module (described in Section 8.3). Then, it instantiates a set of registers to hold values between temporal FSM steps (with struct types shown in Figure 10.1). For example, the Fetch stage will write a value into `rg_Fetch_to_Decode` which

will be read later by the Decode stage. Then, it instantiates four FIFOFs for IMem requests (outgoing) and responses (incoming), and for DMem requests (outgoing) and responses (incoming). As mentioned before, we do not make any assumption about the *latency* of memory requests, *i.e.,* how long it takes the external memory subsystem to consume a request from one of the request FIFOs and enqueue a response into the corresponding response FIFO. Finally, it instantiates the four registers needed for trap-handling, described in Section 2.7.

In the display above we have elided the BEHAVIOR section of the module, which we will describe shortly.

In the INTERFACE section, the `init` method initializes the PC and sets `rg_runstate` to `CPU_RUNNING`, releasing the BEHAVIOR section to start executing. We use the interface transformers discussed in Section 7.5.5 that produce Semi-FIFO "views" of FIFOs to lift the FIFO interfaces into the module interface.

10.4 Help-functions for the Drum CPU module behavior

Before we look at the FSM implementing **Drum** behavior, we first have some `Action` functions that encapsulate some common actions performed in several states in the FSM.

NOTE: In BSV, function definitions do not have to be at the top-level of a file, in fact they can be defined at any nested level. Here, we define these inside a module.

The following function writes an rd-value into the the GPRs, in those cases where the instruction has an rd (`fn_Decode`, described in Section 6.3, computed `has_rd` for each kind of instruction):

src_Drum/CPU.bsv: line 163 ...

```
1   function Action fa_update_rd (RR_to_Retire x1,
2                                 Bit #(XLEN) rd_val);
3      action
4         if (x1.has_rd) begin
5            let rd = instr_rd (x1.instr);
6            gprs.write_rd (rd, rd_val);
7            ...
8         end
9      endaction
10  endfunction
```

The following function is the last action during an instruction's execution, updating the PC:

src_Drum/CPU.bsv: line 177 ...

```
1   function Action fa_redirect_Fetch (Bit #(XLEN) next_pc);
2      action
3         rg_pc   <= next_pc;
4         ...
5      endaction
6   endfunction
```

The following function saves values into registers `rg_epc`, `rg_cause` and `rg_tval`, when a trap is detected. These will later be written into the corresponding CSR registers during trap-handling.

src_Drum/CPU.bsv: line 185 ...

```
function Action fa_setup_exception (Bit #(XLEN) epc,
                                    Bit #(4) cause,
                                    Bit #(XLEN) tval);
   action
      rg_exception <= True;
      rg_epc       <= epc;
      rg_cause     <= cause;
      rg_tval      <= tval;
   endaction
endfunction
```

10.5 The main behavior actions in the Drum CPU module

The following sub-sections show the main actions performed by **Drum**, corresponding to the components in Figure 10.1.

NOTE: In BSV, the type `Action` is a first-class type. This means we can write "action expressions" and bind the result (of type `Action`) to a name, and then refer to that action later by that name. We use this capability below first to define the **Drum** FSM actions; then we will embed these actions in the **Drum** FSM in Section 10.6. We also reuse exactly the same actions in the alternate specification of **Drum**'s behavior using Rules instead of `StmtFSM` in Chapter 14.

10.5.1 FSM action for Fetch

src_Drum/CPU.bsv: line 202 ...

```
Action a_Fetch =
action
   await (rg_runstate != CPU_HALTED);
   ...
   let y <- fn_Fetch (rg_pc,
                      ...
   rg_Fetch_to_Decode <= y.to_D;
   f_IMem_req.enq (y.mem_req);
   ...
endaction;
```

This action applies `fn_Fetch()` (Section 6.2) to the PC, stores the `to_D` part of the result in register `rg_Fetch_to_Decode`, and sends the `mem_req` part of the result to the Instruction Memory by enqueueing it on the outgoing `f_IMem_req` FIFO. The other end (dequeue end) of the FIFO is in the module interface. The outer environment of this module will dequeue it and forward it to memory.

10.5.1.1 BSV: `Action` *vs.* zero-argument `Action` function

We can also write a zero-argument function similar to `a_Fetch`:

```
function Action fa_Fetch ();
   action
      await (rg_runstate != CPU_HALTED);
      let y <- fn_Fetch (rg_pc,
```

```
                        ...
      rg_Fetch_to_Decode <= y.to_D;
      f_IMem_req.enq (y.mem_req);
      ...
   endaction;
endfunction
```

The body of the function is the same as the right-hand side of the `a_Fetch` binding. When using these, we'd say:

`a_Fetch;` or `fa_Fetch ();`

respectively. In both cases the expression is of type `Action`. In the latter case, we have to *apply* the function to zero arguments to get an `Action` result.

This is another example of the general principle in **BSV**: "all types are first-class". In the same way that we can declare variables, function arguments and results of scalar types, like `Bit#(32)`, we can declare variables, function arguments and results of type `Action`. Indeed, the same is true for function types, modules types, rule types, interface types, ... any type. It allows us to use many powerful idioms, such as parameterizing modules by modules, writing functions to generate rules, and so on, which are not possible in Verilog/SystemVerilog and VHDL.

This is a key capability in **BSV** that is inspired by modern functional programming languages like Haskell and OCaml, where we also use the term "higher-order functions".

10.5.2 FSM action for Decode

src_Drum/CPU.bsv: line 229 ...

```
1  Action a_Decode =
2  action
3     let mem_rsp <- pop_o (to_FIFOF_O (f_IMem_rsp));
4     let y       <- fn_Decode (rg_Fetch_to_Decode, mem_rsp, rg_flog);
5     rg_Decode_to_RR <= y;
6     ...
7  endaction;
```

We pop the response from IMem (bind `mem_rsp` to the value at head of the FIFO, and also remove it from the FIFO), then apply `fn_Decode()` (Section 6.3) to the `Fetch_to_Decode` value from the Fetch stage and the IMem response, and store the function result in register `rg_Decode_to_RR`.

10.5.3 FSM action for Dispatch

The next FSM action is the Register-Read and Dispatch stage:

src_Drum/CPU.bsv: line 238 ...

```
1  Action a_Register_Read_and_Dispatch =
2  action
3     // Read GPRs
4     // Ok that read_rs1 and read_rs2 may return junk values
5     //         since not all instrs have rs1/rs2.
```

```
6       let x       = rg_Decode_to_RR;
7       let rs1_val = gprs.read_rs1 (instr_rs1 (x.instr));
8       let rs2_val = gprs.read_rs2 (instr_rs2 (x.instr));
9
10      Result_Dispatch y <- fn_Dispatch (x, rs1_val, rs2_val, rg_flog);
11      rg_Dispatch       <= y;
12      ...
13  endaction;
```

Using the information in `rg_Decode_to_RR` created in the Decode stage, we read the two registers `rs1` and `rs2`. We apply the function `fn_Dispatch` (Section 6.4) and store the result in register `rg_Dispatch`.

Note that we blindly read both registers `rs1` and `rs2`, even though many instructions do not have one or both of these fields. In those situations we'll be reading bogus/irrelevant values, but it does not matter—in the Execute stage we will only use these values in instructions that need them. Recall that while in a software programming language this may seem unnecessary or wasted work, in this hardware context nothing is wasted—the hardware for reading both registers exists; there is nothing lost in using it.

10.5.4 FSM actions for Execute and Retire

After the Dispatch FSM stage, we move on to the Exectute and Retire FSM stages. Figure 10.2 shows the details of the four "flows" that may follow. Each of the flows can have

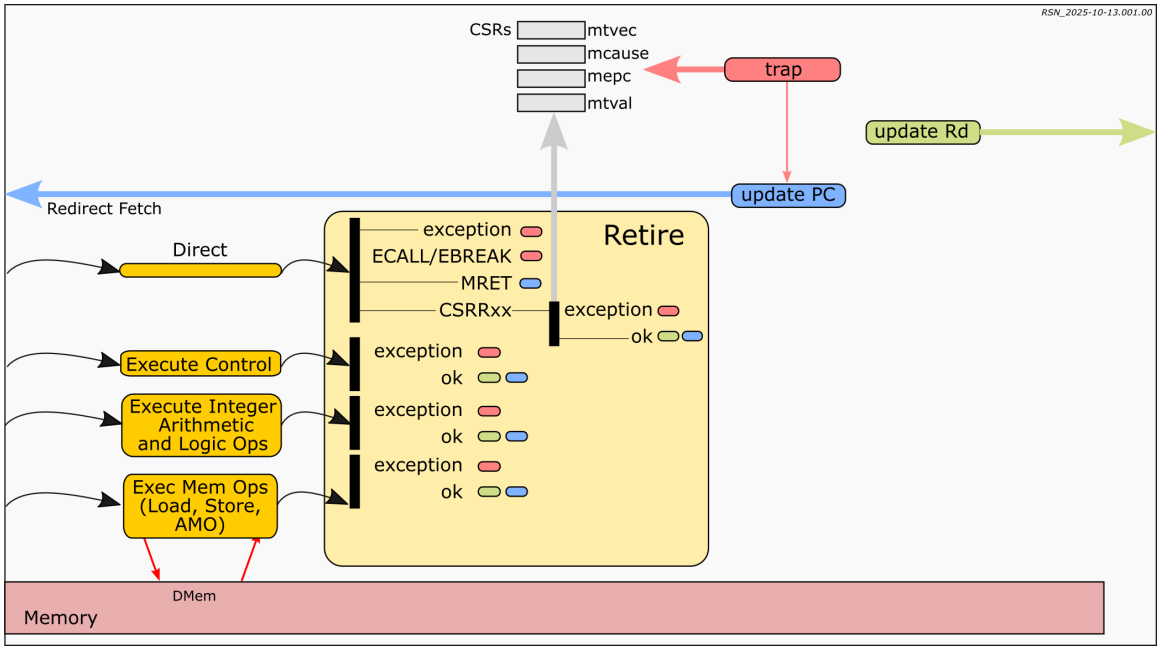


Figure 10.2: Execute and Retire actions in **Drum**

an exception or a normal (OK) result. The Direct flow may need to execute a SYSTEM instruction (ECALL, EBREAK, MRET and CSRRxx). Executing a CSRRxx instruction

may also have an exception or normal result. For each flow, the colored ovals show the actions that may be performed; these correspond exactly to the three help-functions discussed earlier in Section 10.4.

We next elaborate each of these four flows.

10.5.4.1 FSM actions in Direct flow of Execute and Retire

src_Drum/CPU.bsv: line 268 ...

```
Action a_Retire_direct =
action
   let x_direct = rg_Dispatch.to_Retire;
   if (x_direct.exception) begin
      fa_setup_exception (x_direct.pc,        // epc
                          x_direct.cause,     // cause
                          x_direct.tval);     // tval
      ...
   end
   // ----------------
   else if (is_legal_CSRRxx (x_direct.instr)) begin
      match { .exc, .rd_val } <- csrs.mav_csrrxx (x_direct.instr,
                                                  x_direct.rs1_val);
      if (exc)
         fa_setup_exception (x_direct.pc,                  // epc
                             cause_ILLEGAL_INSTRUCTION,    // cause
                             x_direct.instr);              // tval
      else begin
         fa_update_rd (x_direct, rd_val);
         fa_redirect_Fetch (x_direct.fallthru_pc);
         ...
      end
      ...
   end
   // ----------------
   else if (is_legal_MRET (x_direct.instr)) begin
      let new_pc <- csrs.mav_xRET;
      fa_redirect_Fetch (new_pc);
      csrs.ma_incr_instret;
      ...
   end
   // ----------------
   else if (is_legal_ECALL (x_direct.instr)
            || (is_legal_EBREAK (x_direct.instr)
                && (! ebreak_halt)))
      begin
         let cause = ((x_direct.instr [20] == 0)
                      ? cause_ECALL_FROM_M
                      : cause_BREAKPOINT);
         fa_setup_exception (x_direct.pc,    // epc
                             cause,
                             0);             // tval
         csrs.ma_incr_instret;
         ...
      end
   // ----------------
   else if (is_legal_EBREAK (x_direct.instr)
            && ebreak_halt)
      begin
```

```
50             rg_runstate <= CPU_HALTREQ;
51             $display ("CPU: EBREAK halt for debugger at PC %0x", rg_pc);
52          end
53       // ----------------
54       else begin
55          wr_log2 (rg_flog, $format ("CPU.EX.Direct: IMPOSSIBLE"));
56          $finish (1);
57       end
58    endaction;
```

First, for convenience, we give the value `rg_Dispatch.to_Retire` a shorter and more convenient name, `x_direct`, to be used in the rest of this `action-endaction` block.

If we received an exception from the Dispatch stage, we invoke the help-function `fa_setup_exception()` (Section 10.4) to set register `rg_exception` to true and to record the relevant values in the registers `rg_epc`, `rg_epc`, `rg_cause` and `rg_tval`.

If we received a CSRRxx instruction from the Dispatch stage, we invoke the `mav_csrrxx()` method in the `csrs` module to perform the instruction on the relevant CSRs. The result is a 2-tuple (Section 5.3) and we use a `match` construct bind the names `exc` and `rd_val` to the two components. If the CSRRxx instruction failed (`exc` is true), once again we invoke `fa_setup_exception()` to record this. Otherwise, we invoke `fa_update_rd()` to write `rd_val` to a GPR, and we invoke `fa_redirect_Fetch()` to update the PC to the fall-through PC value.

If we received an MRET instruction from the Dispatch stage, we simply invoke `fa_redirect_Fetch()` to resume executing at the PC value provided in `rg_mtvec`.

If we received an ECALL or EBREAK instruction from the Dispatch stage, recall from Section 2.7.1 that these instructions merely invoke exceptions, with "cause codes" `cause_ECALL_FROM_M` and `cause_BREAKPOINT`, respectively (Section 2.7 discussed cause codes). We simply invoke `fa_setup_exception()` to record this. Inside this clause, since we are already assured that the current insruction is either ECALL or EBREAK, we only need to check one bit (`instr[20]`) to distinguish them in order to select the correct cause code.

There is a final "`else`" clause, but between the functionality of the Decode and Dispatch stages, it should be impossible to enter this clause. This is just a bit of defensive programming and can safely be removed; even if not removed, it does not produce any hardware, since it only contains non-synthesizable-to-hardware constructs such as `$display` and `$finish`.

10.5.4.2 Counting retired instructions in CSR `minstret`

The RISC-V specification says that there is a CSR called `minstret` that keeps a count of all retired instructions. This is often used for performance measurement; for example `minstret` divided by time gives us a measure of CPU speed, "instructions/second".

There are only two nuances for which we must take special care. First, any instruction that raises an exception is considered to be "not retired", so we do not increment `minstret` in that case.

Second, suppose `minstret` has the value 1000, and the next instruction we retire is a CSRRxx instruction that writes 600 into CSR `minstret`. Should the final value of `minstret`

be 1001? Or 600? Or 601? Or something else? The RISV-V spec says it should be 600, *i.e.,* an explicit CSRRxx write into `minstret` should override any implicit increment of the CSR.

In the **Drum** code, we invoke:

```
csrs.ma_incr_instret;
```

to increment `minstret` inside the `csrs` module in almost all cases:

in `a_Retire_direct`	for MRET, ECALL and EBREAK but not for CSRRxx
in `a_Retire_Int`	when not an exception
in `a_Retire_Control`	when not an exception
in `a_Retire_DMem`	when not an exception

In the remaining case for `a_Retire_direct`, CSRRxx, we invoke this method—`csrs.mav_csrrxx()`—which performs the CSRRxx instruction, and increments `minstret` if there is no exception and if it did not write to CSR `minstret`.

10.5.4.3 FSM actions in Execute and Retire Control flow

The Control path is a sequence of two actions. First, we have an Execute action invokes `fn_EX_Control` (Section 6.5) on the information provided by Dispatch, and stores the result in register `rg_EX_Control_to_Retire`:

src_Drum/CPU.bsv: line 339 ...

```
1  Action a_EX_Control =
2  action
3     let x = rg_Dispatch.to_EX_Control;
4     let y <- fn_EX_Control (x, rg_flog);
5     rg_EX_Control_to_Retire <= y;
6     ...
7  endaction;
```

Then, the Retire action checks if Execute resulted in an exception or not. If an exception, it invokes `fa_setup_exception` to record that. If not an exception, it invokes `fa_update_rd` to store the output value (this can only be a "return address" computed for JAL and JALR), and it invokes `fa_redirect_Fetch` to resume execution at the `next_pc` value computed by BRANCH/JAL/JALR.

src_Drum/CPU.bsv: line 349 ...

```
1  Action a_Retire_Control =
2  action
3     let x_direct  = rg_Dispatch.to_Retire;
4     let x_control = rg_EX_Control_to_Retire;
5     if (x_control.exception)
6        fa_setup_exception (x_direct.pc,
7                            x_control.cause,
8                            x_control.tval);
9     else begin
10       fa_update_rd (x_direct, x_control.data);
11       fa_redirect_Fetch (x_control.next_pc);
12       csrs.ma_incr_instret;
13       ...
14    end
15    ...
16 endaction;
```

10.5.4.4 FSM actions in Execute and Retire Integer flow

This sub-FSM is similar to the Control sub-FSM: it is also a sequence of two actions. The Execute action invokes `fn_EX_Int` (Section 6.6) on the information provided by Dispatch, and stores the result in register `rg_EX_to_Retire`.

src_Drum/CPU.bsv: line 372 ...

```
Action a_EX_Int =
action
   let x = rg_Dispatch.to_EX;
   let y <- fn_EX_Int (x, rg_flog);
   rg_EX_to_Retire <= y;
   ...
endaction;
```

Then, the Retire action checks if Execute resulted in an exception or not. If an exception, it invokes `fa_setup_exception` to record that. If not an exception, it invokes `fa_update_rd` to store the output value and `fa_redirect_Fetch` to resume execution at the fall-through PC.

src_Drum/CPU.bsv: line 382 ...

```
Action a_Retire_Int =
action
   if (rg_EX_to_Retire.exception)
      fa_setup_exception (rg_Dispatch.to_Retire.pc,
                          rg_EX_to_Retire.cause,
                          rg_EX_to_Retire.tval);
   else begin
      fa_update_rd (rg_Dispatch.to_Retire, rg_EX_to_Retire.data);
      fa_redirect_Fetch (rg_Dispatch.to_Retire.fallthru_pc);
      csrs.ma_incr_instret;
      ...
   end
   ...
endaction;
```

10.5.4.5 FSM actions in Execute and Retire DMem flow

This sub-FSM is similar to Control and Integers sub-FSMs: it is also a sequence of two actions. The Execute action just sends the memory-request provided by the Dispatch stage to memory by enqueueing it on the outgoing `f_DMem_req` FIFO.

src_Drum/CPU.bsv: line 403 ...

```
Action a_EX_DMem =
action
   Mem_Req y = rg_Dispatch.to_EX_DMem;
   f_DMem_req.enq (y);
   ...
endaction;
```

Then, the Retire action pops the memory response from the incoming `f_DMem_rsp` FIFO. If the memory returned an exception, we compute the proper cause code and then invoke `fa_setup_exception()` to record it. If the memory did not return an exception, we invoke `fa_update_rd()` to store any loaded value into the GPRs, and invoke `fa_redirect_Fetch()` to resume execution at the fall-through PC.

src_Drum/CPU.bsv: line 412 ...

```
Action a_Retire_DMem =
action
   let x_direct = rg_Dispatch.to_Retire;
   let mem_rsp <- pop_o (to_FIFOF_O (f_DMem_rsp));
   ...

   Bool exception = ((mem_rsp.rsp_type == MEM_RSP_ERR)
                     || (mem_rsp.rsp_type == MEM_RSP_MISALIGNED));
   if (exception) begin
      Bit #(4) cause = ((mem_rsp.rsp_type == MEM_RSP_MISALIGNED)
                        ? (is_LOAD (x_direct.instr)
                           ? cause_LOAD_ADDRESS_MISALIGNED
                           : cause_STORE_AMO_ADDRESS_MISALIGNED)
                        : (is_LOAD (x_direct.instr)
                           ? cause_LOAD_ACCESS_FAULT
                           : cause_STORE_AMO_ACCESS_FAULT));
      fa_setup_exception (x_direct.pc,                 // epc
                          cause,
                          truncate (mem_rsp.addr));    // tval
   end
   else begin
      let data = mem_rsp.data;
      if (instr_opcode (x_direct.instr) == opcode_LOAD) begin
         if (instr_funct3 (x_direct.instr) == funct3_LB)
            data = signExtend (data [7:0]);
         else if (instr_funct3 (x_direct.instr) == funct3_LBU)
            data = zeroExtend (data [7:0]);
         else if (instr_funct3 (x_direct.instr) == funct3_LH)
            data = signExtend (data [15:0]);
         else if (instr_funct3 (x_direct.instr) == funct3_LHU)
            data = zeroExtend (data [15:0]);
         // TODO: LW in RV64
      end
      fa_update_rd (x_direct, truncate (data));
      fa_redirect_Fetch (x_direct.fallthru_pc);
      ...
   end
   ...
endaction;
```

10.5.5 FSM actions for exceptions

The final action of the **Drum** FSM is used in case any of the Retire flows recorded an exception. It invokes the `mav_exception()` method in the CSRs module, and invokes `fa_redirect_Fetch` so that execution resumes at the trap-vector PC provided in CSR `mt_vec`.

src_Drum/CPU.bsv: line 461 ...

```
Action a_exception =
action
   Bool is_interrupt = False;
   Bit #(XLEN) tvec_pc <- csrs.mav_exception (rg_epc,
                                              is_interrupt,
                                              rg_cause,
                                              rg_tval);
   rg_exception <= False;
```

```
9       fa_redirect_Fetch (tvec_pc);
10      ...
11   endaction;
```

10.6 The Drum CPU module behavior

Having defined all the actions needed by **Drum**, we can now assemble them into the **Drum** FSM. First we define an FSM specification `exec_one_instr` (of type `Stmt`) for executing a single instruction.

```
module mkCPU (CPU_IFC);
   // STATE
   ...
   // ****************************************************************
   // BEHAVIOR
   ...
```

src_Drum/Drum_FSM.bsv: line 9 ...

```
1    Stmt exec_one_instr =
2    seq
3       a_Fetch;
4       a_Decode;
5       a_Register_Read_and_Dispatch;
6
7       // Execute and Retire
8       if (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_DIRECT)
9          a_Retire_direct;
10      else if (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_CONTROL)
11         seq    // BRANCH, JAL, JALR
12            a_EX_Control;
13            a_Retire_Control;
14         endseq
15      else if (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_INT)
16         seq    // LUI, AUIPC, IALU
17            a_EX_Int;
18            a_Retire_Int;
19         endseq
20      else if (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_DMEM)
21         seq
22            a_EX_DMem;
23            a_Retire_DMem;
24         endseq
25      else    // IMPOSSIBLE
26         noAction;
27
28      if (rg_exception)
29         a_exception;
30   endseq;
```

Then, we embed that `Stmt` in a next-level `Stmt` in an infinite while-loop. In the loop we first check if any attached hardware debugger has requested the CPU to be halted and, if not, takes an interrupt if one is pending, otherwise executes one instruction.

Finally, we instantiate that `Stmt` using a `mkAutoFSM` module that will implement the behavior starting immediately when the circuit emerges from reset. The FSM never stops, because of the infinite while-loop.

src_Drum/Drum_FSM.bsv: line 40 ...

```
1    mkAutoFSM (seq
2                  while (True)
3                     if (rg_runstate == CPU_HALTREQ)
4                        ...
5                     else if (can_take_intr)
6                        a_interrupt (cause);
7                     else
8                        exec_one_instr;
9               endseq);
```

```
   // ****************************************************************
   // INTERFACE
   ...
endmodule
```

Recall that each `Action` is instantaneous, and that FSMs sequence actions. In the FSM, the Fetch action `a_Fetch` takes place in one instant; the Decode action `a_Decode` takes place at a later instant; the `a_Register_Read_and_Dispatch` action takes place at an even later instant. After that, the FSM goes down one of four alternative paths (Direct, Control, Int and DMem). Finally, if `rg_exception` is true, it performs `a_exception`.

The `a_Fetch` action tries to enqueue an IMem request to memory. The action is not "enabled" until the FIFO has space available (is not full). When the Fetch action fires it enqueues the memory request, and writes the `rg_Fetch_to_Decode` register atomically.

The `a_Decode` action tries to dequeue an IMem response from memory, read a `Fetch_to_Decode` struct from an input register, and write a `Decode_to_RR` struct on an output register. The action is not "enabled" until the input FIFO has data available (is not empty). When the Decode action fires, these actions are performed, atomically.

Thus, the time interval between the Fetch instant and the Decode instant is unpredictable; it depends on when input FIFOs are not empty and output FIFOs are not full (see dicussion in Section 1.5.4.1 on unpredictable memory latency).

NOTE:

Looking ahead to a topic we'll discuss in more detail in Chapter 13, each module interface method has a so-called *implicit condition*, *i.e.,* an accompanying boolean value value that indicates when the method is READY or not or, to say it another way, whether the method is ENABLED or not. For the FIFO methods "`.first`" and "`.deq`", which are used by "`pop_o`" above, the methods are ready/enabled only when the FIFO is not empty. The "`enq`" method is ready/enabled only when the FIFO is not full.

The Decode action in the FSM is translated by the *bsc* compiler into a BSV *rule*. A rule does not "fire" until all the implicit conditions in its method-calls are enabled. This is why the Decode action implicitly waits until something is available in the FIFO.

EXERCISE: Please see Ex-10-A-Drum-initialization in Appendix E, page E-20. □

10.7 Conclusion

And that is the complete **Drum** CPU! We can compile it with *bsc* into Verilog, connect the CPU interface to a memory system, and run the system to execute RISC-V programs

compiled for the RV32I ISA subset (and, further, **Drum** can recover from illegal instructions due to trap-handling).

10.7.1 But Drum code looks just like C!? Why not code it in C?

Looking at the BSV FSM code for **Drum**, it seems like we could convert it into C code by simply replacing a few BSV idioms with corresponding C idioms (like replacing `begin-end` and `action-endaction` with "`{`" and "`}`", replacing BSV register declarations with C variable declarations, *etc.*). Similarly, each of the pure functions `fn_Fetch`, `fn_Decode`, ... can easily be slightly tweaked into C functions.

This can indeed be done, and the result would be a C simulator for RISC-V! We could compile it with any C compiler and run the software on any platform.

So, why not code **Drum** in C instead of BSV, and change the *bsc* compiler to accept C syntax? It is difficult to compile general-purpose C into hardware. C has many constructs that have no obvious mapping into hardware, such as pointers, the address-of operator "`&`", address arithmetic, `malloc`, and so on. But can't we define a restricted subset of C suitable for hardware design? Compilation is still very difficult. In the suggested re-coding in C above, we lose the distinction between variables used to hold state (registers, register files, FIFOs) and ordinary variables for temporary values (which in hardware are just wires). We also lose the distinction between non-temporal statements (statements within an action block) versus temporally sequenced (actions in an FSM). A C-to-hardware compiler would have to reconstruct this information.

But the killer reason is sequentiality *vs.* concurrency. With **Drum** it is deceptively easy to imagine a simple correspondence with C, because **Drum** is a sequential FSM, and C is a sequential language. When we advance to concurrent FSMs, including pipelined processors like **Fife**, we depart subtantially and deeply from sequential process semantics, and C becomes more and more unsuitable for hardware description.

NOTE:

There are products in the marketplace under the rubric of "High Level Synthesis" (HLS) that translate C source codes into synthesizable Verilog. These indeed work on a carefully defined subset of C, not the full C language. They work best on simple programs that contain nested loops working on dense, rectangular arrays (many image-processing and linear algebra applications can be so characterized), because the compiler is able to analyze the originally sequential semantics of such programs and transform them into highly parallel, highly structured representations that can then be mapped into very stylized hardware (data path plus control FSM). Even with this capability, the programmer may need to provide many additional directives to the compiler to guide it towards good quality hardware. We do not consider this to be a general purpose approach to hardware design.

See also Appendix B for more discussion on this topic.

10.7.2 Adding more RISC-V register-to-register instructions to Drum

The RISC-V instruction set is modular, based on a baseline of integer instructions (RV32I), with many standard optional extensions, such as:

- **M**: Integer multiply and divide instructions

- **F**, **D**: Single- and Double-Precision Floating Point instructions
- Other custom instructions

The **Drum** design of Figure 10.1 can easily be extended to accomodate such register-to-register extensions, by adding more execution stages as illustrated in Figure 10.3.

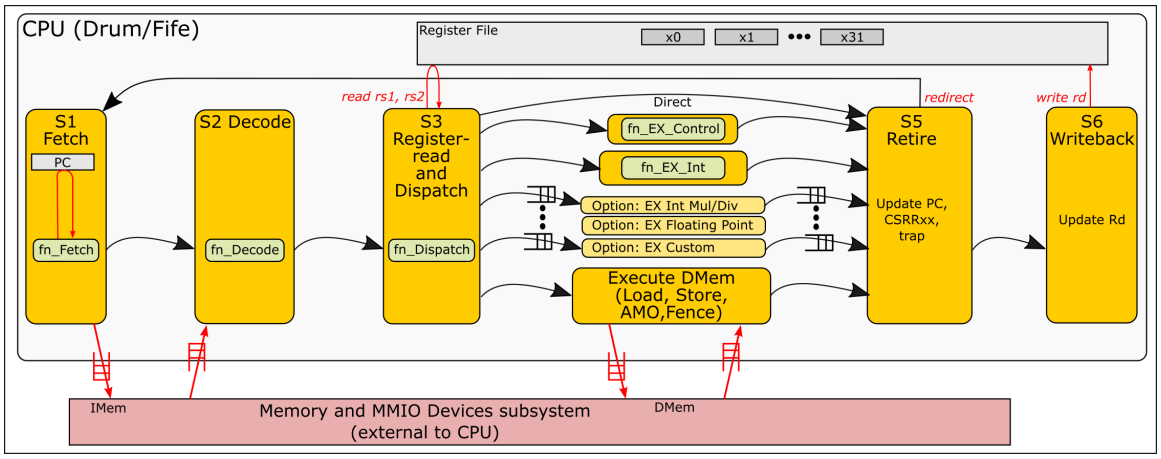


Figure 10.3: Adding more RISC-V instructions

The `OpClass` type in `Inter_Stage.bsv` needs to be extended to handle the new classes of instructions (**M**, **F,D**, Custom). The Decode stage's big if-then-else must be extended to handle these new kinds of instructions. The Register-Read-and-Dispatch stage must be extended to dispatch into the new execution pipelines. It can reuse the existing register-read facilities. The Retire stage needs to field the results from the new execution pipelines and retire those instructions. It can reuse the existing register-write and exception facilities.

To implement the **F** and **D** instructions, we need a second register file similar to the GPRs, to hold the bit representations of floating-point values. Register-read and register-write logic now needs to be cognizant of which register file we are addressing.

The actual functional logic for **M**, **F,D** and/or Custom will be entirely internal to the respective EX_Int_Mul_Div, EX_Floating_Point or EX_Custom stages. Note, these stages can have variable latency; indeed they are likely to take many cycles for their computations, which are more complex than simple integer ops. However, no modification is necessary to the rest of **Drum** (or **Fife**) to accommodate these additional latencies; **Drum** and **Fife** are both "latency-tolerant" designs so it is easy to replace/add units with different latencies.

10.7.3 Adding RISC-V Atomic Memory Op instructions to Drum

The modular RISC-V instruction set also has an "**A**" extension for atomic memory read-modify-write operations, such as AMOSWAP and AMOADD. These ops both read and write memory. For example, AMOSWAP writes a value to memory like STORE, and returns the previous value from that location, like LOAD.

These, too, are easily added to **Drum** and **Fife**. As far as the CPU is concerned, these are merely new memory opcodes, just like the LOAD and STORE opcodes. All the work

is really done outside the CPU in the memory system. In **Drum** and **Fife**, we'd reuse the EX_DMem facilities, just decoding and transmitting the new memory opcodes.

Chapter 11

BSV: Verifying BSV designs

11.1 Introduction

In this chapter we describe general techniques used to verify **BSV** designs of any kind. The next chapter focuses on techniques for verification of CPU implementations.

11.2 Testbenches and DUTs

To debug a **BSV** design, we typically set up a system similar to that shown in Figure 11.1. The **BSV** design that is being tested is usually called the "Design Under Test", or DUT

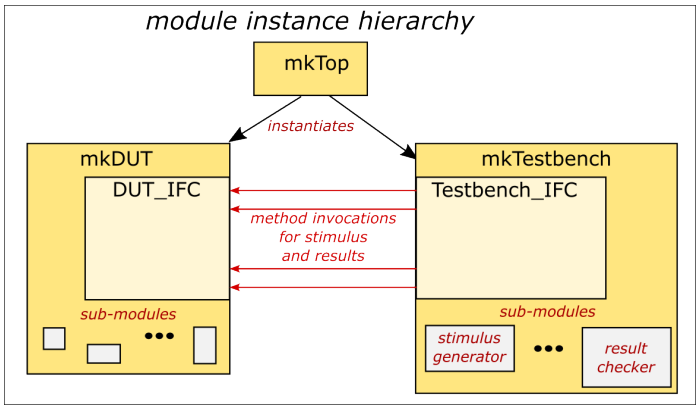


Figure 11.1: A testbench connected to a DUT

for short. The surrounding and/or adjacent modules are called the "Testbench" (or "Test Harness" or "Test Environment").

The top-level module, `mkTop`, has the `Empty` interface, which is just an interface with no methods, and is pre-defined in the *bsc* library.

The testbench interacts with the DUT *via* its interface methods. The DUT interface and the testbench interface are often opposite "duals", *e.g.,* if `DUT_IFC` has `FIFOF_I` sub-interface for input data, the testbench might have a corresponding `FIFOF_O` that delivers that input data; these are connected together by `mkTop`.

Code inside the testbench produces input data for the DUT; this part of the testbench is called the "stimulus generator". Stimulus data can be generated in the testbench, or read from data files.

Other code inside the testbench collects output data from the DUT and checks if it has the expected value corresponding to the stimulus. This part of the testbench is called a "checker". Output data can be checked immediately in the testbench, or recorded into files for offline manual or automated checking. Alternatively, the DUT may just print outputs to the screen (during simulation), which can be visually examined for correctness or recorded for offline manual or automated checking.

BSV designers typically write their testbenches in **BSV**. If the testbench is only used in simulation (not in actual hardware, FPGA or ASIC) then it can read and write files. It can also import C code (see Appendix D) to reuse existing C algorithms and models, and to have full access to operating system services (files, networking, *etc.*).

In the SystemVerilog community there is a mature standard methodology for testbenches called UVM (Unversal Verification Methodology) which exploits the "object-orientd programming" aspects of SystemVerilog for reusability (see Glossary C for some more detail). **BSV** designs can also use UVM testbenches. The whole of Figure 11.1 can be in Verilog/SystemVerilog, with the DUT using the Verilog produced from a **BSV** design using the *bsc* compiler.

11.3 printf-style Debugging

A popular style of debugging **BSV** designs is the same as in debugging software in any programming language: insert "print" statements in the **BSV** code at various places to print out values of interest during simulation. We examine these outputs to identify suspicious values and then perhaps insert and remove print statements to zero-in on the exact place in the **BSV** code where something wrong was computed.

BSV, like Verilog and SystemVerilog, has the following built-in functions to write to files during simulation, analogous to `printf` in C. All of them have `Action` type, and so they can occur in any Action context: bodies of rules, bodies of Action and ActionValue methods, bodies of Action and ActionValue functions.

```
$write    (      format-string, arg, ..., arg )
$display  (      format-string, arg, ..., arg )

$fwrite   ( file, format-string, arg, ..., arg )
$fdisplay ( file, format-string, arg, ..., arg )
```

The first two write to "standard output" (*i.e.,* the terminal), and the latter two write to a specific file which has previously been opened with an Action statement like this:

```
file <- $fopen ("log.txt", "w");
```

The difference between "write" and "display" is merely that the latter appends a newline at the end of the output.

These are similar to C's `printf` and `fprintf` functions. The format string is a string (in double-quotes) with formatting directives for the arguments that follow (`%d` for signed integers, `%b` for binary numbers, `%h` for hexadecimal numbers, *etc.*).

These **BSV** statements for printing only exist in simulation code. They are omitted completely (ignored completely) by synthesis tools that target actual hardware (ASIC or FPGA).[1]

In **BSV**, additionally, you can interleave format strings and arguments, like this:

$display (*format-string*, *arg*, ..., *format-string*, *arg*, ...)

11.3.1 `FShow` for "pretty-printing" enums and structs

In any enum or struct type declaration in **BSV**, one can attach a "`deriving(FShow)`" clause to request the *bsc* compiler to define an "`fshow`" function for the type, with some default formatting. For example, the **Drum**/**Fife** code has such a clause in the declaration of the `Decode_to_RR` struct type:

src_Common/Inter_Stage.bsv: line 52 ...

```
typedef struct {
   Bit #(XLEN)  pc;
   ...
   // If exception
   Bool         exception;  // Fetch exception/ decode illegal instr
   Bit #(4)     cause;
   Bit #(XLEN)  tval;

   // If not exception
   Bit #(XLEN)  fallthru_pc;
   Bit #(32)    instr;
   OpClass      opclass;
   Bool         has_rs1;
   Bool         has_rs2;
   Bool         has_rd;
   Bool         writes_mem;   // All mem ops other than LOAD
   Bit #(XLEN)  imm;          // Canonical (bit-swizzled)
   ...
} Decode_to_RR
deriving (Bits, FShow);
```

The result-type of `fshow()` is of type "`Fmt`", and this type is also allowed as an argument to `$display` (and its variants): Example:

```
   Decode_to_RR y = ...
   Fmt          f = fshow (y);
   $display (      "Decode result is ", f);
   $fwrite  (file, "Decode result is ", f);
```

EXERCISE: Please see Ex-11-A-FShow in Appendix E, page E-21. □

---

[1] This is not because of any fundamental synthesizability difficulty, it is only because there is no standard concept of "file" or "output stream" in hardware designs, not even a serial port.

11.3.2 `Fmt` formatted values

BSV's formatting facilities are actually more powerful than `printf` in C/C++ or `$display` in Verilog/SystemVerilog.

The result-type of `fshow()` is a standard type in **BSV** called `Fmt`, and this is also an acceptable argument in **BSV**'s `$display` functions (as demonstrated above in the last section).

We can create new `Fmt` objects using the built-in pure function `$format()`, and we can combine them (concatenate them) using an infix "`+`" operator. `Fmt` is a first-class type, so we can bind it to variables, pass it as arguments and results of functions, and so on. The following example illustrates defining another function to format a `Decode_to_RR` struct type, an alternative to `fshow` to format it in some other preferred way:

src_Common/Inter_Stage.bsv: line 274 ...

```
function Fmt fshow_Decode_to_RR (Decode_to_RR x);
   Fmt f = $format ("    Decode_to_RR{");
   f = f + $format ("I_%0d", x.xtra.inum);
   f = f + $format (" pc:%08h", x.pc);
   f = f + $format (" instr:%08h", x.instr);
   f = f + $format (" pred:%08h epoch:%0d\n", x.predicted_pc, x.epoch);
   f = f + $format ("            ");
   f = f + $format ("fallthru:%08h ", x.fallthru_pc);
   if (x.exception) begin
      f = f + fshow_cause (x.cause);
      f = f + $format (" tval:%0h", x.tval);
   end
   else begin
      f = f + fshow (x.opclass);
      f = f + $format (" has_{rs1,rs2,rd}:{%0d,%0d,%0d} writes_mem:%0d, imm:%0h",
                       x.has_rs1, x.has_rs2, x.has_rd, x.writes_mem, x.imm);
   end
   f = f + $format (" halt_sentinel:%0d}", x.halt_sentinel);
   return f;
endfunction
```

Note the use of if-then-else to customize the `Fmt` object according to the actual data in the struct (which would not happen in the default `fshow()`, which would simply print all the struct fields). This function can be used just like, and in place of, `fshow`:

```
   Decode_to_RR y = ...
   Fmt          f = fshow_Decode_to_RR (y);
   $display (      "Decode result is ", f);
   $fwrite  (file, "Decode result is ", f);
```

This shows one immediate advantage of the `Fmt` facilities: we can format something once and write it to multiple output streams. By abstracting formatting into a common function, it can be modified easily and all `$display` statements using it can share the benefit.

A second advantage is that actual formatting code, which is often quite verbose, *ad hoc* and messy with a lot of fragments of character strings, can be lifted to a different location in the code, keeping the `$display` statement short and sweet.

11.4 Dynamic assertions

The **BSV** libraries offer a package:

```
import Assert ::  *;
```

which contains the following function:

```
function Action dynamicAssert (Bool b, String s);
```

This can be used in any Action context (*e.g.,* a rule body) to check an expected property each time that Action context is executed. For example the **Drum** CPU code includes the following excerpt:

src_Drum/CPU.bsv

```
1  Action a_Retire_DMem =
2  action
3     ...
4     let mem_rsp <- pop_o (to_FIFOF_O (f_DMem_rsp));
5     dynamicAssert ((mem_rsp.rsp_type != MEM_REQ_DEFERRED),
6                    "Mem req not speculative but got DEFERRED mem response");
7     ...
8  endaction
```

which checks for the unexpected situation where the DMem memory response was "deferred" (deferred memory responses are only possible for speculative memory accesses, which only occur in **Fife** and not in **Drum**). Every time this action is executed (on every DMem response), the boolean condition is tested and, if false, it aborts the simulation after printing the associated string.

These `dynamicAssert` statements have no cost in final real hardware because, like `$display` statements, they exist only in simulation code. Thus, one should not hesitate to use them liberally.

NOTE:

A `dynamicAssert` is no more than an if-B-then-A_1-else-A_2 where
- B is the assertion condition
- Action A_1 is `noAction` (do nothing if the condition is true)
- Action A_2 is `$display(`*string-to-be-printed*`)` followed by a `$finish` to terminate the simulation.

In fact, the if-then-else version is often more convenient because we can write `$display` to also print current values of interest, instead of just a `String`.

11.5 Waveform-style debugging

Many hardware designers like to debug designs using "waveforms", which are a graphical display of how values on buses (bundles of wires) in the design vary over time.

All Verilog, SystemVerilog and VHDL simulators have a facility to write out a "Value Change Dump" (VCD) file, which is a record of how each bus (bundle of wires) in the design changed over time (measured with clock ticks). VCD files can then be viewed as

a graphical display in any waveform viewer. Waveform viewers are bundled with most commercial RTL simulators, but the free and open-source *gtkwave* viewer is also popular.

When simulating in a Verilog simulator, most simulators have command-line or interactive controls to switch VCD dumping on or off.

When simulating in **Bluesim** interactively, the commands:

`sim vcd on`	enables writing out VCDs
`sim vcd off`	disables writing out VCDs

VCD dumping can also be controlled from within a **BSV** program, using these three Actions:

`$dumpvars`	Starts writing out VCDs
`$dumpoff`	Stops writing out VCDs
`$dumpon`	Resumes writing out VCDs

11.6 Summary

Verification in **BSV** is, in principle, the same as in Verilog or SystemVerilog: The DUT is instantiated along with a Testbench that provides stimulus and consumes the output for checking and analysis.

There are many good textbooks available on verification, covering topics such as "code coverage" (how much of the DUT has been tested), constrained-random stimulus generation, so-called "*fuzzing*", creation of so-called "Verification IP" (reusable library components for testbenches), *etc.*

The Testbench can be written in **BSV**, but it can also be written in Verilog or SystemVerilog. If written in **BSV**, the testbench is synthesizable, and can be executed along with the DUT on FPGAs, which is usually many orders of magnitude faster than simulation. If written in Verilog or SystemVerilog, the testbench may or may not be synthesizable. If not synthesizable, it can only be run in simulation.

For **printf**-style debugging, **BSV** enhances traditional `$display()` with a powerful facility of "formatted strings" (expressions of type `Fmt`).

BSV also offers dynamic assertions and VCD waveform dumping.

Chapter 12

RISC-V: Functional verification of CPUs

12.1 Introduction

The last chapter described general techniques used to verify **BSV** designs of any kind. This chapter focuses on techniques for verification of CPU implementations.

Debugging and verifying a CPU implementation (whether it is a hardware implementation or a software simulator) is hard because, as discussed in Section 1.3.1, it is not just a program, but is itself an *interpreter* of programs. Thus we are confronted with two levels, not one. The first-level program (P1) is the CPU implementation (the **BSV** program), which we are simulating or running in hardware. P1, in turn, is interpreting the RISC-V program (P2) that was loaded into the RISC-V CPU's memory. When we observe an error in P2's outputs or behavior, it could be a bug in P2 (the RISC-V program), or a bug in P1 (our RISC-V implementation), or both.

In other words, when we have an interpreter of programs, we conceptually have an exponentially larger space of things to test (test cases). Every RISC-V program P2 is potentially a test case for our implementation P1, along with every potential input to each P2! This is illustrated in Figure 12.1.

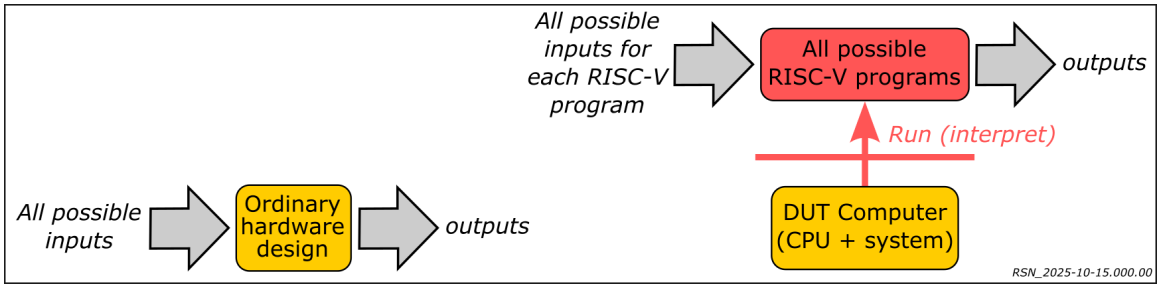


Figure 12.1: Complexity of CPU verification

In addition to this large space of potential tests, the duration of a test can be very long. A bug may exhibit itself only inside an application program running under an operating

system. To reach this point we may have to execute hundreds of millions, or billions of P2 instructions on P1.

Fortunately, for small to medium-sized isolated test programs, we can exploit a key property the RISC-V ISA (and most ISAs): except for interrupts, which are asynchronous events, and a few instructions (such as `rdtime` and `rdcycle`, the sequence of instructions executed by a RISC-V program on a particular initial memory contents is completely deterministic and repeatable. This means that if we excute a RISC-V program P2 repeatedly in our test setup, or even run P2 on different implementations (such as a software simulator and **Drum** and **Fife**), they should all exhibit *exactly* the same sequence of instructions, or "*instruction trace*". If one of them takes a particular conditional BRANCH, then the others should, as well. If one of them traps due to, say, an illegal instruction, then the others should, as well, and they all should vector to exactly the same instruction address for the trap handler. If one of them traps due to, say, an unimplemented memory location, then the others should, too, assuming they have the same (or equivalent) memory setup. We call this a *functional equivalence* because different implementations perform the same function on such a test program, even though they may execute it at vastly different speeds.

Beyond small and medium-sized deterministic test programs, we need to test the CPU in an environment that more closely resembles its eventual operating environment. This may include running operating systems and interacting with devices that may generate timer and device interrupts asynchronously, *i.e.,* at unpredictable moments in time relative to instruction execution. This topic is discussed in Section 12.6.

This chapter only addresses the question of *functional correctness* ("Does the CPU compute correct data?"). An equally important topic is *performance correctness* ("Does the CPU compute results in the allowed/expected time?"). That topic is addressed in Chapter 19.

12.2 Trusted functional simulators ("golden reference models")

Most CPU design teams rely on a "trusted" software functional simulator that can execute RISC-V programs. Such a simulator is written, and is maintained, with a focus that prioritizes clarity and correctness, not speed. This simulator acts as a reference standard, and is also called a "Golden Reference Model". It answers questions such as:

- What is a correct execution when executing a RISC-V program P2?
- What is the correct architectural state (registers, CSRs, memory) before and after each instruction I_j?

We can compare our hardware implementation against this reference to see if our hardware implementation is also behaving correctly.

Often, a trusted simulator is the first thing one implements when designing a new ISA, before starting any hardware design. Once the ISA is frozen, because the functional simulator is only defining functional correctness (not speed), it can be used across multiple hardware CPU design projects, *i.e.,* it is a long-lived resource that can be shared across multiple projects.

For the RISC-V ISA, there are several free and open-source software functional simulators available that we can use as a trusted reference. Appendix A.3 provides URL links to the two most well-known, the **Spike** simulator and the **Sail** simulator. Each of them can be configured for a particular subset of the RISC-V ISA (such as RV32IM, or RV64IMAFDC, with or without privilege levels and virtual memory, *etc.*), and then run RISC-V programs, producing a log/trace of the instructions executed.

It can be useful to be able to modify and customize a trusted software functional simulator. For example, if we are extending the RISC-V ISA with new, custom instructions, then we would like the simulator to support them. Or, we may want extra information detail in the simulator's output trace that was not originally provided. Or, we may use the simulator in non-standard ways, such as the asymmetric tandem verification mode described in Section 12.6.

Being open-source, the **Spike** and **Sail** simulators can be modified for custom purposes. Many companies also have their own trusted software RISC-V functional simulators for greater modifiability and customization (such as **Cissr** from Bluespec, Inc.).

Golden reference models are usually written as RISC-V interpreters in a software programming language (for example, **Spike** is written in C++, and **Sail** is compiled with OCaml). In this case they only run as software on a standard computer.

Golden reference models can also be written in a high-level hardware design language, such as **BSV**. For example, we can think of **Drum** in this way. Because it is a simple FSM, it is extremely simple and clear. The advantage of a hardware golden reference model is that it can be synthesized into hardware and plugged into any system environment where a more powerful CPU implementation will be go.

12.3 RISC-V test programs for verification

An advantage of an open (non-proprietary) ISA like RISC-V is that there is a vast and growing worldwide community of hardware CPU developers—companies, universities, research organizations, students and hobbyists—who share a common need for RISC-V test programs for verification. Many of the test programs they create are shared in free and open-source form.

12.3.1 ISA tests

The oldest and most stable set of test programs originated the the University of California Berkeley with the original RISC-V team, and is now maintained and expanded by RISC-V International (RVI) on their GitHub site at `https://github.com/riscv-software-src/riscv-tests`. These are a set of several hundred test programs written in RISC-V Assembly Language. Each program contains a small set of tests for some particular instruction in the RISC-V ISA. The tests cover RV32I and RV64I and the A, M, F, D and C extensions; machine, supervisor and user extensions; and some system features like PMP (Physical Memory Protection). Most user-level tests can be built to run with or without virtual memory. The tests are provided in source-code form (RISC-V Assembly Language) and need to be compiled with a C compiler (*e.g., gcc*) to produce ELF binaries.

The tests are small (a few thousand instructions) and are all *self-checking*, *i.e.,* they compute something and then check if the result has the expected value. At the end of the test, each program outputs a PASS/FAIL indicator. PASS means it passed all the tests in the file. In the FAIL case, it also outputs the test number within the file that failed.

The tests are small enough that it is feasible manually to compare a trace from our CPU implementation against the assembly-language source code (or the assembly language in the *gcc*-produced "objdump" disassembly of the ELF file) and analyze it for errors.

The tests also run sufficiently quickly (a few minutes at most) that we routinely and automatically rerun all the ISA tests every time we make any significant change to the CPU design's source code or test environment.

12.3.2 ACTs and other test suites

RVI (RISC-V International) is developing a set of tests called ACTs (Architecture Compatibility Tests). Each test is a RISC-V program that runs and produces a final "signature", which is a reliable hash of the final state (PC, registers, CSRs, memory, *etc.*).

If a candidate RISC-V CPU implementation claims to support, say, RV32IM, then it must run the relevant ACTs for RV32IM, and the produced signatures should match the official signature for that test published by RVI (the official signatures, in turn, are produced by RVI by running on a trusted simulator such as **Spike** or **Sail**).

This is an ongoing project. Some relevant links:

https://wiki.riscv.org/pages/viewpage.action?pageId=49872986
https://github.com/riscv-non-isa/riscv-arch-test
https://riscof.readthedocs.io/en/stable/

Another test suite, called "riscv-dv", was developed initially at Google; subsequent stewardship was taken over by the ChipsAlliance non-profit group. These tests generate "random" RISC-V programs (constrained to a certain ISA subset), run the program on a candidate implementation which should generate a trace, and compare against a trace from a trusted functional simulator:

https://github.com/chipsalliance/riscv-dv

In addition, most organizations maintain their own collection of test programs and "regression suites" (programs that previously surfaced a bug in some implementation, which are continuously re-run to ensure that the bug has not resurfaced due to more recent changes).

12.3.3 What does "verified" mean? Levels of assurance and coverage

In the software community, the term "verified program" has historically meant "formally verified program", *i.e.,* that we have a *proof of correctness* of the program against a formal specification of the program. A formal specification is usually written in a specialized formal specification language which are usually highly mathematical, declarative languages.

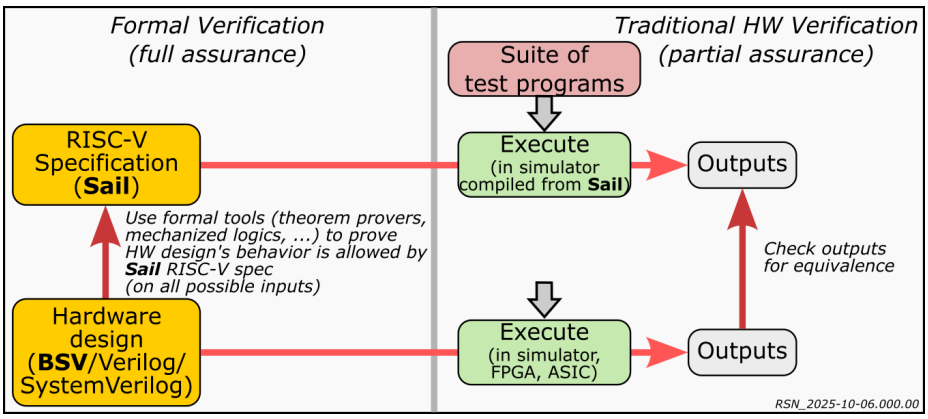


Figure 12.2: Levels of Assurance

A formal proof of correctness uses formal tools (theorem provers, mechanized logics) to show that a candidate implementation correctly computes something prescribed by the formal spec. Formal verification may not involve executing the implementation at all, just analysis of the code for the implementation. Formal verification may be partial (prove that the implementation is correct for certain inputs) or complete (for all inputs). Although there has been huge progress in formal verifcation over the decades, as of 2025 it is still not able to handle very large and complex programs; it is used very successfully for individual or small collections of modules, and for partial correctness.

The hardware community has also been exploring formal verifation of hardware implementations, and there are several tools offered commercially. But, as in in the software community, as of 2025 it is still not able to handle very large and complex designs; it is used quite successfully for individual or small collections of modules, and for partial correctness.

In the hardware community, the term "verification" has historically meant "extensive testing with a large suite of test programs", not formal verification. One can think of the term "level of assurance" as a point on scale ranging from "no assurance" (untested, unproved) to "full assurance" (formally proved correct). Figure 12.2 illustrates the relationship of formal and traditional hardware verification.

As we increase the number and variety of test programs that we run (the test suite), we may increase the level of assurance that we have a correctly implemented design. In the rare and unlikely case that the set of inputs is small enough that we can test all of them, we essentially have full assurance, or a proof of correctness by enumeration. Normally, the space of possible tests is too large even to enumerate, let alone run. Thus the test suite needs to be carefully engineered and selected to *cover* as much of the space as feasible, particularly the space of inputs expected when the design is deployed in the field.

The term "coverage" is a kind of measure of level of assurance. It can be used both in a functional way ("How much of the space of possible inputs has been tested?") or in a more implementation-specific way ("What fraction of the lines of code in the **BSV**/Verilog/SystemVerilog source have we exercised with our tests?").

12.3.4 Testing regimes

Here is a sequence of testing regimes with increasing simulation times but also increasing levels of assurance of the correctness of our CPU implementation:

- Run all the standard "ISA tests" mentioned in Section 12.3.
- Run the ACTs and riscv-dv test suites mentioned in Section 12.3.
- Run a small operating system (such as FreeRTOS or Zephyr). This will check correct handling of timer interrupts, and possibly memory maps and physical memory protection (PMPs).
- Run a "micro-kernel" operating system, such as **SeL4**. This will check virtual memory systems and process-switching.
- Run the kernel of a full-service operating systsem (such as the Linux kernel).
- Run a standard distribution a full-service operating systsem (such as Debian Linux or Ubuntu Linux), *i.e.,* the OS kernel *plus* the pre-load of all the applications and service programs that come with distribution (including block devices and networking).

The latter items are often executed on FPGA because simulation speed may be too slow.

12.4 A testbench for Drum and Fife

Figure 12.3 illustrates the structure of the testbench provided with this book for **Drum** and **Fife**. On the left we see the top-level of the module hierarchy, and on the right we see some

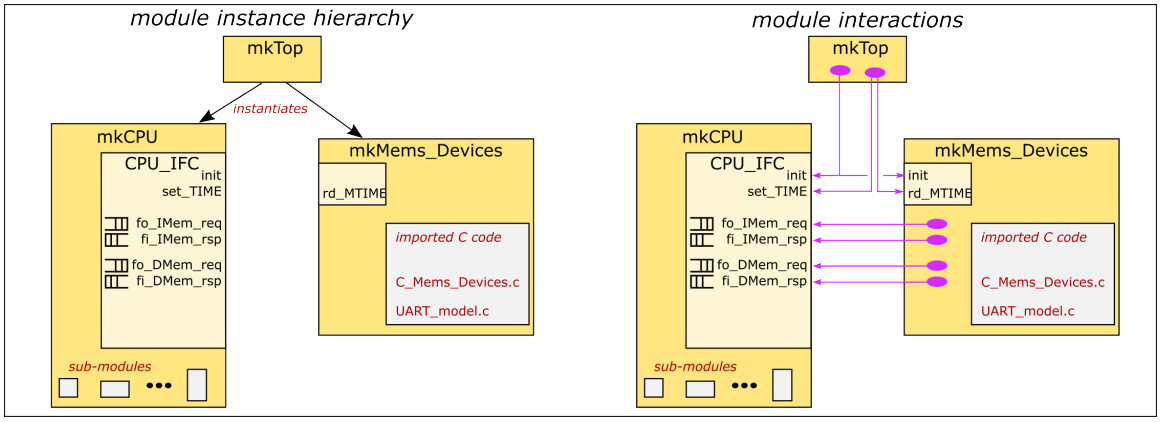


Figure 12.3: Top-level simulation setup for the **Drum** and **Fife** CPUs

module interactions. The top-level module, `mkTop`, has the `Empty` interface.

The interface for **Drum** and **Fife**, `CPU_IFC`, was described in Section 10.2. The most important sub-interfaces are the FIFOs carrying IMem and DMem memory requests and responses. `mkTop` instantiates the `mkCPU` module (which can be either **Drum** or **Fife**) and `mkMems_Devices` modules. An excerpt from `mkTop` is shown below:

src_Top/Top.bsv: line 30 ...

```
(* synthesize *)
module mkTop (Empty);
   ...
   // Instantiate the CPU
   CPU_IFC cpu <- mkCPU;

   // Instantiate the memory model
   Mems_Devices_IFC mems_devices <- mkMems_Devices (cpu.fo_IMem_req,
                                                    cpu.fi_IMem_rsp,
                                                    ...
                                                    cpu.fo_DMem_req,
                                                    cpu.fi_DMem_rsp,
                                                    ...
```

Note that we pass the CPU's IMem and DMem sub-interfaces directly to Mems_Devices module as module parameters; as a result, rules in `mkMems_Devices` can directly access the IMem and DMem FIFO interfaces of the CPU to collect IMem and DMem requests and send back IMem and DMem responses.

The `mkMems_Devices` module implements models for memory, a UART (Universal Asynchronous Receiver/Transmitter, also known as a serial port), a real-time clock, and any other devices expected by the CPU and the RISC-V code running on the CPU. Some of these (memory, UART, **Fife** store-buffer) are implemented by importing C code.

The memory system in the testbench also needs a way to be pre-loaded with the binary RISC-V code of the program that we want the CPU to execute, before the CPU begins executing.

A rule in `mkTop` that fires at the beginning of simulation invokes the methods `cpu.init` and `mems_devices.init`, passing them a struct with some initialization parameters, such as the reset value for the PC (address from which the first instruction will be fetched), and a file descriptor into which logs should be written.

src_Top/Top.bsv: line 80 ...

```
   // Initialize modules
   rule rl_step1 (rg_top_step == 1);
      ...
      let init_params = Initial_Params {pc_reset_value:    'h_8000_0000,
                                        addr_base_mem:     'h_8000_0000,
                                        size_B_mem:        'h_1000_0000,
                                        ...
      cpu.init (init_params);
      mems_devices.init (init_params);
      ...
      rg_top_step <= 2;
   endrule
```

Another rule in `mkTop` fires on every clock. On each clock, it retrieves a value reprsenting the wall-clock time from `mems_devices` and relays it into the CPU:

src_Top/Top.bsv: line 125 ...

```
   (* fire_when_enabled, no_implicit_conditions *)
   rule rl_relay_MTIME;
      let t <- mems_devices.rd_MTIME;
      cpu.set_TIME (t);
   endrule
```

Other than this startup code, `mkTop` plays no further role in execution. Rules inside `mkCPU` operate the CPU, putting out IMem and DMem requests. Rules inside `mkMems_Devices` operate the memory and device models, returning IMem and DMem responses.

We do not intend to describe `mkMems_Devices` in any more detail here, since it is outside the main focus of this book, the **Drum** and **Fife** CPUs. Section D has details on how to import C code into **BSV**. The interested student is welcome to peruse the code in the `src_Top/` directory.

12.5 Symmetric Tandem Verification of CPU implementations

In Section 12.1 we mentioned that the instruction trace for a RISC-V program is mostly deterministic and repeatable across RISC-V implementations. We can exploit this property in a setup called *symmetric tandem verification*, illustrated in Figure 12.4.

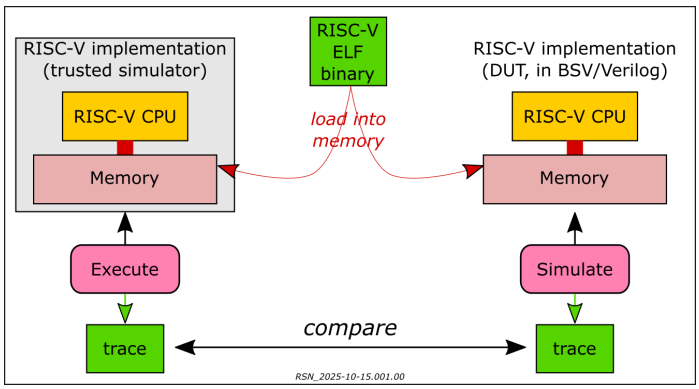


Figure 12.4: Symmetric Tandem Verification

We load the same ELF RISC-V binary file into the memories of a "trusted" simuator and our DUT (Design Under Test) setup, and have them both run the program. We arrange to have both simulators write out a trace of their respective program executions. Then, we compare the two traces. Any difference in the traces is an indicator of a potential bug in the DUT.

For example, we might find that, at a certain conditional BRANCH instruction in P2, the trusted simulator took the branch whereas our implementation did not. This would indicate either that there is some problem with our implementation of the conditional BRANCH instruction, or that there was a problem in some previous instruction that computed one or both of the register values used by the conditional BRANCH instruction. Thus, debugging is a process of identifying exactly which instruction in our implementation "went wrong", *i.e.,* did something different from what the trusted simulator did.

12.5.1 Configuration

The trusted simulator and the DUT setup the should be configured "identically":

- They should support the same RISC-V ISA subset, so that an illegal instruction in one is also an illegal instruction in the other.

- They should have the same (or equivalent) memory systems, so that an illegal or misaligned memory access in one has the same response in the other.
- They do not need to have the same *temporal* behavior. For example, memory access latency in one need not match memory latency in the other, and the "time" taken to execute any particular instruction in one need not match the "time" taken by the other.

12.5.2 Level of detail in traces

Traces can be produced at varying levels of detail. For example, for each instruction executed, we could record:

- just the PC;
- or also the instruction iself;
- or also any values it reads from or writes to registers

More detail means more simulation overhead (slower simulation) to produce the trace. Traces can become very large (*e.g.,* gigabytes when tracing, say, the booting of an OS). But more detail can also provide better "resolution" in identifying the location of a bug. For example, suppose one instruction (I1) computes a wrong value and stores it to memory, where it sits for thousands, perhaps millions of instructions before it is loaded into a register (instruction I2) and, some instructions later, this is used in a conditional BRANCH instruction (instruction I3). The wrong value may cause the trusted simulator and the DUT to diverge, where one takes the branch, the other does not, which we detect because the next PCs are different. If our trace only records PCs, then we will detect a difference only at I3. However if we also record register values, we will detect the difference earlier, at I1 or I2.

One way to reduce excessive detail is to record traces only for a certain window of instructions, say starting at the 10 millionth instruction and for the following one thousand instructions. This would be fine if we were guaranteed that there was no divergence until the 10 millionth instruction, but we have no way of knowing that. A way to address this is:

- Run the trusted simulator withoug producing any trace until 10M instructions, and record a "snapshot" of *the entire architectural state*—PC, all registers, CSRs, all of memory. Then, continue for 1K instructions, producing a trusted trace.
- Initialize the DUT's PC, registers and memory with the values in the snapshot, and then let the DUT execute for 1K instructions, producing a DUT trace to compare with the trusted trace.

This requires infrastructure support in the trusted simulator to be able to dump a snapshot, and in the testbench to be able to initialize its state using the snapshot.

12.5.3 Online *vs.* offline tandem verification

The setup in Figure 12.4 can be performed offline or online:

- **Offline**: Each of the two simulators records its trace in a file, and these files are compared later, manually, or with "`diff`", or with a specialized comparison tool. In fact, for a given test program, the reference simulator need be run only once, and its trace can be saved for future comparisons.
- **Online**: The two simulators are run concurrently (*e.g.,* two processes in an operating sytem). They each generate their trace into a *stream* such as an operating system "pipe" or "tty" or network connection. A comparison tool runs concurrently as a third process, continually consuming the two trace streams and comparing them immediately.

The online setup requires more infrastructure and tooling, but has some advantages. First, it can abort both simulations as soon as it detects a divergence, so we don't unnecessarily continue simulating for a long time. Second, since the compare tool is continually consuming the traces, it need not be recorded in a file, thereby eliminating the "trace-file size" problem.

12.6 Asymmetric tandem verification and "full-system" verification

We said in Section 12.5 that, except for interrupts (asynchronous events), the two traces (from the trusted simulator and the DUT) should be identical. There are some nuances to this claim.

12.6.1 Instructions with non-deterministic results

If a RISC-V program uses the `rdcycle` or `rdtime` instructions, then the results, loaded into registers, will likely be different in the two setups.

12.6.2 Reading uninitialized memory

If the memories in the trusted simulator and the DUT setup have not been initialized identically, then a LOAD instruction can return different results in the two setups, even in "bug-free" RISC-V programs. For example, when traversing a C string (one character per byte), the program may only LOAD aligned 8-byte doublewords, for more efficiency. At the end of the string, only a prefix of the 8 loaded bytes may be part of the string, and the remaining bytes may be "uninitialized" or random. The C program may correctly examine only those bytes that are in the string. However, a tandem verifier will not be aware of this, and may compare the full 8 bytes loaded by the trusted simulator and the DUT, and falsely identify a difference because bytes outside the string happen to be different.

12.6.3 Devices and interrupts

Simulating the DUT may require running code that interacts with devices or device models because they are fundamental to the CPU's intended applications. Thus, the DUT simulation needs to model more of the "full system" in which it is intended to be used. Devices or device models may be unique or proprietary to the intended application domain. Devices

contain memory-mapped locations accessed by the RISC-V program, and may generate interrupts. We cannot expect the trusted simulator (which is often created and maintained by others) to model all these devices.

12.6.4 Asymmetric mode

All the above issues can be handled if we take an "asymmetric" view of tandem verifaction, and if we can configure the trusted simulator to work in "tandem verification mode". This is illustrated in Figure 12.5.

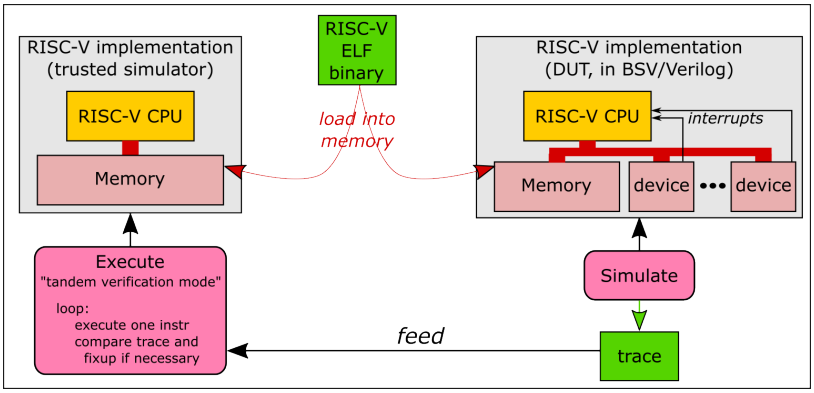


Figure 12.5: Asymmetric tandem verification: dealing with "minor" differences, interrupts (asynchronous events), devices, *etc.*

Here, the DUT simulation produces a CPU execution trace, including a record of the interrupts received and taken. This trace is fed to the trusted simulator which runs in tandem verification mode. In this mode, before simulating each instruction, it examines the next ITEM in the trace, and:

- If ITEM is an interrupt received (bit set in the DUT CPU's CSR MIP), then also set that bit in the trusted simulator's CSR MIP. This brings the trusted simulator's CSR back "in sync" with the DUT, from where we proceed.
- If ITEM is an interrupt taken: check if the trusted simulator can also take this same interrupt at this time (correctly set interrupt bits in CSR MIP, correctly set interrupt-enable bits in CSRs MIE, MSTATUS *etc.*). If so, then perform the interrupt-taking actions in the trusted simulator (save values in CSRs `mepc`, `mcause` and `mtval`, update CSR `MSTATUS`, set the PC to the value in CSR `mtvec`, *etc.*). If this interrupt cannot be taken here, report a divergence and stop.
- Otherwise (ITEM is an instruction-execution), perform the next instruction in the trusted simulator and compare results with ITEM.

  If the instruction is a LOAD, RDTIME or RDCYCLE and the loaded values are different, report the difference, but do not stop, do a "fixup" and continue:

  - Update the loaded value in the trusted simulator to be the same as in the DUT trace. This brings the trusted simulator's registers back "in sync" with the DUT, from where we proceed.

In summary: the trusted simulator need not model any devices at all, just memory. Instead of generating a trace, it checks the DUT-produced trace for correctness.

12.7 Tandem verification with real hardware (FPGA or ASIC)

The discussion above on Tandem Verification assumed that the trusted model and the DUT were both being run in simulation. But, of course, there is nothing simulation-specific about the technique.

If the RISC-V CPU *hardware* is capable of generating traces, and the infrastructure supports streaming that trace out of the hardware to a host machine then, once again, the trace can be compared with that of a trusted simulator.

The trusted model can itself be written in a synthesizable hardware description language like **BSV**. In fact, the **Drum** CPU could be a candidate, because it is an extremely simple implementation, not designed for speed, and so possibly trustworthy. In this case, the entire setup in Figure 12.5 could be in hardware, which is likely to run several orders of magnitude faster than any simulation.

12.8 Randomized Property-based testing (PBT) with TestRIG

For **Fife** and **Drum** we use a symmetric tandem-verification setup called **TestRIG** from the University of Cambridge. We postpone details of this until Chapter 17 which follows our discussion of **Fife**, because some of the more interesting naunces concern pipelining. For now, briefly:

- **TestRIG** generates a random instruction sequence and feeds it to both **Sail** and to our DUT (**Fife** or **Drum**). Both **Sail** and our DUT return a sequence of reports to **TestRIG** summarizing the effects of each instruction; effectively, a kind of trace of the execution of the instruction sequence.

  TestRIG checks if the two traces are identical and reports success or failure.

- **TestRIG** repeats this over a number of random sequences (configurable).

- **TestRIG** uses a Haskell program for sequence generation and comparison. Haskell has a **QuickCheck** library for generic generators, and **TestRIG** customizes this for generating RISC-V instructions.

- When a failure is detected on an instruction sequence, the Haskell **QuickCheck** library also has ways automatically to *shrink* the sequence and retry the test, and repeatedly shrink while it still finds a failure. This can often shrink a failing sequence from thousands of instructions to a handful of instructions, which makes it much easier to diagnose.

We will discuss **TestRIG** for **Fife** and **Drum** in detail in Chapter 17.

Chapter 13

BSV: Rules and their Semantics

13.1 Introduction

"*Rules*" are the fundamental constructs in **BSV** to specify dynamic behavior. Rules appear in the body of **BSV** modules, and they may invoke methods in interfaces of other modules. An interface method is simply a "mini-rule" that is incorporated into a rule from which it is invoked. Rule semantics can be understood in two incremental steps:

- Semantics of a rule in isolation (Section 13.4)
- Semantics of the collection of rules in a **BSV** program (Section 13.5)

Finally, the performance of rules (how long does a computation take?) can be understood by understanding how rules are mapped to clocks (Section 13.6)

13.2 Syntax of a rule and the data types of its components

Figure 13.1 shows the syntactic structure of a rule, with the keywords `rule` and `endrule`.

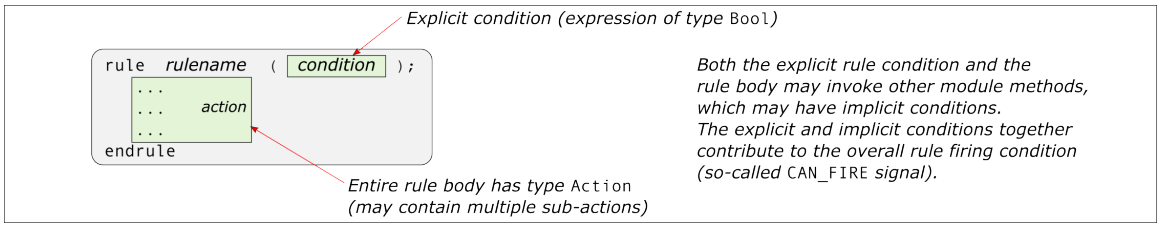


Figure 13.1: Syntactic structure of a rule

The rule's *explicit* condition is an expression of type `Bool`. Recall from Section 4.11 that, since it is not of `Action` or `ActionValue` type, it is guaranteed by **BSV**'s type system therefore to be a pure computation with no side-effects.

The rule body, as a whole, is an expression of type `Action`. It typically consists of multiple statements, including register-writes, FIFO enqueues/dequeues, module method invocations, `let`-bindings, `$displays`, if-then-elses, and so on. Many of the statements/subexpressions will themselves be of type `Action` or `ActionValue`. The overall action of a rule is the set of all sub-actions performed by a rule.

13.3 A rule's implicit condition and overall condition

Both the rule condition and the body may contain invocations of methods in interfaces of other modules. Each method has an "implicit condition", also of type `Bool` indicating whether the method is currently enabled or not. This implicit method output is also called its READY signal. For example, for a standard FIFO f, the f`.first` and f`.deq()` methods have implicit conditions that are true only when the FIFO is non-empty. A method like f`.first`, being pure (not `Action` or `ActionValue`), may be invoked both in rule conditions and in rule bodies. A method like f`.deq`, being of type `Action`, can never be invoked in a rule condition, only in a rule body.

The overall rule condition, also known as its "CAN_FIRE" condition, is a conjunction of the rule's explicit condition and implicit conditions of any invoked methods, whether those methods are in the rule condition or in the rule body. For example, if a rule invokes f`.first` or f`.deq`, the rule's CAN_FIRE condition cannot be true if f is empty.

13.4 Semantics of a rule in isolation

In this section we discuss the semantics of a rule in isolation; in Section 13.5 we will consider the collection of rules that constitute a **BSV** design.

Each rule can be viewed as a pure function (therefore, a combinational circuit) whose inputs come from various methods and which produces outputs for various `Action` and `ActionValue` methods. Each `Action` or `ActionValue` method has an implicit boolean ENABLE argument (separate from its normal arguments and result). An `Action` or `ActionValue` method *performs* its action if its ENABLE argument is true.

Consider the following rule (artificial, just for illustration, not taken from code of **Drum** or **Fife** or any actual design):

```
1   rule rl_compute ((y != 0) && got_x && (f.first == 3));
2    if (y [0] == 1) w <= w + x;
3    x <= x << 1;
4    g.enq (w * f.first);
5   endrule
```

Figure 13.2 illustrates the semantics of this rule in isolation. In the discussion below, we use the words "input" and "output" relative to a method (input to the method, or output from the method).

At the top of the diagram we see outputs from methods feeding the rule: four register-reads[1] and one FIFO `.first`. The latter provides two outputs: a data output from the head of the FIFO (black line) and a READY value (green line) which is the implicit condition of the method, which is true only if the FIFO is not empty. Actually *all* methods have implicit conditions, but a register's `.read` and `.write` methods are always READY, *i.e.,* their implicit conditions are constant true, so we omit them in diagrams.

[1] Recall from Section 7.3.1 mentioning a register in an expression is equivalent to using its `._read` method, and assigning a value to a register is equivalent to using its `._write` method.

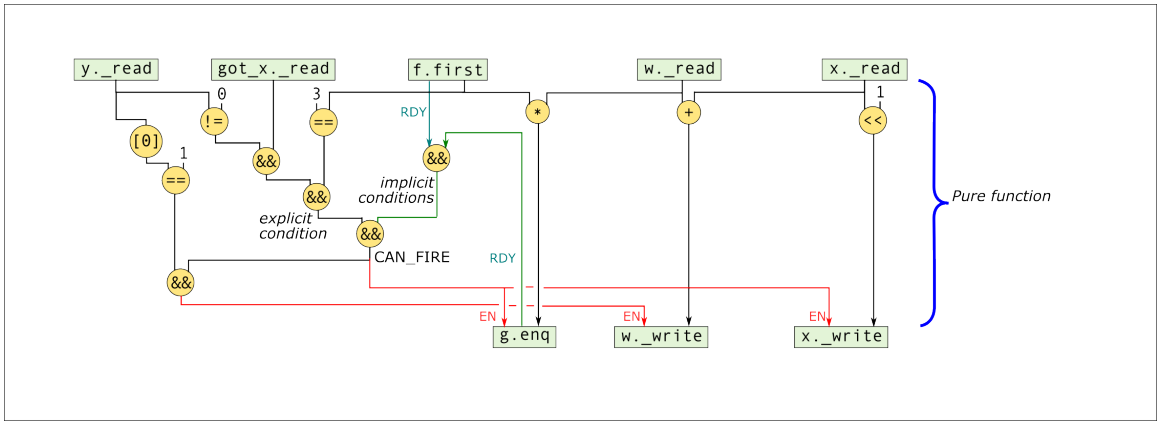


Figure 13.2: Semantics of a rule in isolation

At the bottom of the diagram we see three `Action` methods used in the rule: two register-writes and one FIFO `.enq`. Each method has an input data value (black line) and an implicit input ENABLE value (red line). The FIFO `.enq` method also has a READY output (green line), which is its implicit condition (true only when the FIFO is not full).

In between the top and the bottom is a pure function (therefore, a combinational circuit). On the left half we see that all the explicit-condition expressions are calculated and combined (with "`&&`"), and then further combined with the implicit conditions, to produce the CAN_FIRE signal. This is fed directly to the `g.enq` and `x._write` ENABLE inputs The CAN_FIRE signal is further combined with the calculation of "`y[0]==1`" and this is fed to the ENABLE input of `w._write`

The data inputs to the three `Action` methods are straightforward calculations from inputs.

When the ENABLE input to an `Action` method is true, the method actually performs the action (enqueue into a FIFO, store into a register).

From the diagram we can see that the ENABLEs of `g.enq` and `x._write` are true when FIFO `f` has data available (not empty), when FIFO `g` has space available (not full) and when the explicit condition is true—the rule "fires" and the actions are performed. The ENABLE of `w._write` is true only if `y[0]==1` is also true.

From this description, several things should be clear:

- All `Actions` in a rule are performed *simultaneously*, no matter what textual order they may appear in the rule body. We also say that the actions all occur *in parallel*.
- All `Actions` in a rule are performed *instantaneously*.
- Explicit rule conditions and method implicit conditions are combined to determine whether the rule executes at all.
- Some actions in a rule may be further restricted by if-then-else conditions in the rule.

EXERCISE: Please see Ex-13-A-Simultaneous-Actions in Appendix E, page E-21. □

13.4.1 Hardware representation of a rule in isolation

Figure 13.3 overlays standard symbols in digital hardware for registers and FIFOs onto Figure 13.2, to show how the semantics can map to real hardware in a straightforward way.

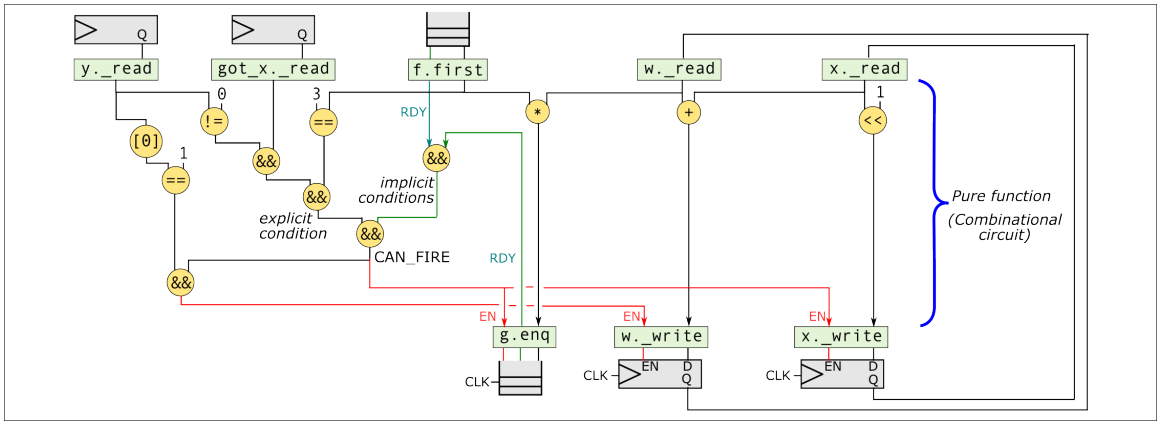


Figure 13.3: Hardware representation of a rule in isolation

BSV registers map directly into Verilog registers which, in turn, map into "D flip flops". The `.read` method output is the same as the "Q" outputs of D flip flops. The `.write` method inputs are the same as the "D" and "EN" inputs of D flip flops. Each register also has a "clock" (CLK) input. On each edge of CLK (*e.g.,* on the positive edge, or so-called "posedge", when going from low to high), if EN is true, then the value on the D input is copied into the register, replacing the previous value. The value in the register is continuosly presented on the Q output wires.

BSV FIFOs are implemented as Verilog modules using registers. The details need not concern us here[2] but, suffice it to say, on a clock edge, if EN is true, the data value is loaded into a register in the FIFO representing the tail of the queue.

In the semantics, we said that all actions of the rule are "simultaneous". In hardware terms: they occur on the same clock edge.

In the semantics, we said that all actions are "instantaneous". In hardware terms this is the standard digital abstraction, as if clock edges are instantaneous, and as if registers load their values at that instant. In practice, because of physics, clock edges are steep but not vertical (they have a small but finite rise times), but standard digital abstraction suppresses this detail.

Standard digital abstraction also treats combinational circuits as instantaneous, as if there is zero delay in producing outputs from inputs. In practice, signals take small but finite time to propagate through wires and gates. Thus, the ENABLE for `w._write` will be available slightly later than the ENABLE for the other two methods, because it goes through a longer combinational path. But in the standard digital abstraction we idealize all this as zero delay.

[2] If you are curious, you can look at the FIFO Verilog codes in the *bsc* library.

NOTE: Although Figure 13.3 is useful in developing intuitions, it is important to understand that **BSV** semantics stands alone, and does not depend on any particular mapping to hardware! Different compilers may map **BSV** to hardware in different ways, for example using multiple clocks for some or all rules. Indeed a compiler could choose to map **BSV** code into so-called "*asynchronous logic*" (which does not have clocks at all).

13.4.2 A rule firing cannot perform the same action more than once

From our description that a rule's actions are semantically simultaneous, it should be clear that the same action cannot be performed more than once in a single rule firing. For example:

```
1    rule rl_foo (...);
2       x <= 2;          x <= 3;
3       f.enq (2);       f.enq (3);
4       g.deq;           g.deq;
5    endrule
```

Each line shows an absurdity: we cannot write two values into the same register at the same instant, nor enqueue two values into the same FIFO at the same instant, nor dequeue two values from the same FIFO at the same instant.

The *bsc* compiler will flag such errors in a program with a message like "Cannot compose actions in parallel".

This may seem a somewhat minor point (would anyone really write such absurdities in their programs?), but understanding it will help when we discuss mapping multiple rules to a clock in Section 13.6.

13.5 Semantics of a collection of rules

The semantics of a collection of rules is deceptively simple: it simply repeats, forever, the single-rule semantics of Section 13.4:

```
while True
    Choose any rule whose CAN_FIRE is true
        Perform the actions in that rule's body
```

Of course, when a rule fires, its actions will have modified some state (a register, a FIFOF, *etc.*). Thus, in the next iteration of this while-loop, a different set of rules may have true CAN_FIRE conditions.

Revisiting our FIFO example, if a rule R1 invokes f`.first` or f`.dequeue`, it cannot fire if f is empty. Some other enabled rule R2 may fire and invoke f`.enqueue`; the FIFO then becomes non-empty, at which R1's CAN_FIRE may become true, and R1 may become eligible to fire.

Observe that the semantics is *one rule at a time*. We emphasize that this is only at the semantic level, in the same sense that RISC-V ISA semantics is one-instruction-at-a-time

and C/C++ semantics is one-statement-at-a-time. Any *implementation* is free to speed things up with concurrency and/or reordering, provided they are consistent with the one-at-a-time semantics so that the programmer has no surprises.

NOTE:

> Observe that rule-level semantics is non-deterministic: if several rules' CAN_FIRE is true, we can choose any one. This is sometimes shocking and scary to the **BSV** newcomer, but it is in fact very common in formal specification systems (including all those cited below, because forcing a schedule (a particular way to choose enabled rules) is usually an *over-specification* and instead should be left as implementation leeway. Proving any correctness property of a program using the non-deterministic semantics proves it for *all* possible schedules, and is thus more general than a proof for a specific schedule.
>
> Nevertheless, please keep calm and carry on; the *bsc* compiler removes all non-determinism (in a predictable way) when it compiles to hardware.

We cannot emphasize enough that *the semantics is enough for reasoning about functional correctness of all* **BSV** *programs*[3], *i.e.,* "does the program compute what we expect it to compute?", without appealing to clocks and clocked digital hardware! This includes all of **Drum** and **Fife**; in fact, the **BSV** code for **Drum** and **Fife** does not mention any clock, and in this book we do not mention clocks and temporal properties of **Drum** and **Fife** until much later, in Chapters 18 and 19.

NOTE:

> This, again, can be shocking and scary to the newcomer to **BSV** who has already learned some digital hardware design. In traditional teaching of digital hardware design, one often introduces clocks and clocked logic practically in Lecture 1, and this suffuses the thinking completely from that point onward.
>
> Again, please keep calm and carry on; the **BSV** view will grow on you and, over time, becomes the simpler and more intuitive view!

This semantics of rules is well-known in the Theoretical Computer Science literature, broadly falling under the rubric of "*Term Rewriting Systems*", because it is a very simple, clean, self-contained computation model (like Turing Machines and Lambda Calculus) ([1, 14, 15, 23]).

Several formal specification systems for concurrent programming use this computation model: well-known examples include:

- Guarded Commands Language (Dijktra [7])
- UNITY (Chandy and Misra [4])
- Event-B (Metayer, Abrial and Voisin [17])
- TLA+ (Lamport [16])

13.6 Mapping rules to a clock, for real-time behavior

With the rule semantics of Section 13.5, we only have an abstract view of time—in an execution we can say whether this rule firing was *before* that rule firing, or *after*; no more. In particular, we cannot ascribe any real-time measure to it (microseconds, nanoseconds, ...).

Digital hardware systems are driven by one or more *clock signals*. Figure13.4 illustrates a

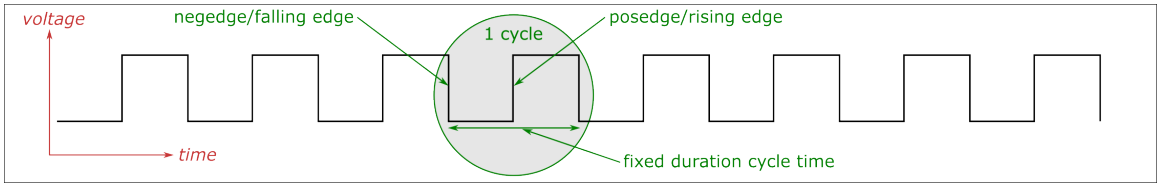


Figure 13.4: A clock signal

clock signal. A clock signal is an electrical signal (a voltage on a wire), where the voltage-change over time has the shape of a so-called "square wave", which repeats, or "cycles", at a fixed time interval (the "cycle time"), indefinitely. Each cycle has a falling edge (or negedge) and a rising edge (or posedge). Modern FPGAs typically run from 10s to 100s of MHz ("megahertz", of millions of cycles per second), and modern ASICs can run at up to several GHz ("gigahertz", or billions of cycles per second. Thus, a clock is a real-time reference signal.

Standard digital state elements (*e.g.,* registers) update their values only and exactly on a clock edge (posedge or negedge), and hold that value for the full next cycle, when they may change again. Usually all elements in a circuit react to the same edge, either the posedge or the negedge; for simplicity we'll assume the posedge.

Specifically, if a D flip flop's EN input is high (1, true) at the posedge, it updates its state to contain the value on the D input; if the EN input is low (0, false), it does nothing, retaining its previous value. The value in the flip flop is continuously driven on its Q output.

13.6.1 Constraints on mapping rules to a clock

To map the collection of rules in a **BSV** design to clocked digital hardware, as a first approximation, suppose that each rule was mapped as suggested in Figure 13.3. Then, conceptually, at each posedge, all enabled rules (whose CAN_FIRE is true) will fire and perform their output actions.

But this would be wrong, for two reasons:

- *Action Conflicts* or *R*esource Conflicts: Two different rules may invoke the same action/actionvalue method (try to write the same register, or enqueue onto the same FIFO, dequeue the same FIFO, *etc.*). As discussed in Section 13.4.2, this is clearly not feasible at the same instant (same posedge).

- *Ordering Conflicts*: The *ordering* can be inconsistent with rule semantics. Consider the execution of these two rules:

BSV
```
rule rl_r1 (...);
   x <= y + 1;
endrule
```

BSV
```
rule rl_r2 (...);
   y <= x + 2;
endrule
```

[3]There is a nuance regarding how one defines functional correctness when we compute just one deterministic outcome of a non-deterministic program, but we ignore that here.

According to the one-rule-at-a-time semantics, either rule `rl_r1` precedes `rl_r2` or *vice versa*. In either case, register state-update by one rule is observed by the other rule. Whereas, if we execute the rule actions at the same instant, neither rule observes the update by the other rule. Thus, sequential execution and parallel, instantaneous execution will produce inconsistent results. (See also Exercise Ex-13-A-Simultaneous-Actions on page E-21.)

Recall in Figure 13.2 that a rule's CAN_FIRE signal controls its execution completely—if it is false, the rule does nothing. We take advantage of this observation in Figure 13.5, where we pass the CAN_FIRE values of all rules through a *Rule Controller* function which returns, for each CAN_FIRE, a corresponding WILL_FIRE value.

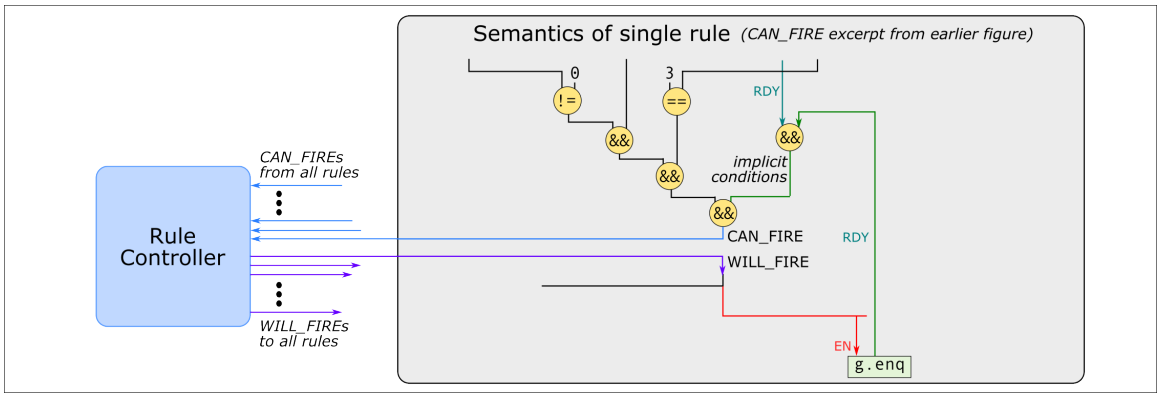


Figure 13.5: Controlling rule execution (CAN_FIRE excerpt from Figure 13.3)

Whenever a pair of rules `rl_r1` and `rl_r2` would conflict if executed simultaneously, the Rule Controller ensures that it never happens—if both CAN_FIREs are true, the Rule Controller forces one of the WILL_FIREs to be false, *i.e.,* it suppresses one of the two rules.

13.6.2 The Rule Controller produced by the *bsc* compiler, and reasoning about performance

The abstract description of the Rule Controller above allows for many possible implementations of the Rule Controller; the only criterion for correctness is that the resulting rule execution in hardware should be consistent with one-rule-at-a-time semantics.

The *bsc* compiler produces a simple, state-free, combinational circuit for the Rule Controller. First, it produces a *linear* ordering of all the rules in the program, that results in minimal conflicts (for example, if rule rl_1 reads a register and rule rl_2 writes the same register, then it tries to place rl_1 earlier than rl_2 in the ordering, because executing the rules simultaneously would be consistent with that order, *i.e.,* would not conflict.

Second, for any rule rl_1 before rule rl_2 in the ordering and which would conflict if executed simultaneously, the WILL_FIRE of rl_1 is negated and AND-ed with the CAN_FIRE of rl_2 to suppress the latter rule.

A compile-time flag to the *bsc* compiler will make it dump the "schedule", *i.e.,* the linear ordering of rules and the conditions under which one rule may suppress another.

With this information, we can reason about the real-time perfomance of a **BSV** program, *i.e.,* to address the question: "does the program compute something within the number of clocks we expect it to be computed?" Note, our unit of time is clock cycles, not real-time *per se* (seconds), because **BSV** has no way to know the actual speed (1 MHz? 1 GHz?) of the clock you might supply to your hardware circuit.

13.6.3 Explicit control of rule ordering, and controller optimizations

We mentioned in the previous section that the *bsc* compiler produces a linear ordering of all the rules in a program, attempting to minimize the number of conflicts. This is a heuristic, because producing an "optimal" ordering is undecidable. Further, when the compiler sees a conflict between two rules, it may have no criterion to choose which one has priority, *i.e.,* which one will suppress the other. In such situations, the **BSV** programmer can provide explicit *attributes* on rules to guide the compiler's choice.

For example:

BSV
```
rule rl_r1 (...);
   ...
endrule

(* descending_urgency = "rl_r1, rl_r2" *)
rule rl_r2 (...);
   ...
endrule
```

Here, the `descending_urgency` attributes advises the compiler to treat rule `rl_r1` with higher priority. The attribute is written just before a rule; it can name this rule and any rules earlier in the source text.

Another attribute advises the compiler, when there is no conflict, to force `rl_r1` to suppress `rl_r2`:

BSV
```
(* preempts = "rl_r1, rl_r2" *)
```

A third attribute asserts to the compiler that the CAN_FIRE of two rules are mutually exclusive, *i.e.,* can never simultaneously be true (the compiler tries to prove such properties by itself, but cannot always do so), which permits the compiler to simplify the controller.

BSV
```
(* mutually_exclusive = "rl_r1, rl_r2" *)
```

In this case, the compiler also generates code that checks the mutual-exclusivity property during simulation.

13.7 `StmtFSM` can always be translated into rules

As mentioned in Section 9.2, `StmtFSM` does not add any fundamental semantic power to **BSV**; anything expressible with `StmtFSM` can also be expressed using rules. Here we show a small example:

BSV

```
Stmt s = while (b)
            seq
               action1;
               if (c)
                  action2;
               else
                  action3;
               action4;
            endseq
```

Here is a translation into rules:

BSV

```
// rg_step == 7 means "idle"
// To start the FSM, environment sets rg_step <= 0
Reg #(Bit #(3)) rg_step <- mkReg (7);

rule rl_while (rg_step == 0);
   rg_step <= (b ? 1 : 100);
endrule

rule rl_A1 (rg_step == 1);
   action1;
   rg_step <= (c ? 2 : 3);    // if-then-else
endrule

rule rl_A2 (rg_step == 2);
   action2;
   rg_step <= 4;
endrule

rule rl_A3 (rg_step == 3);
   action3;
   rg_step <= 4;
endrule

rule rl_A4 (rg_step == 4);
   action4;
   rg_step <= 0;    // back to top of loop
endrule
```

This translation illustrates (a) that `StmtFSM` is just a higher-level abstraction on rules, and (b) that `StmtFSM` can often be much more readable than the corresponding rules—the while-loop, the sequencing, and the if-then-else structure are not readily apparent from the rules.

In Chapter 14 we will show how the FSM for **Drum** from Chapter 10 can be hand-translated directly into rules. Indeed, the *bsc* compiler performs a similar translation of `StmtFSM` into rules.

13.8 When to use `StmtFSM` and when to use rules

This raises the question: when should we use `StmtFSM` and when should we use rules?

For some, it is matter of personal taste; some like the higher-level abstraction of `StmtFSM`, while others like the uniformity of using rules exclusively.

At the very small scale of a single `Action` (perhaps containing sub-actions), this can be the body of a single rule, and this is often more succinct and clear than using `StmtFSM` for just one action. Many modules have a single rule in their body (we will see many such examples in **Fife**, in later chapters).

Ability to debug is sometimes another consideration. For explicit rules, the programmer supplies rule names, which are presumably carefully chosen for clarity. The rules generated by the *bsc* compiler to implement a `StmtFSM`, on the other hand, have names that are compiler-generated, and may not be so clear (to help legibility, the compiler includes the source file line number in the rule name). Readability of rule names affects us most in compiler-generated error and warning messages, *e.g.,* a message that two sub-actions cannot be placed in the same rule, or a message about the relative schedule-order of two rules.

Another difference is in performance. When the *bsc* compiler translates `StmtFSM` into rules, it may have to make pessimistic assumptions about when it can iterate a loop, or when it can terminate a `par-endpar` block. In these cases, it may be possible to save a clock cycle by writing with rules instead.

13.8.1 Use rules for unstructured processes

The situation where rules are often much clearer than `StmtFSM` is when we have an unstructured process, *i.e.,* a process that is not easily expressed with a proper nesting of `Stmt` constructs (sequencing, if-then-else, and loops).

Suppose we modify the example in Section 13.7 as follows: after action2, if a certain condtion "`c2`" holds, we can immediately go back to the top of the loop.

It is possible, but not easy to modify the `Stmt` to express this "goto top of loop" requirement.[4] On the other hand, the rules version is almost trivial to modify—we just have to modify one rule:

BSV

```
rule rl_A2 (rg_step == 2);
   action2;
   rg_step <= (c2 ? 0 : 4);
endrule
```

[4]There is a long-standing theoretical Computer Science result from the 1960s that arbitrary control-flows can be converted into "goto-free" structured programs; but this transformation involves introducing many boolean variables to remember deeply nested conditions, and the resulting programs are not necessarily very readable or efficient.

Chapter 14

RISC-V: the Drum unpipelined CPU (using Rules instead of `StmtFSM`)

14.1 Introduction

In Section 10.6 we showed the complete behavior of the **Drum** CPU module, coded using BSV's `StmtFSM` construct for FSMs.

In this short chapter we show a manual translation of that FSM into BSV Rules. Our first objective is to reinforce the fact that `StmtFSM` does not add any fundamental new computational capability to BSV—anything that could be coded with `StmtFSM` can also be coded with Rules. `StmtFSM` is just a convenient, more readable view of rules, when the FSM follows a structured flow.

In other situations, where the FSM does not follow a structured flow, and certainly where we have multiple cooperating FSMs, Rules are often the more convenient medium of expression. These ideas are explored in the exercises the end of the next section.

14.2 The Drum CPU module behavior with Rules

First we define an enumeration type to give symbolic labels to all the **Drum** FSM actions:

src_Drum/CPU.bsv: line 71 ...

```
1  typedef enum {
2     A_FETCH,
3     A_DECODE,
4     A_REGISTER_READ_AND_DISPATCH,
5     A_EX,
6     A_RETIRE_CONTROL,
7     A_RETIRE_INT,
8     A_RETIRE_DMEM,
9     A_EXCEPTION
10 } CPU_ACTION
11 deriving (Bits, Eq, FShow);
```

Then, we instantiate a register `rg_action` to hold the label of the next action to be performed, initialized to `A_FETCH`. We manually transform each FSM action into a rule. Each

rule's explicit condition checks `rg_action`, and the rule-body updates `rg_action` to enable the next action. After the final rule to execute an exception, we assign `A_FETCH` again to `rg_action`, and the whole process repeats itself, forever.

src_Drum/Drum_Rules.bsv: line 9 ...

```
1   Reg #(CPU_ACTION) rg_action <- mkReg (A_FETCH);
2
3   rule rl_fetch ((rg_runstate == CPU_RUNNING)
4                  && (! can_take_intr)
5                  && (rg_action == A_FETCH));
6      a_Fetch;
7      rg_action <= A_DECODE;
8   endrule
9
10  rule rl_decode (rg_action == A_DECODE);
11     a_Decode;
12     rg_action <= A_REGISTER_READ_AND_DISPATCH;
13  endrule
14
15  rule rl_register_read_and_dispatch (rg_action == A_REGISTER_READ_AND_DISPATCH);
16     a_Register_Read_and_Dispatch;
17     rg_action <= A_EX;
18  endrule
19
20  rule rl_Retire_direct ((rg_action == A_EX)
21                         && (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_DIRECT));
22     a_Retire_direct;
23     rg_action <= A_EXCEPTION;
24  endrule
25
26  // BRANCH, JAL, JALR
27  rule rl_EX_Control ((rg_action == A_EX)
28                      && (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_CONTROL));
29     a_EX_Control;
30     rg_action <= A_RETIRE_CONTROL;
31  endrule
32
33  rule rl_Retire_Control (rg_action == A_RETIRE_CONTROL);
34     a_Retire_Control;
35     rg_action <= A_EXCEPTION;
36  endrule
37
38  // LUI, AUIPC, IALU
39  rule rl_EX_Int ((rg_action == A_EX)
40                  && (rg_Dispatch.to_Retire.exec_tag ==  EXEC_TAG_INT));
41     a_EX_Int;
42     rg_action <= A_RETIRE_INT;
43  endrule
44
45  rule rl_Retire_Int (rg_action == A_RETIRE_INT);
46     a_Retire_Int;
47     rg_action <= A_EXCEPTION;
48  endrule
49
50  // rule rl_EX_DMem (rg_action == A_EX_DMEM);
51  rule rl_EX_DMem ((rg_action == A_EX)
52                   && (rg_Dispatch.to_Retire.exec_tag == EXEC_TAG_DMEM));
53     a_EX_DMem;
54     rg_action <= A_RETIRE_DMEM;
```

```
55    endrule
56
57    rule rl_Retire_DMem (rg_action == A_RETIRE_DMEM);
58       a_Retire_DMem;
59       rg_action <= A_EXCEPTION;
60    endrule
61
62    rule rl_exception (rg_action == A_EXCEPTION);
63       if (rg_exception)
64          a_exception;
65       rg_action <= A_FETCH;
66    endrule
```

Please study the FSM version (in Section 10.6) and the above code side-by-side, to see how straightforward it is manually to translate a `StmtFSM` into Rules.

14.2.1 Optimizing the Drum rules

If we execute the **Drum** CPU on a RISC-V program and study the detailed, cycle-by-cycle execution trace, we will see many possibilities to eliminate "wasted" cycles.

- In rule `rl_Decode` we may detect an exception—the IMem response itself could be an exception; or Decode may find the fetched instruction to be illegal. In case of an exception, instead of proceeding to Register-Read-and-Dispatch, we could directly process the exception and return to Fetch.

- Similarly, rules `rl_EX_Control`, `rl_EX_Int` may raise exceptions; in these cases, too, we could directly deal with the exception and return to Fetch.

- The Retire Direct CSRRxx action may raise an exception, which is handled in the final exception action, which also accesses the CSRs (to update trap registers and return the trap-vector PC). These could be fused into a single method in the CSR module, after which we could return directly to Fetch.

- Because we are using (a subset of) standard CSRs, the only exceptions in a CSRRxx instruction can be because we are addressing an unsupported CSR, or we are attempting to update a read-only CSR. Both these questions can be decided in Decode, in which case we can assume that, in Retire, the CSRRxx treatment will not raise an exeption, and can go straight to Fetch.

- The Decode and Register-Read-and-Dispatch rules could be "fused" into a single rule. Currently, the former rule leaves information in a register which is used by the latter rule. If fused, the information can be directly passed as a value, and the intervening register could be eliminated.

  Fusing rules together like this comes at a cost: the combinational logic in a rule body becomes more complex. This, in turn, may require us to clock it at a slower speed. Thus, while we may have "saved time" by eliminating a "tick", we may have made the ticks longer (slower clock). Careful analysis of the actual numbers is needed to judge whether such a transformation is worth it or not.

These optimizations are easier to implement in the Rules version of **Drum** (instead of the `StmtFSM` version), because the flow now becomes less structured, with ad-hoc jumps from some rules back to Fetch, *etc.*. `StmtFSM` can only express structured, properly-nested flows.

EXERCISE: Please see Ex-14-A-Optimizing-Drum-Rules in Appendix E, page E-22. □

14.3 Conclusion

And that is the complete **Drum** CPU behavior, written using Rules! The reason this was so easy, and why this chapter is so short, is that we reused all the `Action` definitions defined earlier for the FSM version of **Drum**.

Chapter 15

RISC-V: the Fife pipelined CPU: Principles

15.1 Introduction

In this chapter we turn our attention to **Fife**, the pipelined CPU. Our focus here is purely on identifying new problems raised by pipelining, and proposing solutions. In the next chapter we will look at BSV code to implement **Fife**.

Figure 15.1 annotates the abstract execution algorithm in Figure 2.16 with some specifics for the pipelined implementation in **Fife**. Unlike **Drum**, where each yellow box was one

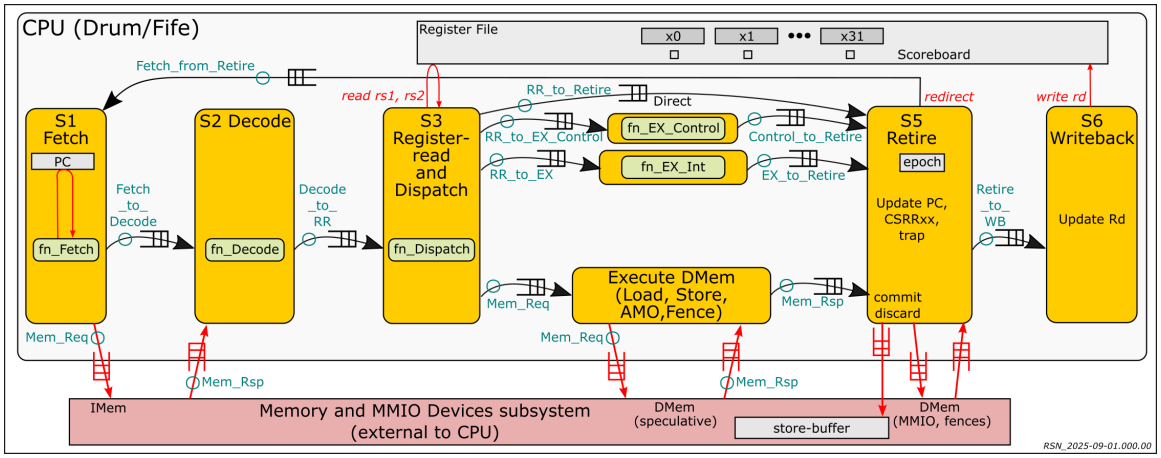


Figure 15.1: Pipelined interpretation of RISC-V instructions (Fig. 2.16 with some annotations)

step in a sequential process, for **Fife** we interpret each yellow box as containing its own infinite process. There are now half-a-dozen or more processes in the diagram (one for each yellow box), all running *concurrently*. Each black arrow in the diagram represents a flow of *messages* sent from one stage to another. These messages are sent via FIFO buffers (depicted by annotations in the figure).

Each stage is an infinite loop, consuming incoming messages and producing outgoing messages. Thus, while the Writeback stage is working on instruction n, the Retire, Execute, Register-Read, Decode and Fetch stage(s) may be working on instructions $n+1$, $n+2$, $n+3$, $n+4$, and $n+5$, respectively. Thus, there is a sequence, or train, of instructions flowing through the diagram from left to right.

Pipelining raises four new problems, and these are the focus of this chapter:

- Continuing to Fetch, with PC Prediction and Epochs
- Managing Register Read/Write Hazards with a Scoreboard
- Retiring outputs of the Execute Stages in Order with Tags
- Allowing Memory Ops to be Pipelined with a Store Buffer

15.2 Continuing to Fetch, with PC Prediction and Epochs

What should the Fetch stage do after issuing a request to IMem for instruction n? To issue another request, it needs to know the PC of instruction $n+1$, but there are several uncertainties about that next PC:

- The current Fetch itself can raise an exception (trap) if its PC is misaligned, is an unsupported memory address, *etc.* In this case the next PC will the trap-handler PC instead of the "normal" next PC.
- If it is a BRANCH instruction, until it reaches the Execute Control stage we do not know the branch target PC address, nor if the branch is taken or not.
- If it is a JAL or JALR instruction, until it reaches the Execute Control stage we do not know the target PC address.
- Many instructions can raise an exception (trap) (illegal instructions, BRANCH/JAL/JALR with misaligned target addresses, DMem ops with misaligned addresses or unsupported addresses, *etc.*); in these cases the next PC will the trap-handler PC instead of the "normal" next PC.
- The CPU may choose to respond to an external interrupt, in which case the next PC will be the interrupt-handler's address.

Note, the Fetch stage knows nothing about instruction n other than its PC. The instruction itself is not known until IMem sends its response to the Decode stage (and assuming the Fetch does not raise an exception).

15.2.1 PC Prediction in the Fetch Stage

A standard solution is for the Fetch stage to *predict* the next PC, *i.e.,* make a guess about the next PC. Since all RISC-V RV32 instructions are 32-bits wide (4 bytes), and *most* of them "fall-through" to the next adjacent instruction, a simple prediction is: PC+4. This prediction will be correct for most instructions, but will be wrong for BRANCH instructions that take the branch, for JAL/JALR instructions, and for any instruction that traps. When the prediction is wrong, the instructions that follow immediately are called "mispredicted" or "wrong-path" instructions.

NOTE:

RISC-V instructions are all 32-bits wide, so PC+4 is a reasonably good guess. In ISAs that have variable-length instructions, prediction may be more complicated. Even in RISC-V, when implementing the **C** (Compressed Instructions) extension, some instructions may be 16-bits wide, raising similar complications.

Earlier we said "the Fetch stage knows nothing about instruction n other than its PC". This is not strictly true–the CPU may have fetched this PC before (*e.g.,* this PC is inside a loop, or in a procedure that is called repeatedly). Knowledge of past behavior can improve the current prediction. Most predictors in modern processors use past history to improve and "tune" their branch predictors dynamically while executing the program. Designing good branch predictors is a deep topic for which there are many good textbooks (for example, [9]).

PC prediction can be seen as a kind of "machine learning". The CPU's past execution history constitutes the "training data" for a model, and the model is then asked to predict the next PC for the current PC.

15.2.2 Identifying and Flushing Wrong-path Instructions

Clearly, we need to identify and flush wrong-path instructions from the pipeline. Figure 15.2 shows the general scheme.

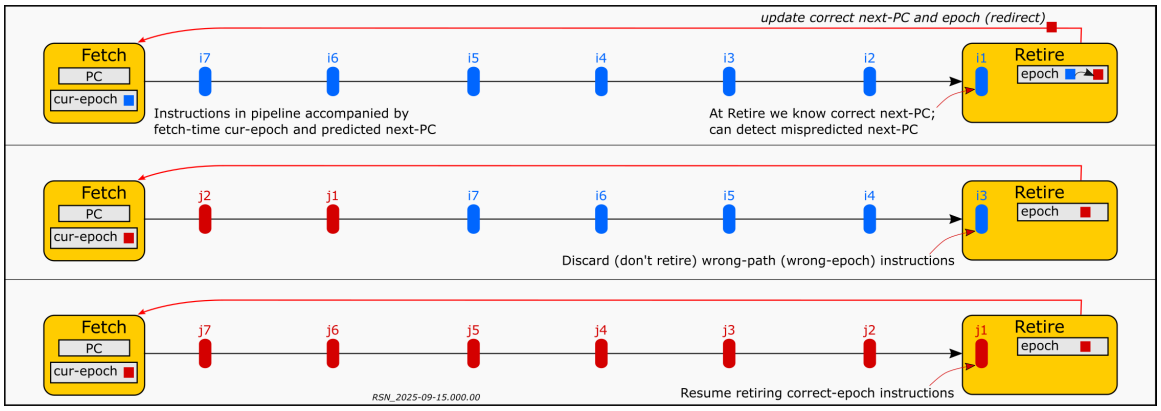


Figure 15.2: Using epochs to identify and discard mispredicted instructions

Suppose the Fetch stage issues requests for two instructions i_1 at address a_1 and i_2 at address a_2, where a_2 is predicted from a_1. When issuing the request for a_1, the Fetch stage can pass along a_2 to the Decode stage, from which point it can accompany i_1 as it journeys through the pipeline (*i.e.,* every instruction is accompanied by its next-PC prediction).

When i_1 reaches the Retire stage, we know the correct next PC (Trap handler PC? PC+4? Branch-taken target? JAL/JALR target?). By comparing this actual next PC with a_2, we know whether the successor to i_1 was predicted correctly or not.

If we find that the prediction was correct, there is nothing more to be done; we allow the pipeline flow to proceed (to retire i_2).

If we find that the prediction was wrong, then two things must happen:

- We need to *redirect* the Fetch stage to start fetching from the correct next-PC. This involves sending a message from the Retire stage back to the Fetch stage containing the correct next-PC. Suppose the first instruction fetched after this redirection is j_1.

- Instruction i_2, and possibly following instructions i_3, i_4, ... until j_1 are wrong-path instructions, and must be flushed.

When the Retire stage starts flushing wrong-path instructions i_2, i_3, i_4, ... how does it know when it has reached the end of the wrong-path sequence? In other words, how does it know when it sees j_1? This is precisely the purpose of the `rg_epoch` register shown in Figure 15.1.

Think of `rg_epoch` as a counter that continuously counts upward. Suppose the current value is e_1. As described above, when the Retire stage recognizes an instruction whose successor has been mispredicted, we send a redirection message to the Fetch stage with the corrected PC. The Retire stage also increments e_1 and sends the incremented value as part of the redirection. Each time the Fetch stage is redirected, it remembers the new epoch value. It also sends this value down the pipeline, accompanying each instruction fetched with this value.

Now, flushing wrong-path instructions in the Regire stage is easy: i_2, i_3, i_4, ... will be accompanied by the old epoch value e_1, whereas the first correct-path instruction j_1 will be accompanied by the new epoch value $e_1 + 1$. Thus, the Retire stage knows exactly which instructions are wrong-path and it can discard them.

EXERCISE: Please see Ex-15-A-Epoch-Bitwidth in Appendix E, page E-22. □

15.2.3 Terminology: Speculative Instructions and Commits

Before an instruction has reached the Retire stage, there is always the possibility that an earlier instruction that is ahead of it in the pipeline will branch/jump to a PC that was not predicted, or will trap, making this instruction irrelevant. Until this moment, we say that the instruction is still "*speculative*". When it reaches the Retire stage, we say that it's side-effects can be "*committed*".

15.2.4 Speculative instructions should not have any side-effects

It is not enough for the Retire stage just to discard mispredicted instructions. Instructions have side-effects: they may modify registers and write to memory. We must ensure that speculative instructions make no modifications that are visible to right-path instructions that follow, until they reach the Retire stage. The details of how this is accomplished will be seen in Section 15.5 and Section 15.6.

15.3 Managing Register Read/Write Hazards with a Scoreboard

Suppose instruction i_1 writes to register x_7, and the following instruction i_2 reads from register x_7. Instruction i_1's write to x_7 only happens in the Writeback stage. If i_2 were to follow behind i_1 immediately, it will be in the Exec stage, and would have already read x_7

earlier when it was in the Register-Read stage. In other words, it would have read a *stale* or obsolete value x_7. This is called a Read-Write *hazard*, or a read-after-write *dependency*.

In this situation, we need to make i_2 wait in the Register-Read stage until i_1 has completed its update of x_7. This is precisely the purpose of the `scoreboard` shown in Figure 15.1. The general scheme is shown in Figure 15.3.

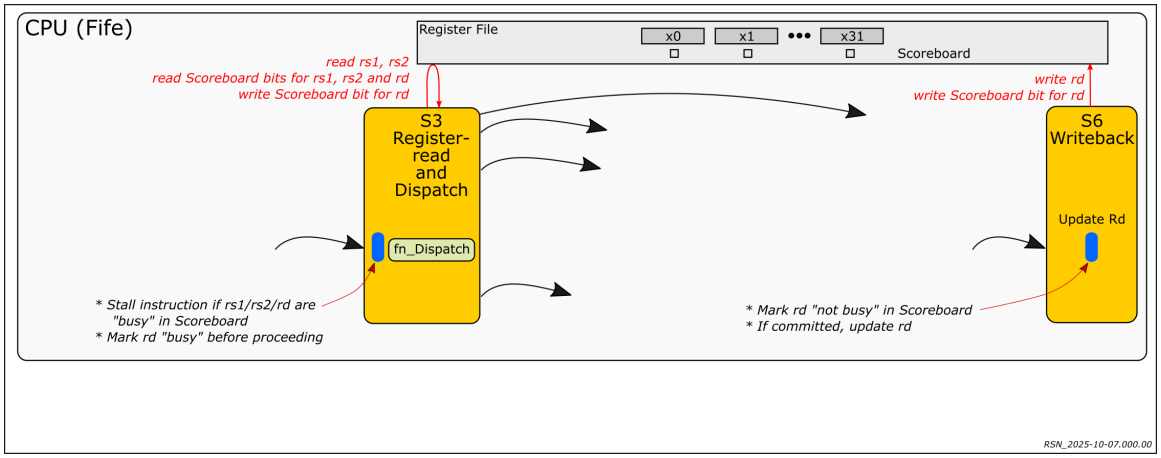


Figure 15.3: Using a scoreboard to manage register read/write hazards

The `scoreboard` is an array of 32 1-bit registers (one bit for each GPR). When an instruction (such as i_1) passes through the Register-Read stage, if it writes to register x_7, we set the corresponding bit 7 in the scoreboard to 1, indicating that x_7 is "busy". When i_1 reaches the Writeback stage and writes to the register, it also resets the scoreboard bit 7 to 0, indicating that x_7 is "not busy".

When an instruction (such as i_2) reaches the Register-Read stage and wants to read a register (such as x_7), if the corresponding scoreboard bit says it is busy, then the Register-Read stage "*stalls*", *i.e.,* it waits until the scoreboard condition is cleared (by i_1 in the Writeback stage).

While i_2 is waiting in the stalled Register-Read stage, note that the following Execute stage may become "empty", *i.e.,* there is no instruction occupying that stage. We refer to this as a "*pipeline bubble*".

EXERCISE: Please see Ex-15-B-Hazards in Appendix E, page E-22. □

EXERCISE: Please see Ex-15-C-Write-Write-Hazards in Appendix E, page E-22. □

EXERCISE: Please see Ex-15-D-Write-Write-ordering in Appendix E, page E-23. □

15.3.1 Releasing Scoreboard Reservations for Uncommitted Instructions

When the Writeback stage discards an instruction due to mis-speculation, that instruction may be one which normally would have written a result value into register x_j in the register file. If so, when it passed through the Register-Read-and-Dispatch stage, it would have marked register x_j as "busy" in the scoreboard. Now, when we discard the instruction, even though we do not write any value into register x_j, we still need to release the "busy" reservation on x_j (otherwise, any succeeding instruction trying to read x_j will get stuck in the Register-Read stage. Said another way, the reservation in the scoreboard is a side-effect of this instruction that needs to be rolled back.

15.3.2 Bypassing

Digital hardware usually runs in time units of "clock cycles". The Writeback stage writes a GPR (possibly) and writes the scoreboard (to mark it "not-busy"). The Register-Read stage reads zero to two GPRs, reads the scoreboard (to check "not-busy") and writes the scoreboard (to set "busy").

For ordinary registers a write is only visible on the next clock cycle. Thus, if the register file and the scoreboard registers are just ordinary registers, the Register-Read stage cannot observe "not-busy" or the new register value until one clock after the Writeback stage performed its writes. This does not affect correctness, but slows the performance of the CPU.

It is possible to design some extra circuitry around the scoreboard and register file so that the Register-Read stage can observe "not-busy" and the new register value on the *same* clock cycle as when Writeback performs its writes. This technique is generically called "*bypassing*" or "*short-circuiting*".

EXERCISE: Please see Ex-15-E-Bypassing in Appendix E, page E-23. □

An even more advanced form of bypassing (with much more circuit complexity) would be:

- Eliminate the scoreboard; do not stall an instruction in the Register-Read stage, but allow it to move into its appropriate Execute stage, and stall it there if necessary. This frees up the Register-Read stage to process the next instruction, which may move into a different Execute stage.
- When Writeback writes a register value, broadcast it to the different Execute stages to enable instructions there that are stalled on this register value.

15.4 Retiring outputs of the Execute Stages in Order with Tags

As shown in Figure 15.4, each yellow box in the Execute stage is an independent pipeline handling a certain subset of the instruction set. For example, "Execute Control" handles BRANCH, JAL and JALR instructions. "Execute Integer Arithmetic and Logic Ops" handles LUI, AUIPC, and all arithmetic and logic instructions. "Exec Mem Op" handles LOAD

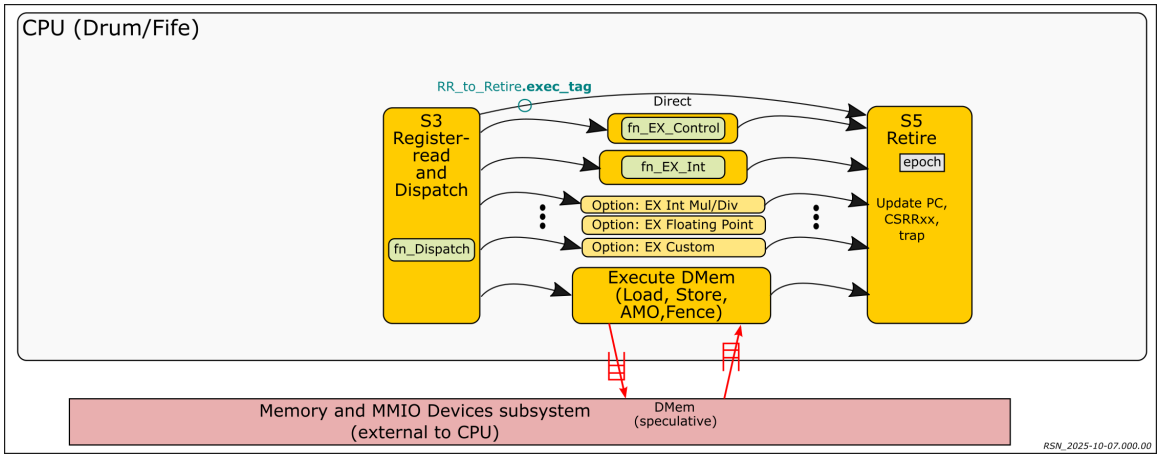


Figure 15.4: Multiple concurrent pipelines in the Execute stage, of varying latencies

and STORE instructions. If we extend **Fife** to handle the **M** ISA extension, we would have a pipeline for integer multiply and divide instructions. If we extend **Fife** to handle the **F** and **D** ISA extension, we would have a pipeline for floating point arithmetic. The Register-Read-and-Dispatch stage sends information into these pipes depending on the kind of instruction.

Instructions may have different latencies in traversing these Execute pipes. For example, Control and Integer ops may typically traverse in one clock, but multiplication, division, floating point and memory ops may take more clocks. The latency variation may be data dependent: for example multiplication/division may recognize the special case where an operand is 0 or 1 and return a result quickly. A memory op may return quickly on a cache hit, and take more time on a cache miss.

The Retire stage needs to gather the outputs from the Execute stages and retire them in the proper order. But, because of varying latency, availability of data is not an indication of the proper order.

The solution to this "ordering" problem is *tags*. In Figure 15.1 we see there is also a *direct path* from Register-Read-and-Dispatch to Retire. We pass a tag on this path *for every instruction*. For example if the instruction is a BRANCH instruction, the Register-Read-and-Dispatch stage sends information into Execute Control, but it also sends a tag `EXEC_TAG_CONTROL` on the direct path to Retire, indicating that it has just dispatched an instruction into Execute Control.

Thus, the sequence of tags on the direct path tells Retire exactly the order in which to service the various Execute pipes. Retire always looks at tag on the direct path first. For example, if Retire sees a `EXEC_TAG_DMEM` tag on the direct path, it knows that it must next look for an output from the Exec Memory Ops pipe, even if outputs are already available on the Execute Control and/or Execute Integer pipes from later instructions.

15.4.1 Full out-of-order execution and reorder buffers

The multiple concurrent execute pipelines in Figure 15.4 are independent and, because they may vary in complexity and latency, may execute instructions "out-of-order" with respect to each other. However, we assume that in each of the execute pipelines, the

instructions flowing through that pipeline are executed (and complete) in order (FIFO-like ordering). Because of this, our simple tagging scheme in the Retire stage is adequate to retire instructions across the execute pipelines in the correct order.

Other microarchitecture designs use something called a "reorder buffer" (ROB), which can be regarded as an array of slots for completed instructions in the proper order. Each slot is initially marked "empty". Every instruction has a unique ordering number which is mapped to a particular index in the ROB. When its execution is complete, it is placed in the ROB at the correct index and marked "complete". A separate process drains the ROB sequentially, waiting if necessary for the next slot to reach the completed state.

15.5 Allowing Memory Ops to be Pipelined, with a Store Buffer

Consider the Execute DMem (memory ops) stage in Figure 15.5, which issues LOAD and

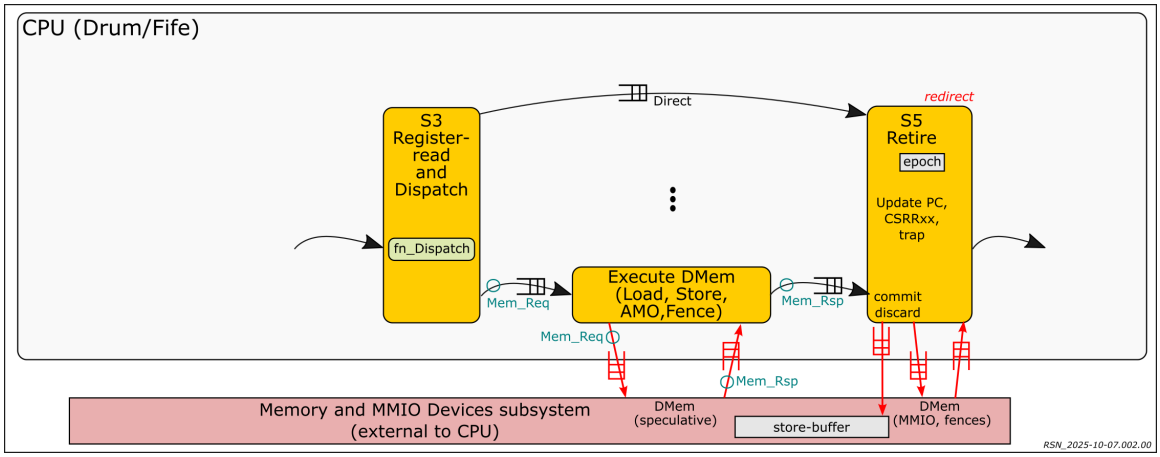


Figure 15.5: Speculative and non-speculative DMem access, with a Store Buffer

STORE requests to memory and collects their responses. At this stage, the instruction is still speculative; it may be discarded when it reaches the Retire stage. We must ensure that STORE instructions do not yet modify memory permanently.

The mechanism for this purpose is the "*store buffer*" shown in the diagram. This is a buffer in front of the memory system (between the CPU and the memory system).

- The store buffer is itself a queue.
- When we execute a STORE instruction, the address, data and size are appended to the STORE buffer queue. When the instruction finally reaches the Retire stage, the Retire stage sends a final "commit/discard" message to the store-buffer. If a commit, the STORE at the head of the store-buffer is committed to memory and we dequeue it from the store-buffer. If a discard, we just dequeue it from the store-buffer.
- When we execute a LOAD instruction, we first check the store-buffer if there have been any recent updates to the address in question, and then go to the memory system behind it if necessary.

15.5.1 What about LOADs and STOREs to non-memory-like devices (MMIO)?

In RISC-V there are no separate instructions for input and output to devices. Devices contain "device registers" which are placed at particular "memory addresses" and are accessed from the CPU just like memory, with LOAD and STORE instructions. We say that the device registers are "mapped" to those addresses. These accesses can control and configure the device, and move data between the CPU and the device. Such a scheme is known as MMIO, for Memory-Mapped Input-Output.

Device registers, although addressed like memory locations, may behave quite differently from memory, in several ways:

- LOADs may have side-effects. For example, a LOAD from a memory location does not (observably) disturb anything, but a LOAD from a device could switch on an LED, or increment a counter.
- LOADs may not be idempotent (two identical LOADs cannot be coalesced into one). For example, two successive LOADs from a memory location return the same value, whereas two successive LOADs from, say, a UART's "receiver buffer register" may return two successive (and different) characters from a keyboard.
- STOREs may have additional side-effects. A STORE to a memory location merely stores the value there. A STORE to a device register may display it on a screen, start a motor, release a wheel brake, and so on.
- STOREs may not be idempotent (two identical STOREs cannot be coalesced into one), because each may have its own side-effect.
- In memory, a LOAD returns the value stored there by the most recent STORE. For a UART device, on the other hand, a STORE may display a character on a screen whereas a subsequent LOAD from the same address may return a character from the keyboard.

For these reasons, it is dangerous to perform any LOAD or STORE speculatively on non-memory devices. The "Execute Memory Ops" stage does not even attempt the access; it simply defers such a request for future execution by the Retire stage. The decision whether to defer a request or not is based on the address (each address is for memory or .

Once the LOAD/STORE instruction has reached the Retire stage and we know for sure that it is not speculative, the Retire stage performs the memory operation: it sends the request to memory, collects the response, and retires the instruction.

EXERCISE: Please see Ex-15-F-Store-Buffers in Appendix E, page E-23. □

15.6 The Retire Stage

Figure 15.6 summarizes the actions to be taken by **Fife**'s Retire stage.

Much of this is the same as in Figure 10.2 for **Drum**. The new additional details are:

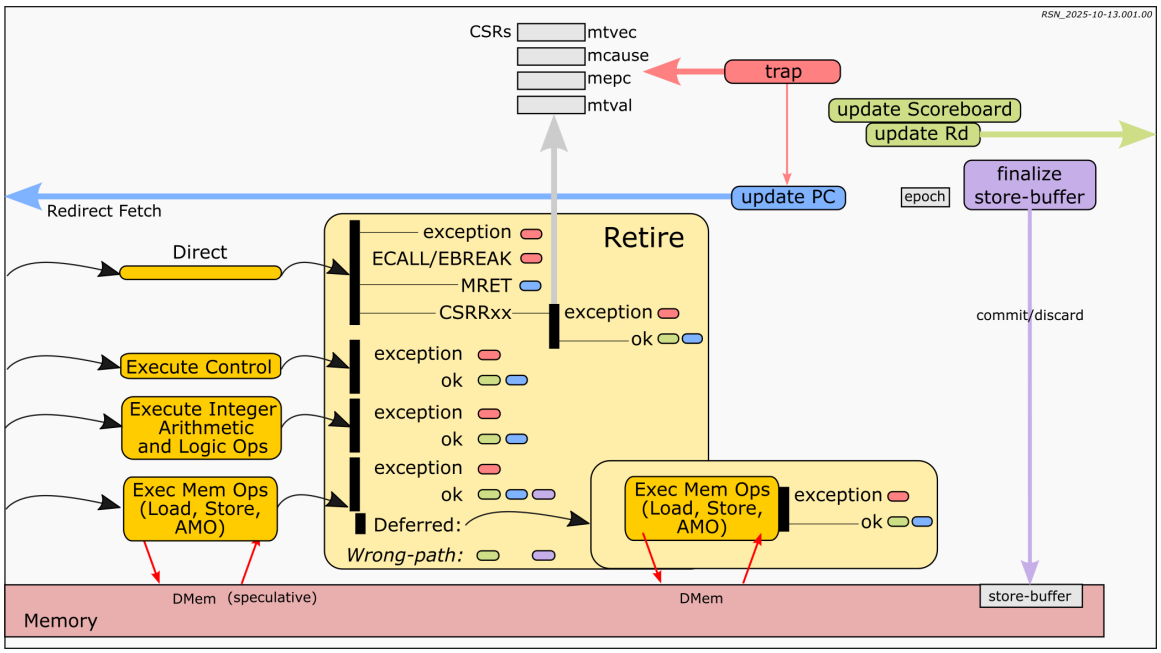


Figure 15.6: Retire actions in **Fife**

- The "update PC" operation now sends a message to redirect Fetch *only* if the successor to the current PC had been mispredicted. Recall that we carry the predicted PC value along with the instruction through the pipe, so here we can compare the now-known actual next PC with the predicted PC.

  If we redirect, we increment the `epoch` number and send the new epoch number along with the redirection. Any subsequent instructions with the old epoch number are "wrong path" instructions and must be discarded.

- Wrong path: any of the Execute pipes can produce a wrong-path instruction (accompanying epoch does not match current epoch). In each such case, we discard the instruction, but there are two more needed actions:
  - If the instruction has an `Rd`, we would have taken a scoreboard reservation for it in the Register-Read stage; we must now release that scoreboard reservation. We extend `update_rd` to perform a scoreboard-release without a register-write.
  - In the case of a non-deferred STORE instruction, we would have placed an entry in the store-buffer. We use "finalize store buffer" to discard that entry.

- Correct path non-deferred STORE instruction: we would have placed an entry in the store-buffer. We use "finalize store buffer" to commit that entry to memory.

- "Deferred" memory request from DMem (because it was for an non-memory-like address such as MMIO): If so, the second Exec Mem Ops box in the figure now performs the DMem operation.

Chapter 16

RISC-V: the Fife pipelined CPU code

16.1 Introduction

In this chapter we study BSV code to implement the principles that were discussed in Chapter 15. We repeat Figure 15.1 here, for reference.

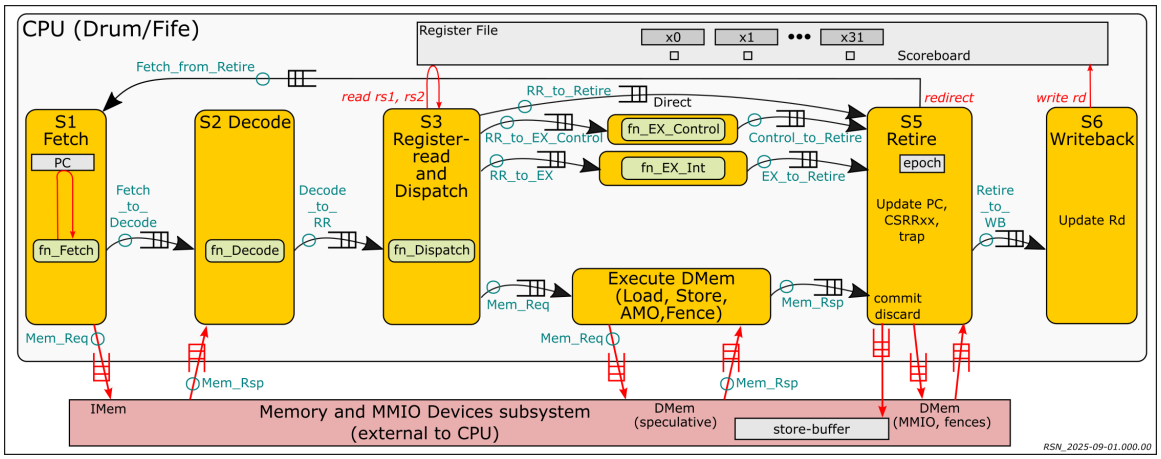


Figure 16.1: Pipelined interpretation of RISC-V instructions (Fig. 2.16 with some annotations)

16.2 The Fife top-level CPU module

The code for the top-level **Fife** CPU module is actually simpler than the code for the **Drum** CPU module, because it simply instantiates sub-modules for each stage and connects them:

src_Fife/CPU.bsv: line 51 ...

```
1 (* synthesize *)
2 module mkCPU (CPU_IFC);
3    // ================================================================
4    // STATE
5    ...
6    Fetch_IFC       stage_F          <- mkFetch;
```

```
7     Decode_IFC      stage_D          <- mkDecode;
8     RR_WB_IFC       stage_RR_WB      <- mkRR_WB;
9     EX_Control_IFC  stage_EX_Control <- mkEX_Control;  // Branch, JAL, JALR
10    EX_Int_IFC      stage_EX_Int     <- mkEX_Int;      // Integer ops
11    Retire_IFC      stage_Retire     <- mkRetire;
12
13    // ----------------
14    // Forward flow connections
15
16    // Fetch->Decode->RR-Dispatch, and direct path RR-Dispatch->Retire
17    mkConnection (stage_F.fo_Fetch_to_Decode,  stage_D.fi_Fetch_to_Decode);
18    mkConnection (stage_D.fo_Decode_to_RR,     stage_RR_WB.fi_Decode_to_RR);
19    mkConnection (stage_RR_WB.fo_RR_to_Retire, stage_Retire.fi_RR_to_Retire);
20
21    // RR-Dispatch->various EX
22    mkConnection (stage_RR_WB.fo_RR_to_EX_Control,
23                  stage_EX_Control.fi_RR_to_EX_Control);
24    mkConnection (stage_RR_WB.fo_RR_to_EX_Int,
25                  stage_EX_Int.fi_RR_to_EX_Int);
26
27    // Various EX->Retire
28    mkConnection (stage_EX_Control.fo_EX_Control_to_Retire,
29                  stage_Retire.fi_EX_Control_to_Retire);
30    mkConnection (stage_EX_Int.fo_EX_Int_to_Retire,
31                  stage_Retire.fi_EX_Int_to_Retire);
32
33    // ----------------
34    // Backward flow connections
35
36    // Fetch<-Retire (redirection)
37    mkConnection (stage_Retire.fo_Fetch_from_Retire, stage_F.fi_Fetch_from_Retire);
38    // RR-Dispatch<-Retire (register writeback)
39    mkConnection (stage_Retire.fo_RW_from_Retire, stage_RR_WB.fi_RW_from_Retire);
40
41    // ================================================================
42    // BEHAVIOR: all normal running behavior is inside the above modules
43
44    // ****************************************************************
45    // BEHAVIOR: Debugger support
46
47    `include "CPU_Dbg.bsv"
48
49    // ****************************************************************
50    // INTERFACE
51
52    method Action init (Initial_Params initial_params);
53       ...
54       stage_F.init (initial_params);
55       stage_D.init (initial_params);
56       stage_RR_WB.init (initial_params);
57       stage_EX_Control.init (initial_params);
58       stage_EX_Int.init (initial_params);
59       stage_Retire.init (initial_params);
60
61       $display ("================================================================");
62       $display ("%s: starting execution at PC %0h",
63                 cpu_name, initial_params.pc_reset_value);
64       ...
65    endmethod
```

```
66
67    // IMem
68    interface fo_IMem_req = stage_F.fo_Fetch_to_IMem;
69    interface fi_IMem_rsp = stage_D.fi_IMem_to_Decode;
70
71    // DMem, speculative
72    interface fo_DMem_S_req    = stage_RR_WB.fo_DMem_S_req;
73    interface fi_DMem_S_rsp    = stage_Retire.fi_DMem_S_rsp;
74    interface fo_DMem_S_commit = stage_Retire.fo_DMem_S_commit;
75
76    // DMem, non-speculative
77    interface fo_DMem_req = stage_Retire.fo_DMem_req;
78    interface fi_DMem_rsp = stage_Retire.fi_DMem_rsp;
79
80    // Set TIME
81    method Action set_TIME (Bit #(64) t) = stage_Retire.set_TIME (t);
```

This is practically a direct textual description of Figure 16.1. The STATE section first instantiates the pipeline stages shown in the figure. There is no explicit module corresponding to Execute Memory Ops—the DMem request is sent out directly from `stage_RR_RW` and the DMem response is collected directly by `stage_Retire`.

The STATE section then instantiates the "forward-flow" connections between modules (left to right in the figure), using the *bsc* library module `mkConnection` to connect a `FIFOF_O` interface (producer) to a `FIFOF_I` interface (consumer), which was discussed in Section 7.5.6. These module connections are in the STATE section because, as discussed in Section 7.5.6 `mkConnection` is just a module instantiation.

The next few lines instantiate the "backward-flow" connections.

In the INTERFACE section, after the `init` method, the next two lines are the flows of IMem requests from the Fetch stage to memory and IMem responses from memory to the Decode stage. These just lift interfaces from `stage_F` and `stage_D` to the CPU interface, as is.

The next three lines are for *speculative* DMem access, which we discussed in Section 15.5: the flow of DMem requests from `stage_RR` to memory, the flow of DMem responses from memory to `stage_Retire`, and the flow of "commit/discard" messages from `stage_Retire` to the store-buffer to discharge STOREs that are waiting in the store-buffer.

The last two lines are for *non-speculative* DMem access, which we discussed in Section 15.5.1.

Note that the module interface `CPU_IFC` is exactly the same as in **Drum** (although **Drum** has no need for, and does not use the `DMem_S` speculative interfaces). Thus, in a system context, we can directly substitute **Drum** for **Fife** and vice versa. Generalizing this idea, we can develop other CPUs and substitute them, as well.

We next go through the code for the individual stage modules.

16.3 How we connect stages

Section 7.5.7 discussed a technique to connect FIFO-like connections between separately compiled stage modules (we repeat Figure 7.2 here for reference):

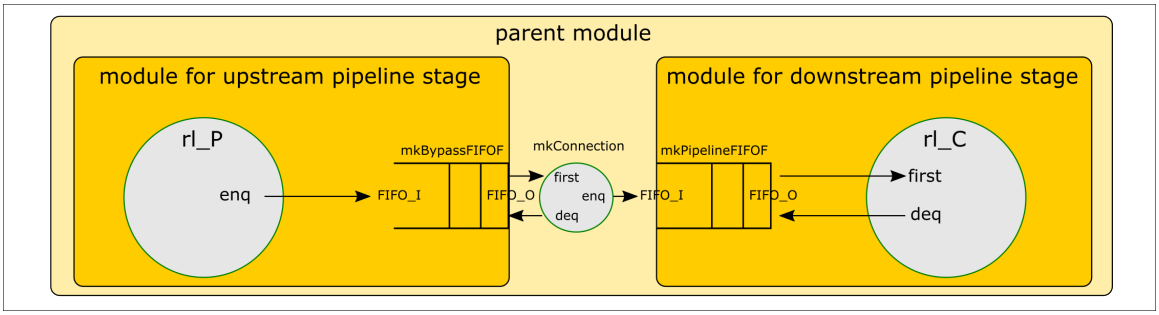


Figure 16.2: How we connect **Fife** stages

- Instantiate a `mkBypassFIFOF` module in the producer stage module; use its `FIFOF_I` semi-FIFO interface internally, and lift its `FIFOF_O` semi-FIFO interface to the stage module's interface.
- Instantiate a `mkPipelineFIFOF` module in the consumer stage module; use its `FIFOF_O` semi-FIFO interface internally, and lift its `FIFOF_I` semi-FIFO interface to the stage module's interface.
- In the parent module (`mkCPU`), use `mkConnection` to connect the lifted `FIFOF_O` and `FIFOF_I` interfaces.

The benefits of this technique are discussed in detail in Section 18.5. Suffice it to say, here, that:

- Despite there being two FIFOs, data can traverse from producer to consumer in one clock tick, as desired.
- The structure allows the producer and consumer to be compiled independently by *bsc*, with no "rule-scheduling" constraints leaking across stage boundaries.
- There are no combinational paths crossing the stage boundary (through the two FIFOs).
- The structure allows us to reason about (and prove) correctness of each stage completely independently of other stages.

All of these are pleasing "modularity" properties of the design.

16.4 The Fetch stage

The Fetch stage module interface is shown below. Apart from the `init` method, the remaining sub-interfaces correspond to the FIFO-labeled arrows in Figure 16.1: the outgoing `FIFOF_O` interfaces to memory (IMem) and the Decode stage, and the incoming `FIFOF_I` for redirections from the Retire stage.

src_Fife/S1_Fetch.bsv: line 32 ...

```
1  interface Fetch_IFC;
2     method Action init (Initial_Params initial_params);
3
4     // Forward out
5     interface FIFOF_O #(Fetch_to_Decode)  fo_Fetch_to_Decode;
6     interface FIFOF_O #(Mem_Req)          fo_Fetch_to_IMem;
7
```

```
   // Backward in
   interface FIFOF_I #(Fetch_from_Retire) fi_Fetch_from_Retire;
endinterface
```

The Fetch stage module code is shown below.

src_Fife/S1_Fetch.bsv: line 53 ...

```
(* synthesize *)
module mkFetch (Fetch_IFC);
   // ----------------------------------------------------------------
   // STATE
   ...
   Reg #(S1_RunState) rg_runstate <- mkReg (S1_HALTED);

   // Forward out
   FIFOF #(Fetch_to_Decode) f_Fetch_to_Decode <- mkBypassFIFOF;
   FIFOF #(Mem_Req)         f_Fetch_to_IMem   <- mkBypassFIFOF;

   // Backward in
   FIFOF #(Fetch_from_Retire) f_Fetch_from_Retire <- mkPipelineFIFOF;

   // PC and epoch registers
   Reg #(Bit #(XLEN))     rg_pc        <- mkReg (0);
   Reg #(Bit #(W_Epoch))  rg_epoch     <- mkReg (0);
   ...
   // ----------------------------------------------------------------
   // BEHAVIOR

   // Forward flow
   rule rl_Fetch_req ((rg_runstate == S1_RUNNING)
                      && (! f_Fetch_from_Retire.notEmpty)
                      ...
      // Predict next PC
      let pred_pc = rg_pc + 4;
      let y <- fn_Fetch (rg_pc, pred_pc, rg_epoch, rg_inum, rg_arch_inum, rg_flog);
      f_Fetch_to_Decode.enq (y.to_D);
      f_Fetch_to_IMem.enq (y.mem_req);

      rg_pc        <= pred_pc;
      ...
   endrule

   // Backward flow: redirection from Retire
   rule rl_Fetch_from_Retire ((! rg_oiaat) || (! rg_oiaat_fetch));
      let x <- pop_o (to_FIFOF_O (f_Fetch_from_Retire));
      rg_pc        <= x.next_pc;
      rg_epoch     <= x.next_epoch;
      ...
   endrule

   // ----------------------------------------------------------------
   // INTERFACE

   method Action init (Initial_Params initial_params) if (rg_runstate == S1_HALTED);
      rg_pc       <= initial_params.pc_reset_value;
      rg_runstate <= S1_RUNNING;
      ...
   endmethod

```

```
53     // Forward out
54     interface fo_Fetch_to_Decode = to_FIFOF_O (f_Fetch_to_Decode);
55     interface fo_Fetch_to_IMem   = to_FIFOF_O (f_Fetch_to_IMem);
56
57     // Backward in
58     interface fi_Fetch_from_Retire = to_FIFOF_I (f_Fetch_from_Retire);
59  endmodule
```

The STATE section instantiates various registers and FIFOs. Ignore `rg_runstate` for now, it is only used by a hardware debugger for halting the CPU.

Next, the BEHAVIOR section contains two *rules* (Chapter 13). In rule `rl_Fetch_req`, the explicit condition is the expression:

```
(rg_running && (! f_F_from_Retire.notEmpty))
```

The implicit condition of rule `rl_Fetch_req`'s comes from the methods that it invokes, namely `f_Fetch_to_Decode.enq ()` and `f_Fetch_to_IMem.enq ()`. The rule will fire only when the explicit condition is true, and when both FIFOs are enabled to enqueue (have space). When it fires, it performs the composite action that comprises all the actions in the rule body:

- enqueue the value of `y.to_D` into FIFO `f_Fetch_to_Decode`;
- enqueuethe value of `y.mem_req` into FIFO `f_Fetch_to_IMem`;
- write the predicted PC value `pred_pc` into register `rg_pc`, and

The right-hand side expressions (`rg_pc+4` and `fn_Fetch(...)` are all combinational circuits, and "`y`" is not a register, just a name for the wires carrying the output value of `fn_Fetch()`.

NOTE:

> The function `fn_Fetch()` is exactly the same as the one used in the Fetch stage of **Drum** (was described in Section 6.2).
>
> The types of the messages passed to the Decode stage (`y.to_D` of type `F_to_D`) and to memory (`y.mem_req` of type `Mem_Req`) are the same as in **Drum**.

All these actions are semantically *instantaneous* and *simultaneous*. Note that when the rule's implicit and explicit conditions are true, all the actions are peformed; if false, none of them are performed, *i.e.,* the rule is "atomic".

In summary, rule `rl_Fetch_req` computes an IMem memory request from the PC and sends it to memory; it sends auxiliary information to the Decode stage; and it updates the PC and inum in preparation for the next Fetch (the next firing of the rule)..

The second rule in the module is `rl_Fetch_from_Retire`. It receives, in `x`, a redirection message from the Retire stage, and updates the PC and epoch accordingly. This rule has no explicit conditions; its single implicit condition comes from `f_Fetch_from_Retire`'s implicit condition that we cannot pop a value from the FIFO until it is non-empty, *i.e.,* this rule only fires when a redirection message is available. When it fires, it performs three actions atomically/instantaneously/simultaneously:

- It dequeues `x` from the FIFO `f_F_from_Retire` (the dequeue action is inside the `pop_o` function),

- It updates `rg_pc` with the new PC in the redirection message,
- It updates `rg_epoch` with the new epoch in the redirection message.

Note, `rl_Fetch_from_Retire` updates two registers `rg_pc` and `rg_epoch` and, *concurrently*, `rl_Fetch` reads both those registers. Because rule actions are atomic, we are guaranteed that `rl_Fetch` will not see inconsistent values in those two registers, where one has been updated but the other has not yet been updated.

Finally, the INTERFACE section of the module is simple. After the `init` method, we simply lift the FIFO interfaces to the `mkFetch` module interface.

16.4.1 Prioritizing rule `rl_Fetch_from_Retire` over `rl_Fetch_req`

What would happen if we omit the condition "`(!f_Fetch_from_Retire.notEmpty)`" in `rl_Fetch_req`? It would not affect the correctness of this module at all. If we omit the condition, then both rules could be enabled simultaneously, and it is non-deterministic which one will fire first. Actually, the hardware is deterministic because the *bsc* compiler will make a prioritization choice, but its choice is unspecified, and so semantically it is non-deterministic (*e.g.,* a new release of the compiler, or a later change in other code in the module may prioritize differently). But, for correctness, the choice *does not matter*. Because of the atomicity of rules, the rules will never *interleave* their behavior—either one goes before the other, or vice versa. We can quickly reason that, in either case, we get functionally correct behavior.

The only difference will be in performance: if both rules are enabled, then, by definition, `rl_Fetch` is fetching "wrong-path" instructions due to an earlier misprediction (which is why we have a redirection). We know that wrong-path instructions will be discarded and have no effect. So if `rl_Fetch` is prioritized, it will fetch one more wrong-path instruction, with no impact on correctness.

By adding the "`(!f_Fetch_from_Retire.notEmpty)`" condition to `rl_Fetch`, we are explicitly prioritizing `rl_Fetch_from_Retire` when both are enabled, thus avoiding `rl_Fetch_req` performing a wasted wrong-path fetch.

16.5 The Decode stage

The Decode stage module interface is shown below. Apart from the `init` method, the remaining sub-interfaces correspond to the FIFO-labeled arrows in Figure 16.1: the incoming `FIFOF_I` interfaces from the Fetch stage and memory (IMem), and the outgoing `FIFOF_O` interface to the Register-Read stage.

src_Fife/S2_Decode.bsv: line 32 ...

```
1  interface Decode_IFC;
2     method Action init (Initial_Params initial_params);
3
4     // Forward in
5     interface FIFOF_I #(Fetch_to_Decode)  fi_Fetch_to_Decode;
6     interface FIFOF_I #(Mem_Rsp)          fi_IMem_to_Decode;
7
```

```
8     // Forward out
9     interface FIFOF_O #(Decode_to_RR)  fo_Decode_to_RR;
10    endinterface
```

The Decode stage module code is shown below.

src_Fife/S2_Decode.bsv: line 47 ...

```
1     (* synthesize *)
2     module mkDecode (Decode_IFC);
3        // ================================================================
4        // STATE
5        ...
6        // Forward flows in
7        // Depth should be > F=>IMem=>D path latency
8        FIFOF #(Fetch_to_Decode)  f_Fetch_to_Decode <- mkSizedFIFOF (4);
9        FIFOF #(Mem_Rsp)          f_IMem_to_Decode  <- mkPipelineFIFOF;
10
11       // Forward flow out
12       FIFOF #(Decode_to_RR) f_Decode_to_RR <- mkBypassFIFOF;
13
14       // ================================================================
15       // BEHAVIOR
16
17       rule rl_Decode (! f_Fetch_to_Decode.first.halt_sentinel);
18          Fetch_to_Decode  x        <- pop_o (to_FIFOF_O (f_Fetch_to_Decode));
19          Mem_Rsp          rsp_IMem <- pop_o (to_FIFOF_O (f_IMem_to_Decode));
20
21          Decode_to_RR y <- fn_Decode (x, rsp_IMem, rg_flog);
22
23          f_Decode_to_RR.enq (y);
24          ...
25       endrule
26       ...
27       // ================================================================
28       // INTERFACE
29
30       method Action init (Initial_Params initial_params);
31          rg_flog <= initial_params.flog;
32       endmethod
33
34       // Forward flows in
35       interface fi_Fetch_to_Decode = to_FIFOF_I (f_Fetch_to_Decode);
36       interface fi_IMem_to_Decode  = to_FIFOF_I (f_IMem_to_Decode);
37       // Forward flows out
38       interface fo_Decode_to_RR = to_FIFOF_O (f_Decode_to_RR);
39    endmodule
```

The STATE section instantiates FIFOs for incoming and outgoing flows.

The BEHAVIOR section has the single rule `rl_Decode`, whose implicit conditions will make it wait for both incoming FIFOs `f_Fetch_to_Decode` and `f_IMem_to_Decode` to be non-empty, and for its outgoing FIFO `f_Decode_to_RR` to have space. When the rule fires, it:

- pops `x` and `rsp_Mem` from the two FIFOs, respectively;
- applies function `fn_Decode()` to those values (this is the *same* `fn_Decode()` that was used in the Decode stage of **Drum**, and described in Section 6.3), and

- sends the result y of type `Decode_to_RR` on to the Register-Read stage.

Note that `fn_Decode`'s `Decode_to_RR` output has a boolean `has_rd` field which is:

- false for those instructions that do not have an rd field;
- true for those instructions that do have an rd field, *and where* `rd` *is not zero.*

The INTERFACE section is again straightforward, just lifting the FIFO interfaces to this module's interface.

16.5.1 Balancing concurrent paths in a pipeline

In Figure 16.1 we see that there are two concurrent FIFO-like paths from Fetch to Decode: one direct, and the other via memory (IMem). Fetch produces two outputs together, one for each path. The paths re-converge at Decode, where Fetch's first output "meets" the corresponding response from memory.

The path via IMem is also FIFO-like because good memory systems are themselves pipelined; it is possible for Fetch to issue several consecutive memory requests before the first response arrives at Decode. We say that multiple IMem transactions may be "in flight". Also, in Section 1.5.4.1 we discussed how memory latency can be variable and unpredictable, so the number of transactions in flight may vary.

If we wish to allow up to n IMem transactions in flight, then the direct Fetch-to-Decode path must be capable of buffering up to n `Fetch_to_Decode` items. If not, rule `rl_Fetch` in Fetch will get stuck: its `f_Fetch_to_Decode` FIFO will be full, and so the "`enq`" method's implicit condition will become false, preventing the rule from firing.

This is the reason, in Decode, the incoming `f_Fetch_to_Decode` FIFO is instantiated with `mkSizedFIFOF(4)`, which is a FIFO with the capacity to hold four items[1] We also use the term "balancing" for this—the number of items that can be "in flight" in the two paths should be matched.

Why did we choose the number 4? Why not something smaller, or larger? Greater capacity requires more hardware resources, which argues for a smaller number, but a smaller number may affect performance where Fetch has to wait only because of limited capacity. We would like to choose the smallest number that covers the number of IMem transactions that may be in flight (memory latency, or depth of IMem pipeline). But that number varies with memory system design and, even for a particular design, it varies over time—it depends on the particular RISC-V program running, the memory system's caches, hits and misses, cache interactions, cache miss latencies, and so on[2] Thus, there is no obvious "optimal" choice for `mkSizedFIFOF` capacity; we must "tune" the number by running the CPU on desired applications, with different FIFO capacities, observe the effect on performance and resources, and pick an "acceptable" capacity.

[1] We could also have, equivalently, instantiated the `f_Fetch_to_Decode` FIFO in Fetch with `mkSizedFIFO`, or split the buffering capacity between the two stages.

[2] With virtual memory, it will also depend on TLB misses and the latency of page table walks.

16.6 The Register-Read and Dispatch (and Register-Write) stage

The Register-Read and Dispatch module contains (instantiates) the GPRs register file (Section 8.2) and the scoreboard to manage read-write hazards (Section 15.3). In the forward flow:

- It stalls (waits) if the instruction has Rs1, Rs2 or Rd, and these are busy according to the scoreboard.
- It reads Rs1 and Rs2 registers for the current instruction.
- It sets the scoreboard for the current instruction's Rd to 1, marking it "busy" (if the instruction has an Rd).
- It uses information from Decode to dispatch to the four Execute pipes. We always (for every instruction) send an execution tag and additional information on the direct pipe. Depending on the instruction, it may also send information into one of the other Execute pipes:
  - Execute Control
  - Execute Integer
  - memory (a DMem request)

This stage also participates in a backward flow, because the GPRs register file and scoreboard are instantiated here. When an instruction is completed, the Retire stage sends a message to this module to update those components: release an rd reservation in the scoreboard and write-back an rd register value.

The interface for the Register-Read and Dispatch stage module is shown below. Apart from the `init` method, the remaining sub-interfaces correspond to the FIFO-labeled arrows in Figure 16.1: the incoming `FIFOF_I` interface from the Decode stage, the outgoing `FIFOF_O` interfaces direct to Retire and to Execute Control, Execute Int and to DMem, and an incoming `FIFOF_I` from Retire for updating the register file and scoreboard, which are instantiated inside this module.

src_Fife/S3_RR_S6_WB.bsv: line 38 ...

```
interface RR_WB_IFC;
   method Action init (Initial_Params initial_params);

   // Forward in
   interface FIFOF_I #(Decode_to_RR)  fi_Decode_to_RR;

   // Forward out
   interface FIFOF_O #(RR_to_Retire)      fo_RR_to_Retire;
   interface FIFOF_O #(RR_to_EX_Control)  fo_RR_to_EX_Control;
   interface FIFOF_O #(RR_to_EX)          fo_RR_to_EX_Int;
   interface FIFOF_O #(Mem_Req)           fo_DMem_S_req;

   // Backward in
   interface FIFOF_I #(RW_from_Retire)  fi_RW_from_Retire;

   // For debugger
   method Action      gpr_write (Bit #(5) rd, Bit #(XLEN) v);
   method Bit #(XLEN) gpr_read  (Bit #(5) rs);
endinterface
```

16.6.1 BSV: Vectors for the Scoreboard

In Section 15.3 we discussed the general principles of a scoreboard, and described it as an array of 1-bit values. In BSV the following type is used to represent an array of n items, each of which is of type t:

```
Vector #(n, t);
```

Note: in order to use this type in any BSV code file, the file must import the `Vector` library:

```
import Vector ::  *;
```

So, we can define a `Scoreboard` type as follows:

```
typedef Vector #(32, Bit #(1)) Scoreboard;
```

The BSV Vector-library function:

```
replicate (v)
```

creates a value of type `Vector #(n, t)` where n is inferred from the context and `v` has the type t. All n items in the value are equal to `v`. Thus, we can instantiate a scoreboard like this, where all the vector elements are initialized to 0:

```
Reg #(Scoreboard) rg_scoreboard <- mkReg (replicate (0));
```

In hardware, a `Vector#(32,Bit#(1))` value occupies exactly 32 bits, *i.e.,* the size of the vector times the size of each element. So, why not use `Bit #(32)` instead? It's a matter of programming taste:

- The same syntax `v[j]` works both for bit-selection from `Bit#(n)` and `Vector#(n,Bit#(1))`.
- With `Bit#(n)`, a jth bit can also be selected using shift-and-mask operations: `((v >> j) & 1)`.
- The jth bit of `Vector#(n,Bit#(1))` can be updated using simple assignment
  `v [j] = new_value;`
- The jth bit of `Bit#(n)` can be updated using shift and mask operations:
  `(v | (1 << j))` to set the jth bit to 1, and
  `(v & (~ (1 << j)))` to resset the jth bit to 0

We can convert a `Vector #(32, Bit #(1))` value into a `Bit#(32)` value with:

```
pack (v)
```

and we can convert a `Bit#(32)` value into a `Vector #(32, Bit #(1))` value with:

```
unpack (v)
```

16.6.2 The Register-Read and Dispatch (and Register-Write) module

The first part of the `mkRR_RW` module, instantiating its STATE, is shown below:

src_Fife/S3_RR_S6_WB.bsv: line 62 ...

```
(* synthesize *)
module mkRR_WB (RR_WB_IFC);
   ...
   // Forward in
   FIFOF #(Decode_to_RR) f_Decode_to_RR <- mkPipelineFIFOF;

   // Forward out
   FIFOF #(RR_to_Retire)     f_RR_to_Retire     <- mkBypassFIFOF;  // Direct
   FIFOF #(RR_to_EX_Control) f_RR_to_EX_Control <- mkBypassFIFOF;
   FIFOF #(RR_to_EX)         f_RR_to_EX_Int     <- mkBypassFIFOF;
   FIFOF #(Mem_Req)          f_DMem_S_req       <- mkBypassFIFOF;

   // Backward in
   FIFOF #(RW_from_Retire) f_RW_from_Retire <- mkPipelineFIFOF;

   // General-Purpose Registers (GPRs)
   GPRs_IFC #(XLEN)  gprs <- mkGPRs_synth;
   ...

   // Scoreboard for GPRs
   Reg #(Scoreboard) rg_scoreboard <- mkReg (replicate (0));
```

It instantiates FIFOs for all the forward and backward flows, and it instantiates the GPRs register file `gprs` (Section 8.2) and the `scoreboard` (Section 15.3). The second part of the `mkRR_RW` module, implementing its BEHAVIOR for the forward flow, is shown below:

src_Fife/S3_RR_S6_WB.bsv: line 88 ...

```
   // ================================================================
   // BEHAVIOR: Forward

   rule rl_RR_Dispatch ((! f_RW_from_Retire.notEmpty)
                        && (! f_Decode_to_RR.first.halt_sentinel));
      ...
      let x       = f_Decode_to_RR.first;
      let instr   = x.instr;
      let opclass = x.opclass;
      let rs1     = instr_rs1 (instr);
      let rs2     = instr_rs2 (instr);
      let rd      = instr_rd  (instr);

      let scoreboard = rg_scoreboard;
      let busy_rs1   = (x.has_rs1 && (scoreboard [rs1] != 0));
      let busy_rs2   = (x.has_rs2 && (scoreboard [rs2] != 0));
      let busy_rd    = (x.has_rd  && (scoreboard [rd]  != 0));
      Bool stall     = (busy_rs1 || busy_rs2 || busy_rd);

      if (stall) begin
         // No action
         ...
      end
      else begin
         ...
         f_Decode_to_RR.deq;
```

```
27
28          // Read GPRs.
29          // Ok even if instr does not have rs1 or rs2
30          // values used only if relevant.
31          let rs1_val = gprs.read_rs1 (rs1);
32          ...
33
34          let rs2_val = gprs.read_rs2 (rs2);
35          ...
36
37          // Dispatch to one of the next-stage pipes
38          Result_Dispatch y <- fn_Dispatch (x, rs1_val, rs2_val, rg_flog);
39
40          // Update scoreboard for Rd
41          if (x.has_rd) begin
42             scoreboard [rd] = 1;
43             rg_scoreboard <= scoreboard;
44          end
45
46          // Direct to Retire
47          f_RR_to_Retire.enq (y.to_Retire);
48
49          // Dispatch
50          case (y.to_Retire.exec_tag)
51             EXEC_TAG_DIRECT:  noAction;
52             EXEC_TAG_CONTROL: f_RR_to_EX_Control.enq (y.to_EX_Control);
53             EXEC_TAG_INT:     f_RR_to_EX_Int.enq (y.to_EX);
54             EXEC_TAG_DMEM:    f_DMem_S_req.enq (y.to_EX_DMem);
55          endcase
56          ...
57
58       end
59    endrule
```

In line 6, we observe the first element in the `f_Decode_to_RR` FIFO. Note, the `.first` method is non-destructive, *i.e.,* it merely observes and *does not dequeue* the first element from the FIFO. This is because, if we must stall, it needs to be available again the next time the rule fires.

The next several lines compute the stall condition by checking the scoreboard for whether rs1, rs2 or rd are busy (if the instruction has rs1, rs2 or rd, respectively). Note that an instruction with an rd field, but where rd==0, will not stall because Decode will have set `has_rd` to false.

If we stall, the rule takes no action; everything will be retried the next time it fires.

If we do not stall, then we dequeue the `f_Decode_to_RR` FIFO. We read values of the rs1 and rs2 registers. Note, if the instruction does not have an rs1 or rs2, here we will be reading some random registers according to the bits that happen to be in the rs1 and rs2 bit-positions in the instruction. This does not matter; in the Execute stage each instruction only *uses* these values if the instruction has an rs1 and/or rs2.

Next, we apply the function `fn_Dispatch()` (discussed in Section 6.4, and is the same one we use in **Drum**) to the inputs, which computes `y`, containing the structs to be sent on the direct path (`y.to_Retire`), to Execute Control (`y.to_Control`), and to Execute Int and Execute DMem (`y.toEX`).

If the instruction has an rd, we reserve it on the scoreboard. Note that if an instruction has an rd field, but rd=0, Decode would have marked `has_rd` false, and so we never take a reservation on register `x0`.

We enqueue `y.to_Retire` on the direct flow (FIFO `f_RR_to_Retire`).

The remaining code is a nested if-then-else that sends information into the appropriate Execute pipe (Execute Control, Execute Integer, or DMem).

EXERCISE: Please see Ex-16-A-Register-Read-and-Dispatch in Appendix E, page E-24. □

EXERCISE: Please see Ex-16-B-Rule-Conditions in Appendix E, page E-24. □

EXERCISE: Please see Ex-16-C-Debugging-Stalled-Rules in Appendix E, page E-24. □

The third part of the `mkRR_RW` module, implementing its BEHAVIOR for the backward flow, is shown below:

src_Fife/S3_RR_S6_WB.bsv: line 187 ...

```
1  // ================================================================
2  // BEHAVIOR: Backward: reg write from retire
3
4  rule rl_RW_from_Retire;
5     let x <- pop_o (to_FIFOF_O (f_RW_from_Retire));
6
7     Scoreboard scoreboard = rg_scoreboard;
8     scoreboard [x.rd] = 0;
9     rg_scoreboard <= scoreboard;
10
11    if (x.commit) begin
12       gprs.write_rd (x.rd, x.data);
13       ...
14    end
15    ...
16 endrule
```

We pop the message `x` from the `f_RW_from_Retire` FIFO. We perform its specified scoreboard-release for register rd. If the rd value is to be committed, we write it into GPR [rd]. The fourth and final part of the `mkRR_RW` module, defining its INTERFACE, is shown below:

src_Fife/S3_RR_S6_WB.bsv: line 209 ...

```
1  // ================================================================
2  // INTERFACE
3
4  method Action init (Initial_Params initial_params);
5     $display ("GPRs: simple, non-bypassed");
6     rg_flog <= initial_params.flog;
7  endmethod
8
9  // Forward in
10 interface fi_Decode_to_RR = to_FIFOF_I (f_Decode_to_RR);
11
```

```
12    // Forward out
13    interface fo_RR_to_Retire     = to_FIFOF_O (f_RR_to_Retire);
14    interface fo_RR_to_EX_Control = to_FIFOF_O (f_RR_to_EX_Control);
15    interface fo_RR_to_EX_Int     = to_FIFOF_O (f_RR_to_EX_Int);
16    interface fo_DMem_S_req       = to_FIFOF_O (f_DMem_S_req);
17
18    // Backward in
19    interface fi_RW_from_Retire = to_FIFOF_I (f_RW_from_Retire);
20
21    // For debugger
22    method Action gpr_write (Bit #(5) rd, Bit #(XLEN) v);
23       gprs.write_dm (rd, v);
24    endmethod
25
26    method Bit #(XLEN) gpr_read (Bit #(5) rs);
27       return gprs.read_dm (rs);
28    endmethod
29 endmodule
```

It simply lifts the FIFO interfaces to this module's interface.

16.7 The Execute Control stage

The interface for the Execute Control stage module is shown below. Apart from the `init` method, the remaining sub-interfaces correspond to the FIFO-labeled arrows in Figure 16.1: the incoming `FIFOF_I` interface from the Register-Read and Dispatch stage and the outgoing `FIFOF_O` interface to the Retire stage.

src_Fife/S4_EX_Control.bsv: line 31 ...

```
1  interface EX_Control_IFC;
2     method Action init (Initial_Params initial_params);
3
4     // Forward in
5     interface FIFOF_I #(RR_to_EX_Control)     fi_RR_to_EX_Control;
6     // Forward out
7     interface FIFOF_O #(EX_Control_to_Retire) fo_EX_Control_to_Retire;
8  endinterface
```

The Execute Control stage module is shown below:

src_Fife/S4_EX_Control.bsv: line 42 ...

```
1  (* synthesize *)
2  module mkEX_Control (EX_Control_IFC);
3     Reg #(File) rg_flog <- mkReg (InvalidFile);    // debugging
4
5     // Forward in
6     FIFOF #(RR_to_EX_Control)      f_RR_to_EX_Control     <- mkPipelineFIFOF;
7     // Forward out
8     FIFOF #(EX_Control_to_Retire)  f_EX_Control_to_Retire <- mkBypassFIFOF;
9
10    // ================================================================
11    // BEHAVIOR
12
13    rule rl_EX_Control;
14       let x <- pop_o (to_FIFOF_O (f_RR_to_EX_Control));
```

```
      let y <- fn_EX_Control (x, rg_flog);
      f_EX_Control_to_Retire.enq (y);
      ...
   endrule

   // ================================================================
   // INTERFACE

   method Action init (Initial_Params initial_params);
      rg_flog <= initial_params.flog;
   endmethod

   // Forward in
   interface fi_RR_to_EX_Control     = to_FIFOF_I (f_RR_to_EX_Control);
   // Forward out
   interface fo_EX_Control_to_Retire = to_FIFOF_O (f_EX_Control_to_Retire);
endmodule
```

After instantiating the forward flow input and output FIFOs, the rule `rl_EX_Control` simply applies the function `fn_Control` to each input and enqueues the output. This function was described in Section 6.5, and is the same one we use in **Drum**. Finally, the interface, after the `init` method, simply lifts the FIFO interfaces to the interface of this module.

16.8 The Execute Integer Ops stage

The interface for the Execute Integer stage module is shown below. Apart from the `init` method, the remaining sub-interfaces correspond to the FIFO-labeled arrows in Figure 16.1: the incoming `FIFOF_I` interface from the Register-Read and Dispatch stage and the outgoing `FIFOF_O` interface to the Retire stage.

src_Fife/S4_EX_Int.bsv: line 35 ...

```
interface EX_Int_IFC;
   method Action init (Initial_Params initial_params);

   // Forward in
   interface FIFOF_I #(RR_to_EX)     fi_RR_to_EX_Int;
   // Forward out
   interface FIFOF_O #(EX_to_Retire) fo_EX_Int_to_Retire;
endinterface
```

The Execute Integer stage module is shown below:

src_Fife/S4_EX_Int.bsv: line 46 ...

```
(* synthesize *)
module mkEX_Int (EX_Int_IFC);
   Reg #(File) rg_flog <- mkReg (InvalidFile);    // debugging

   // Forward in
   FIFOF #(RR_to_EX)      f_RR_to_EX_Int     <- mkPipelineFIFOF;
   // Forward out
   FIFOF #(EX_to_Retire)  f_EX_Int_to_Retire <- mkBypassFIFOF;

   // ================================================================
```

```
   // BEHAVIOR

   rule rl_EX_Int;
      let x <- pop_o (to_FIFOF_O (f_RR_to_EX_Int));
      let y <- fn_EX_Int (x, rg_flog);
      f_EX_Int_to_Retire.enq (y);
      ...
   endrule

   // ================================================================
   // INTERFACE

   method Action init (Initial_Params initial_params);
      rg_flog <= initial_params.flog;
   endmethod

   // Forward in
   interface fi_RR_to_EX_Int = to_FIFOF_I (f_RR_to_EX_Int);

   // Forward out
   interface fo_EX_Int_to_Retire = to_FIFOF_O (f_EX_Int_to_Retire);
endmodule
```

After instantiating the forward flow input and output FIFOs, the rule `rl_EX_Int` simply applies the function `fn_EX_Int` to each input and enqueues the output. This function was described in Section 6.6, and is the same one we use in **Drum**. Finally, the interface, after the `init` method, simply lifts the FIFO interfaces to the interface of this module.

16.9 The Execute Memory Ops stage (speculative DMem)

There is no explicit code for an Execute Memory Ops stage. The forward-path rule `rl_RR_Dispatch` in the Register-Read-and-Dispatch stage, described in Section 16.6, directly enqueues a memory request that goes out to memory. The Retire stage, to be discussed in Section 16.10, consumes the corresponding memory response.

16.10 Fife: the Retire stage

This module is longer than the others only because it takes care of many possible cases as outlined in Figure 16.3 (which repeats Figure 15.6 here for reference).

Here is the interface definition for this module:

src_Fife/S5_Retire.bsv: line 34 ...

```
interface Retire_IFC;
   method Action init (Initial_Params initial_params);

   // Forward in
   interface FIFOF_I #(RR_to_Retire)          fi_RR_to_Retire;
   interface FIFOF_I #(EX_Control_to_Retire)  fi_EX_Control_to_Retire;
   interface FIFOF_I #(EX_to_Retire)          fi_EX_Int_to_Retire;

   // DMem, speculative
   interface FIFOF_I #(Mem_Rsp)               fi_DMem_S_rsp;
```

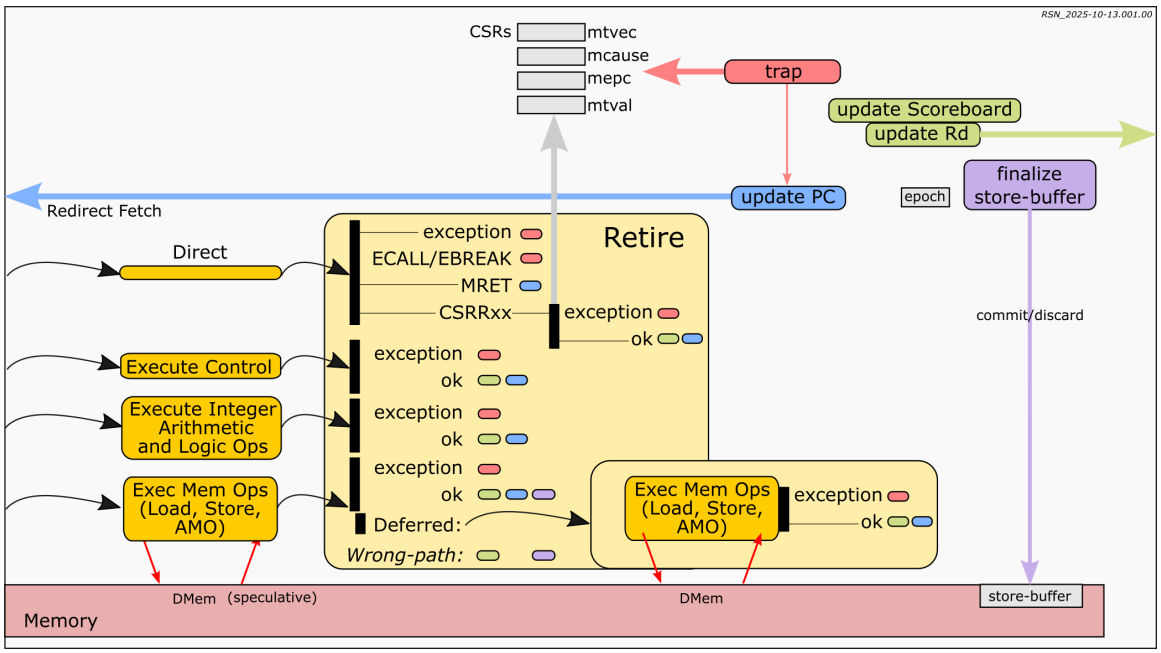


Figure 16.3: Actions in the "Retire" stage of **Fife** (same as Fig. 15.6)

```
11      interface FIFOF_O #(Retire_to_DMem_Commit) fo_DMem_S_commit;
12
13      // DMem, non-speculative
14      interface FIFOF_O #(Mem_Req)  fo_DMem_req;
15      interface FIFOF_I #(Mem_Rsp)  fi_DMem_rsp;
16
17      // Backward out
18      interface FIFOF_O #(Fetch_from_Retire)  fo_Fetch_from_Retire;
19      interface FIFOF_O #(RW_from_Retire)     fo_RW_from_Retire;
20
21      // Set TIME
22      (* always_ready, always_enabled *)
23      method Action set_TIME (Bit #(64) t);
24
25      // Set MIP.MTIP
26      (* always_ready, always_enabled *)
27      method Action set_MIP_MTIP (Bit #(1) v);
28      ...
29  endinterface
```

The first four `FIFO_I` sub-interfaces correspond to the black arrows entering the Retire module from the execute pipes (left of Retire module in the figure).

The next `FIFO_O` interface, `fo_DMem_S_commit`, is the purple arrow sending commit/discard messages to the store-buffer.

The next two sub-interfaces are red arrows connecting the second Exec Mem Ops box to memory (non-speculative DMem request and response).

The last two `FIFO_O` sub-interfaces are the blue arrow at the top of the figure carrying redirections to the Fetch stage, and the green arrow carrying Register-Writes to the `RR_RW` module.

This module has a mixture of modes. Normally it is in pipeline mode, receiving instructions from the Execute stages and retiring them. In certain circumstances, it goes into FSM mode, similar to **Drum**:

- Execute non-speculative DMem ops (MMIO/non-memory-like), FSM-sequencing through issuing a request to memory and handling the response.
- Execute a CSRRxx insruction, which may raise an exception, which must then be handled.
- Execute traps (for exceptions or interrrupts).

We define a "module mode" to reflect these modes:

src_Fife/S5_Retire.bsv: line 83 ...

```
typedef enum {MODE_PIPE,        // Normal pipeline operation
              MODE_DMEM_RSP,    // Handle Non-speculative DMem response
              MODE_EXCEPTION
} Module_Mode
deriving (Bits, Eq, FShow);
```

The first part of the Retire stage `mkRetire` module is shown below.

src_Fife/S5_Retire.bsv: line 102 ...

```
(* synthesize *)
module mkRetire (Retire_IFC);
   Reg #(File) rg_flog <- mkReg (InvalidFile);    // debugging

   // Control-and-Status Registers (CSRs)
   CSRs_IFC csrs <- mkCSRs;

   // For managing speculation, redirection, traps, etc.
   Reg #(Epoch) rg_epoch  <- mkReg (0);

   // Forward in
   // Depth of f_RR_to_Retire should be > longest EX pipe
   FIFOF #(RR_to_Retire)          f_RR_to_Retire         <- mkSizedFIFOF (8);
   FIFOF #(EX_Control_to_Retire)  f_EX_Control_to_Retire <- mkPipelineFIFOF;
   FIFOF #(EX_to_Retire)          f_EX_Int_to_Retire     <- mkPipelineFIFOF;
   FIFOF #(Mem_Rsp)               f_DMem_S_rsp           <- mkPipelineFIFOF;

   // Forward out
   FIFOF #(Retire_to_DMem_Commit) f_DMem_S_commit  <- mkBypassFIFOF;

   // Backward out
   FIFOF #(Fetch_from_Retire) f_Fetch_from_Retire <- mkBypassFIFOF;
   FIFOF #(RW_from_Retire)    f_RW_from_Retire    <- mkBypassFIFOF;

   // Non-speculative DMem reqs and rsps
   FIFOF #(Mem_Req)  f_DMem_req  <- mkBypassFIFOF;
   FIFOF #(Mem_Rsp)  f_DMem_rsp  <- mkPipelineFIFOF;

   Reg #(Module_Mode) rg_mode <- mkReg (MODE_PIPE);

   // Regs to set up exception handling
   Reg #(Bit #(XLEN)) rg_epc   <- mkRegU;
   Reg #(Bit #(4))    rg_cause <- mkRegU;
   Reg #(Bit #(XLEN)) rg_tval  <- mkRegU;
```

It instantiates the CSR registers (Section 8.3), and register `rg_epoch` to keep track of the epoch (Section 15.2), and then FIFOs for all the incoming and outgoing channels. Finally,

it instantiates register `rg_mode` to hold the current module mode, and a couple of registers to hold exception information until we can perform the exception actions.

16.10.1 Common facilities used by many rules

Before we study the rules in the module, we first encapsulate, in `Action` functions, some common actions performed in many rules.

The following function captures the actions to be taken whenever we need to redirect the Fetch stage to start fetching from a different PC. This can be due to a misprediction of the successor of the current instruction, or due to an exception or MRET.

src_Fife/S5_Retire.bsv: line 163 ...

```
function Action fa_redirect_Fetch (Bool         mispredicted,
                                   Bool         haltreq,
                                   RR_to_Retire x1,
                                   Bit #(XLEN)  next_pc);
   action
      ...
      if (((rg_runstate == S5_RUNNING) && mispredicted)
          || haltreq)
         begin
            let next_epoch = rg_epoch + 1;
            rg_epoch <= next_epoch;
            let y = Fetch_from_Retire {next_pc:    next_pc,
                                       next_epoch: next_epoch,
                                       ...
                                       };
            f_Fetch_from_Retire.enq (y);
         ...
         end
   endaction
endfunction
```

The boolean `mispredicted` indicates whether the prediction was correct or not. If the prediction was correct, no action is taken. Otherwise, we increment `rg_epoch`, and send a redirection message to the Fetch stage with the correct PC and the new epoch. The consumption of this message in the Fetch stage was discussed in Section 16.4.

The following function captures the actions to be taken for updating an instruction's destination rd register. If the instruction has an rd register, it assembles a `RW_to_Retire` message and sends it to the `RR_RW` module. The consumption of this message in the Register-Read and Dispath (and Register-Write) stage was discussed in Section 16.6.

src_Fife/S5_Retire.bsv: line 200 ...

```
// * unreserve the scoreboard
// * if commit, also write the rd_val
function Action fa_update_rd (RR_to_Retire x1,
                              Bool         commit,
                              Bit #(XLEN)  rd_val);
   action
      if (x1.has_rd) begin
         let y = RW_from_Retire {rd:         instr_rd (x1.instr),
                                 commit:     commit,
                                 data:       rd_val,
                                 ...
```

```
                                    };
            f_RW_from_Retire.enq (y);
            ...
         end
      endaction
   endfunction
```

The following function sends a commit/discard message to the speculative DMem store-buffer, if the instruction writes to memory:

src_Fife/S5_Retire.bsv: line 228 ...

```
   function Action fa_retire_store_buf (RR_to_Retire x1,
                                        Mem_Rsp      mem_rsp,
                                        Bool         commit);
      action
         if (x1.writes_mem && (mem_rsp.rsp_type == MEM_RSP_OK)) begin
            let y = Retire_to_DMem_Commit{commit: commit,
                                          ...
            f_DMem_S_commit.enq (y);
         end
      endaction
   endfunction
```

The `f_RR_to_Retire` (direct path) FIFO contains an entry for *every* instruction. Information in the first element tells us how to retire it, including which of the execute pipes need to be examined. The following definitions capture some properties of this first element.

src_Fife/S5_Retire.bsv: line 245 ...

```
   // mispredictions and in-order merging of all pipes.
   RR_to_Retire x_rr_to_retire = f_RR_to_Retire.first;

   Bool wrong_path = (x_rr_to_retire.epoch != rg_epoch);
   Bool is_Direct  = (x_rr_to_retire.exec_tag == EXEC_TAG_DIRECT);
   Bool is_Control = (x_rr_to_retire.exec_tag == EXEC_TAG_CONTROL);
   Bool is_Int     = (x_rr_to_retire.exec_tag == EXEC_TAG_INT);
   Bool is_DMem    = (x_rr_to_retire.exec_tag == EXEC_TAG_DMEM);
```

The incoming instruction is a wrong-path instruction if its accompanying epoch does not match our `rg_epoch` register.

16.10.2 Rule to retire wrong-path instructions (all paths; discard)

For a wrong-path instruction, we dequeue it from `f_RR_to_Retire`.

If it has an Rd, we send a discard-scoreboard-reservation message to the RR-RW module using `fa_update_rd()`.

If it was a Control, Int or DMem instruction, we dequeue it from the corresponding incoming FIFO. In the DMem case, if it did a speculative STORE, we also send a "discard" message to the head of the store-buffer using `fa_retire_store_buf()`.

src_Fife/S5_Retire.bsv: line 288 ...

```
   rule rl_Retire_wrong_path ((rg_mode == MODE_PIPE)
                              && wrong_path
                              && (! x_rr_to_retire.halt_sentinel));
```

```
4        f_RR_to_Retire.deq;
5
6        // Unreserve/commit rd if needed
7        fa_update_rd (x_rr_to_retire, False, ?);
8
9        // Discard related pipe
10       if (is_Control) f_EX_Control_to_Retire.deq;
11       if (is_Int)     f_EX_Int_to_Retire.deq;
12       if (is_DMem) begin
13          let mem_rsp <- pop_o (to_FIFOF_O (f_DMem_S_rsp));
14          // Send 'discard' (False) to store-buf, if needed
15          fa_retire_store_buf (x_rr_to_retire, mem_rsp, False);
16       end
17       ...
18    endrule
```

16.10.3 Rules to retire from direct path

The following subsections handle instruction-retirement for the cases where the direct-path has all the necessary information, i.e., where the instruction was not dispatched into the Execute Control, Execute Int or Execute DMem paths.

16.10.3.1 Rule to retire CSRRxx instructions (direct path)

The following rule handles CSRRxx instructions, which arrive on the direct path:

src_Fife/S5_Retire.bsv: line 342 ...

```
1    rule rl_Retire_CSRRxx (retire_Direct
2                           && is_legal_CSRRxx (x_rr_to_retire.instr));
3       // Note: mav_csrrxx takes care of csrs.ma_incr_instret as well
4       match { .exc, .rd_val } <- csrs.mav_csrrxx (x_rr_to_retire.instr,
5                                                   x_rr_to_retire.rs1_val);
6       // Unreserve/commit rd if needed
7       fa_update_rd (x_rr_to_retire, (! exc), rd_val);
8
9       if (! exc) begin
10         f_RR_to_Retire.deq;
11
12         Bool mispredicted = (x_rr_to_retire.predicted_pc
13                              != x_rr_to_retire.fallthru_pc);
14         fa_redirect_Fetch (mispredicted,
15                            (rg_runstate == S5_HALTREQ),
16                            x_rr_to_retire,
17                            x_rr_to_retire.fallthru_pc);
18         ...
19      end
20      else begin
21         rg_epc   <= x_rr_to_retire.pc;
22         rg_cause <= cause_ILLEGAL_INSTRUCTION;
23         rg_tval  <= x_rr_to_retire.instr;
24         rg_mode  <= MODE_EXCEPTION;
25      end
26      ...
27   endrule
```

The rule's explicit condition restricts it to fire only when we are in PIPE mode and when the next instruction is not a wrong-path instruction (epoch is correct), is a Direct-path instruction, is not an exception, and is indeed a CSRRxx instruction.

If so, it applies `csrs.csrrxx()` (described in Section 8.3, and the same one we used in **Drum**), and updates the rd register with the result.

If the CSRRxx operation did not raise an exception, it redirects Fetch this instruction's successor was mispredicted.

If the CSRRxx operation raised an exception, we save the details in some registers and switch from PIPE mode into EXCEPTION mode; this will enable rule `rl_exception` (Section 16.10.7) to handle it.

Why postpone exception-handling to the later rule? Why not perform that rule's action here? The reason is to avoid contention on the CSR registers. Rule `rl_Retire_CSRRxx` invokes method `csrs.mav_csrrrxx()` and rule `rl_exception` invokes method `csrs.mav_exception()`. By separating these invocations into two separate rules, we avoid contention between these two CSR module methods.

EXERCISE: Please see Ex-16-D-CSR-Method-Conflict in Appendix E, page E-24. □

EXERCISE: Please see Ex-16-E-Resolving-a-CSR-Method-Conflict in Appendix E, page E-25. □

16.10.3.2 Rule to retire MRET instructions (direct path)

The following rule handles MRET instructions, which arrive on the direct path:

src_Fife/S5_Retire.bsv: line 378 ...

```
1  rule rl_Retire_MRET (retire_Direct
2                       && is_legal_MRET (x_rr_to_retire.instr));
3     f_RR_to_Retire.deq;
4     let new_pc <- csrs.mav_xRET;
5     Bool mispredicted = True;
6     fa_redirect_Fetch (mispredicted,
7                        (rg_runstate == S5_HALTREQ),
8                        x_rr_to_retire,
9                        new_pc);
10    csrs.ma_incr_instret;
11    ...
12 endrule
```

Again, pay careful attention to the rule's explicit condition which restricts it to fire only when we are in PIPE mode and when the next instruction is not a wrong-path instruction (epoch is correct), is a Direct-path instruction, is not an exception, and is indeed an MRET instruction.

The rule body is simple—always redirect to the PC found in the CSR MEPC (Section 2.7).

16.10.3.3 Rule to retire ECALL and EBREAK instructions (direct path)

The following rule handles ECALL and EBREAK instructions, which arrive on the direct path:

src_Fife/S5_Retire.bsv: line 397 ...

```
rule rl_Retire_ECALL_EBREAK (retire_Direct
                             && (is_legal_ECALL (x_rr_to_retire.instr)
                                 || (is_legal_EBREAK (x_rr_to_retire.instr)
                                     && (! ebreak_halt))));
   rg_epc   <= x_rr_to_retire.pc;
   rg_cause <= ((x_rr_to_retire.instr [20] == 0)
                ? cause_ECALL_FROM_M
                : cause_BREAKPOINT);
   rg_tval  <= 0;
   csrs.ma_incr_instret;
   rg_mode <= MODE_EXCEPTION;
   ...
endrule
```

The rule's explicit condition restricts it to fire only when we are in PIPE mode and when the next instruction is not a wrong-path instruction (epoch is correct), is a Direct-path instruction, is not an exception, and is indeed an ECALL or EBREAK instruction.

ECALL and EBREAK are just deliberately-invoked exceptions, the only difference being the exception-code we pass in the `mcause` CSR (Section 2.7.1). We save appropriate exception values and switch from `MODE_PIPE` to `MODE_EXCEPTION`, which will handle the exception.

EXERCISE: Please see Ex-16-F-ECALL-EBREAK in Appendix E, page E-25. □

16.10.3.4 Rule to retire exceptions from the direct path

The following rule handles exceptions which arrive on the direct path:

src_Fife/S5_Retire.bsv: line 432 ...

```
rule rl_Retire_Direct_exception (retire_Direct_Exception);
   rg_epc   <= x_rr_to_retire.pc;
   rg_cause <= x_rr_to_retire.cause;
   rg_tval  <= x_rr_to_retire.tval;
   rg_mode <= MODE_EXCEPTION;

   log_Retire_Direct_exc (rg_flog, x_rr_to_retire);
   ...
endrule
```

The rule's explicit condition restricts it to fire only when we are in PIPE mode and when the next instruction is not a wrong-path instruction (epoch is correct), is a Direct-path instruction, and is an exception.

The rule does nothing more than save the exception information in registers and move from `MODE_PIPE` to `MODE_EXCEPTION` so that the exception will be handled by `rl_exception`.

16.10.4 Rule to retire from the Execute Control path

The following rule handles instructions (BRANCH, JAL, JALR) that arrive on the Execute Control path. In Execute Control, it could have succeeded or it could have raised an exception (if the control-transfer target PC was misaligned).

src_Fife/S5_Retire.bsv: line 453 ...

```
1  rule rl_Retire_EX_Control (retire_Control);
2     let x2 <- pop_o (to_FIFOF_O (f_EX_Control_to_Retire));
3
4     // Unreserve/commit rd if needed
5     fa_update_rd (x_rr_to_retire, (! x2.exception), x2.data);
6
7     if (! x2.exception) begin
8        f_RR_to_Retire.deq;
9
10       // Redirect Fetch PC if mispredicted
11       Bool mispredicted = (x_rr_to_retire.predicted_pc != x2.next_pc);
12       fa_redirect_Fetch (mispredicted,
13                          (rg_runstate == S5_HALTREQ),
14                          x_rr_to_retire,
15                          x2.next_pc);
16       csrs.ma_incr_instret;
17       ...
18    end
19    else begin
20       rg_epc   <= x_rr_to_retire.pc;
21       rg_cause <= x2.cause;
22       rg_tval  <= x2.tval;
23       rg_mode <= MODE_EXCEPTION;
24    end
25    ...
26 endrule
```

We dequeue the direct-path information (`f_RR_to_Retire`) and we pop the Execute Control path information (`f_EX_Control_to_Retire`).

We invoke `fa_update_rd()` to update the destination register (JAL and JALR may save a "return address" in rd) and its scoreboard reservation. Recall from Section 16.10.1 that `fa_update_rd()` will not send any update message if the instruction does not have an rd (or if rd is 0). The "`(!x2.exception)`" argument to `fa_update_rd()` ensures that if there was an exception only the scoreboard reservation will be released, and no register value will be written.

If there was no Execute Control exception, we redirect the Fetch stage if mispredicted. At this point we know both what was predicted as the successor PC and what is the actual successor PC for this instruction, so we can compare these to determine if there was a misprediction.

If there was an Execute Control exception, we save relevant values in registers and move from `MODE_PIPE` to `MODE_EXCEPTION` which will enable `rl_exception` to perform the required actions.

16.10.5 Rule to retire from the Execute Integer path

The following rule handles LUI, AUIPC and Integer instructions that arrive on the Execute Integer path. Note, none of these standard RISC-V instructions raise any exceptions. However, in this rule we assume the possibility of an instruciton in case, in future, we extend **Fife**'s supported ISA with new non-standard Integer instructions that could raise an exception.

src_Fife/S5_Retire.bsv: line 485 ...

```
1  rule rl_Retire_EX_Int (retire_Int);
2     EX_to_Retire x2 <- pop_o (to_FIFOF_O (f_EX_Int_to_Retire));
3
4     // Unreserve/commit rd if needed
5     fa_update_rd (x_rr_to_retire, (! x2.exception), x2.data);
6
7     if (! x2.exception) begin
8        f_RR_to_Retire.deq;
9
10       // Redirect Fetch PC if mispredicted
11       Bool mispredicted = (x_rr_to_retire.predicted_pc
12                            != x_rr_to_retire.fallthru_pc);
13       fa_redirect_Fetch (mispredicted,
14                          (rg_runstate == S5_HALTREQ),
15                          x_rr_to_retire,
16                          x_rr_to_retire.fallthru_pc);
17       csrs.ma_incr_instret;
18       ...
19    end
20    else begin
21       rg_epc   <= x_rr_to_retire.pc;
22       rg_cause <= x2.cause;
23       rg_tval  <= x2.tval;
24       rg_mode <= MODE_EXCEPTION;
25    end
26    ...
27 endrule
```

We dequeue the direct-path information (`f_RR_to_Retire`) and we pop the Execute Int path information (`f_EX_Int_to_Retire`).

We invoke `fa_update_rd()` to update the destination register and its scoreboard reservation. Recall from Section 16.10.1 that `fa_update_rd()` will not send any update message if the instruction does not have an rd (or if rd is 0). The "`(!x2.exception)`" argument to `fa_update_rd()` ensures that if there was an exception only the scoreboard reservation will be released, and no register value will be written.

If there was no Execute Integer exception, we redirect the Fetch stage if mispredicted. We can compare the predicted successor PC for this instruction with its fall-through PC to determine if there was a misprediction.

If there was an Execute Integer exception, we save relevant values in registers and move from `MODE_PIPE` to `MODE_EXCEPTION` which will enable `rl_exception` to perform the required actions.

16.10.6 Rules to retire from the Execute DMem path, or perform deferred DMem request

The following rules handle instructions that arrive on the Execute DMem path. This path:

- could have raised an exception (bad address, misaligned, memory-permission violation);
- or, it could have performed a speculative LOAD/STORE/AMO successfully, but if it wrote to memory, the value is still sitting in the store-buffer and needs to be committed;
- or, it could have been deferred because the memory address was a non-memory-like location (*e.g.,* MMIO) that did not allow speculation.

The first two cases are handled by the first rule below; the deferred case is handled by rules described subsequently.

16.10.6.1 Retire speculative and exception from DMem

The following rule handles incoming DMem exceptions and successful speculative results.

src_Fife/S5_Retire.bsv: line 521 ...

```
1  rule rl_Retire_EX_DMem (retire_DMem
2                          && (f_DMem_S_rsp.first.rsp_type != MEM_REQ_DEFERRED));
3
4     let x2 <- pop_o (to_FIFOF_O (f_DMem_S_rsp));
5
6     Bool exception = (x2.rsp_type != MEM_RSP_OK);
7
8     // Sign-extend data if necessary
9     Bit #(XLEN) rd_val = 0;
10    if ((instr_opcode (x_rr_to_retire.instr) == opcode_LOAD)
11        && (instr_rd (x_rr_to_retire.instr) != 0))
12       begin
13          rd_val = truncate (x2.data);
14          if (instr_funct3 (x_rr_to_retire.instr) == funct3_LB)
15             rd_val = signExtend (rd_val [7:0]);
16          else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LBU)
17             rd_val = zeroExtend (rd_val [7:0]);
18          else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LH)
19             rd_val = signExtend (rd_val [15:0]);
20          else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LHU)
21             rd_val = zeroExtend (rd_val [15:0]);
22          // TODO: LW in RV64
23       end
24
25    // Unreserve/commit rd if needed
26    fa_update_rd (x_rr_to_retire, (! exception), rd_val);
27
28    if (! exception) begin
29       f_RR_to_Retire.deq;
30
31       // Send 'commit' (True) to store-buf, if needed
```

```
32          fa_retire_store_buf (x_rr_to_retire, x2, True);
33
34          // Redirect Fetch PC if mispredicted
35          Bool mispredicted = (x_rr_to_retire.predicted_pc
36                               != x_rr_to_retire.fallthru_pc);
37          fa_redirect_Fetch (mispredicted,
38                             (rg_runstate == S5_HALTREQ),
39                             x_rr_to_retire,
40                             x_rr_to_retire.fallthru_pc);
41          csrs.ma_incr_instret;
42          ...
43       end
44       else begin
45          rg_epc   <= x_rr_to_retire.pc;
46          rg_cause <= ((x2.rsp_type == MEM_RSP_MISALIGNED)
47                       ? (is_LOAD (x_rr_to_retire.instr)
48                          ? cause_LOAD_ADDRESS_MISALIGNED
49                          : cause_STORE_AMO_ADDRESS_MISALIGNED)
50                       : (is_LOAD (x_rr_to_retire.instr)
51                          ? cause_LOAD_ACCESS_FAULT
52                          : cause_STORE_AMO_ACCESS_FAULT));
53          rg_tval  <= truncate (x2.addr);
54          rg_mode <= MODE_EXCEPTION;
55       end
56       ...
57    endrule
```

Note that the rule condition excludes deferred requests (they are handled in another rule, described in the next section).

We dequeue the direct-path information (`f_RR_to_Retire`) and we pop the Execute DMem path information (`f_EX_DMem_S_Rsp`).

We invoke `fa_update_rd()` to update the destination register and its scoreboard reservation. Recall from Section 16.10.1 that `fa_update_rd()` will not send any update message if the instruction does not have an rd (or if rd is 0). The "`(!x2.exception)`" argument to `fa_update_rd()` ensures that if there was an exception only the scoreboard reservation will be released, and no register value will be written.

If there was no Execute Dmem exception, we send a "commit" message to the store buffer and redirect the Fetch stage if mispredicted. Note that "`fa_retire_store_buf()`" will send a commit message only if the instruction wrote memory (Section 16.10.1). We can compare the predicted successor PC for this instruction with its fall-through PC to determine if there was a misprediction.

If there was an Execute DMem exception, we save relevant values in registers and move from `MODE_PIPE` to `MODE_EXCEPTION` which will enable `rl_exception` to perform the required actions. The RISC-V "cause code" is computed based on the kind of DMem exception and the kind of instruction. We do not need to use "`fa_retire_store_buf()`" to send a "discard" message to the store-buffer because a DMem instruction, if it raises an exception, will not perform any speculative stores.

EXERCISE: Please see Ex-16-G-Rd-vs-Store-Buffer in Appendix E, page E-25. □

16.10.6.2 Rule to handle deferred DMem requests (from Execute DMem path)

The following rule handles memory requests that were deferred by the Execute DMem stage because there could not be performed speculatively.

src_Fife/S5_Retire.bsv: line 588 ...

```
rule rl_Retire_DMem_deferred (retire_DMem
                              && (f_DMem_S_rsp.first.rsp_type == MEM_REQ_DEFERRED));
   let x2 <- pop_o (to_FIFOF_O (f_DMem_S_rsp));

   // Issue DMem request
   let mem_req = Mem_Req {req_type: x2.req_type,
                          size:     x2.size,
                          addr:     x2.addr,
                          data:     x2.data,
                          ...
                          };
   f_DMem_req.enq (mem_req);
   rg_mode <= MODE_DMEM_RSP;    // go to await response
   ...
endrule
```

We pop the Execute DMem result (`f_DMem_S_rsp`). We do not dequeue the direct-path (`f_RR_to_Retire`) because we will need that informnation when we later handle the DMem response; we leave it at the head of the FIFO.

We merely construct the memory request now, and send it to memory (enqueue it on `f_DMem_req`). We also switch out of `MODE_PIPE` into `MODE_DMEM_RSP` which enables rule `rl_Retire_DMem_rsp` (described next) to await the memory response.

16.10.6.3 Rules to retire responses for deferred DMem requests

The following rule handles the corresponding responses from memory.

src_Fife/S5_Retire.bsv: line 611 ...

```
rule rl_Retire_DMem_rsp (rg_mode == MODE_DMEM_RSP);
   let x2 <- pop_o (to_FIFOF_O (f_DMem_rsp));

   Bool exception = ((x2.rsp_type == MEM_RSP_ERR)
                     || (x2.rsp_type == MEM_RSP_MISALIGNED));

   if (exception) begin
      rg_epc   <= x_rr_to_retire.pc;
      rg_cause <= ((x2.rsp_type == MEM_RSP_MISALIGNED)
                   ? (is_LOAD (x_rr_to_retire.instr)
                      ? cause_LOAD_ADDRESS_MISALIGNED
                      : cause_STORE_AMO_ADDRESS_MISALIGNED)
                   : (is_LOAD (x_rr_to_retire.instr)
                      ? cause_LOAD_ACCESS_FAULT
                      : cause_STORE_AMO_ACCESS_FAULT));
      rg_tval  <= truncate (x2.addr);
      rg_mode <= MODE_EXCEPTION;

      // Unreserve rd if needed
      fa_update_rd (x_rr_to_retire, False, ?);
   end
```

```
22       else if (x2.rsp_type == MEM_REQ_DEFERRED) begin
23          ...
24          // IMPOSSIBLE. Non-speculative requests cannot be deferred
25          $finish (1);
26       end
27       else begin
28          f_RR_to_Retire.deq;
29
30          // Sign-extend data if necessary
31          Bit #(XLEN) rd_val = truncate (x2.data);
32          if ((instr_opcode (x_rr_to_retire.instr) == opcode_LOAD)
33              && (instr_rd (x_rr_to_retire.instr) != 0))
34             begin
35                if (instr_funct3 (x_rr_to_retire.instr) == funct3_LB)
36                   rd_val = signExtend (rd_val [7:0]);
37                else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LBU)
38                   rd_val = zeroExtend (rd_val [7:0]);
39                else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LH)
40                   rd_val = signExtend (rd_val [15:0]);
41                else if (instr_funct3 (x_rr_to_retire.instr) == funct3_LHU)
42                   rd_val = zeroExtend (rd_val [15:0]);
43                // TODO: LW in RV64
44             end
45
46          // Unreserve/commit rd if needed
47          fa_update_rd (x_rr_to_retire, True, rd_val);
48
49          // Redirect Fetch to correct mispredicted PC
50          Bool mispredicted = (x_rr_to_retire.predicted_pc
51                               != x_rr_to_retire.fallthru_pc);
52          fa_redirect_Fetch (mispredicted,
53                             (rg_runstate == S5_HALTREQ),
54                             x_rr_to_retire,
55                             x_rr_to_retire.fallthru_pc);
56          csrs.ma_incr_instret;
57          ...
58
59          // Resume pipeline behavior
60          rg_mode <= MODE_PIPE;
61       end
62       ...
63    endrule
```

We can now dequeue the direct-path information (`f_RR_to_Retire`) and the memory-response (`f_DMem_Rsp`).

If the response returned an exception, we compute the RISC-V exception `cause` code, save values in registers, and move to `MODE_EXCEPTION` so that `rl_exception` can handle it subsequently.

The memory system should *never* return a `MEM_REQ_DEFERRED` for these requests through the non-speculative memory interface; responses can only be OK or an exception. The middle "`else-if`" clause is just a bit of defensive programming in case of a buggy memory system.

If the response was OK, we invoke `fa_update_rd()` to update the destination register and its scoreboard reservation. Recall from Section 16.10.1 that `fa_update_rd()` will not send

any update message if the instruction does not have an rd (or if rd is 0). The second argument is True because this not the exception case.

Finally, we redirect the Fetch stage if mispredicted, and return to `MODE_PIPE` to resume pipeline processing. We can compare the predicted successor PC for this instruction with its fall-through PC to determine if there was a misprediction.

EXERCISE: Please see Ex-16-H-Retiring Deferred DMem in Appendix E, page E-25. □

16.10.7 Common Rule to handle exceptions

This is a common rule used to handle exceptions.

src_Fife/S5_Retire.bsv: line 688 ...

```
1  rule rl_exception (rg_mode == MODE_EXCEPTION);
2     f_RR_to_Retire.deq;
3
4     Bool is_interrupt = False;
5     Bit #(XLEN) tvec_pc <- csrs.mav_exception (rg_epc,
6                                                is_interrupt,
7                                                rg_cause,
8                                                rg_tval);
9     Bool mispredicted = True;
10    fa_redirect_Fetch (mispredicted,
11                       (rg_runstate == S5_HALTREQ),
12                       x_rr_to_retire,
13                       tvec_pc);
14    rg_mode <= MODE_PIPE;
15    ...
16 endrule
```

In all the previous rules, when an exception is detected, we save relevant values in registers `rg_epc`, `rg_cause` and `rg_tval` and switch the module mode to `MODE_EXCEPTION`, which enables this rule. It invokes `csrs.mav_exception()` which updates the relevant CSRs, and returns the value CSR `mtvec` (trap-vector PC). Here, we simply redirect to that PC, and return to pipeline processing in `MODE_PIPE`.

16.10.8 Fife module interface definition

The final INTERFACE section, as with other modules, has an `init` method, and then lifts the various FIFO interfaces into sub-interfaces for this module.

src_Fife/S5_Retire.bsv: line 715 ...

```
1  // ================================================================
2  // INTERFACE
3
4  method Action init (Initial_Params initial_params);
5     csrs.init (initial_params);
6     rg_flog <= initial_params.flog;
7  endmethod
8
9  // Forward in
```

```
10    interface fi_RR_to_Retire         = to_FIFOF_I (f_RR_to_Retire);
11    interface fi_EX_Control_to_Retire = to_FIFOF_I (f_EX_Control_to_Retire);
12    interface fi_EX_Int_to_Retire     = to_FIFOF_I (f_EX_Int_to_Retire);
13
14    // DMem, speculative
15    interface fi_DMem_S_rsp    = to_FIFOF_I (f_DMem_S_rsp);
16    interface fo_DMem_S_commit = to_FIFOF_O (f_DMem_S_commit);
17
18    // DMem, non-speculative
19    interface fo_DMem_req = to_FIFOF_O (f_DMem_req);
20    interface fi_DMem_rsp = to_FIFOF_I (f_DMem_rsp);
21
22    // Backward out
23    interface fo_Fetch_from_Retire  = to_FIFOF_O (f_Fetch_from_Retire);
24    interface fo_RW_from_Retire     = to_FIFOF_O (f_RW_from_Retire);
25
26    // Set TIME
27    method Action set_TIME (Bit #(64) t) = csrs.set_TIME (t);
```

16.11 Conclusion

And that is the complete **Fife** CPU! There is actually very little in this chapter that is RISC-V specific; these pipeline structures are needed for a pipelined CPU for *any* ISA. All our RISC-V-specific discussions were completed in the chapters leading up to **Drum**, and here we have simply reused all those RISC-V-specific common functions.

There is quite a lot of scope for performance optimization here, even keeping the same pipeline structure. For example:

- The sooner we can deliver a redirection from Retire to Fetch, the less time the CPU will waste fetching and discarding wrong-path instructions.
- In a message from Retire to the Register-Read and Dispatch stage to update rd and release the scoreboard, the sooner we can deliver that value to any waiting instruction, the less time wasted in waiting on the scoreboard.
- In Retire, the `rl_exception` actions can be folded in to the rule where the exception is detected, avoiding another delay, except possibly in the CSRRxx case, where there may be contention with CSR access.

We will discuss how to do some of these optimizations in Chapter 18.

It is not straightforward to judge whether an optimization is worth it or not. Many optimizations add hardware cost (silicon area and energy consumption). Some optizationw, while reducing the number of clocks for a computation, may require slower clock speeds because of longer combinational paths, thus negating some of the speed advantage.

Because of the high variability in the mix of instructions across different programs, some optimizations may have more effect on some programs than on others.

Ultimately, the value of most optimizations cannot be judged in the abstract, only in specific contexts: What are the actual application programs that will be run? Does this optimization improve the performance of those optimizations? At what hardware and energy cost?

Chapter 17

PBT: Property-based Testing



17.1 Introduction

... to be written ...

Chapter 18

BSV: Rules and Methods II: Improved performance with CRegs (Concurrent Registers)

18.1 Introduction

In Section 13.6.1 we discussed constraints on mapping rules into clocked digital hardware. In particular, executing two rules on the same clock edge may result in an ordering conflict, and therefore the compiler produces a Rule Controller to suppress simultaneous firing. This has some consequences on the performance of BSV programs, depending on the primitive modules they use. We illustrate this problem in Section 18.2, and then show a general-purpose solution in Section 18.3 using "*Concurrent Registers*" (or CRegs).

Finally, in Section 18.6 we mention an older solution called `RWire`s. We deprecate use of `RWire`s in favor of `CReg`s, but they are still supported by the compiler (Drum and Fife codes do not use any `RWire`s).

18.2 A motivating example for CRegs: an up-down counter for credit-based flow-control

Figure 18.1 shows a generic scheme for so-called "credit-based flow control", which is widely used in many scenarios in hardware systems for streaming data between loosely-coupled agents.

The transmitter streams a series of packets to a receiver over a loosely-coupled one-way link. In order to avoid over-running the receiver's buffer, we initialize an up-down counter with the size of the receiver's buffer. Think of these as "credits", *i.e.,* as long as the transmitter has credits available, it can send packets without danger of over-running the receiver's buffer. For each packet sent, it decrements the counter ("remaining available credit"). If the credits ever fall to zero, the transmitter must stop sending (stall) until the credits are replenished.

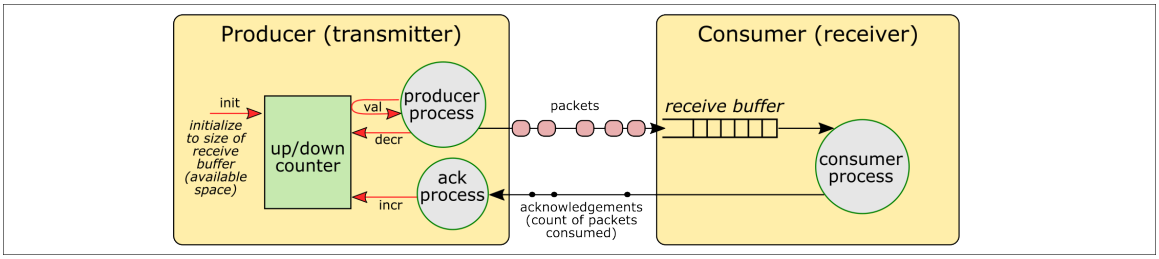


Figure 18.1: Credit-based flow control with an up/down counter

The receiver consumes packets asynchronously from its buffer. As it frees up space, it indicates this in "acknowledgments" streamed on a one-way link in the opposite direction to the transmitter. At the transmitter, the counter is incremented according to these acknowledgements ("replenish credits"). If the transmitter had been stalled due to lack of credits, it can now resume transmitting.

Consider the following "Up-Down Counter" module interface:

```
1  interface Up_Down_Counter_IFC;
2     method Action   incr;
3     method Action   decr;
4     method Bit #(4) val;
5  endinterface
```

The specification for any module implementing this is this: internally, the module should contain a register that can hold values in the range 0..15, and, the methods do the following:

- `incr` increments the counter if it is < 15
- `decr` decrements the counter if it is > 0
- `val` returns the current value of the counter

Here is a proposed module to implement the interface:

```
1  module mkUp_Down_Counter_I (Up_Down_Counter_IFC);
2     // STATE
3     Reg #(Bit #(4)) rg_counter <- mkReg (15);
4
5     // ----------------------------------------------------------------
6     // INTERFACE
7     method Action incr () if (rg_counter != 15);
8        rg_counter <= rg_counter + 1;
9     endmethod
10
11    method Action decr () if (rg_counter != 0);
12       rg_counter <= rg_counter - 1;
13    endmethod
14
15    method Bit #(4) val;
16       return rg_counter;
17    endmethod
18 endmodule
```

18.2.1 Semantic and Performance Analysis of `mkUp_Down_Counter_I` when mapped to clocked hardware

Suppose we have three rules, each invoking one of the three methods on the same module. Which pairs of rules can fire on the same clock?

- `val` *and* `incr`?

  Yes: Clocked execution is consistent with rule-at-a-time semantics where the rule with `val` fires before the rule with `incr`. (And similary, a rule invoking `val` can fire on the same clock as a rule invoking `decr`.)

- `incr` *and* `decr`?

  No, for both reasons discussed in Section 13.6.1. There would be an action/resource conflict because both rules cannot write the same register at the same instant. There would be an ordering confict because rule-at-a-time semantics demands that one rule's update is visible to the other (in whichever order they may fire).

The key takeaway is that `incr` and `decr` cannot be invoked on the same clock (same instant). To revisit our credit-based flow-control example: on each clock we can either send a packet or receive an acknowledgement, but not both.

18.3 Concurrent Registers `CRegs`, and a Faster Up-Down Counter

A Concurrent Register, or `CReg`, is a module provided by the *bsc* library. This is an excerpt from the library documentation:

```
1 module mkCReg #(parameter Integer n,
2                 parameter a_type resetval)
3               (Reg#(a_type) ifc[])
```

The interface is an *array* of register interfaces, indicated by the square brackets. A `CReg` module is instantiated with syntax like this:

```
1   // parameter n is 2, resetval is 15
2   Array #(Reg #(Bit #(4))) crg_counter <- mkCReg (2, 15);
```

This instantiates a `CReg` whose interface is an array of 2 register interfaces, and whose reset value is 15. Each interface can be accessed by indexing: `crg_counter [0]` and `crg_counter [1]`.

The key properties of a `CReg x` with an array of `n` register interfaces is:

- All the methods can be invoked in the same clock.
- A read at the j'th register interface,*i.e.,* `x[`j`]._read` returns the latest of:
  - the value v_{j-1}, if `x[`$j-1$`]._write(`v_{j-1}`)` is being invoked;

- else the value v_{j-2}, if `x[`$j-2$`]._write(`v_{j-2}`)` is being invoked;
- ...
- else the value v_1, if `x[1]._write(`v_1`)` is being invoked;
- else the value v_0, if `x[0]._write(`v_0`)` is being invoked;
- else the value in the register.

- The register value is updated with the latest of:
  - the value v_{n-1}, if `x[`$n-1$`]._write(`v_{n-1}`)` is being invoked;
  - else the value v_{n-2}, if `x[`$n-2$`]._write(`v_{n-2}`)` is being invoked;
  - ...
  - else the value v_1, if `x[1]._write(`v_1`)` is being invoked;
  - else the value v_0, if `x[0]._write(`v_0`)` is being invoked;
  - else the current value in the register.

18.3.1 A possible hardware implementation of a CReg

Figure 18.2 shows a possible hardware implementation of a CReg. On the left is a regular

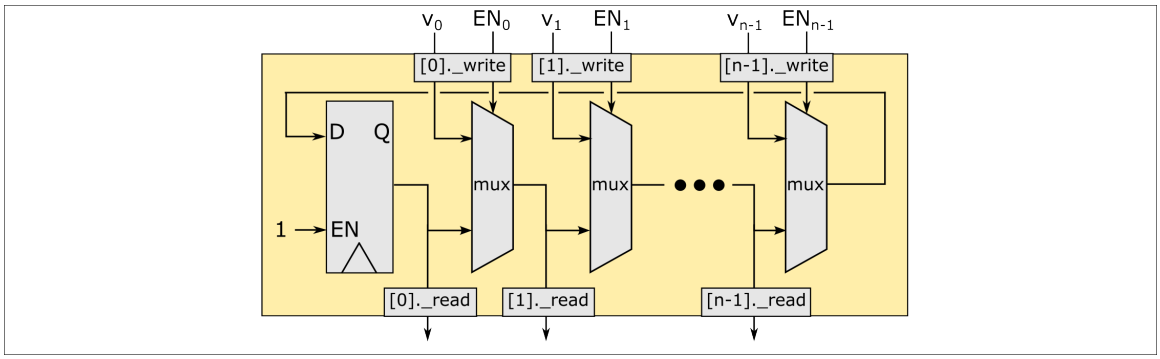


Figure 18.2: A possible hardware implementation of a CReg

D flip flop. To its right is a combinational circuit comprised of a series of n multiplexers, where n is the interface array size for the CReg. The top and bottom depict the n `.write` and `.read` methods, respectively.

The jth multiplexer selects either the output of the previous element (D flip flop or multiplexer), or the data argument of the `[`j`]._write` method. The selection is controlled by the EN input of the `[`j`]._write` method. Thus, if the `[`j`]._write` method is currently being invoked, that data is fed onward, otherwise the data from the previous element.

The `[`j`]._read` method returns the value before the j'th multiplexer.

A little study of the diagram will show that it indeed implements the semantics described in the previous section.

18.3.2 A Faster Up-Down Counter, using a `CReg`

Here is an alternative module implementing our Up-Down Counter interface, using a `CReg`:

```
module mkUp_Down_Counter_I (Up_Down_Counter_IFC);
   // STATE
   Array #(Reg #(Bit #(4))) crg_counter <- mkCReg (2,15);

   // ----------------------------------------------------------------
   // INTERFACE
   method Action incr (rg_counter != 15);
      rg_counter [0] <= rg_counter [0] + 1;
   endmethod

   method Action decr; (rg_counter != 0);
      rg_counter [1] <= rg_counter [1] - 1;
   endmethod

   method Bit #(4) val;
      return rg_counter [0];
   endmethod
endmodule
```

Now, three rules invoking the three methods can all be invoked on the same clock. Further, when they are invoked in the same clock, it answers the question, "What value does `val` return?":

> The original value in the CReg?
> Or the value after the increment?
> Or the value after the decrement?
> Or the value after the increment and decrement?

Per the semantics of `CReg`, it returns `rg_counter[0]`, which is the original value in the CReg. If we were to return `rg_counter[1]`, it would return the value after the increment and before the decrement. If our CReg had an array of 3 register interfaces intead of 2, we could return `rg_counter[2]` which would be the value after both the increment and decrement.

18.4 Example: Using a CReg for the RISC-V CSR `mcycle`

One of the RISC-V CSRs is `mcycle`. This CSR is incremented by the hardware automatically on every clock, to count total clock cycles. But this CSR can also be written from RISC-V code using a CSRRxx instruction (which was described in Section 2.7.2). Suppose the CSR contains the value n, and we are executing a CSRRxx instruction that wants to write m to the CSR. After the clock instant, should the value be $n + 1$? Or m? Or $m + 1$? Or something else? The RISC-V specification says it should be m, the value written by the CSRRxx instruction.

This can be cleanly expressed using a CReg.

from src_Common/CSRs.bsv

```
   Array #(Reg #(Bit #(64))) csr_mcycle <- mkCReg (2, 0);
```

The automatic increment in every cycle is performed by this rule in the CSRs module:

from src_Common/CSRs.bsv

```
   rule rl_count_cycles;
      csr_mcycle [0] <= csr_mcycle [0] + 1;
   endrule
```

To execute a CSRRxx instruction, a rule in the CPU invokes a method in the CSRs module which contains this code:

from src_Common/CSRs.bsv

```
   csr_mcycle [1] <= csr_val;
```

The use of the indexes [0] and [1] ensure that when a CSRRxx instruction is executed, its value will override the incremented value written by `rl_count_cycles`.

18.5 PipelineFIFOs and BypassFIFOs

Consider the following module implementing a 1-element FIFO with a `FIFOF` interface:

```
module mkFIFOF (FIFOF #(Bit #(32)));
   Reg #(Bit #(32)) rg_data <- mkRegU;
   Reg #(Bool)      rg_full <- mkReg (False);

   // ----------------
   // INTERFACE

   method Bool notEmpty ();
      return rg_full;
   endmethod

   method Bit #(32) first () if (rg_full);
      return rg_data;
   endmethod

   method Action deq () if (rg_full);
      rg_full <= False;
   endmethod

   method Bool notFull ();
      return (! rg_full);
   endmethod

   method Action enq (Bit #(32) x) if (! rg_full);
      rg_data <= x;
      rg_full <= True;
   endmethod

   method Action clear;
      rg_full <= False;
   endmethod
endmodule
```

Consider a producer rule `rl_P` that invokes `enq`, and a consumer rule `rl_C` that invokes `first` and `deq`, as shown in Figure 18.3.

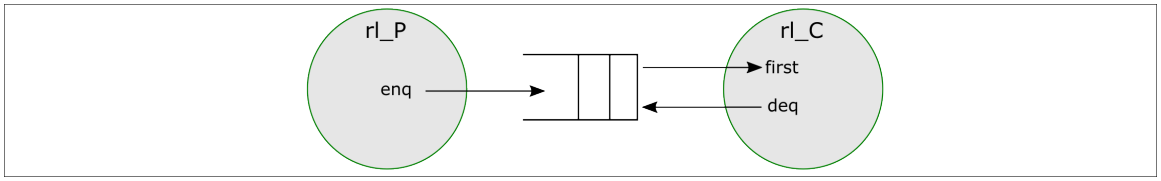


Figure 18.3: A producer and consumer connected with a FIFO

These rules cannot fire at the same instant (on the same clock) because both of them read and write register `fg_Full` (because both `enq` and `deq` read and write the register).

Let us use a CReg to relax this constraint, in two different ways.

18.5.1 `mkPipelineFIFOF`

```
1  module mkPipelineFIFOF (FIFOF #(Bit #(32)))
2     Reg #(Bit #(t))      rg_data  <- mkRegU;
3     Array #(Reg #(Bool)) crg_full <- mkCReg (3, False);
4
5     // ----------------
6     // INTERFACE
7
8     method Bool notEmpty ();
9        return crg_full [0];
10    endmethod
11
12    method Bit #(32) first () if (crg_full [0]);
13       return rg_data;
14    endmethod
15
16    method Action deq () if (crg_full [0]);
17       crg_full [0] <= False;
18    endmethod
19
20    method Bool notFull ();
21       return (! crg_full [1]);
22    endmethod
23
24    method Action enq (Bit #(32) x) if (! crg_full [1]);
25       rg_data      <= x;
26       crg_full [1] <= True;
27    endmethod
28
29    method Action clear;
30       crg_full [2] <= False;
31    endmethod
32 endmodule
```

Again, consider a producer rule `rl_P` that invokes `enq`, and a consumer rule `rl_C` that invokes `first` and `deq`. Now, these rules *can* fire together at the same instant (on the same clock).

Because of the choice of indexes we can see that, in the equivalent rule-at-a-time semantics, `rl_C` fires *before* `rl_P`. Thus, even if the FIFO was full at the start of the clock, `rl_P` can

still `enq` into the FIFO, *provided* `rl_C` is firing on the same clock. Per the rule-at-a-time semantics, `rl_C` fires first, which empties the FIFO, *i.e.,* `rl_P` sees the FIFO as empty and is therefore able to `enq` into it.

Note also that another rule invoking `clear`, too, can fire on the same clock as `rl_P` and `rl_C` and, because of its index, logically fires "last", and so will leave the FIFO in a finally empty state.

If we examine the Verilog generated for `mkPipelineFIFOF`, we will see that the READY signal for `enq` incorporates the ENABLE signal of the `deq` method (because if the FIFO is full from the previous clock, in this clock `enq` can only be invoked if `deq` is also being invoked). Thus, there is a *combinational path* backward through the FIFO (a path that involves only wires and gates and no state-element) from the `deq` method to the `enq` method. This is illustrated in Figure 18.4.

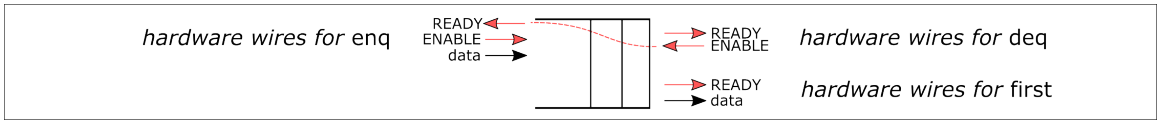


Figure 18.4: Combinational path through `mkPipelineFIFOF`

This kind of FIFO is called a "Pipeline FIFO" because it is an ideal candidate for the FIFO between stages of a pipeline. It allows the downstreanm stage (the consumer) and the upstream stage (the producer) to fire on the same clock, advancing data in a piplined manner. It is so useful that it is available in the *bsc* library in the `SpecialFIFOs` package.

EXERCISE: Please see Ex-17-A-PipelineFIFOs in Appendix E, page E-25. □

18.5.2 `mkBypassFIFOF`

In this version, we change the CReg indexes used by the methods.

```
1  module mkPipelineFIFOF (FIFOF #(Bit #(32)));
2     Array #(Reg #(Bit #(32))) crg_data <- mkCRegU (2);
3     Array #(Reg #(Bool))      crg_full <- mkCReg (3, False);
4
5     // ----------------
6     // INTERFACE
7
8     method Bool notEmpty ();
9        return crg_full [1];
10    endmethod
11
12    method Bit #(32) first () if (crg_full [1]);
13       return crg_data [1];
14    endmethod
15
16    method Action deq () if (crg_full [1]);
17       crg_full [1] <= False;
18    endmethod
```

```
19
20    method Bool notFull ();
21       return (! crg_full [0]);
22    endmethod
23
24    method Action enq (Bit #(32) x) if (! crg_full [0]);
25       crg_data [0] <= x;
26       crg_full [0] <= True;
27    endmethod
28
29    method Action clear;
30       crg_full [0] <= False;
31    endmethod
32 endmodule
```

Again, consider a producer rule `rl_P` that invokes `enq`, and a consumer rule `rl_C` that invokes `first` and `deq`. These rules *can* fire together at the same instant (on the same clock).

Because of the choice of indexes we can see that, in the equivalent rule-at-a-time semantics, `rl_P` fires *before* `rl_C`. Thus, even if the FIFO was empty at the start of the clock, `rl_C` can still read `first`, and `deq` the FIFO, *provided* `rl_P` is firing on the same clock. Per the rule-at-a-time semantics, `rl_P` fires first, which fills the FIFO, *i.e.,* `rl_C` sees the FIFO as full and is therefore able to use `first` and `deq`.

As a separate observation, we can see that another rule invoking `clear`, too, can fire on the same clock as `rl_P` and `rl_C` and, because of its index, logically fires "last", and so will leave the FIFO in a finally empty state.

If we examine the Verilog generated for `mkBypassFIFOF`, we will see that the READY signal for `first` and `deq` incorporates the ENABLE signal of the `enq` method (because, if the FIFO is empty at the previous clock, in this clock `first` and `deq` can only be invoked if `enq` is also being invoked). Further, when `enq` and `first` are invoked in the same clock, the data argument to `enq` is passed straight though to the data result of `first`. Thus, there are several *combinational paths* forward through the FIFO (paths that only involve wires and gates and no state-element) from the `enq` method to the `first` and `deq` methods. This is illustrated in Figure 18.5.

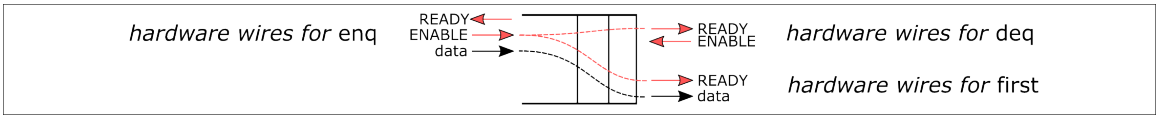


Figure 18.5: Combinational paths through `mkBypassFIFOF`

This kind of FIFO is called a "Bypass FIFO" because of the way data can "bypass" the FIFO from `enq` to `first`. It is so useful that it is available in the *bsc* library in the `SpecialFIFOs` package.

18.5.3 Back-to-back compositions of `BypassFIFOF` and `PipelineFIFOF`

An interesting component is a back-to-back composition of the two FIFOs discussed in the previous sections. Consider this code fragment:

```
module mk... (...);
   FIFOF #(Bit #(32)) f_bypass   <- mkBypassFIFOF;
   FIFOF #(Bit #(32)) f_pipeline <- mkPipelineFIFOF;

   // Producer rule (into f_bypass's enq side)
   rule rl_P;
      ... f_bypass.enq (x);
   endrule

   // Connect f_bypass's first/deq side to f_pipeline's enq side
   mkConnection (to_FIFO_O (f_bypass), to_FIFO_I (f_pipeline));

   // Consumer rule (from f_pipeline's first/deq side)
   rule rl_C;
      let y = f_pipeline.first;
      f_pipeline.deq;
   endrule
endmodule
```

This is illustrated in Figure 18.6. This composition has some pleasant properties:

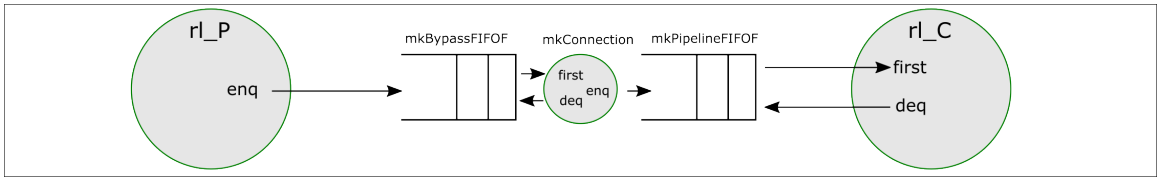


Figure 18.6: A producer and consumer connected with a composed BypassFIFO-PipelineFIFO.

- Normally, if we connect two FIFOs back-to-back like this, it will always take a minimum of two ticks for a datum to traverse through (from `enq` to `first`/`deq`—one tick in the first FIFO and one tick in the second. With this composition on the other hand, because of the Bypass FIFO, it can traverse in a single tick.
- Like the ordinary *bsc* library `mkFIFOF`, and unlike `mkPipelineFIFOF` and `mkBypassFIFOF` by themselves, the producer rule `rl_P` side and the consumer rule `rl_C` have no ordering constraints (they are conflict free). They can fire in the same clock and can go in either logical order.
- There are no combinational paths through the pair of FIFOs! One can verify this by generating the Verilog for the composed FIFOs and analyzing it (which, fortunately, does not have to be done by hand, because when the *bsc* compiler generates Verilog for a module, it helpfully reports combinational paths, if any, through the module.

These properties make it attractive to use this composition to connect stages in a pipeline, enabling a *modular* separation of stages. We can place `mkBypassFIFOF` in the module for the upstream stage, place `mkPipelineFIFOF` in the module for the downstream stage, and connect them in the parent module with `mkConnection`. This is illustrated in Figure 18.7. We use this structure extensively and exclusively in Fife to connect its stages.

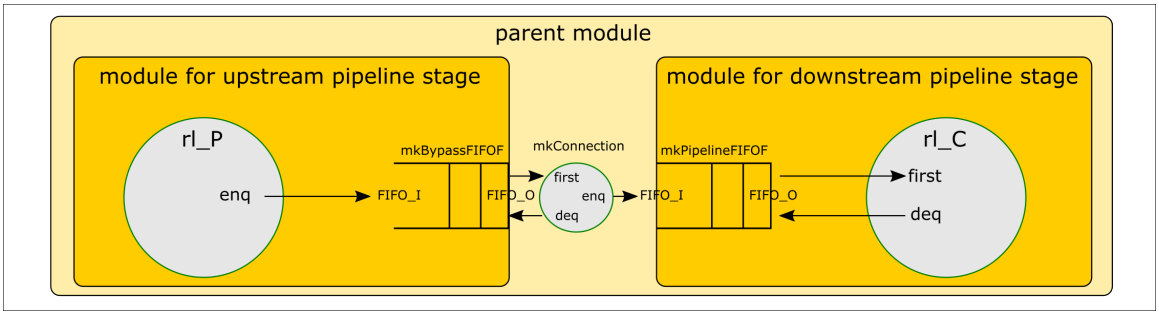


Figure 18.7: Pipeline-stage modularity enabled by composing a BypassFIFO and a PipelineFIFO

The only downside is that the composition has two data registers, one each in `mkBypassFIFOF` and `mkPipelineFIFOF`. In many applications, this does not matter much, *i.e.,* the overall state size is dominated by other components, and this addition is in the noise.

EXERCISE: Please see Ex-17-B-Pipeline-FIFO-Composition in Appendix E, page E-26. □

18.6 Alternatives to CRegs: RWires and their variants (deprecated)

In digital hardware,

- a register communicates a value across clocks (from one clock to succeeding clocks), and
- a wire communicates a value within a clock (from state element to gate, from gate to gate, and from gate to state element).

A BSV register is a standard digital hardware register. A BSV CReg plays the role of both register and wires, communicating values flexibly within a clock and across clocks, depending on whether its methods are invoked within a clock or across clocks. The concept of CRegs was first proposed by Daniel Rosenband and Arvind at MIT [20, 21] (they were originally called "Ephemeral History Registers" or EHRs).

Before the invention of CRegs, BSV had a facility to manage intra-clock communication, called the "`RWire`". These are still available in BSV, along with several specializations: "`Wire`", "`BypassWire`", "`DWire`", and "`PulseWire`". Please see the *bsc* library documentation for detailed information.

CRegs are semantically preferable to RWires because they fit cleanly into rule-at-a-time semantics, where a CReg can be treated as an ordinary register. The semantics of a BSV program with CRegs can be explained purely at the rule-at-a-time level, with no mention of clocks. RWires, on the other hand, are only meaninful with clocks, and are therefore semantically messier.

Since the inclusion of CRegs into the *bsc* library, the need for `RWire` and its variants has mostly disappeared.

Chapter 19

RISC-V: Optimizing Drum and Fife

19.1 Introduction

There are three physical dimensions on which we may want to optimize a CPU design:

- Time (performance): The speed (wall-clock time) at which a desired application is executed by the CPU.
- Space (area/resources): On ASICs, the silicon area occupied by the design. For ASICS this may be measured in square millimeters or in "number of gates". For FPGAs this may be measured in LUTs (Lookup Tables), BRAMs, DSPs and so on. Smaller is better, for many reasons (better silicon yields, lower power consumption, *etc.*.
- Energy: the amount of energy consumed to execute a desired application. Smaller is better.

These are not independent dimensions; improving one dimension often costs something in another dimension. Ultimately, a competitive product specification for a particular target market will define the acceptable boundaries for each dimension.

We will not say much in this book about energy optimization, even though it is of course increasingly important in modern times, both in the macro sense (reduce energy consumption for a greener planet) and in the micro sense (mobile devices and IoT (Internet of Things) devices work on battery power or energy harvested from the environment). Modern designs have many techniques to slow down or switch off clocks, and reduce circuit-supply voltage for less critical or idle circuits, both of which reduce energy consumption.

Regarding Performance, it is important to keep in mind that the ultimate number that matters is *application performance*, *i.e.,* how long does it take to execute an application of interest. One will often hear other numbers cited as, such as clock speed, instructions per clock (IPC) or its inverse clocks per instruction (CPI), or total number of instructions. All these contribute to application performance; they do not individually determine application performance. Conceptually, the total execution time for an application is:

<exec time> = <total number of instrs> × 1/<clock-speed> × CPI

the units being:

seconds = instructions × seconds/cycle × cycles/instruction

For a given application and a given set of input data, total number of instructions is a function of the ISA and compiler quality. ISAs like x86 or the older DEC PDP-11, DEC Vax, and Motorola 68000 were *Complex Instruction Sets*, where a single "CISC" instruction could express more work, such as a memory read, an integer op, and a memory write. RISC-V is a *Reduced Instruction Set*, where that same work needs separate "RISC" instructions for LOAD, Integer and STORE. And, of course, a better compiler may produce a smaller program (fewer instructions) from the same source code.

CPI is not a constant. First, it may vary inherently across different kinds of instructions—an integer ALU instruction may take fewer cycles than a memory-access instruction or a floating-point instruction. Second, even for a particular kind of instruction, CPI may vary; for example, the number of cycles for a memory-access instruction may depend on hits/misses in caches, hits/misses in virtual memory TLBs (Trqnslation Lookaside Buffers), *etc.*. Third, an integer that reads a register may stall for a number of cycles because a preceding instruction has not yet written the register. Thus, in the above formula, CPI is just an average CPI across the application.

Clock speed may not be constant; in some implementations, clocks are slowed down, or even switched off for non-performance-critical components during intervals that do not demand high performance (*e.g.,* "idling").

The terms in the formula above need simultaneously to be optimized for the best product. They are not independent; higher clock speeds restrict how much "circuit work" can be done in a single clock which, in turn, affects microarchitecture (*e.g.,* may need to split an operation into multiple pipeline stages); which, in turn, can affect instructions/cycle. A CISC ISA may get more work done with fewer instructions, but a RISC ISA may be implementable with much higher clocks speeds and more pipeline parallelism.

Instructions/cycle depends on microarchitecture. The more parallelism we can exploit, the higher the instructions/cycles that can be achieved. A common term to refer to this measure is "ILP" (Instruction-Level Parallelism).

For example, in both Drum and Fife, any particular instruction takes five or more cycles, from Fetch to Writeback. Drum, having an FSM microarchitecture, therefore retires one instruction every five or more cycles. Fife, which executes many instructions in parallel in its pipelined microarchitecture, can often retire one instruction per cycle.

An advanced microarchitecture with more parallelism is the *superscalar* microarchitecture, which may fetch and execute 2 or 4 (or more) instructions at a time.

Another advanced microarchitecture with more parallelism is the *out-of-order* microarchitecture which may, for example, have multiple integer execution units, and allow multiple integer instructions to execute at a time, or even out-of-order depending on when their input data is available. The cache in their DMem may support "non-blocking caches" or "hits-under-misses" *i.e.,* for two memory accesses a_1 and a_2 that arrive in that order, it permits a_2 to be serviced while a_1 may be awaiting a cache-refill due to a cache-miss.

Superscalarity and Out-of-order microarchitectures are beyond the scope of this book. In the rest of this chapter we will discuss improving the resources and performance of Drum and Fife.

19.2 Pipeline traces and visualization to analyze performance

Before trying to optimize a CPU, we must first analyze and understand the existing implementation thoroughly so that we can then identify opportunities for improvement.

The first step is to produce a *pipeline trace*, which has more fine-grain detail than a mere *instruction trace* (which only records the trace of instructions retired). We record, for every instruction, its transit through *each* stage of the FSM (Drum) or pipeline (Fife).

For example, if we examine the Fife code for `rule rl_Decode` we see an invocation of a chain of functions:

src_Fife/S2_Decode.bsv

```
     log_Decode (rg_flog, y, rsp_IMem);
```

⟶

src_Common/Fn_Decode.bsv

```
   function Action log_Decode (File flog, Decode_to_RR y, Mem_Rsp rsp_IMem);
      ...
      ftrace (flog, y.inum, y.pc, y.instr, "D", $format(""));
      ...
```

⟶

src_Common/Utils.bsv

```
function Action ftrace (File         flog, ...)
         ...
         $fdisplay (flog, "Trace %0d %0d %0h %0h %s", cur_cycle,
                    inum, pc, instr, label, adhoc);
```

This writes a line like this into the logfile produced by Drum and Fife when they are simulated:

log.txt

```
...
Trace 6 2 80000008 fe012e23 D
...
```

which records the fact that on clock tick 6 (cycle 6), the 2nd instruction, whose PC is `0x8000_0008`, and whose 32 bits are `0xfe01_2e23`, was processed in the Decode stage. Similarly, we write a trace item for every interesting microarchitectural event for each instruction.

By studying this code, manually or with analysis software, we can get an idea of exactly how many cycles each instruction takes, and where we may be "losing cycles", if any.

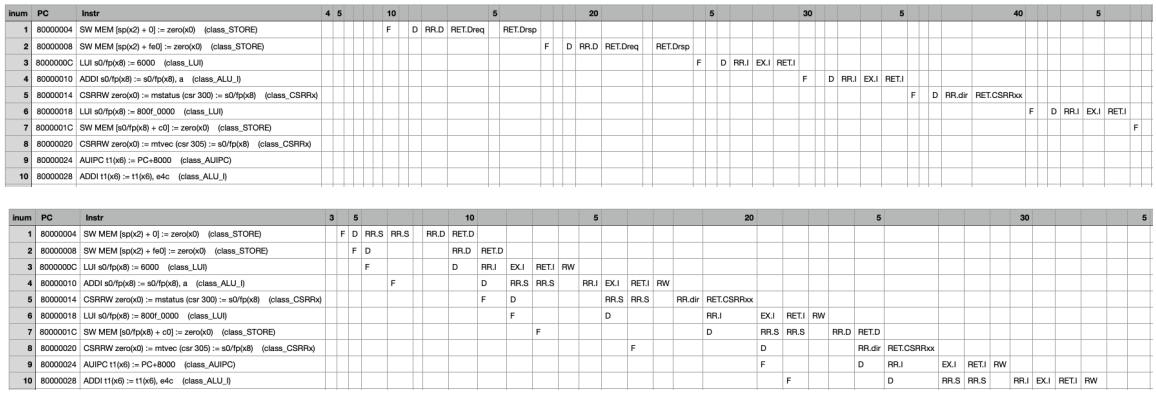
This trace can also be processed and displayed in a "pipeline visualization" tool. If we take the pipeline trace `log.txt` produced by Drum or Fife, we can run it through the Python program `Log_to_CSV.py` provided with Drum and Fife:

log.txt

```
$ Tools/Log_to_CSV/Log_to_CSV.py  log.txt  0 100
```

This selects from instruction 0 through instruction 100 from the pipeline trace file, and produces a file `log.txt.csv` in the standard "comma-separated variables" data input format that is accepted by most spreadsheed programs (OpenOffce, Microsoft Excel, Mac Numbers, Google Sheets).

Figure 19.1 shows the display, in the Mac Numbers spreadsheet application, for the pipeline trace of up to the first few instructions for the "Hello World!" C program running on Drum and Fife, respectively:



inum	PC	Instr	4	5			10				5				20					5				30				5				40				5		
1	80000004	SW MEM [sp(x2) + 0] := zero(x0) (class_STORE)					F		D	RR.D	RET.Dreq			RET.Drsp																								
2	80000008	SW MEM [sp(x2) + fe0] := zero(x0) (class_STORE)											F		D	RR.D	RET.Dreq			RET.Drsp																			
3	8000000C	LUI s0/fp(x8) := 6000 (class_LUI)																			F		D	RR.I	EX.I	RET.I												
4	80000010	ADDI s0/fp(x8) := s0/fp(x8), a (class_ALU_I)																									F		D	RR.I	EX.I	RET.I						
5	80000014	CSRRW zero(x0) := mstatus (csr 300) := s0/fp(x8) (class_CSRRx)																															F		D	RR.dir	RET.CSRRxx		
6	80000018	LUI s0/fp(x8) := 800f_0000 (class_LUI)																																				F	
7	8000001C	SW MEM [s0/fp(x8) + c0] := zero(x0) (class_STORE)																																					F
8	80000020	CSRRW zero(x0) := mtvec (csr 305) := s0/fp(x8) (class_CSRRx)																																					
9	80000024	AUIPC t1(x6) := PC+8000 (class_AUIPC)																																					
10	80000028	ADDI t1(x6) := t1(x6), e4c (class_ALU_I)																																					

inum	PC	Instr	3	5					10					5				20					5				30					5
1	80000004	SW MEM [sp(x2) + 0] := zero(x0) (class_STORE)		F	D	RR.S	RR.S		RR.D	RET.D																						
2	80000008	SW MEM [sp(x2) + fe0] := zero(x0) (class_STORE)			F	D				RR.D	RET.D																					
3	8000000C	LUI s0/fp(x8) := 6000 (class_LUI)				F				D	RR.I	EX.I	RET.I	RW																		
4	80000010	ADDI s0/fp(x8) := s0/fp(x8), a (class_ALU_I)					F				D	RR.S	RR.S		RR.I	EX.I	RET.I	RW														
5	80000014	CSRRW zero(x0) := mstatus (csr 300) := s0/fp(x8) (class_CSRRx)									F	D				RR.S	RR.S		RR.dir	RET.CSRRxx											
6	80000018	LUI s0/fp(x8) := 800f_0000 (class_LUI)										F				D				RR.I	EX.I	RET.I	RW								
7	8000001C	SW MEM [s0/fp(x8) + c0] := zero(x0) (class_STORE)											F							D	RR.S	RR.S		RR.D	RET.D						
8	80000020	CSRRW zero(x0) := mtvec (csr 305) := s0/fp(x8) (class_CSRRx)															F				D				RR.dir	RET.CSRRxx					
9	80000024	AUIPC t1(x6) := PC+8000 (class_AUIPC)																			F				D	RR.I	EX.I	RET.I	RW		
10	80000028	ADDI t1(x6) := t1(x6), e4c (class_ALU_I)																				F				D	RR.S	RR.S		RR.I	EX.I	RET.I	RW

Figure 19.1: Visualization of the per-instruction stage events in Drum and Fife

In both displays, the vertical axis, going downwards, shows instruction number 1, 2, 3, ... The horizontal axis, going to the right, shows the PC and disassembled instruction in the first two columns, followed by clock ticks. For each instruction, we see its instruction numbrer, PC and dissassembly, and then the Drum/Fife stage events at the clock tick where the event happened.

We can clearly see the difference between FSM sequenced (Drum) and pipelined (Fife) behavior. In Drum, after the first instruction has completed ("RET.Drsp" = Retire DMem Response) in tick 6, the Fetch ("F") for the second instruction occurs in tick 7. In Fife, after the Fetch for the first instruction on tick 4, the Fetch for the second instruction occurs in tick 5, the very next tick.

Considering IPC, we can see that in Drum, we have barely completed 6 instructions at tick 46, whereas Fife is has finished 10 instruction by tick 33. The tradeoff, as suggested earlier, is that Drum takes far fewer resouces (LUTs, gates) in silicon compared to Fife.

For each instruction, after its Fetch, horizontal gaps indicate lost cycles, *e.g.,* an instruction stalling at Register Read because it is waiting for an earlier instruction to write that register value, or a memory access waiting for the memory response. In Fife, we can also see which instructions were discarded (more lost cycles) due to PC misprediction or traps.

19.3 Optimization opportunities in Drum and Fife

The following sections discuss a number for optimization opportunities for Drum and Fife. Whether they actually produce improvements or not is usually difficult to predict just from observation or analysis, because an optimization's improvement in one dimension may come at a cost in another and, further, optimizations may interact positively or negatively. Ultimately, one must implement the optimizations and *measure* the actual impacts on clock speed, IPC, application performance, resources, and energy consumption, *etc.* and then make a deployment choice based on actual product priorities.

Each of these opportunties is a potential project for the reader of this book, starting with the existing Drum or Fife code, for more practice with RISC-V and **BSV**. Almost all of them them require no new **BSV** concepts beyond what has already been covered.

19.3.1 Drum and Fife: Fusing Decode and RR-Dispatch

More FSM steps in Drum or more stages in Fife (longer pipelines) will, in general, enable higher clock speeds (because the circuits in each stage have less to do, and so have less circuit delay). But more stages increase the latency (number of ticks) for each instruction, and the resources (more register or FIFO buffering between stages).

In some cases it is worthwhile going in the opposite direction, fusing what used to be two stages into one. A functionally easy pair to fuse is Decode and RR-Dispatch. In Drum, the Decode action looks like this:

src_Drum/CPU.bsv

```
Action a_Decode =
action
   let mem_rsp <- pop_o (to_FIFOF_O (f_IMem_rsp));
   let y       <- fn_Decode (rg_Fetch_to_Decode, mem_rsp, rg_flog);
   rg_Decode_to_RR <= y;
   ...
endaction;
```

It writes it output into the `rg_Decode_to_RR` register. The Register-Read-and-Dispatch action looks like this and reads from the register:

src_Drum/CPU.bsv

```
Action a_Register_Read_and_Dispatch =
action
   let x       = rg_Decode_to_RR;
   ...
endaction
```

We can fuse these into a single action, like this:

```
Action a_Decode_and_Register_Read_and_Dispatch =
action
   // Decode part
   let mem_rsp <- pop_o (to_FIFOF_O (f_IMem_rsp));
   let y       <- fn_Decode (rg_Fetch_to_Decode, mem_rsp, rg_flog);
   // RR-and-dispatch part
   let x       = y;
```

```
    ...
endaction
```

directly passing the output of Decode (`y`) as `x` in Register-Read-and-Dispatch. This allows us to eliminate the `rg_Decode_to_RR` register. In the main Drum FSM, we replace the original two sequenced actions:

```
    a_Decode;
    a_Register_Read_and_Dispatch;
```

with a single action:

```
    a_Decode_and_Register_Read_and_Dispatch;
```

The same fusion can be done in Fife as well: we fuse `rl_Decode` (in package `S2_Decode`) with `rl_RR_Dispatch` (in package `S3_RR_RW`) into a single rule, and eliminate the FIFOs connecting the two modules (through which we were passing the `Decode_to_RR` intermediate value).

In both cases, we reduce, by 1 tick, the transit through Decode and Register-Read-and-Dispatch, and we eliminate the intermediate buffer (register or FIFOs), both of which appear to be improvements.

On the other hand, the depth of the combined stage combinational circuit is now the sum of the depths in the original two stages; this may reduce the clock speed at which we can run it.

Fusion may not reduce the achievable clock speed if some other stage already limits the achievable speed. In this case we would say that the pipeline was originally "unbalanced" and that fusion has brought it closer to balance (all stages having approximately the same combinational delay). Fusion has merely utilized some timing "slack" that was already available in the two stages.

19.3.2 Drum and Fife: Fusing some Retire actions

In both Drum and Fife, exception handling is done in one place, by itself. In Drum, it is the last, separate action in the FSM, `a_exception`, in module `mkCPU` in `src_Drum/CPU.bsv`. In Fife, it is in rule `rl_exception` in module module `mkRetire` in `src_Fife/S5_Retire.bsv`.

In both cases, the exception is actually detected in earlier rules, which save values in three registers `rg_epc`, `rg_cause` and `rg_tval` which are then used by the exception handling action.

Similar to the fusion idea of Section 19.3.1, the exception-handling action can be fused directly into the earlier actions where the exception was detected, and the three registers can be eliminated.

This transformation to reduce ticks for exception-handling may be less important because, almost by definition, exceptions should be rare, and therefore saving this tick may not affect application performance very much.

19.3.3 Drum (rules version): short-circuiting stages

In the "rules" version of Drum (`src_Drum/Drum_Rules.bsv` and described in Chapter 14), the "retire" rules (`rl_Retire_direct`, `rl_Retire_Control` `rl_Retire_Int` and `rl_Retire_DMem` set the next action to `A_EXCEPTION`.

This, in turn, enables the next rule `rl_exception` which tests whether there actually was an exception, and if not, sets the next action to `A_FETCH`.

This structure emerged due to a blind mechanical mimicing of the structure of the `StmtFSM` version of Drum.

Instead, in each of the "retire" rules, we can replace:

```
    rg_action <= A_EXCEPTION;
```

with:

```
    rg_action <= (rg_exception ? A_EXCEPTION : A_FETCH);
```

allowing it to "jump" back to FETCH immediately, instead of spending another tick in rule `rl_exception`.

This kind of "jump" is not possible in `StmtFSM`, which only allows structured compositions of sequencing and if-then-else.

19.3.4 Drum: using narrower inter-stage buffers

Consider the output of the `fn_Dispatch` in `src_Common/Fn_Dispatch.bsv`, with the following type:

src_Common/Fn_Dispatch.bsv: line 29 ...

```
1 typedef struct {
2    RR_to_Retire      to_Retire;
3    RR_to_EX_Control  to_EX_Control;
4    RR_to_EX          to_EX;
5    Mem_Req           to_EX_DMem;
6 } Result_Dispatch
7 deriving (Bits, FShow);
```

In Drum, this result is stored in register `rg_Dispatch`. The width of this buffer is the sum of the widths of each of the four component types:

```
      Number of bits in to_Retire
+     Number of bits in to_EX_Control
+     Number of bits in to_EX
+     Number of bits in to_EX_DMem
```

The `to_Retire` field has type `RR_to_Retire`, which contains an `exec_tag` field, whose type is:

src_Common/Inter_Stage.bsv: line 82 ...

```
typedef enum {EXEC_TAG_DIRECT,
              EXEC_TAG_CONTROL,
              EXEC_TAG_INT,
              EXEC_TAG_DMEM
} Exec_Tag
deriving (Bits, Eq, FShow);
```

Observe that `exec_tag` determines which of fields in the result of `fn_Dispatch` other than `to_Retire` are relevant:

- When `exec_tag` is `XEC_TAG_CONTROL` then only the `to_EX_Control` field is relevant.
- When `exec_tag` is `XEC_TAG_INT` then only the `to_EX` field is relevant.
- When `exec_tag` is `XEC_TAG_DMEM` then only the `to_EX_DMem` field is relevant.

With this observation, we can see that, in `rg_Dispatch`, we can have a common set of bits shared by these three cases. Then, the number of bits in `rg_Dispatch` could be reduced to:

```
    Number of bits in to_Retire
+   max ( Number of bits in to_EX_Control
          Number of bits in to_EX
          Number of bits in to_EX_DMem )
```

BSV has a convenient type-notation to express this "overlaying of alternatives". In this case, it would look like this:

```
typedef struct {
   RR_to_Retire      to_Retire;    // without the exec_tag field

   union tagged {
      void              EXEC_TAG_DIRECT;
      RR_to_EX_Control  EXEC_TAG_CONTROL;
      RR_to_EX          EXEC_TAG_INT,
      Mem_Req           EXEC_TAG_DMEM
   } exec_tag

} Result_Dispatch
deriving (Bits, FShow);
```

The new nested `exec_tag` field combines the previous "enum" type and the 3 struct fields into a single "union" indicating the alternatives for each tag.

However, note that this sharing requires a MUX on the input to the register which routes the different possible substructs into the shared bits. This could add significant combinational circuit cost.

19.3.5 Fife: saving FIFO resources

In Section 16.3 with Figure 16.2 and in Section 18.5.3 with Figure 18.7 we described how we connect Fife stages in a modular way, using a pair of FIFOs—a BypassFIFO in one stage (module) and a PipelineFIFO in the other, connected using `mkConnection` in the parent

CPU module. We also explained how, while improving modularity, separate compilation and stage independence, it also costs us in resources: inside the pair of FIFOs are two data registers for what is essentially used as a 1-element FIFO.

We may now selectively optimize these FIFO pairs, replacing such a pair by a single FIFO. The upside is that the buffer now has just one register for the buffer. The downsides are (a) we now have a scheduling constraint across the buffer, where the rule on one side has to be scheduled before the rule on the other side, and (b) we now have a combinational path spanning both both stages. The longer combinational path may reduce achievable clock speed.

Whether we use a PipelineFIFO or a BypassFIFO for this purpose depends on where it is used, because of the scheduling constraint. Usually we will need a PipelineFIFO in a forward path and a BypassFIFO in a reverse path because, for pipeline behavior we need to schedule a downstream rule before an upstream rule.

19.3.6 Fife: Reducing the misprediction penalty

In Fife, in Fetch stage, after issuing an IMem read-request for the instruction using the address in `rg_pc`, it "predicts" the next instruction address to be `rg_pc+4`. That prediction could of course be wrong, if the previous instruction is a BRANCH or JUMP that takes it somewhere else, or if it traps, which takes it to the instruction address of the trap-handling code.

As described in Section 15.2, we manage these "mispredictions" using "epochs". At any given time, we are in some particular prediction epoch. In Fetch, we attach the current epoch and the current next-instruction prediction to the instruction. In Retire, we can definitively check the next-instruction prediction. If mispredicted, we increment the epoch redirect Fetch with the correct PC and new epoch. Retire starts discarding all subsequent instructions thqt have the old epoch, because they are all in the "misprediction shadow", and resumes normal operation only it sees the first instruction carrying the new epoch.

The time wasted on these discarded instructions is called the "*misprediction penalty*". Clearly, we would like to reduce this wasted time.

19.3.6.1 Save a tick for Fetch redirection using CRegs

In package `S1_Fetch`, rule `rl_Fetch_req` reads `rg_pc` for the instruction address used for IMem requests, and reads `rg_epoch` to accompany the instruction down the pipe (A).

Concurrently, `rl_Fetch_from_Retire` updates `rg_pc` and `rg_epoch` with a redirected PC and new epoch (B).

When `rg_pc` is a standard register, the update from (B) is visible to (A) only on the next tick.

Suppose:

- we replace `rg_pc` and `rg_epoch` by CRegs `crg_pc` and `crg_epoch` ((Section 18.3);
- we replace the register-writes (B) with a writes to `crg_pc[0]` and `crg_epoch[0]`, and
- we replace the register-reads (A) with reads from `crg_pc[1]` and `crg_epoch[1]`.

Then, the updates in (B) is visible to (A) in the *same* tick.

Caution: we now have combinational paths (through the CReg) from (B) into (A), with a potential negative impact on clock speed.

19.3.6.2 Save a tick for Fetch redirection by eliminating backward FIFO

Currently, the redirection from Retire back to Fetch spends a tick in a FIFO (see FIFOs `f_Fetch_from_Retire` in Retire and in Fetch stages).

We can eliminate this tick by having Retire directly write into registers `rg_pc` and `rg_epoch` for the redirection update.

Caution: we have reduced the isolation of the two stages Retire and Fetch, resulting in longer combinational paths, with a potential negative impact on clock speed.

19.3.6.3 Quicker reaction to redirection by Register-Read and Dispatch

Currently, mispredicted instructions go through all their usual processing before being discarded at Retire. This may include several delays and possibly use resources:

- Stall at Register-Read due to read/write hazards.
- Reserve an Rd register, which then has to be un-reserved by Retire when the instruction is discarded.
- Spend multiple cycles in some execution stages (memory-access, integer multiply/divide, floating-point, ...).
- For DMem ops, "pollute" the cache with unnecessary accesses
- For DMem STORE ops, use up an entry in the store-buffer which then has to be discarded by Retire when the instruction is discarded.

If, in addition to the Retire⟶Fetch redirection message, we also feed the message to the Decode stage and/or the Register-Read-and-Dispatch stage, they can immediately start discarding wrong-epoch instructions, or convert them into no-ops. In either case, they no longer encounter the delays or use resources as listed above.

19.3.6.4 Better next-PC prediction

The Fetch stage currently performs a fixed next-PC prediction like this:

src_Fife/S1_Fetch.bsv

```
    let pred_pc = rg_pc + 4;
```

This is not bad, since in most codes there are medium to long stretches of "straight-line" code between BRANCH and JUMP instructions. Applications in the "scientific" domain have a lot of well-structured loops operating on vectors and rectangular matrices; in these applications it is often possibly to "unroll" loops significantly, producing *very* long sequences of straight-line code (hundreds of instructions).

Still, prediction can be improved further; in modern processors, prediction success rates can be in the high 90% range, close to 100%.

Let us restructure the code slightly. Suppose, in this module `mkFetch`, we have instantiated a "`pc_predictor`" module with a "`pc_predict`" ActionValue method

src_Fife/S1_Fetch.bsv
```
    let pred_pc <- pc_predictor.predict (rg_pc);
```

If the `pc_predict` method simply increments its argument by 4 and returns it, we are at status quo, *i.e.,* equivalent to the original code. But now we can replace the `pc_predictor` module with more sophisticated versions for better prediction.

Branch Target Buffers One technique is to add a "*Branch-Target Buffer*" (BTB), which is a table associating the PC of a BRANCH instructions with the next-PC when the BRANCH was recently executed.

Consider a simple loop like this:

```
        initialize loop index and limit
B1:     conditional BRANCH to B3 if loop index > limit
        ...
        loop body and increment loop index
        ...
B2:     unconditional BRANCH to B1
B3:
```

Suppose the loop iterates 100 times. Our simple PC+4 prediction will fail on every loop iteration at B2, and once at B1 at the end of the loop, *i.e.,* there will be 101 mispredictions.

When the branch at B1 or B2 reaches Retire, we know whether it was mispredicted or not. If mispredicted, then in the redirection message sent back to Fetch, we also indicate that this was a BRANCH instruction. In Fetch, in rule `rl_Fetch_from_Retire` where we update the PC and epoch, we also invoke a method `branch_train()` in the `pc_predictor` module informing it about the BRANCH's PC and the actual next-PC, and this pair is stored in the BTB table.

In the `pc_predict(pc)` method, we consult the BTB for the argument PC. If we find a match, we return the associated next-PC; else, we return the default PC+4 prediction.

Now let us consider again our loop example. The first time we arrive at B2 we will mispredict PC+4, because B2 is not yet in the BTB, but we will also enter (B2,B1) into the BTB. In subsequent arrivals at B2, we will find B2 in the BTB and correctly predict that the next-PC is B1. Regarding B1, for the first 100 arrivals we will predict PC+4 (the default prediction) which is correct, but at the 101st (last) arrival this will be a misprediction.

The net outcome is that for the whole loop, where previously we had 101 mispredictions, the BTB reduces it to just 2 mispredictions. Also remember that "101" was based on the number of loop iterations (we assumed 100)—for a 1000-iteration loop, we would have 1001 mispredictions. The simple BTB has reduced it to a constant 2 mispredictions, no matter how many iterations.

Adding hysteresis to the BTB Consider executing our example loop a second time (perhaps it is nested inside an outer loop, or is in a function that has been called twice). Now, B2 will have no mispredictions, because the BTB already has a (B2,B1) entry. The first time

we encounter B1 it will be mispredicted, because the last time we executed the loop, at loop exit we would have entered (B1,B3) into the BTB, whereas now we need PC+4. This misprediction will restore B1's prediction back to PC+4, and so for subsequent iterations, B1 will predict correctly. On the last iteration, B1 will again mispredict as PC+4, instead of B3. Thus, we have two mispredictions for the loop.

One technique is to add a little "hysteresis" or "delay"into the BTB. From Retire, we were sending a redirection message back to Fetch only on mispredictions. Now, suppose Retire *always* sends a message to Fetch for BRANCH instructions; in the case of a correct prediction, this is not a redirection but merely a confirmation of correct prediction. For each entry in the BTB, suppose we associate a 1-bit counter. When we create a new entry in the BTB, we enter (pc1,pc2,0). When we subsequently receive a confirmation for this prediction, we increment it to (pc1,pc2,1). The first time we receive a misprediction/redirection for pc1, we decrement it to (pc1,pc2,0), *i.e.,* we don't change the BTB prediction, we merely reduce the "confidence level" of the prediction. The second time we receive a misprediction/redirection for pc1, we change the prediction to the redirected pc. Thus the 1-bit counter "delays" the change in the BTB.

Now consder again our example loop. In the first execution of the loop (with no entry for B1 in the BTB), from the first iteration we get (B1,B1+4,0) in the BTB, and from the second iteration onwards we have (B1,B1+4,1). After the 100th iteration, B1 will mispredict (it branches to B3). The BTB entry will decrement to (B1,B1+4,0), but we do not change the prediction. Now, in the second execution of the loop, in the first interation, we will predict correctly, with the BTB entry now incrementing again to (B1,B1+4,1). We will mispredict only after the final iteration, on the loop exit. Thus, in repeated executions of this loop, we have reduced the mispredictions from 2 to 1. If this loop is executed 1000 times (it is nested inside another loop), that reduction of mispredictions goes from 2000 to 1000.

Control-flow-indexed BTBs In many RISC-V programs we will find that we can more accurately predict the outcome of a BRANCH if we knew how we arrived at that BRANCH, *i.e.,* the instruction trace that preceded that BRANCH. We don't need every instruction in the preceding trace, just a trace of recent BRANCHes, *i.e.,* something that summarizes the *control flow* leading up to the BRANCH. Some BTBs are indexed by a short control-flow trace ending at the BRANCH PC, instead of just the BRANCH PC by itself.

Return Address Stacks JAL instructions are mostly used for subroutine calls, and JALR insructions are mostly used for subroutine returns (also subroutine calls to "distant" addreses or computed addresses) . By convention, compilers use RISC-V register `x1` (also known as `ra`) for the saved "return address" (saved in a call and used in a return). Thus, when we see a JAL or JALR whose Rd register field is `x1`, it is probably a subroutine call. When we see a JALR instruction whose `rs1` field is `x1` and whose 12-bit immediate field is 0, it is probably a subroutine return. These observations (that a JAL or JALR is likely a call or return) can detected in the Decode stage; when the instruction reaches Retire, it can pass this information to Fetch on the redirect path, and there it can be recorded in the PC predictor sub-module.

In the PC predictor module, we can maintain a small local stack called the "Return Address Stack") or RAS. When asked to predict for a PC that is known to be a subroutine call, in

addition to the prediction we can push PC+4 on the RAS. When asked to predict for a PC that is known to be a subroutine return, we can pop the prediction from the RAS. In effect, the RAS is "shadowing" or "mimicing" what happens on the actual program stack for return-addresses.

Final remarks on PC prediction The PC-predictor module requires careful engineering. First, the `pc_predict` method must be fast—it is invoked on every cycle by `rl_Fetch` in the Fetch stage. Thus, BTBs and RASs cannot be too large, otherwise it will not be possible perform lookups at speed. Updates to the BTB and RAS (due to fresh information from Retire) must occur concurrently with lookups, but not interfere with lookups.

There is also a potential security problem with PC predictors. PC predictors, in their BTBs and RASs, are recording some abtracted information about the behavior of a program. Suppose we switch to another (malicious) process without clearing this information. That progam may be able to execute code that measures whether certain branches are predicted or not, and thereby glean some information about the orignal progam. These kinds of "side-channel" attacks have become quite famous in recent years. One solution is to clear the PC predictor state completely on a process-switch. Another is to partition the PC predictor state and to reserve and use different partitions for different processes.

In general next-PC prediction can be viewed as an "online machine-learning" problem. It is a "machine-learning" problem because we are "training" the PC predictor with a data set (past trace of the program), and then we are making predictions based on that training. It is "online" because training is interleaved with use (as opposed to "offline", where training and use are completely separate phases). It is also "online" because training is happening on live data, not a previously collected data set. These observations suggest some ideas for exploration:

- Training does not have to be online. One can run a program on a simulator and save precise prediction data to be pre-loaded into the online predictor. This could be done on a shorter run of the program (smaller input data) if the prediction data remains stable.
- Modern machine-learning techniques could be applied to the prediction problem.

19.3.7 Fife: Reducing the register-hazard penalty

The register-hazard penalty is the number of cycles an instruction I2 may stall (idle wait) in the Register-Read stage because one or more of its input or output registers are "busy"—there is an earlier instruction I1 ahead of it in the pipeline that will be writing to one of those registers. This "stall" situation is detected and managed using the scoreboard (Section 15.3).

19.3.7.1 Fife Bypassing: Save a tick in GPRs and Scoreboard using CRegs

In package `S3_RR_RW`, rule `rl_RR_Dispatch` reads `rg_scoreboard` to detect whether it should stall, and `gprs` when it can proceed (A).

Concurrently, `rl_RW_from_Retire` updates `rg_scoreboard` and `gprs` with new information from Retire (B).

When `rg_scoreboard` is a standard register, and `gprs` do ordinary reads/writes into its underlying `RegFile`, the updates from (B) is visible to (A) only on the next tick.

Suppose we replace `rg_scoreboard` with a 2-port CReg `crg_scoreboard` (Section 18.3). In `rl_RW_from_Retire` we write port [0] of the scoreboard, and in `rl_RR_Dispatch` we read port [1] of the scoreboard. Then, reading the scoreboard (A) can receive the updated value from (B) in the same cycle.

We need to do something similar for the GPRs. In module `mkGPRs` we add a 2-port CReg:

src_Common/GPRs.bsv

```
Array #(Reg #(Tuple2 #(Bit #(5), Bit #(xlen)))) crg <- mkCReg (2, tuple2 (0, 0));
```

and we modify the methods as follows:

src_Common/GPRs.bsv

```
method Bit #(xlen) read_rs1  (Bit #(5) rs1);
   match { .rd, .rd_val } = crg [1];
   return ((rs1 == rd) ? rd_val : regfile.sub (rs1));
endmethod

... similarly read_rs2 ...

method Action write_rd (Bit #(5) rd, Bit #(xlen) rd_val);
   let v = ((rd == 0) ? 0 : rd_val);
   crg [0] <= tuple2 (rd, rd_val);
endmethod
```

Thus, when `write_rd` and `read_rd` are invoked in the same cycle on the same register (`rd` and `rs1`, respectively), the write-value `rd_val` is passed directly to the read-method in the same cycle (otherwise the read method gets it from the register file, as usual).

With these two changes, we save one tick in the register-hazard delay.

Caution: we now have combinational paths (through the CRegs) from (B) into (A), with a potential negative impact on clock speed.

19.3.7.2 Save a tick for register update by eliminating backward FIFO

Currently, the register update from Retire back to Register-Read-and-Dispatch spends a tick in a FIFO (see FIFOs `f_RW_from_Retire` in Retire and in Register-Read-and-Dispatch stages).

We can eliminate this tick by having Retire directly write into registers `rg_scoreboard` and `gprs`.

Caution: we have reduced the isolation of the two stages Retire and Register-Read-and-Dispatch, resulting in longer combinational paths, with a potential negative impact on clock speed.

19.3.7.3 Fife: Dispatching multiple instructions that write to the same Rd

Suppose we have two instructions, I1 followed by I2, that both write to the same register Rd. In the current code, I1 reserves Rd in the scoreboard, and I2 stalls if I1 has not released

the reservation (not yet performed its write into the GPRs). But I2 does not need the value written to the GPR; so why did we stall? Suppose there is a third insruction I3 whose Rs1 == Rd. If we had allowed I2 to proceed, then when I1 finishes, it will release the Rd reservation, which will allow I3 to proceed. But this is wrong—I3 needs to wait for I2 to write a value into the register, not take II's value.

Conceptually, we can think of the reservation bit for a register as a 1-bit counter indicating how many instructions (0 or 1) downstream expect to write Rd. We can generalize this idea. If we use 2 bits for the reservation in the scoreboard, we can allow 0 to 3 instructions downstream that expect to write Rd. For I2, instead of stalling if the reservation bit is 1, we now stall if the reservation bits value is 3. If the reservation bits value is < 3, we increment them and allow I2 to proceed, *i.e.,* we have eliminated a stalling cycle. When I1 and I2 retire, we decrement the reservation bits. For I3, we stall until the reservation bits value is non-zero.

19.3.8 Drum and Fife: Reducing memory system delays

Memory systems are outside the scope of this book; this section is mostly informational, to familiarize the reader with memory features of interest. We recommend [9] for a more thorough discussion of these topics.

A pipelined memory is one where the path through the memory system can itself sustain one memory request per tick. This allows the CPU independently to pump in several memory requests into the memory request channel, while concurrently pumping out memory responses for previous requests from the memory response channel. Ideally, the memory system has a *throughput* or *bandwidth* of one request per tick.

Memory *latency* is the delay from when the CPU issues an IMem or DMem request until it receives the corresponding IMem or DMem response.

19.3.8.1 TCMs (Tightly Coupled Memories)

Small computer systems (embedded, IoT) often use a small TCM ("Tightly Coupled Memory") for their memory system. TCMs are usually implemented in SRAM (static RAM, BRAMs in FPGAs) which can be accessed in 1 tick, so in these memory systems it is fairly straightforward to have a 1-tick latench and also sustain a 1-request-per-tick throughput.

19.3.8.2 Caches

However, most modern systems have large memories based on DRAM (Dynamic RAM), which can take many cycles to access both because DRAMs are slower than SRAMs, and because requests and responses may have to traverse many layers of interconnect between the CPU and the DRAM. To alleviate this delay, most systems use *caches* close to the CPU. Caches are often implemented in SRAMs so that, when a request "hits" in the cache (*i.e.,* the addressed word is already in the cache), it is usually possible to perform the access in 1 tick. On a cache "miss", on the other hand, the cache system has to "refill" the missing data by fetching it from DRAM, and this can take many cycles. The delay incurred here is called the "cache miss penalty".

Non-blocking caches are able to process another memory request R2 while it is servicing a cache miss for an earlier request R1, *i.e.,* while it is busy fetching a cache line from DRAM for R1. These caches are more complex; simple caches process one request at a time.

19.3.8.3 Virtual Memory

Computer systems that run operating systems (including Linux, Windows, MacOS, ...) typically also support *virtual memory* (RISC-V "S" and "U" privilege levels with a virtual memory scheme such as "Sv32" or "Sv39").

When virtual memory is active, every address sent in an IMem or DMem request is a "virtual address" (VA). The memory system first map the VA this into a "physical address" (PA) by traversing a mapping data structure called a "Page Table" (PT) using a procedure called a "Page Table Walk" (PTW). The PA is then processed as before, *i.e.,* looked up in a cache.

19.3.8.4 TLBs (Translation Lookaside Buffers)

A single VA-to-PA translation, involving a single PTW, involves multiple memory references, and so can take many cycles. This is an unacceptable penalty to pay for each IMem and DMem memory access, and so all such systems employ a "Translation Lookaside Buffer" (TLB) which can be regarded as a cache of previous translations. TLBs are typically small and implemented in discrete logic or SRAMs, and so can also be accessed in 1 tick, and sustain througput of 1 translation per tick, provided we have a "hit" in the TLB. On a TLB miss, we have to perform a PTW to refill the TLB with the cached result; this takes several cycles and is the "TLB miss penalty".

19.3.8.5 TLBs and Caches

Some memory systems compose the TLB and Cache sequentially, *i.e.,* lookup a VA in the TLB, and use the resulting PA for cache access. In these systems, the minimum overall memory access latency is 2 ticks, one for each of those two steps.

Some systems access the TLB and the cache in parallel, reducing memory-access latency to 1 tick. But how can the cache be accessed without the TLB ouput PA? They exploit the fact that certain bits remain the same between the VA and its PA. Virtual memory systems operate at the granularity of "*pages*", consecutive sequence of typically 4K or 8K bytes, aligned to an 4KB/8KB page address boundary. Thus, the "intra-page" bits remain the same between the VA and the PA, and can be used to index the cache lookup before knowing the TLB result. However, this limits the number of bits that can be used to index the cache, *i.e.,* the number of "sets" in the cache. To increase the size of the cache, one may have to increase the size of each set (more "ways", or more "set-associative").

Chapter 20

BSV: Suggested further study



20.1 Introduction

20.2 Alternative simulators: Bluesim, other Verilog sims

20.3 First-class modules

20.4 Polymorphism

20.5 Typeclasses. Conversion of Integer literals. Bits/pack/unpack

20.6 Bluecheck

20.7 Tagged unions, pattern-matching

20.8 Multiple Clock Domains

20.9 Importing RTL

20.10 BH alternative syntax

Chapter 21

RISC-V: Suggested further study



21.1 Introduction

In this chapter we provide suggestions for further study of RISC-V topics. Each of these can also be seen as suggested exercises for the interested student.

21.2 Implementing RV64I instead of RV32I

21.3 M Extension

Integer Multiply and Divide.

21.4 F and D Extensions

IEEE single- and double-precision floating point.

21.5 C Extension

Compressed instructions

Implementation: wholly in Decode stage. Can be shared between Fife and Drum.

Affect of compressed instructions on Fetch.

Affect of compressed instructions on PC-prediction.

Affect of non-32-bit alignment of compressed instructions.

21.6 A extention

AMO operations

LR, SC, AMOxxx instructions and their implementation in the memory system.

21.7 Advanced branch prediction

Is a form of online machine-learning (past history is the “training data”).

Branch instruction taken/not-taken hints.

Hysteresis in prediction.

Branch-Target buffers (BTBs)

Return-Address Stacks (RASs)

21.8 Register renaming: towards out of order processing

An alternative to the counter in the scoreboard.

Have more phys regs than 32. Maintain a table that dynamically maps logical register num to phys reg num. This is ”virtualizing” the registers.

In S3, for each instruction, allocate a new phys register num for its Rd, use that phys num in the instr going forward, and update the log-phys map.

In case of trap or misprediction, have to restore the logical-to-physical map.

21.9 Advanced bypassing: towards dataflow and out-of-order processing

In S3, do not stall any instruction for register-hazards, just forward each instruction into its appropriate S4 pipe, with each input register either having its value or a marker saying ”not yet available”.

In each S4 pipe, stall if the head of the queue does not yet have its input values.

In each S4 pipe, as soon as an output register value is ready, broadcast it to the other S4 pipes, which update any entries awaiting that register value. This may release the stalling of the head entry, allowing it to execute.

Without register renaming, we have to keep the ”counter” from the scoreboard. With register renaming, there is no need, since each physical register will have only one outstanding writer.

In each exec unit, we can treat the pending instrs as a set, not FIFO. I.e., execute any instr that is ”ready”. Then we have OOO dataflow processing.

21.10 Memory systems: TCMs, Caches, PMPs

Separate I- and D-caches.

FENCE.I: for "manual" I- and D-Mem coherence.

FENCE: flushing caches for for devices.

Multi-level caches and cache hierarchies.

Non-blocking caches.

Cache-coherence.

Memory Protection with PMPs.

21.11 Memory Systems: Virtual Memory

Page Tables, TLBs, Virtual Memory.

21.12 Performance measurement

CSRs TIME, MCYCLE.

Other "hpmcounter" CSRs for other events. Counter enables.

21.13 Testing

ISA tests.

Tandem Verification

Sail formal model.

ACTs (sp ?)

21.14 Interrupts

- General concepts: CSRs MIP and MIE; minimal MSTATUS with interrupt-enable bits
- Interrupts are initially disabled using the MSTATUS.interrupt-enable bit immediately; CSRxx can be used to re-enable.
- MMIO addresses MTIME, MTIMECMP.
- Interrupts are handled just like traps; the only question is: when to check for interrupts and respond.
- How does MIE bit return to 0?

PLIC, CLIC

Interrupt/trap delegation.

21.15 Linux and server-class capability

Multiple privilege levels: Machine, Supervisor, User

21.15.1 Hypervisor support

21.15.2 RISC-V ISA Formal Specification

Sail model

Appendix A

Resources: Documents and Tools

This appendix describes all the resources relevant to this course.

A.1 GitHub

We will be using GitHub extensively. Course materials will be provided in a public GitHub repository, and GitHub's "discussion" facilities can be used to for questions and answers, visible to all.

For students who do not already know how to use GitHub, we will teach the basics.

More detailed documentation can be found starting at: https://docs.github.com/en/get-started/quickstart

A.2 RISC-V ISA (Instruction Set Architecture) Specifications

We will refer to the Unpriviledged ISA very frequently, so you may wish to download a copy of the PDF for your laptop, and/or print a copy. The Privileged ISA document is not needed until later.

- "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA".
  Bibliography entry [27] contains a link to `riscv.org` from which to download a PDF.
- "The RISC-V Instruction Set Manual Volume II: Privileged Architecture"
  Bibliography entry [28] contains a link to `riscv.org` from which to download a PDF.

The *formal specification* of the RISC-V ISA is written in the Sail formal-specification language, and can be found at https://github.com/riscv/sail-riscv.

- "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA".
  Bibliography entry [27] contains a link to `riscv.org` from which to download a PDF.
- "The RISC-V Instruction Set Manual Volume II: Privileged Architecture"
  Bibliography entry [28] contains a link to `riscv.org` from which to download a PDF.

A.3 RISC-V Trusted Simulators and Reference Programs for Testing Implementations

The most well-known trusted simulator for RISC-V is the Spike simulator, a free and open-source simulator that is written in C++ and very carefully maintained by the RISC-V community as the standard "reference model" for RISC-V execution. The Spike simulator can be found at: https://github.com/riscv-software-src/riscv-isa-sim

Another trusted simulator is the Sail model (the Sail model is the official "formal specification" for RISC-V. The Sail model and simulator can be found at: https://github.com/riscv/sail-riscv.

Spike is usually more up-to-date with the latest ratified ISA extensions, compared to the Sail model.

RISC-V International maintains a set of standardized tests that are useful in testing new CPU implementations. The following repository— https://github.com/riscv-software-src/riscv-tests— contains several hundred small test programs, written in RISC-V Assembly Language, organized by ISA extension: RV32I, RV64I, A, M, F, D and C extensions, Machine/Supervisor/User mode, *etc.*

A.4 RISC-V Assembly Language Manuals

We will not do very much assembly language programming, and we will teach whatever notation we need during the course.

There are several RISC-V Assembly Language manuals available online, and some in bookstores; download them only if you prefer a local copy:

- "RISC-V Assembly Programmer's Manual", Palmer Dabbelt, Michael Clark and Alex Bradbury.

  Bibliography entry [5] contains a link to online manual.

- "RISC-V ASSEMBLY LANGUAGE Programmer Manual Part I", Shakti RISC-V Team, Indian Institute of Technology, Madras, India. Please see bibliography entry [22] for link from which to download a PDF.

- "An Introduction to Assembly Programming with RISC-V", Edson Borin.

  Bibliography entry [3] contains a link from which to download a PDF.

- "RISC-V Assembly Language", Anthony J. Dos Reis.

  Bibliography entry [8]. Available in bookstores.

A.5 RISC-V GNU tools, including `riscv-gcc` compiler

We will be using the GNU tool chain, specifically the *gcc* compiler and linker, and the *objdump* tool for disassembling an ELF file.

During the course we will show you how to install and use these tools.

The use of these tools is mostly the same as when targeting any target architecture, including well-known architectures like x86 and ARM; the student can find voluminous tutorial materials available on the GNU tool chain on web and in books.

gcc has some specific options for RISC-V; these are documented here:

- https://gcc.gnu.org/onlinedocs/gcc/RISC-V-Options.html
- https://gcc.gnu.org/onlinedocs/gcc/gcc-command-options/machine-dependent-options/risc-v-options.html

It is also useful to know how to use the GNU debugger tool, *gdb*. Again, the student can find voluminous tutorial materials available on on web and in books.

A.6 BSV

In this course, we design the hardware of our RISC-V pipelined CPU using the High Level Hardware Description Language **BSV**. The reasons for our choice (instead of using Verilog, SystemVerilog or VHDL) are discussed in more detail in Appendix B of this document, as well as in the Introduction of the "BSV by Example" book described below.

No advance knowledge of **BSV** is needed for this course; we will teach all necessary **BSV** concepts during the course as we go along.

However, for those who would like to study **BSV** on their own, or wish to view additional **BSV** materials, the following sections provide some resources.

A.6.1 "BSV By Example" book (free downloadable PDF)

This book takes the student through a series of small, targeted **BSV**: examples:

BSV by Example, by Rishiyur S. Nikhil and Kathy R. Czeck, 2010.

Quoting from the Introduction:

> "This book is intended to be a gentle introduction to BSV."
>
> " This book tries to take you into the BSV language one small step at a time. Each section includes a complete, executable (and synthesizable) BSV program, and tries to focus on just one feature of the language"

A bound copy of the book can be purchased on Amazon, but a PDF copy of the book and a tar file containing all the BSV program examples in the book can be downloaded for free from the GitHub BSVLang repository at:

https://github.com/BSVLang/Main/tree/master/Tutorials/BSV_Training

- Book (PDF):
  repository/`Tutorials/BSV_by_Example_Book/bsv_by_example.pdf`
- Machine-readable version of all examples in the book:
  repository/`Tutorials/BSV_by_Example_Book/bsv_by_example_appendix.tar.gz`

A.6.2 BSV Tutorial

A **BSV** self-paced tutorial is available in the GitHub BSVLang repository:

https://github.com/BSVLang/Main/tree/master/Tutorials/BSV_Training

in the directory *repository*/`Tutorials/BSV_Training/` which looks like this:

```
BSV_Training/
    Build/
    Example_Programs/
        Common
        Eg02a_HelloWorld
        ...
        Eg03a_Bubblesort
        ...
        Eg04a_MicroArchs
        ...
        Eg05a_CRegs_Greater_Concurrency
        ...
        Eg06a_Mergesort
        ...
        Eg09a_AXI4_Stream
    Reference
```

Each of the `Eg*` directories cotains a complete example, along with documentation explaining the example, and instructions on how to compile and Verilog-simulate it. The `Reference` directory contains a collection of lecture slide decks explaining the **BSV** language.

A.6.3 MIT Course Material

Massachusetts Institute of Technology (MIT) periodically teaches courses on using **BSV** for digital hardware design. The following link:

http://csg.csail.mit.edu/6.375/6_375_2013_www/handouts.html

contains downloadable material:

- PDFs of slide decks for 12 lectures
- PDFs of slide decks for 4 tutorials classes
- PDFs and codes for 6 laboratories

A.6.4 University of Cambridge Examples

Prof. Simon Moore of University of Cambridge, UK, uses **BSV** in his teaching and research. Several of his **BSV** examples can be found here:

https://www.cl.cam.ac.uk/~swm11/examples/bluespec/

These examples are somewhat more advanced than the ones in the previous sections.

A.6.5 *bsc* download and installation; *bsc* and BSV manuals

bsc is free and open-source, and can be downloaded and installed as described in **BSV**'s GitHub web site https://github.com/B-Lang-org/bsc.

On the main page of that repository you will find links to the following documents (same links also given here):

- The "**BSV** Language Reference Guide" [25]. This document describes the syntax and semantics of **BSV**.

  PDF: https://github.com/B-Lang-org/bsc/releases/latest/download/BSV_lang_ref_guide.pdf

- The "BSC Libraries Reference Guide" [26]. This document describes the extensive set of libraries and IP (Intellectual Property blocks) available to the **BSV** user.

  PDF: https://github.com/B-Lang-org/bsc/releases/latest/download/bsc_libraries_ref_guide.pdf

- The "BSC User Guide" [24]. This document describes how to use the *bsc* compiler, which compiles our hardware descriptions written in **BSV** into Verilog (which can then be simulated or synthesizes using standard Verilog tools).

  PDF: https://github.com/B-Lang-org/bsc/releases/latest/download/bsc_user_guide.pdf

We will be using the Language Reference Guide and Librares Reference Guide extensively, so you may wish to download a copy for your laptop.

A.7 Verilator (or other Verilog simulator)

We will be doing Verilog simulations extensively during this course. For low cost (free), and uniformity, we will be using Verilator.

During the course, we will show you how to install Verilator and use it.

The Verilator web site, https://www.veripool.org/verilator/, contains instructions on how to install Verilator, and also links to PDF and HTML manuals for Verilator. Version 5.004, or any more recent version, will be suitable.

You can use other Verilog simulators if you prefer, but you should independently know how to use them because we cannot offer support during the course. Some possibilities:

- Icarus Verilog, also known as "iverilog". This is a very good, free and open-source, easy-to-use Verilog simulator, but is quite slow compared to other Verilog simulators and so may be less useful for large designs.

  https://steveicarus.github.io/iverilog/usage/getting_started.html

- Commercial simulators from Synopsys, Cadence or Siemens/Mentor Graphics), Aldec, and others. Each of these needs a paid license.

A.8 Amazon AWS

All hands-on work in this course will be run on the Amazon AWS cloud. This way, everyone in the course has a common, stable, predictable environment and we do not have to waste any time dealing with the countless variations in environments found on different laptops and servers.

During the course, we will explain all necessary concepts as we go along, including how to set them AWS instances and use them.

The Amazon AWS cloud offers, on the "AWS Marketplace" a vast variety of choices for virtual machines or, to use AWS terminology, *instances*. We expect to use the following kinds of instances:

A: An instance running the latest version of Ubuntu (Linux).
B: A so-called "F1 instance", also running Ubuntu. F1 instances have attached FPGAs.
C: An instance running the so-called "AWS FPGA Developer AMI" available in the AWS Marketplace. This runs CentOS (Linux) and comes pre-installed with Xilinx Vivado tools, which we will use for creating FPGA bitfile images during the course.

In Amazon's pricing, (B) is the most expensive, and so we will use that only when we actually run on FPGA. For general development and simulation activities, we'll use (A) which is much cheaper. We will use (C) whenever we're creating a new FPGA bitfile image.

The standard Amazon documentation is can be found here:

- "Set up to use Amazon EC2"
  https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/get-set-up-for-amazon-ec2.html
- "Tutorial: Get started with Amazon EC2 Linux instances"
  https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html

A.9 Xilinx Vivado

The FPGAs on Amazon AWS F1 instances are Xilinx Ultrascale FPGAs. Thus, when we build bitfiles on Amazon AWS, we will be using Xilinx Vivado tools (which are provided by AWS for zero incremental cost on AWS FPGA Developer AMI instances, see A.8).

During this course, we will explain all necessary concepts as we go along.

When building a bitfile, it is particularly useful to understand how to interpret the Vivado timing and resource reports. The timing report indicates:

- whether or not our design has successfully met our desired frequency target (MHz), and
- if it did not, which part of our circuit is the likely culprit, which needs to be fixed.

The resource report indicates the "size" or our design (how may LUTs, flip-flops, BRAMs, DSPs, *etc.*).

For more details, Xilinx has extensive documentation for which a good starting point is the "Vivado Design Suite Overview" at

https://docs.xilinx.com/r/en-US/ug910-vivado-getting-started/Vivado-Design-Suite-Overview.

A.10 RISC-V textbooks

This course is self-contained, and it is not necessary to acquire any textbooks.

The following list is provide only as a courtesy and convenience. All these books are written using the RISC-V instruction set as examples, and are available in bookstores.

- "The RISC-V Reader: An Open Architecture Atlas", David Patterson and Andrew Waterman, Strawberry Canyon, 2017. Available in bookstores.

  Bibliography entry [6].

- "Computer Organization and Design RISC-V Edition (2nd Edition): The Hardware Software Interface", David A. Patterson and John L. Hennessy Morgan Kaufman, 2020. Available in bookstores.

  Bibliography entry [18].

- "Computer Architecture: A Quantitative Approach, 6th Edition", John L. Hennessy and David A. Patterson, Morgan Kaufmann, 2017. Available in bookstores.

  Bibliography entry [9]. This is the "classic" textbook on computer architecture, a more advanced textbook.

Appendix B

Why BSV?

The BSV language is a modern, high-level, hardware description language with a strong formal semantic basis. It is *fully synthesizable*, *i.e.,* the *bsc* compiler can also compile your source code into Verilog [11], which we regard as the "assembly language" of hardware design. That Verilog code can then be further compiled by ASIC synthesis tools such as Synopsys' Design Compiler or FPGA synthesis tools such as Xilinx Vivado or Altera Quartus, for implementation in ASICs and FPGAs, respectively.

BSV is very suitable for describing architectures precisely and succinctly, and has all the conveniences of modern advanced programming languages such as expressive user-defined types, strong type checking, polymorphism, object orientation and even higher order functions during static elaboration.

All computation in BSV is expressed using "Rules". For many people, this takes a little acclimatization because it is *very* different from traditional programming models (such as in C++ or Java) which are based on sequential processes. But over time, it becomes *the* natural way to think about hardware computation, which is based on massive, fine-grained, heterogeneous parallelism. Complex and high-speed hardware designs are full of very subtle issues of concurrency and ordering, and BSV's computational model is one of the best vehicles with which to study and understand this.

Modern hardware systems-on-a-chip (SoCs) have so much hardware on a single chip that it is useful to conceptualize them, analyze them and design them as *distributed systems* rather than as globally synchronous systems (the traditional view), *i.e.,* where architectural components are loosely coupled and communicate with messages, instead of attempting instantaneous access to global state. This is because the delay in communicating a signal across a chip is now comparable to the clock periods of individual modules. Again, BSV's computational model is well suited to this style of design.

A key to architecting complex systems and reusable modules, whether in software or in hardware, is powerful *interfaces*. Module interfaces in BSV are object-oriented (based on methods), polymorphic/generic, and capture certain computational protocols. This facilitates creating highly reusable modules, enables quick experimentation with alternatives structures, and allows designs to be changed gracefully over time as the requirements and specifications evolve.

Architectural models written in BSV are fully executable. They can be simulated in the **Bluesim**^{TM} simulator; they can be synthesized to Verilog and simulated on a Verilog simu-

lator; and they can be further synthesized to run on FPGAs or be etched into ASIC silicon, as illustrated in Fig. 1.2. Even when the final target is an ASIC, the ability to run on FPGAs enables early architectural exploration, early development of the software that will later run on the ASIC, and much more extensive and early verification of the correctness of the design. Students are also very excited to see their designs actually running on real FPGA hardware.

In this book, we teach the use of BSV for the design of complex hardware modules and systems by going in detail through a series of examples, and exploring basic concepts as needed along the way, such as combinational circuits, pipelines, data types, modularity and complex concurrency. At every stage the student is encouraged to run the designs at least in simulation, but preferably also on FPGAs.

By the end of the course we will have seen all the source code for a complete simple, pipelined RISC-V CPU along with a small "SoC" (System-on-a-Chip) including an interconnect, a connection to memory, and a few devices such as a UART.

Who uses BSV?

BSV has been used in teaching and research at major universities, including MIT (Massachusetts Institute of Technology, USA), Universisty of Cambridge (UK), Indian Institute of Technology, Madras (India), Indian Institute of Technology, Mumbai (India), Seoul National University (South Korea), University of Texas at Austin (USA), Carnegie Mellon Univerity (USA), Georgia Institute of Technology (USA), Cornell University (USA) and Technical University of Darmstadt (Germany).

BSV has been used to design major IP components in commercial ASICs from Texas Instruments, ST Microelectronics and Google. It has been used for FPGA-based modeling at IBM, Intel, Qualcomm, Microsoft Research, several DARPA projects, and others. It is being used for commercial RISC-V processors from InCore Semiconductors (Shakti line of RISC-V processors, India) and The C-DAC (Center for Development of Advance Computing) (Vega line of RISC-V processors, India).

B.1 Why BSV instead of some other Hardware Design Language?

The rest of this chapter is intended for those interested in comparing BSV's approach to other approaches (Verilog, SystemVerilog, VHDL, and SystemC), and can be safely skipped by others who just want to get on with learning BSV.

One may be curious why the material in this book could not have been covered using one of the more widely known languages for hardware design: Verilog [11], VHDL [10], SystemVerilog [13], SystemC [12]. There are several reasons, outlined below.

In the following paragraphs, we will refer to all the above languages, or at least their synthesizable subsets, as "RTL" (Register Transfer Level languages).

B.1.1 A better computational model

Paradoxically, the formal definitions of the semantics of traditional hardware design languages (HDLs)— Verilog, SystemVerilog, VHDL and SystemC— are not in terms of hardware concepts, but in terms of *software simulation* on conventional computers. Like traditional software programming languages, they are defined in terms of sequential statement execution, with traditional conditionals, loops, and procedure calls and returns, reading and writing conventional variables. Programs can have multiple concurrent *processes* (e.g., "always blocks" in Verilog), but each of them is defined with traditional sequential programming semantics.

Digital hardware, on the other hand, has a quite different computation model. It consists of hundreds, if not thousands of concurrent "state machines" that transform the current state of the hardware, implemented using registers, memories and FIFOs. By and large, there is no sequencing of these state machines based on program counters or statement sequences. Rather, these state machines are independent and "reactive", *i.e.,* each one performs an action whenever certain conditions hold, e.g., when a register holds a particular value, or a value is available in a FIFO, etc.

To bridge this rather large gap from conventional sequential processes to concurrent reactive state machines requires a major mental shift. One must severely restrict code to only a much smaller so-called *synthesizable subset* of a conventional HDL. Processes are restricted to simple clocked loops: "`always @posedge CLK ...`", also known as an "always-block". Even more draconian is a transposition away from the natural concurrent state machine view to a *state element-centric* view: even though a state element may be read and written by multiple state machines, all updates to that state element must be concentrated in a single always-block, usually in a large conditional construct (if-then-else, case, ...) that describes all the different contributions of different state machines. This transposition, from the natural state machine-centric view to the rather unnatural state element-centric view, is necessary because in the the synthesizable subset there is no synchronization between always-blocks; the programmer has to plan every detail of how to resolve (arbitrate) competing updates to each state element.

In other words, in conventional HDLs, neither the simulation view (sequential processes) nor the synthesizable view (state element-centric always blocks) are a natural way to model hardware behavior.

BSV programs, instead, directly express the natural model of hardware— concurrent state machines. Each "rule" in BSV is a reactive state transition that awaits some condition on the hardware state and then takes an action to transforms the state. Further, each rule is an *atomic transaction*, *i.e.,* the details of how one arbitrates competing accesses from multiple rules to common shared state is left to the compiler. This kind of arbitration logic, which is hand-written in other HDLs, is a major source of bugs.

In BSV, unlike in other HDLs, the semantics are identical whether you execute in simulation or in hardware—there is no mental gear shift necessary, and simulation behavior is always identical to synthesized hardware behavior.

Finally, the Rules computation model uniquely encourages *refinement*, a powerful design methodology. We initially create a high-level, approximate model of a target design, using a few large rules. Both the level of micro-architectural detail and the range of functionality

are approximated (abstracted). Often such a model can be written in less than a day, and it can immediately be executed to verify functional correctness. Then, over time, we incrementally add architectural detail—for example, pipeline registers and state machines with more, smaller steps—and the original rules (large step state transitions) are replaced by more, smaller rules (small step state transitions). The atomic semantics of rules makes this a robust methodology, *i.e.*,, a refinement does not have a large ripple effect. This is in quite dramatic contrast to the difficulty in changing RTL, which is notoriously brittle and unforgiving.

Refinement allows early and continous confidence in functional correctness and completeness, since we execute the code very frequently. Refinement allows mid-course corrections in functionality, after observing execution on real data. Refinement allows separating *functionality* from *performance*, achieving functionality early and holding it constant while we improve performance to meet a performance target (by target performance we mean some desired targets for speed, area, and power).

B.1.2 Modern language features

The field of programming languages has seen tremendous progress since the early days (1950s). Modern high-level languages have advanced type systems (polymorphism, typeclasses and overloading, functional types, and so on). Modern high-level languages have strong mechanisms for encapsulation and abstraction (such as object-orientation) which promote the separation of concerns between externally visible behavior and internal representation choices. Modern high-level languages make frequent use of higher-order functions—functions whose arguments and results can themselves be functions and data structures whose components can be functions.

Unfortunately, practically none of these powerful features are present in the synthesizable subsets of conventional HDLs[1]. BSV, on the other hand, adopts the full power of the Haskell functional programming language [19]: algebraic types, functional types, polymorphic types, typeclasses, higher-order functions, and recursive and monadic static elaboration. This delivers unprecedented expressive power, type safety and type flexibility in a hardware design language.

B.1.3 Comparison with C++-based High Level Synthesis

Recently, some tools have become available under the rubric of "High Level Synthesis" (HLS) that claim to shield you from this mental gear shift from simulation to hardware. Designs are written in a traditional sequential programming language (typically C++), and an HLS tool automatically compiles this into a hardware implementation. While beautiful in concept, there are many serious limitations in practice, which are discussed below.

[1] SystemVerilog and SystemC have object-orientation, polymorphism, and overloading, but these are typically used only in simulation for verification environments of hardware designs, not for actual hardware design itself.

B.1.3.1 C++ codes need significant rewriting

C++ HLS tools will rarely accept arbitrary, off-the-shelf C++ codes and produce good hardware implementations. C++ codes often requre significant restructuring to achieve good results.

First, the tools only accept a limited subset of C++ syntax. In particular, these tools are very averse to any kind of pointer-based argument passing or data structures, unless all the pointers can be resolved by the compiler (*i.e.,*, the compiler statically knows the addresses represented by the pointers). This is because, while C++ normally executes on machines that provide the abstraction of a single large memory with a single address space (so a pointer is fundamentally an address, and dynamic allocation and relocation are easy), hardware designs typically use hundreds or thousands of individual memory units, from registers to register files to SRAMs, DRAMs, Flash memories, and ROMs, each with its own address space.

Second, most C++ codes written for conventional execution rely deeply on sequential execution. For example, they may re-use a variable (multiple reads and writes in different phases of the code). Many of these programming techniques, often a good idea for higher performance and smaller memory footprints in conventional execution, are exactly the opposite of what is needed for hardware implementation, which is highly parallel.

Overall, for good results, one must develop a keen sense of the hardware implementation impact of various "styles" of writing C++ code. Small changes in style can mean the difference between a terrible implementation and an acceptable one. One vendor insists that any team adopting their tool should not consist solely of C++ experts, but must also include hardware engineers.

B.1.3.2 Narrow range of applicability due to automatic parallelization

C++ is, by official definition, a completely sequential language. Hardware, on the other hand, relies on massive, fine-grain parallelism. It is the HLS tool that has to pull off this magical transformation.

C++ HLS tools rely on a body of knowledge called CDFG Analysis (Control and Data Flow Graph Analysis). After parsing and typechecking, the C++ program is represented internally in a data structure called the CDFG. This CDFG, initially directly reflecting the sequential nature of the source, is analyzed and transformed into a parallel representation from which, eventually, hardware is generated.

It turns out that this transformation only works well for a narrow range of program structures—cleanly nested `for`-loops with fixed iteration bounds, operating on dense rectangular arrays. Of course, many signal-processing and image-processing applications do have this structure, and C++ HLS tools have found their greatest success in this arena.

But the moment we step outside this sweet spot, towards sparse arrays or programs that are highly control-dominated, these tools fall off a cliff. Most hardware design in fact involves components that don't fall into the C++ HLS sweet spot: CPUs, cache systems, switched interconnects, flash memory and disk controllers, high-speed I/O controllers for Ethernet, PCIe, USB, and so on. For example, we are unaware of any project using C++ HLS for CPU design, whereas there are over a dozen such projects using BSV.

B.1.3.3 Lack of "Algotecture": Architectural transparency and predictability

Most people with some training in Computer Science are familiar with the idea that Algorithms are Job One—when writing performance-critical software, the first-order concern is to design a good algorithm. Further, creating a good algorithm is a creative act; compilers don't automatically create good algorithms for you[2].

Unfortunately, because most of our codes run on classical von Neumann machines, many people forget that, when the execution platform changes, our old algorithms may no longer be any good—the assumptions about the cost of fundamental operations may no longer valid and in fact may be wildly different, requiring a complete re-think of the algorithm.

This bring up a fundamental difference between software design and hardware design. In software, you are given a particular target architecture (CPU, GPU, cluster, vector machine, ...), and the designer's job is to design a good algorithm for that fixed architecture. In hardware, on the other hand, the designer's job is to design the algorithm and the architecture *jointly*. In other words, for hardware designers, algorithm and architecture are joined at the hip; it is meaningless to separate these activities. We thus use the term *Algotecture* to describe this integrated activity.

Unfortunately, most C++ HLS tools provide very narrow visibility and control into architecture. For example, directives for loop unrolling and loop fusion may allow you to express some variation in iterative *vs.* parallel *vs.* pipelined structures. But, basically, it's the tool that chooses the architecture, and you have some weak knobs to guide its choices. A common syndrome with C++ HLS tools is that one quickly produces an implementation, but it is terrible in area or performance, and this is followed by a *long* tail of activity in which the designer tweaks the knobs every which way in an effor to beat it down into the desired performance envelope.

In contrast, with BSV, architectural choices (like algorithmic choices) are in the hands of the designer, where it should be. There are no surprises with respect to architecture; performance is never a mystery, and the designer can quickly improve it and converge to an acceptable solution.

B.1.3.4 Summary

In summary, it is our experience that BSV is a much better language for complex hardware design, whether control or data oriented, whether for modeling or architectural exploration or final implementation, or for synthesizable on-FPGA verification transactors. Following the philosophy of DSLs (Domain Specific Languages), BSV is very much an expressive DSL beautifully suited for hardware design, whereas sequential C++ is certainly not (it was never intended to be!).

[2] Of course, there is research in this area, but this starts entering the realm of Artificial Intelligence.

Appendix C

Glossary

2's Complement See entry for "Two's Complement".

ACTs Architecture Compatibility Tests. A test suite under development by RVI to verify that a given implementation complies with a particular subset of the RISC-V ISA. A candidate implementation must run the relevant ACTs and produce correct "expected" output signatures.

API Application Programming Interface. Term commonly used in many programming languages, methodologies and protocols to describe the set of functions/procedures/methods used to interact with a module/object by external entitities (from outside the module/object). The API clearly separates external concerns from internal concerns. External concerns are about "what" a method does or sequence of methods do: what are their argument and result types, and what do they (abstractly) achieve. Internal concerns are about "how" methods do what they are supposed to do. This separation of concerns also allow transparently substituting a module implementation with an alternate implementation (*e.g.,* for greater efficiency) without disturbing the external context.

ASIC Application-Specific Integrated Circuit. A kind of electronic device that represents a desired digital circuit directly in silicon and has been fabricated for that purpose (not customizable and general-purpose like an FPGA).

BRAM Block RAM. A memory component in an FPGA, usually implemented with SRAM.

BSV, BH An open-source, modern, High-Level HDL. Two optional syntaxes (choose to one's taste): BSV has traditional Verilog-like syntax, BH has traditional Haskell-like syntax.

BTB Branch Target Buffer. A component of a PC-predictor module. See Section 19.3.6.4.

CISC Complex Instruction Set Computer (see also, for contrast, "RISC"). Refers to an ISA where a single instruction may express quite a bit of computation, such as memory accesses, ALU ops, and even iteration.

CPI Cycles per Instruction. Please see entry for its inverse, IPC.

CPU Central Processing Unit. The computational element of a computer.

CReg Concurrent Register. See Section 18.3. Also known as EHRs (Ephemeral History Registers).

CSRs Control and Status Registers. These are special registers in the ISA, most of which are accessibly only while executing at higher privilege levels (Machine and Supervisor). Certain key CSRs play a central role in disciplined transition between privilege levels, in virtual memory, and in memory protection.

DRAM Dynamic Random Access Memory. A kind of silicon chip that implements memory. Compared to SRAM, is larger (number of bits), denser (bits per silicon area), cheaper ($ per bit), uses less power (watts per bit) and is more complex to operate (needing regular refreshing *etc.*). Usually off-chip (not part of an ASIC or an FPGA).

DUT Design Under Test. Term commonly used by hardware designers to indicate the artefact being designed, to contrast in particular with the "testbench" or "test harness" which surrounds the design during testing. DUT is often articulated as a word in its own right (pronounced "Dutt", as in the famous name in Bombay cinema https://en.wikipedia.org/wiki/Nargis_Dutt).

EHR Ephemeral History Register. The original name for BSV's CRegs, when developed by Daniel Rosenband and Arvind at MIT [20, 21].

FPGA Field Programmable Gate Array. A kind of electronic device that has configurable circuits that can be customized to represent any desired digital design. These are catalog parts available from several vendors.

FPGA Board A circuit board containing one or more FPGAs, a power supply, and DRAM memories. Often contains other facilities such as GPIO, UARTs, JTAG, PCIe bus connections, Ethernet connection, USB connection, Flash memory, and so on.

FSM Finite State Machine. A sequential process that moves ("transitions") from one state to another in a fixed repertoire of states. Transitions may loop back to earlier states, and may conditionally select one of a set of alternative next-states.

GPIO General Purpose Input Output. An electronic device attached to a computer system. When the CPU stores a byte/word to a GPIO address, the bits of the word appear as electronic signals from the device, and can be used as an *actuator*—switch on/off a back of LED lamps, a relay, a motor, *etc.*.. When the CPU loads a byte/word from a GPIO address, it can read the state of a *sensor*—switches, photocells, motor speed, temperature, *etc.*.

GPR General Purpose Register. For RISC-V, just a synonym for the basic register set holding integers. They are "general purpose" in the sense that software is free to use them in any way (in contrast with some earlier ISAs that restricted certain registers to certain roles, such as holding addresses).

HDL Hardware Design Language. A language in which one can represent circuits, and for which there are tools that can render a program into actual circuits for FPGAs and ASICs. Examples include: BSV, BH, Chisel, Verilog, SystemVerilog, VHDL.

HLHDL High-Level Hardware Design Language. An HDL with higher-levels of abstraction and more powerful constructs and semantics compared to the traditional HDLs Verilog, SystemVerilog and VHDL, in the same sense that modern software programming languages (Java, Python, Javascript, Haskell, OCaml, ...) have higher-levels of abstraction than C/C++ which, in turn, have higher levels of abstraction than Assembly Language. Examples include BSV, BH (the Haskell-syntax variant of BSV), Chisel, and HLS.

HLS High Level Synthesis. The term typically used for tools and methodology that compile C/C++/SystemC programs into hardware. HLS can be fragile in that it works best only on certain subsets of C/C++ ("simple rectangular loop and array" algorithms), and require certain coding styles and directives.

ILP Instruction-Level Parallelism. A measure of how many instructions can be executed in parallel without violating the canonical sequential (one-instruction-at-a-time) semantics of an ISA.

IPC Instructions Per Clock (or its inverse, CPI, or clocks per instruction). A component, together with clock speed (cycles per second), of a CPU's application performance. Application performance depends on clock speed, IPC, and total number of instructions.

ISA Instruction Set Architecture. A specification of instructions: how an instruction is coded in bits; "architectural state" (PC, registers *etc.*); what it means to execute an instruction; assembly language syntax. The specification is described independently of any particular implementation, traditionally in a manual with text and diagrams, occasionally and recently also in a formal-specification language.

An ISA can (and typically does) have many possible implementations, varying widely in speed, size, power, cost, technology (ASIC, FPGA), *etc.* Examples of famous ISAs and vendors who supply implementations include RISC-V (diverse vendors), x86 (Intel and AMD), ARM (Arm, Apple, Samsung, others), Sparc (Sun, Oracle, Fujitsu, others), MIPS (MIPS, Inc.), Power and PowerPC (IBM, others), ...

Microarchitecture The structural and behavioral details of an ISA implementation that are *below* the level of abstraction of the ISA, *i.e.,* not demanded by the ISA but chosen by the implementor for practical reasons (speed, power, area, cost, ...). Examples: pipelines, branch prediction, scoreboards, register renaming, out-of-order execution, superscalarity, instruction fission and fusion, replicated execution units, store-buffers, ...

MMIO Memory-Mapped Input-Output. In RISC-V, the CPU reads and writes registers in a device using ordinary LOAD and STORE instructions. The memory system interprets the addresses to direct such requests to a device. Using LOADs and STOREs, the CPU can control the device, send data to the device and retrieve data from the device.

OOO Out-Of-Order. Refers to advanced CPU microarchitectures which replicate functional units (computational and memory access units); and allow each functional unit immediately to execute any instruction that is "ready" (because its inputs are available), even if that is not in program order. Often combined with SuperScalarity, to achieve vastly more instruction-level parallelism (ILP) compared to a simple in-order pipeline.

OS Operating System. Can vary from small, embedded, real-time OSs such as FreeRTOS, to more capable embedded OSs like Zephyr, to secure micro-kernels like seL4, to full-featured OSs like Linux, Windows, MacOS, Solaris, AIX, *etc.*

PBT Peanut Butter on Toast. No, seriously: Property-Based Testing. This was a technique that was pioneered by Haskell's *QuickCheck* (https://en.wikipedia.org/wiki/QuickCheck), and facilities for which have now been replicated in numerous other programming languages. It is an automated technique that repeatedly generates random stimulus for the DUT and, for each such input, tests one or more *properties* which are formal logical predicates expressing some required correctness property of the DUT. The stimuli are typically generated in increasing size order, where "size" depends on the type of stimulus. When a property failure is encountered, the system attempts to "shrink" the failing stimulus by randomly omitting parts of the stimulus, in the hope of finding a smaller stimulus that exhibits the same failure. The results can be quite dramatic, often producing very small stimuli that trigger some bug.

PTW Page Table Walk. A function in systems supporting virtual memory, to translate virtual memory addresses into physical memory addresses (not described in this book). Requires multiple memory references to descend a "tree" data structure called the Page Table.

RAM Random Access Memory. So called because successive addresses can be unrelated (in contrast with memory technologies that might only support, for example, sequential address accesses).

RAS Return Address Stack. A component of a PC-predictor module. See Section 19.3.6.4. (Within the field of DRAM technology can also stand for Row Address Strobe.)

RISC Reduced Instruction Set Computer (see also, for contrast, "CISC"). Refers to an ISA that separate memory-access instructions from computational instructions, and where each instruction is amenable to fast pipeline implementations.

RISC-V A particular standard ISA. Originated circa 2008-2010 in research at University of California, Berkeley, and subsequently spun out (2010s) into an international non-profit consortium "RISC-V International" (RVI) headquartered in Switzerland (https://riscv.org).

Unlike other well-known ISAs, the RISC-V ISA is an *open* standard, *i.e.,* implementors do not need to pay any license fee in order to use the ISA, which is one of the factors behind its wide adoption by hundreds of vendors.

ROB Reorder buffer. An array into which each completed instruction is placed, at a specific index that represents its proper ISA order. Each slot is initially empty, and becomes full when a completed instruction is placed there. As separate process drains completed instructions from this array in sequential order.

RTOS Real-Time Operating System. A typically small operating sytem for small embedded systems. Compared to, say, Linux,

- It may not support multiple privilege levels (Machine, Supervisor, User).
- It may not support multiple processes, or a variable number of processes.
- It may not support virtual memory.

- It may not support memory protection across processes.
- It may only support a limited repertoire of devices.

RTL Register-Transfer Level/Language. This is a level of abstraction of describing hardware that assumes that the available primitive components are clocked registers and combinational circuits for multiplexers, and basic arithmetic and logic functions (adders, subtractors, boolean operations, shifters, *etc.*).

This is a higher level of abstraction than AND/OR/XOR/NOT gates which, in turn, are a higher level of abstraction than transistors which, in turn, are a higher level of abstraction than silicon regions. Each layer of abstraction is automatically compiled to a lower layer using various tools.

RVI RISC-V International. See entry for RISC-V.

Slack A measure in digital circuits for the amount by which a combinational circuit exceeds the delay requirement for a target clock speed. For example, if the target clock speed is 250 Mhz, *i.e.,* has a 4 ns period, and a combinational circuit between registers only takes 3.2 ns, then it is said to have a "positive slack" of 0.8 ns.

SoC System-on-a-chip. Refers to a complete computing system on a chip, including one or more CPUs (with MMUs and caches), shared caches, interconnects, DRAM interface, JTAG, accelerators and devices, *etc.*

SRAM Static Random Access Memory. A kind of silicon chip that implements memory. See DRAM above for comparison. Usually on-chip in an ASIC or an FPGA.

Superscalar . Refers to advanced CPU microarchitectures which fetch and execute multiple instructions at a time (typically 2, 4, 8). Often combined with OOO (Out-of-order), to achieve vastly more instruction-level parallelism (ILP) compared to a simple in-order pipeline.

SystemVerilog One of the major HDLs. Originally created in the 2000s as a proper superset of Verilog (and thereby subsuming Verilog), and incorporating many features from VHDL; incorporated some modern features from object-oriented software programming languages (principally used in verification testbenches in simulation only); then an IEEE standard that has gone through several versions. Can be used for both analog and digital circuits. Some features can only be used in simulation (a "synthesizable subset" can be rendered into hardware).

TCM Tightly Coupled Memory. Usually an SRAM (Static RAM) used directly as the memory of the CPU, with no cache in front of it.

TLB Translation Look-aside Buffer. A component used in systems supporting virtual memory, to speed up translation of virtual memory addresses into physical memory addresses (not described in this book.)

Two's Complement A particular representation of positive and negative integers in bits (binary) that makes it possible to perform both addition and subtraction using the same hardware. Wikipedia has a good discussion: https://en.wikipedia.org/wiki/Two%27s_complement

UART Universal Asynchronous Receiver/Transmitter. An electronic device attached to a computer system through which the CPU can read ASCII characters from a keyboard and send ASCII characters to a display screen. Typically used for the main console of a computer system.

UVM Universal Verification Methodology. A standard methodology and technology in SystemVerilog for testbenches for verification. Exploits the "object-orientd programming" aspects of SystemVerilog for reusability. Wikipedia has an introduction (https://en.wikipedia.org/wiki/Universal_Verification_Methodology). There are dozens of tutorials, textbooks and UVM IP providers, both open-source and proprietary.

Verilog One of the two grand old HDLs (the other is VHDL). Originally created in the 1980s; then an IEEE standard that has gone through several versions; then subsumed by SystemVerilog. Can be used for both analog and digital circuits. Some features can only be used in simulation (a "synthesizable subset" can be rendered into hardware).

VCD Value Change Dump. A file written out during simulation (Verilog simulation or **Bluesim** simulation) that contains a clock-time-stamped record of every change on every bus (bundle of wires) in the hardware design. This file can then be viewed graphically in any waveform viewer. Waveform viewers are bundled into most commercial simulators. Alternatively, *gtkwave* is a popular free and open-source waveform viewer.

VHDL One of the two grand old HDLs (the other is Verilog). Originally created in the 1980s; then an IEEE standard that has gone through several versions. Many features were adopted by SystemVerilog. Can be used for both analog and digital circuits. Some features can only be used in simulation (a "synthesizable subset" can be rendered into hardware).

Appendix D

BSV: Importing C/C++ functions into BSV simulations

D.1 Introduction

This facility is only used in BSV simulation (compiling C code to hardware is very difficult even in limited contexts, see SectionB.1.3) and it is principally used in *testbenches* for designs.

There are many reasons to import C functions into BSV (whether simulating in **Bluesim** or in Verilog simulation):

- Many applications begin life as a C/C++ model, such as a C/C++ algorithm that we are trying to accelerate in hardware, or an algorithm that we are still prototyping in C/C++ while developing the rest of the system.
- C/C++ is useful for testbench components: They will run much faster than Verilog simulation, and so will not be a performance burden on simulating the Verilog of interest. They have full-service access to data files and operating-system services.
- Even for an actual hardware component for which we have BSV or Verilog code, we may temporarily substitute it with a much faster C model while our focus is on testing other parts of the hardware.

In Drum and Fife code, the whole CPU design is written in BSV. However, to test the CPU we need to connect it to a memory system, a UART, *etc.* and for these we use models written in C. Modeling the memory system in C makes it easy to include code for pre-loading ELF and memhex files.

A C function can be imported into BSV with the simple steps described in the next sections.

BSV can only import C functions directly. If you need to use a C++ function, write a C wrapper function that is invoked from BSV; that C function can then invoke your C++ function.

An imported C function invoked in BSV is semantically instantaneous, never a temporal process. It is imported with `Action` or `ActionValue` type or a pure (combinational) type.[1]

D.2 In BSV code, declare a BSV version of the C function

In the BSV code, the imported C function will be invoked exactly like a normal BSV function. We declare the header of this "BSV" function (*i.e.,* just the result type, function name and arguments with their types, and not the function body), preceded by the phrase `import "BDPI"`:

```
import "BDPI"
function   BSV-type   function-name ( BSV-type   arg
                                      ...
                                      BSV-type   arg );
```

The argument- and result-passing conventions are simple, adjusting for the C's limitation that arguments and results be can at most 64-bits wide:

- BSV values of sizes up to 8, 16, 32 or 64 bits are passed as arguments and results of C type `uint8_t`, `uint16_t`, `uint32_t` and `uint64_t`, respectively, for both arguments and results, in the same corresponding positions.
- For a BSV argument value of size greater than 64 bits, it is passed to C as a pointer to memory containing the BSV value.
- For a BSV *result value* of size greater than 64 bits, the corresponding C function prototype changes slightly:
  - It gains an extra first argument which gets a pointer to memory which should be filled by the C function with the BSV value to be returned.
  - Because of this, it has a `void` return type.

D.3 Compile the BSV code with the *bsc* compiler

If you are compiling for **Bluesim**, there is no change to how you invoke *bsc*.

If you are compiling to Verilog, provide the additional flag "`-use-dpi`" on the command line. This will generate standard SystemVerilog `import "DPI-C"` declarations in the generated Verilog.

[1] Of course an invoked C function, before it returns, could start a C "pthread" which then runs concurrently with the BSV simulation.

D.4 Linking

For **Bluesim** linking, simply provide the C file(s) implementing imported C functions as an additional command-line arguments when you again invoke *bsc* for linking.

For Verilog linking, in the imported C code we recommend adding the following for each of the imported C functions:

```
#ifdef __cplusplus
// 'C' linkage is necessary for linking with Verilator object files
extern "C" {
    ... imported C function's prototype ...
}
#endif
```

This directs the C/C++ compiler to compile the C function with C argument-passing conventions instead of C++ argument passing, which is necessary for SystemVerilog DPI-C. Verilog simulator tools like Verilator compile the C code with a C++ compiler which, by default, would use C++ argument-passing.

For Verilator linking, simply provide the C/C++ file(s) implementing the imported C functions as additional arguments to the `verilator` command.

For other Verilog simulators, the linking details may vary; please consult their respective manuals or experts on how to link-in C code under the SystemVerilog DPI-C standard.

D.5 Recommendations for arguments and results of imported C/C++ functions

The following recommendations circumvent potential complications when using imported C code in BSV.

D.5.1 Only use BSV types corresponding to C types

The data passed between BSV and C have the standard BSV packed representations in bits. It can be tricky in C to deal with, say, a BSV 13-bit value that, in C, is a `uint16_t`. We recommend only using BSV arguments/results whose sizes are multiples of 8-bits so that they map exactly to bytes in C.

Even for BSV structs and vectors, only use structs and vectors whose elements are 8-bit byte-sized. Accessing/updating the components in C is vastly easier with these constraints.

If an original BSV type T does not have "byte-aligned" components, we often define a new type $T2$ with byte-aligned components and copy values from T to $T2$ before the call (for arguments) or from $T2$ to T (for results). These is extra, technically unnecessary "copying" of data, but the resulting simplification of the C code is well worth it.

For structs, be careful that C structs may have "gaps" or "padding" between fields to improve word-alignment, whereas BSV structs are tightly packed. Thus a BSV struct and a C struct, though they may look identical, may have different data representations.

D.5.2 Use `ActionValue#(t)` for imported C function's result

In an `import "BDPI"` declaration the type of the result the function may be `Action`, or `ActionValue#(t)` or some type t. As discussed in Section 4.11, in BSV the last case (not an action or actionvalue) is taken as a strong guarantee of mathematical purity (and absence of side-effects); the `bsc` compiler may merge multiple invocations that have the same arguments, and it may move a function elsewhere in the code with different control conditions. These optimizations can lead to nasty surprises when the C function is not really pure (including core-dumps if it relies on prior initializations).

We recommend normally using `ActionValue#(t)` for the result of any imported C function (or `Action` if the C result is `void`) to avoid surprises.

Use a non-action/actionvalue return type *only* if you are absolutely sure that the C function is pure and does not need any other initializations before invocation (*e.g.,* the C library `toupper` character transfomer or `cos()` trigonometric function). Even for these, be safe and use an actionvalue function.

D.6 Example: Memory Model for Drum and Fife

For Fife and Drum, because our focus is on the CPU module, we implement the memory system in C for convenience. In addition to the speed reason (not slowing down simulation), C is also more convenient for reading in memhex32 and ELF files.

src_Top/Mems_Devices.bsv: line 273 ...

```
1 import "BDPI"
2 function ActionValue #(Bit #(96)) c_mems_devices_req_rsp (Bit #(64) inum,
3                                                           Bit #(32) req_type,
4                                                           Bit #(32) req_size,
5                                                           Bit #(64) addr,
6                                                           Bit #(32) client,
7                                                           Bit #(128) wdata);
```

The corresponding C function prototype, in a C file is:

```
1  #ifdef __cplusplus
2  // 'C' linkage is necessary for linking with Verilator object files
3  extern "C" {
4  void c_mems_devices_req_rsp (uint8_t        *result_p,
5                               const uint64_t  inum,
6                               const uint32_t  req_type,
7                               const uint32_t  req_size_code,
8                               const uint64_t  addr,
9                               const uint32_t  client,
10                              uint8_t        *wdata_p);
11 }
12 #endif
```

Appendix E

Exercises

NOTE: *This appendix may change are we improve or add new exercises. Please ensure you have the latest version!*

For all the exercises you will need an installation of two free and open-source tools, the *bsc* compiler and the Verilator compiler.

For *bsc*, see: https://github.com/B-Lang-org/bsc

Download and installation info
BSV Language Reference Guide (PDF)
BSV Libraries Reference Guide (PDF)
bsc User Guide (PDF)
(PDFs also available in *bsc* installation directory).

For Verilator, see https://www.veripool.org/verilator

Download and installation info
Verilator user manual

Chapter 3: BSV: Overview of the BSV program structure

Ex-03-A-Hello-World

(Refer Section 3.2.)

Please see directory: `Exercises/Ex-03-A-Hello-World/`

1. Observe the version of bsc and verilator in your installation by typing these commands (your versions may later versions):

```
$ bsc -v
Bluespec Compiler, version 2024.01 (build ae2a2fc)
This is free software; for source code and copying conditions, see
https://github.com/B-Lang-org/bsc

$ verilator --version
Verilator 5.018 2023-10-30 rev v5.018-38-g344f87abe
```

Observe a a full list of command-line options for bsc and verilator by typing:

```
$ bsc --help
$ verilator --help
```

The *Bluespec Compiler (BSC) User Guide* is a detailed guide on using *bsc*.

2. Compile and run the given **BSV** program `Top.bsv` using both Bluesim simulation and Verilator Verilog simulation.

   For help:

   ```
   $ make    or    $ make help
   ```

   For Bluesim:

   ```
   $ make b_compile
   $ make b_link
   $ make b_sim
   Hello, World!                          # output
   ```

   For Verilator:

   ```
   $ make v_compile
   $ make v_link
   $ make v_sim
   Hello, World!                          # output
   ```

   For each of the above "`make`" commands, make sure you understand what it does, by examining the Makefile, or using the `--dry-run` flag like this, or examining the terminal when you actually run 'make'.

   ```
   $ make <target> --dry-run
   ```

You can also type "`make b_all`" to or "`make v_all`" to perform all three actions: compile, link and run (simulate).

Once you understand these actions, you should be able to create your own Makefiles for bsc and Verilator.

3. Comment-out the `$finish ()` statement from the code in `Top.bsv`, recompile and run, and observe the behavior.

Ex-03-B-Top-and-DUT

(Refer Section 3.3.2.)

Please see directory: `Exercises/Ex-03-B-Top-and-DUT/`

1. Compile, link, and run the given example, in either Bluesim simulation or in Verilator Verilog simulation, using the Makefile commands (same as last exercise: `b_compile`, `b_link`, `b_run`, `v_compile`, `v_link`, `v_run`).

2. Try the compile steps above after removing the "`-u`" command-line flag and observe the behavior.

3. Change `Bit#(11)` to `Bit#(8)` in `DUT.bsv`; retry the compile-link-run, and observe the behavior.

4. Write down a BSV hexadecimal integer literal whose value is equal to decimal 22 (hexadecimal integers in BSV use the same notation as in Verilog and SystemVerilog (see book Section 4.2).

5. Add another `$display()` statement that reports the number of years that have elapsed from the publication of Kernighan and Ritchie's book to today.

Ex-03-C-Module-and-Interface

(Refer Section 3.4.3.)

Please see directory: `Exercises/Ex-03-C-Module-and-Interface/`

1. Compile, link and run the given program (`Top.bsv`, `DUT.bsv`) using the Makefile.

2. In `DUT.bsv`, in "`export DUT_IFC (..)`", remove the ellipses "`(..)`".

   Try to compile-link-run, and observe the behavior.

3. Define a third string, say `"publisher"`, in the DUT package; add it as a third tuple component to the `m_title_authors` method output; add a `$display()` line in `mkTop` to print it.

4. In `DUT.bsv`, define a second module `mkDUT2` with the same `DUT_IFC` interface. This module's methods should return information about the following book:

> "Introduction to VLSI systems" Carver Mead and Lynn Conway Addison-Wesley, January 1, 1980

In `Top.bsv`, instantiate `mkDUT2` in addition to instantiating `mkDUT`.

Add some `$display` statements to print information from this module.

Some nice quotes from the book:

> "We believe that only by carrying along the least amount of unnecessary mental baggage at each step in such a study, will the student emerge with a good overall understanding of the subject."

> "An atmosphere of excitement and anticipation pervades this field."

Chapter 4: BSV: Combinational circuits for the RISC-V Stage functions

Ex-04-A-Bit-Vectors: Bit-vectors

(Refer Sections 4.6-4.6.2.)

Please see directory: `Exercises/Ex-04-A-Bit-Vectors/`

1. Note that, compared to previous exercises, the Makefile uses more bsc flags to organize source files, intermediate files, etc.

   When you use any of the "make" commands, it shows the *bsc* command that it performs. Study the `bsc` commands and make sure you understand all the command-line flags.

   ```
   bsc -help
   ```

   shows a one-line help for each command-line arg. The *BSC Users Guide* provides more details.

2. Compile and run the code using Verilator simulation or Bluesim simulation.

3. `src_BSV/Top.bsv` contains suggestions of several variations to try.

Ex-04-B-Bit-Vectors-Slicing: Slicing Bit-vectors

(Refer Section 4.6.3.)

Please see directory: `Exercises/Ex-04-B-Bit-Vectors-Slicing/`

1. Compile and run the code using Verilator simulation or Bluesim simulation.

2. `src_BSV/Top.bsv` contains suggestions of several variations to try.

Ex-04-C-Bit-Vectors-Operations: Operations on Bit-vectors

(Refer Section 4.9.)

Please see directory: `Exercises/Ex-04-C-Bit-Vectors-Operations/`

1. Build and run the code using Verilator simulation or Bluesim simulation.
2. Try variations: change bit-widths and values of a and b.
3. Try variations: change all `Bit #(n)` to `Int #(n)`.

Ex-04-D-Decode-Imm: Functions to decode immediates

(Refer Section 4.9.)

1. Write functions with the following types:

```
function Bit #(12) instr_imm_I (Bit #(32) instr);
function Bit #(12) instr_imm_S (Bit #(32) instr);
function Bit #(13) instr_imm_B (Bit #(32) instr);
function Bit #(20) instr_imm_U (Bit #(32) instr);
function Bit #(21) instr_imm_J (Bit #(32) instr);
```

   These pick out the "immediate" fields in I-, S-, B-, U- and J-type instructions, respectively, and rearrange the bits as appropriate to form the proper immediate value.

   Note: These functions exist already in the file `Code/src_Common/instr_Bits.bsv`. We recommend either writing your solutions without looking at those solutions or, study those solutions to understand them, put them aside, and then try writing them yourself from scratch.

2. Write a testbench that invokes each of these functions with 32-bit constants (representing instructions) and prints out the inputs and outputs. Compile and run them and visually inspect the outputs for correctness.

Ex-04-E-is-legal-XXX: Functions to check legality of instructions

(Refer Section 4.10.2.)

1. In the file `Code/src Common/Instr Bits.bsv` locate the functions:

```
instr_opcode()
instr_funct3()
is_legal_BRANCH()
```

   and make sure you understand them thoroughly.

2. Observe that the file contains similar functions, such as:

`is_legal_JAL()`	for the JAL opcode
`is_legal_JALR()`	for the (JALR opcode
`is_legal_OP()`	for LUI, AUIPC, ADD, SLT, OR, AND, ... opcodes
`is_legal_OP_IMM()`	for ADDI, SLTI, ..., ORI, ANDI, ... opcodes
`is_legal_LOAD()`	for LB, LH, LW, LBU, LHU opcodes
`is_legal_Mem()`	for SB, SH, SW opcodes

   which, in turn, use functions like:

   ```
   instr_rs1()
   str_rs2()
   str_rd()
   ```

   which are also in the file.

   Study the code and check that the functions are correct (refer to the "RV32I Base Instruction Set" listing in "Chapter 24 RV32/64G Instruction Set Listings" in the RISC-V Unprivileged ISA specification document, which lists 40 RV32I instructions).

   A visual check is sufficient, but feel free to write a testbench and tests and run them if you feel inspired to do so.

Ex-04-F-FSM-Testbench: FSMs for testbenches

(Refer Section 4.12.)

Please see directory: `Exercises/Ex-04-F-FSM-Testbench/`

1. Compile and run the code using Verilator simulation or Bluesim simulation.

2. Add more actions to the testbench to test more 32-bit values with `is_legal_BRANCH()`.

3. Delete the "`Bit #(7) opcode_BRANCH`" constant definition and the `is_legal_BRANCH()` function from Top.bsv. Instead, replace them with one line:

   ```
   import Instr_Bits :: *;
   ```

   Replace the existing Makefile (which just includes `../Include.mk`) with your own (by just copying `../Include.mk` as `./Makefile`).

   In your Makefile, edit the BSCPATH definition to include a path to where the `Instr_Bits.bsv` file is located. Replace:

   ```
   BSCPATH = src_BSV:+
   ```

   with

   ```
   BSCPATH = ../../Code/src_Common:src_BSV:+
   ```

   Make sure your Makefile still works: compile and run the test as in the previous exercises.

4. Modify the testbench to test more 32-bit values with other `is_legal_XXX()` functions.

5. The testbench constructs 32-bit BRANCH instructions with lines like this:

```
Bit #(32) instr_BEQ = {7'h0, 5'h9, 5'h8, 3'b000, 5'h3, 7'b_110_0011};
```

Instead, define a function to *create* a 32-bit BRANCH instruction, given a 13-bit immediate field, rs2 and rs1 values, and a funct3 field:

```
function Bit #(32) mkBRANCH_Instr (Bit #(13) pc_offset,
                                   Bit #(5)  rs2,
                                   Bit #(5)  rs1,
                                   Bit #(3)  funct3);
   ... <fill in this code> ...
endfunction
```

Refer to Figure 2.5 in the book (or to RISC-V Unpriviliged ISA spec document) for how a 13-bit PC-offset is encoded into the 12-bit immediate of a B-type instruction. Your code should also check that bit [0] of the PC-offset is zero.

In the testbench, use this function to create BRANCH instructions, instead of lines like this:

```
Bit #(32) instr_BEQ = {7'h0, 5'h9, 5'h8, 3'b000, 5'h3, 7'b_110_0011};
```

Also: instead of a constant like `3'b000`, use the symbolic name `funct3_BEQ` that is defined in the file `Instr_Bits.bsv`.

Ex-04-G-Enums-Muxes: Enums and Muxes

(Refer Sections 4.13-4.15.)

1. Write a testbench for the `instr_opclass()` function. Try it with different 32-bit instructions to produce the op class, and print out the op class. When printing the op class, try printing it as an integer, and also symbolically using `fshow()`.

2. Write a new version of the `instr_opclass()` function that expresses a parallel MUX instead of a priority MUX (see Figure 4.6). The key ideas are:

   - Define a value `x_CONTROL` that is either the bit-representation of `OPCLASS_CONTROL`, or a zero bitvector (of the same bit-width) if the boolean values of `is_legal_BRANCH()`, `is_legal_JAL()` and `is_legal_JALR()` are all false. We can implement this by converting that boolean result to a 1-bit value and then replicating it to the width of the `OpClass` type, then bitwise-AND'ing this with the bits of `OPCLASS_CONTROL`.
   - Similarly, define values `x_INT`, `x_MEM` and `x_SYSTEM` that are either `OPCLASS_INT`, `OPCLASS_MEM`, `OPCLASS_SYSTEM` or 0 depending on whether the instruction is an integer, memory or system instruction.
   - Finally, bitwise-OR the four `x_XXX` values together and convert the bits back into the `OpClass` type to produce the result.

3. Test your new version of the `instr_opclass()` function in your testbench.

4. There are three ways to write conditionals: if-then-else, "(eb ? e1 : e2)", and case expressions. Try these alternatives in any of the previous examples or in the Drum/Fife code files.

Chapter 5: Structs, tuples, memory requests and responses

Ex-05-A-Structs: structs

Please see the directory: `Exercises/Ex-05-A-Structs/`
In this exercise we are going to use some actual Drum/Fife code; specifically, these files:

```
Code/src_Common/Instr_Bits.bsv
Code/src_Common/Mem_Req_Rsp.bsv
```

(1) In `Makefile`, study and understand the new additions for this exercise.

Hint: think about the "`import`" statements in `src_BSV/Top.bsv`.

In order to build, you will likely have to edit the definition of `BOOK_REPO` in the `Makefile` to point at the correct directory in your environment.

Study and understand the code in `Top.bsv`.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

(2) Comment-out the `import Instr_Bits` and `import Mem_Req_Rsp` statements, one at a time, and try to recompile.

Compilation will fail: in each case, observe and understand the error message.

(3) Comment-out the `data` field definition in the struct definition.

Recompile, and observe and understand the compiler message.

Recompile and run. What is the difference between the value printed for the `data` field and what was printed earlier?

(4) Make two copies of the `$display()` statement. In the two new copies replace:

```
      fshow (mem_req)
with  mem_req
and   fshow_Mem_Req (mem_req)
respectively
```

Note, `fshow_Mem_Req()` is defined in the file `Code/src_Common/Mem_Req_Rsp.bsv`.

Recompile and run. Observe and understand the differences in the outputs from the original and from the two new statements.

(5) In file `Mem_Req_Rsp.bsv`, locate the function `fshow_Mem_Req_Type()`. Understand the code (see Section 11.3.2 which explains the `Fmt` type and the `$format()` function).

After the existing `$display()` statement, add a line like this:

```
$display ("mem_req.req_type = ",
          fshow_Mem_Req_Type (mem_req.req_type));
```

Recompile and run. Observe and understand the behavior.

Compare the value printed for the `req_type` field in the two `$display()` statements.

(6) After the `$display()` statement, add a line that updates the `mem_req_type` field of the struct by assigning `funct5_STORE` to the field (replacing its current value of `funct5_LOAD`). Then, replicate the `$display()` statement to print the new value of the struct.

Recompile and run (you should see output of original and new `$display`). Observe and understand the behavior.

Ex-05-B-Mem-Req-Rsp: Memory Requests and Responses

Please see the directory: `Exercises/Ex-05-B-Mem-Req-Rsp/`
In these exercises we will write a testbench that sends read requests to a memory model and examines the memory responses.

We are going to use some additional actual Drum/Fife code; specifically, these files:

`Code/src_Common/Utils.bsv`
(for `Initial_Params` struct definition)
`Code/src_Top/Mems_Devices.bsv`
(memory/devices model top-level)
`Code/src_Top/C_Mems_Devices.c`
(memory model implementation in C)
`Code/src_Top/UART_model.c`
(UART model implementation in C)
`Code/vendor/bluespec_BSV_Additional_Libs/Semi_FIFOF.bsv`
(for connections to memory model)

(1) In `Makefile`, study and understand the new additions for this exercise.

Hint 1: think about the `import` statements in `src_BSV/Top.bsv`.

Hint 2: In file `Code/src_Top/Mems_Devices.bsv` there are some `import "BDPI"` statements that refer to C functions that are defined in the C files mentioned above.

In order to build, you will likely have to edit the definition of `BOOK_REPO` in the `Makefile` to point at the correct directory in your environment.

Study and understand the code in `Top.bsv`.

When the program is run, the statement:

```
mems_devices.init (init_params);
```

will initialize the memory model and load it with data from `test.memhex32`.

Study and understand the contents of file `test.memhex32`. It contains 16 instructions from the `Hello World!` program, starting at address 0x_8000_0000.

Study the file:

`Tools/Hello_World_Example_Code/hello.RV32.bare.objdump`

and locate these 16 instructions. There you can see the assembly language for these instructions.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

(2) There are two `Top.bsv` files visible to the *bsc* compiler:

- In this exercise: `./src_BSV/Top.bsv)`
- In Drum/Fife: `Code/src_Top/Top.bsv`

When we compile, why does *bsc* pick the former and not the latter?

(3) In `./src_BSV/Top.bsv`, in module `mkTop`, just before `mkAutoFSM`, define a function to encapsulate and generalize the `a_req` action:

```
1       function Action a_req (Mem_Req_Type t,
2                              Mem_Req_Size s,
3                              Bit #(64)    a,    // addr
4                              Bit #(64)    d,    // data
5                              Bit #(64)    i);   // inum
6         ... body is a_req action with above arguments instead of constants ...
7       endfunction
```

Similarly, define a function to encapsulate and generalize the `a_rsp` action:

```
1       function Action a_rsp ();
2         ... body is a_rsp action ...
3       endfunction
```

Replace the original `a_req` and `a_rsp` actions with calls to these functions. Note, you won't need '`action-endaction`' brackets around these 1-line calls.

Recompile and run and verify that the output is the same as before.

(4) Add an additional memory request and response for the next instruction (`addr` = 0x_8000_0004, `inum` = 2):

```
1         // Original addr
2         a_req (..., 'h_8000_0000, ..., 1);
3         a_rsp;
4         // Next instruction addr
5         a_req (..., 'h_8000_0004, ..., 2);
6         a_rsp;
```

Recompile and run and verify that the output is as expected (matches contents of `test.memhex32`).

(5) Add more calls to `a_req()`/`a_rsp()` (or replace the earlier ones):

- Vary the `Mem_Req_Size` argument to be, variously, 1, 2 and 4 bytes.
- For the 2 and 4 bytes sizes, try both aligned and unaligned addresses.

In each case, recompile and run, and observe and understand the behavior.

(6) In each run so far, observe that the memory system announces the range of addresses that it implements for the memory model:

```
1    INFO: c_mems_devices_init
2      Mem system model
3       ADDR_BASE_MEM:  0x80000000 SIZEB_MEM:  0x10000000 (268435456) bytes
```

Note that the memory size is much larger than the contents of `test.memhex32`, which means that most of the locations remain uninitialized.

Add more calls to `a_req()`/`a_rsp()` (or replace the earlier ones):

- Try a LOAD from address 0x_8000_0040
  (This address is implemented by the memory model, but is not defined in `test.memhex32`).
- Try a LOAD from address 0x_9000_0000
  (This address is not implemented by the memory model).

In each case, recompile and run, and observe and understand the output.

(7) Add more calls to `a_req()`/`a_rsp()` (or replace the earlier ones):

- Do a STORE to some memory location.
- Then do a LOAD to read it back, and verify that the loaded value is indeed what we stored.
- Do LOADs with addresses on either side of the STORE address/size and verify that the STORE did not disturb those locations.

In each case, recompile and run, and observe and understand the output.

(8) In each run so far, observe that the memory system announces the range of MMIO addresses that it implements for the UART model:

```
1    INFO: c_mems_devices_init
2       ...
3       ADDR_BASE_UART: 0x60100000 SIZEB_UART: 0x00001000 (4096) bytes
```

The UART models a standard 16550 UART, which has several memory-mapped registers, but for this test we will only use 0x_6010_0000 which is the "transmit buffer". Every time we write a byte to the transmit buffer, it gets "transmitted" and displayed on the screen.

Add more calls to `a_req()`/`a_rsp()` (or replace the earlier ones): do a series of `a_req()` calls, one for each character in "`Hello World!\r\n`". Each call should be for a 1-byte STORE, to address 0x_6010_0000, with data being the ASCII hex code of that character.

Recompile and run, and observe and understand the behavior.

[Not so easy!] Why is "`Hello World!\r\n`" all printed together, instead of each character being printed as soon as we do the corresponding STORE? *Hint:* See `"// Write the char to the output line buffer"` and lines 526-535 that follow, in `UART_Model.c`.

(9) When printing "`Hello World!\r\n`" (*i.e.,* 14 characters), we alternated 14 `a_req()`'s with 14 `a_rsp()`'s, like this:

```
a_req(); a_rsp(); ...; a_req(); a_rsp(); // 14 pairs
```

Reorganize the code so that we first have all 14 `a_req()`'s followed by all 14 `a_rsp()`'s:

```
a_req(); ...; a_req(); // 14 requests a_rsp; ...; a_rsp(); // 14 responses
```

Recompile and run, and observe and understand the behavior.

Hint: the path in and out of memory is pipelined (so we can send in multiple requests before retrieving a response). But it has a finite depth, so when it "fills up", the pipeline gets stuck; it cannot accept any more requests until we drain some responses.

(10) Reorganize the code from this:

```
a_req(); ...; a_req(); // 14 requests a_rsp; ...; a_rsp(); // 14 responses
```

to this:

```
par seq a_req(); ...; a_req(); endseq seq a_rsp; ...; a_rsp(); endseq endpar
```

The '`par-endpar`' block allows its two components, each of which is a sequence of 14 actions, to run *concurrently.*

Recompile and run, and observe and understand the behavior.

(11) Study the C files to see how the memory model and UART model are implemented.

Chapter 6: Core functions

Ex-06-A-Fetch

Please see the directory: `Exercises/Ex-06-A-Fetch/`
This is a minimal "unit testbench" for `fn_Fetch()`. It instantiates the memory model used by Drum and Fife. The testbench uses `fn_Fetch()` to generate memory requests for simulating instruction-fetch.

As in Ex-05-B-Mem-Req-Rsp, the file 'test.memhex32' contains 16 instructions from the `Hello World!` program starting at address 0x_8000_0000. When the program is run, the statement:

```
mems_devices.init (init_params);
```

will initialize the memory model and load it with data from `test.memhex32`.

(1) Exercise baseline

Study and understand the code in `Top.bsv`.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

(2) Add a few more calls to `a_req()` and `a_rsp()` to fetch a few more instructions from successive addresses 0x_8000_0004, ...0008, ... *etc.*.

Recompile and run. Observe and understand the behavior.

(3) Memory access errors

Add a few more calls to `a_req()` and `a_rsp()` to fetch a few more instructions from:

- An unimplemented address (*e.g.,* 0x_9_0000_0000)
- A misaligned address (*e.g.,* 0x_8000_0002)

Recompile and run. Observe and understand the behavior.

(4) Scope of identifiers

In `Top.bsv`, the functions `a_req()` and `a_rsp()` are defined inside module `mkTop` after the instantiations of FIFOs `f_req` and `f_rsp`.

Move the instantiations of FIFOs `f_req` and `f_rsp` so that they come *after* the two functions.

Recompile and run. Observe and understand the behavior.

(5) Interfaces can be passed as function arguments

Move the two function definitions `a_req()` and `a_rsp()` to be *outside* and *before* module `mkTop`'s definition, as follows:

- Add an argument to `a_req()`:

  ```
  function Action a_req (... inum, FIFOF #(Mem_Reqs) f_reqs);
  ```

  and move this function out of the module, before the module, at the top-level of the package.
- Add an argument to `a_rsp()`:

  ```
  function Action a_rsp (FIFOF #(Mem_Rsps) f_rsps);
  ```

  and move this function out of the module, before the module, at the top-level of the package.

In `mkAutoFSM`,

- in each invocation of `a_req()`, add `f_reqs` as an argument.
- in each invocation of `a_rsp()`, add `f_rsps` as an argument.

Recompile and run. Verify that the behavior is the same as before.

This testbench performs a few so-called "directed tests" for `fn_Fetch()`. By generalizing the FSM to be a loop and to generate random addresses, the testbench can be made into a stronger "unit testbench" that tests many more possibilities. See Section "3.8.1 LFSR" in the *bsc* Libraries Reference Guide for facilities to generate pseudo-random numbers.

Ex-06-B-Decode

Please see the directory: `Exercises/Ex-06-B-Decode/`
This is a minimal unit testbench for `fn_Decode()`. It builds on the previous exercise, using an output of `fn_Fetch()` and the memory response as its inputs.

(1) Exercise baseline.

Study and understand the code in `Top.bsv`. It will be useful to "diff":

- `Ex-06-A-Fetch/src_BSV/Top.bsv`
- `Ex-06-B-Decode/src_BSV/Top.bsv`

to see what has changed.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

Verify that the output of `fn_Decode()` is correct for the fetched instruction. (Note, the program fetches a 32-bit instruction from address 0x_8000_0000 (the particular instruction can be seen in file `test.memhex32`). Locate this instruction in the "objdump" file:

`Code/Tools/Hello_World_Example_Code/hello.RV32.bare.objdump`

which shows the assembly language code for the instruction.)

(2) Repeat, for a few more instructions at other addresses.

(3) Repeat, for a few more instructions at:

- An unimplemented address (*e.g.,* 0x_9_0000_0000)
- A misaligned address (*e.g.,* 0x_8000_0002)

(4) Create a new `test.memhex32` file with just 4 bytes of data at 0x_8000_0000. Edit the testbench to fetch just this one instruction.

Repeat:

- Edit the 4 bytes to represent a different instruction that will take the test program into a different branch of the if-then-else's in `fn_Decode()`.
- Recompile and run. Verify that the output of `fn_Decode` is as expected.

Ex-06-C-Dispatch

Please see directory: `Exercises/Ex-06-C-Dispatch/`
This is a minimal unit testbench for `fn_Dispatch()`. It builds on the previous exercise, using the output of `fn_Decode()` as its input.

(1) Exercise baseline

Study and understand the code in `Top.bsv`. It will be useful to "diff":

- `Ex-06-B-Decode/src_BSV/Top.bsv`
- `Ex-06-C-Dispatch/src_BSV/Top.bsv`

to see what has changed.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

Verify that the output of `fn_Dispatch()` is correct for the fetched instruction. (Note, the program fetches a 32-bit instruction from address 0x_8000_0000 (the particular instruction can be seen in file `test.memhex32`). Locate this instruction in the "objdump" file:

`Code/Tools/Hello_World_Example_Code/hello.RV32.bare.objdump`

which shows the assembly language code for the instruction.)

(2) The output of `fn_Dispatch()` (type `Result_Dispatch`) has four sub-structs:

```
RR_to_Retire       to_Retire;
RR_to_EX_Control   to_EX_Control;
RR_to_EX           to_EX;
Mem_Req            to_EX_DMem;
```

The `to_Retire` sub-struct has a field `exec_tag` of type `Exec_Tag`.

The `to_EX_Control` sub-struct is only valid when `exec_tag == EXEC_TAG_CONTROL`.
The `to_EX` sub-struct is only valid when `exec_tag == EXEC_TAG_INT`.
The `to_EX_DMem` sub-struct is only valid when `exec_tag == EXEC_TAG_DMEM`.

The code currently `$display`'s all the sub-structs, whether valid or not. Replace this code with if-then-else's so that sub-structs are only `$display`'ed when they are valid.

Recompile and run to verify the code is running correctly.

(3) Repeat, for a few more instructions at other addresses.

(4) Repeat, for a few more instructions at:

- An unimplemented address (*e.g.,* 0x_9_0000_0000)
- A misaligned address (*e.g.,* 0x_8000_0002)

(5) Create a new `test.memhex32` file with just 4 bytes of data at 0x_8000_0000. Edit the testbench to fetch just this one instruction.

Repeat:

- Edit the 4 bytes to represent a different instruction that will take the test program into a different branch of the if-then-else's in `fn_Dispatch()`.
- Recompile and run. Verify that the output of `fn_Dispatch()` is as expected.

Ex-06-D-Ex-Control

Consider `fn_EX_Control` in Section 6.5.

(1) The BRANCH, JAL and JALR clauses set the `exception` field to true if the next PC is not aligned.

What would happen if we did not set the exception field here?

(See "Section 2.5 Control Transfer Instructions" in the Unprivileged ISA specification document [27] for a discussion of why we set it here.)

(2) Prove (informally) that the three-way if-then-else in `Fn_EX_Control` will catch all cases, *i.e.,* that we never need a final "`else`" clause. This requires reviewing `fn_Decode` and tracking the flow of information through the Register-Read-and-Dispatch stage (including `fn_Dispatch`), into `fn_Control`.

(3) In `Fn_EX_Control`, can we change the final "`else if`":

```
else if (is_JALR (instr)) begin
```

into a simple "`else`", *i.e.,* omit the the `is_JALR` check?

```
else begin
```

What might be the hardware implication of such a change?

(4) In the final section of `fn_EX_Control`, why do we have: `data: x.fallthrough_pc`?

Hint: review the semantics of JAL and JALR instructions.

What if this is a BRANCH instruction?

What if this is a JAL or JALR instruction with `rd` = 0?

Please see the directory: `Exercises/Ex-06-D-Ex-Control/`
This is a minimal "unit testbench" for `fn_Ex_Control()`. It instantiates the memory model used by Drum and Fife. It prepares input data for `fn_Ex_Control()` using `fn_Fetch()`, `fn_Decode()`, and `fn_Dispatch()`.

(5) Study and understand the code in `Top.bsv`. It will be useful to "diff":

- `Ex-06-C-Dispatch/src_BSV/Top.bsv`
- `Ex-06-D-Ex-Control/src_BSV/Top.bsv`

to see what has changed.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

Verify that the output of `fn_EX_Control()` is correct for the fetched instructions. (Note, the program fetches two 32-bit instructions from addresses 0x_8000_0000 and 0x_8000_003c (the particular instruction can be seen in file `test.memhex32`). Locate this instruction in the "objdump" file:

`Code/Tools/Hello_World_Example_Code/hello.RV32.bare.objdump`

which shows the assembly language code for the instructions.) The first of these two instructions is not a control instruction, and the second is a control instruction (BNE).

For the two BNE instructions, we invoke `a_rsp (0,0)` and `a_rsp (0,1)`. The two arguments represent the rs1 and rs2 values. BNE should be False for the first one, True for the second (so the next PC will be different for the two cases).

(6) Create a new `test.memhex32` file with just 4 bytes of data at 0x_8000_0000. Edit the testbench to fetch just this one instruction.

Repeat:

- Edit the 4 bytes to represent a different control instruction (try BEQ, BNE, BLT, BGE, BLTU, BGEU, JAL, and JALR).
- Recompile and run. Verify that the output of `fn_EX_Control()` is as expected.

Ex-06-E-Ex-Int

Consider `fn_IALU` in Section 6.6.

(1) Lines 16-53 contain two if-statements without "else" clauses. We could instead have written it this way:

```
1   Bit #(XLEN) y = 0;
2   if (opcode == opcode_OP) begin
3      ...
4      ... y = ...
5   end
6   else if (opcode == opcode_OP_IMM) begin
7      ...
8      ... y = ...
9   end
10  return y;
```

Discuss the hardware tradeoffs between writing it in these two ways. *H*ints: Consider:

- Sequentiality of if-then-else.
- Ability (or not) to prove exhaustiveness of conditions in nested if-then-else.
- Ability (or not) to prove mutual-exclusivity of conditions in nested if-then-else.
- Discussion in Section 4.14.1 on parallel and sequential multiplexers (mux). Note: in our code, we have explicitly coded a parallel mux.

(2) Justify the presence or absence of the "`pack`" operator in each "case" arm in `fn_IALU`.

(3) The RISC-V ISA has ADD, SUB and ADDI instructions, but no SUBI instruction. Why not?

(4) Suppose we want to extend `fn_IALU` so it also works when XLEN=64 (*i.e.,* for RV64I). What needs to change to accommodate this?

Hint: it only matters in the shift-amount of the shift instructions, where the shift-amount can be 6-bits wide instead of 5-bits (allowing a maximum of 63-bit shifts instead of 31 bits).

Please see the directory: `Exercises/Ex-06-E-Ex-Int/`
This is a minimal "unit testbench" for `fn_Ex_Int()`. It instantiates the memory model used by Drum and Fife. It prepares input data for `fn_Ex_Int()` using `fn_Fetch()`, `fn_Decode()`, and `fn_Dispatch()`.

(5) Study and understand the code in `Top.bsv`. It will be useful to 'diff':

- `Ex-06-C-Dispatch/src_BSV/Top.bsv`
- `Ex-06-D-Ex-Int/src_BSV/Top.bsv`

to see what has changed.

Compile and run the code using Verilator simulation or **Bluesim** simulation. Observe and understand the behavior.

Verify that the output of `fn_EX_Int()` is correct for the fetched instruction. (Note, the program fetches a 32-bit instruction from address 0x_8000_0000 (the particular instruction can be seen in file `test.memhex32`). Locate this instruction in the "objdump" file:

`Code/Tools/Hello_World_Example_Code/hello.RV32.bare.objdump`

which shows the assembly language code for the instruction.)

For the instruction, we invoke `a_rsp (0,0)`. The two arguments represent the rs1 and rs2 values (which may or may not be relevant for different int opcodes).

(6) Create a `test.memhex32` file with just 4 bytes of data at 0x_8000_0000. Edit the testbench to fetch just this one instruction.

Repeat:

- Edit the 4 bytes to represent a different integer instruction (try various Int opcodes, with different rs1 and rs2 values).
- Recompile and run. Verify that the output of `fn_EX_Int()` is as expected.

Chapter 7: BSV: Modules and Interfaces: Registers, Register Files and FIFOs

Ex-07-A-Interface-Transformers

(Please refer to Section 7.5.5.)

Write a similar function to view the input side of a `FIFOF` interface as a `FIFOF_I` interface.

Ex-07-B-Polymorphic-Types

(Please refer to Section 7.6.)

The types `Mem_Req` and `Mem_Rsp` (Sections 5.2 and 5.4.3) are monomorphic. Write polymorphic versions of these types that are parameterized on `xlen` for the `addr` field.

Ex-07-C-Synthesizable-Modules

(Please refer Section 7.6.1.)

(1) In `mkRISCV_GPRs_V` replace the explicit type declaration of `ifc` with the `let` keyword.

(2) Write a monomorphic wrapper for the BSV library `mkFIFOF` module that specializes it into a FIFO that only carries `Bool` values. Add an annotation so the wrapper becomes a separate Verilog module. Compile it and study the generated Verilog.

Chapter 8: RISC-V: Modules for GPRs and CSRs

Ex-08-A: GPR `x0` is constant zero

(Please refer Section 8.2.)

Why did the RISC-V ISA designers specify that GPR `x0` is constant zero?

Hint: Many computations require a constant zero value. For example, in C, a `for`-loop iterating over an array requires an intitial index of zero. Think about how you would produce a value zero in a register without using `x0`.

Why not reserve more GPRs for other common constant values (such as "1")?

Ex-08-B: GPR `x0` as a destination register

(Please refer Section 8.2.)

Why would one use `x0` as a destination register (`Rd` field of an instruction)?

Hint:

- Think about JUMP instructions where we are not interested in saving a "return address" (unconditional jumps, returns from subroutines, tail-recursion, continuation-passing-style, ...
- Think about CSR and AMO instructions where we are not interested in the value returned to a register.

Ex-08-C: GPR Register Files

(Please refer Section 8.2.)

(1) In `mkGPRs` we write the value when j is zero, but we ignore it on reads. Write a variant where, in `write_rd`, we alway write 0 when the index `rd` is zero, and the read methods no longer check if `rs1` or `rs2` are 0.

In this variant, what happens if we try to read `x0` before it is written for the first time?

Compare the circuitry generated in the original and in the variant. Why might we choose one over the other?

(2) In `mkGPRs` suppose we use `mkRegFile(1,31)` instead of `mkRegFileFull`. What needs to change to accommodate this?

Ex-08-D: CSRs

(Please refer Section 8.3.)

Write a testbench module that instantiates `mkCSRs` and tests its interface methods.

- Invoke the `mav_csrrxx()` method to write some CSRs and read them back and print the results.
  - Start by testing `mtvec`, `mepc`, `mcause`, and `mtval`.
  - Try reading and writing to a CSR address that is illegal (unimplemented CSR).
  - Try reading and writing to a CSR that is read-only (*e.g.,* `MVENDORID`).
- Use `mav_csrrxx()` to write `mtvec`; invoke `mav_exception` method and check that it returns what was written in `mtvec`. Use `mav_csrrxx()` to read `mepc`, `mcause` and `mtval` and check that they are correct.
- Use `mav_csrrxx()` to write `mepc`; invoke `read_epc` and check that we get the correct value.
- Invoke `ma_incr_instret`, then use `mav_csrrxx()` to read CSRs MINSTRET and MINSTRETH and check that we get the correct value.
- Invoke `set_TIME`, then use `mav_csrrxx()` to read CSRs TIME and TIMEH and check that we get the correct value.

[Advanced; optional] Why do we have CSRs TIME and TIMEH, but not MTIME and MTIMEH?
Hint: Read Section 3.2.1 in the Privileged ISA specification, as a starting point.

Chapter 10: RISC-V: The Drum CPU

Ex-10-A: Drum initialization

(Please refer Section 10.6.)

What might happen if we omitted the "`await!(rg_running)`" statement in the Drum CPU? (Try it in simulation!)

Hint: The FSM may start running before the PC has been initialized ...

Chapter 11: BSV: Verifying BSV designs

Ex-11-A: `FShow`

(Please refer Section 11.3.1.)

Write a small test program that constructs a struct value (of type `Decode_to_RR`, say), and then prints it in two different ways:

```
Decode_to_RR y = ... construct a Decode_to_RR struct value ...

$display ("y         is ", y);
$display ("fshow (y) is ", fshow (y));
```

Observe and comment on the difference.

Chapter 13: BSV: Rules and their Semantics

Ex-13-A: All `Actions` in a rule are simultaneous

(Please refer Section 13.4.)

Consider the following two alternative ways of writing a rule (that differ only in the order of the rule-body statements):

BSV
```
rule rl_r1 (... condition ...);
 x <= y + 1;
 y <= x + 2;
endrule
```

BSV
```
rule rl_r2 (... condition ...);
 y <= x + 2;
 x <= y + 1;
endrule
```

Sketch the semantic view (and possible hardware) for the two alternatives. Is there any semantic difference between the two rules?

If the values of registers `x` and `y` are 10 and 20 respectively, what are their values after the rule fires once? What would the answer(s) be with the following similar-looking C statements?

C/C++
```
{
    x = y + 1;
    y = x + 2;
}
```

C/C++
```
{
    y = x + 2;
    x = y + 1;
}
```

Chapter 14: RISC-V: the Drum unpipelined CPU (using Rules instead of `StmtFSM`)

Ex-14-A: Optimizing Trum Rules

(Please refer Section 14.2.1.)

Implement some or all the ideas in Section 14.2.1 and measure the results (circuit size, achievable clock speed, application program speed).

Chapter 15: RISC-V: the Fife Pipelined CPU: Principles

Ex-15-A: Bit-width of `rg_epoch`

(Please refer Section 15.2.)

We have described `rg_epoch` as a counter that is incremented on each recognition of a misprediction. If the register contents have type `Bit #(n)`, then this will wrap-around to 0 after 2^n increments. Is this a problem?

If `rg_epoch` contains a `Bit #(n)` value, how small can n be?

Ex-15-B: Hazards

(Please refer Section 15.3.)

For two consecutive instructions I_1 and I_2,

- I_1 may want to write register x_7 and I_2 may want to read x_7,
- I_1 may want to write register x_7 and I_2 may write to write x_7,
- I_1 may want to read register x_7 and I_2 may want to read x_7,
- I_1 may want to read register x_7 and I_2 may want to write x_7.

Above, we motivated scoreboards with the first scenario. What about the other three scenarios?

Ex-15-C: Write-Write Hazards

(Please refer Section 15.3.)

Write-write hazards can be treated just like read-after-write hazards. Alternatively the 1-bit in the scoreboard for a register (say x_7) can be generalized into an n bit up/downcounter, indicating the number of instructions that have been allowed into Execute pipelines that intend to write x_7. The Retire stage decrements this counter; the Register-Read stage stalls an instruction if these counters (for its input registers) are non-zero; and the Register-Read stage increments this counter for an instruction's destination register.

Implement a scoreboard module with this scheme. What should happen if a counter reaches its maximum value? How many bits should each counter have?

Ex-15-D: Write-Write ordering

(Please refer Section 15.3.)

In the counter-based scoreboard of the previous exercise, if there are multiple instructions in the Execute stages that intend to write x_7, in what order can those writes occur? What would be the consequences of a wrong order?

Hint: The answer is in Section 15.4.

Ex-15-E: Bypassing

(Please refer Section 15.3.2.)

(A) Implement a scoreboard unit with bypassing/short-circuiting as described in the above note.

Hint: Needs BSV "Concurrent Registers" (Section 18.3).

(B) What are the implications of bypassing/short-circuiting on the length of combinational paths in a design, and the consequent effect on achievable clock frequency?

Ex-15-F: Store Buffers

(Please refer Section 15.5.)

(A) When the Retire stage sends a "commit/discard" message, how do we know that it applies to the pending STORE at the head of the store-buffer queue?

(B) For a speculative LOAD, its value may come from memory or from the store-buffer. In what order should it scan these sources?

(C) Does the Retire stage need to send both "commit" and "discard" messages? If requests are accompanied by an instruction-number, the Retire stage could only send "commit" messages accompanied by the instruction-number. The store-buffer can discard all pending STOREs with earlier instruction numbers.

Discuss the implications of such a design on the size of the store-buffer.

(D) Register reads/writes and memory reads/writes both raise similar "hazards".

Explore a "scoreboard"-like solution for memory accesses, and comment on the possibilities and difficulties for doing this.

Explore a "store-buffer"-like solution for register accesses, and comment on the possibilities and difficulties for doing this.

Chapter 16: RISC-V: the Fife pipelined CPU code

Ex-16-A: Rule Atomicity

(Please refer Section 16.6.2.)

Consider this hypothetical scenario: suppose the `stall` condition is false. Then, we need to dequeue `f_Decode_to_RR` and do one or more enqueues into `f_RR_to_Retire` and `f_RR_to_EX_Control`, `f_RR_to_EX_Int` or `f_DMem_S_req.enq`. Is it possible that we perform the dequeue and then are unable to perform the enqueue(s) because the corresponding output FIFO happens to be full?

Hint: rule atomicity

Ex-16-B: Rule conditions

(Please refer Section 16.6.2.)

Write a boolean expression representing the overall firing condition for the rule. Briefly: all FIFO-modifying actions (dequeue, enqueue) have implicit conditions, but for each FIFO, that condition is only relevant if the conditions on the surrounding if-then-else's select that action.

Ex-16-C: Debugging Stalled Rules

(Please refer Section 16.6.2.)

When debugging the implementation, it is useful to know if, due to some coding mistake, rule `rl_RR_Dispatch` is stalled forever. For example, for some instruction with an rd, if the Retire stage did not send back the rd's scoreboard-release, that register will be forever "busy".

Add a register to count consecutive stalls, initially 0. In the rule, whenever we successfully dispatch an instruction, reset the counter to 0. Whenever we stall, increment the stall counter, and if the stall-counter reaches some chosen threshold value, prints debugging messages and executes `$finish` to terminate simulation.

Ex-16-D: CSR method conflict

(Please refer Section 16.10.3.1.)

The provided implementation of the CSR registers (Section 8.3) will in fact prevent the two methods `csrs.mav_csrrxx()` and `csrs.mav_exception()` from being invoked in a single action, *i.e.,* in a single rule, due to method-ordering constraints. Try this experiment: copy the body of `rl_exception` into `rl_Retire_CSRRxx`; try compiling it with `bsc`, and observe the compiler's complaint message.

Ex-16-E: Resolving a CSR method conflict

(Please refer Section 16.10.3.1.)

Modify the CSRs module method `mav_csrrxx()` to combine the actions of the original `mav_csrrrxx()` with the actions of `mav_exception()`. The result type will have to be extended (say, a 3-tuple) to also return the trap-vector PC in case of an exception. Modify `rl_Retire_CSRRxx` so that it immediately does a Fetch-redirect with the trap-vector PC in case of an exception. There is no longer any need to shift out of `MODE_PIPE` into `MODE_EXCEPTION`.

This will save a cycle in handling CSRRxx instructions. But what is the impact on hardware complexity in the `mkCSRs` module?

Ex-16-F: ECALL/EBREAK

(Please refer Section 16.10.3.3.)

In the exercises in Section 16.10.3.1, we discussed why, in `rl_Retire_CSRRxx` we postponed exception handling to the separate rule `rl_exception`: `rl_Retire_CSRRxx` itself accesses the CSRs, and `rl_exception` accesses the CSRs, and we wanted to avoid contention.

Here, `rl_Retire_ECALL_EBREAK` does not otherwise access the CSRs, so there is no such contention, so why not handle the exception directly here (copy the body of `rl_exception` here) and save a cycle? Discuss the impact on hardware structure. Try it, examine the generated Verilog, and examine the performance impact (which will depend on the RISC-V program being run, of course).

Ex-16-G: Update Rd *vs.* Retire Store Buffer

(Please refer Section 16.10.6.1.)

Inside `rl_Retire_EX_DMem` we invoke "`fa_update_rd()`" always, but we invoke "`f_retire_store_buf()`" only when there was no exception. Justify this.

Ex-16-H: Retiring Deferred DMem

(Please refer Section 16.10.6.3.)

This rule fields a memory response, but does not send any commit/discard message to the store-buffer. Why not?

Chapter 17: BSV: Rules and Methods II: Improved performance with CRegs (Concurrent Registers)

Ex-17-A: `mkPipelineFIFOF`

(Please refer Section 18.5.1.)

The example code in the text for `mkPipelineFIFOF` had a particular choice of indexes for the dequeue-side methods, the enqueue-side methods and for the clear method, respectively, corresponding to the first row of the table below:

0	1	2
`notEmpty, first, deq`	`notFull, enq`	`clear`
`notEmpty, first, deq`	`clear`	`notFull, enq`
`clear`	`notEmpty, first, deq`	`notFull, enq`

What changes in the behavioral semantics (ordering) of the FIFO if we choose the indexes in the second or third row of the table instead (moving the `clear` method between the dequeue and enqueue sides, or ahead of both of them? *Hint:* think about simultaneous invocation of `clear` with either an `enq`, or a `deq`, or both.

Ex-17-B: Composition of `mkBypassFIFOF` and `mkPipelineFIFOF`

(Please refer Section 18.5.3.)

What happens if we reverse the composition—use `mkPipelineFIFOF` upstream and `mkBypassFIFOF` downstream?

- How many ticks will it take for a datum to traverse the pair?
- Will `rl_P` and `rl_C` still be conflict-free (and therefore can fire in either logical order)?
- Are there any combinational paths through the pair?
- Consider combinational paths in rule `rl_P` ending at the `enq` method of the first FIFO, and combinational paths in rule `rl_C` starting at the `first` method of the second FIFO. What is the difference between these when we compose `mkBypassFIFOF` $\longrightarrow$ `mkPipelineFIFOF` *vs.* `mkPipelineFIFOF` $\longrightarrow$ `mkBypassFIFOF`?

Index of BSV topics

/*, start of block comment (until-*/), 4-3
//, start of comment-to-end-of-line, 4-3
? (don't care literal value), 4-6
$display statements, 3-1

AAAA_AAAA, the default don't care value, 4-6
Action
    action-endaction blocks, 9-5
    as a first-class type, 10-5
    Type of actions, 9-5
    Type of expression with side-effect and no return value, 4-14, 7-5
ActionValue
    Type of experssion with side-effect and return value, 4-14
assertions for debugging, 11-5

BH (alternate Haskell-like syntax), 1-8
Binary notation for integer literals, 4-2
Bit
    Integer type, 4-8
Bit Vectors, 1-9, 4-5
    extend, 4-7
    truncate, 4-7
    zeroExtend, 4-7
    operators on, 4-7
    slices of, 4-6
Bool, 4-8
    operators on, 4-8
    value True and False, 4-8
bus, 1-9
    hardware bundle of wires, 4-12

case expression or statement, 4-20
Combinational circuits, 1-9, 4-12
    data types, 4-15
    purity, 4-13
Combinational primitives, 4-12
Comments
    block, from /* to matching */, 4-3
    to end-of-line, starting with //, 4-3
Conditional compilation, 4-21
Connecting FIFOs, 7-11

Data types, 4-3
deriving
    Bits, 5-4
    Fshow, 5-4
deriving Bits, 4-16
deriving Eq, 4-16
deriving FShow, 4-16
Digital Abstraction, The, 1-10
$display has Action type, 9-5
Don't care literal value ?, 4-6
Drum
    as an FSM, 9-2
    as an FSM concurrent with its memory system, 9-3
DUT, 11-1

endpackage, 3-3
enum types, 4-15
export statement, 3-7
Exporting types abstractly, 3-7

False (Bool constant), 4-2
field
    of a struct, 5-1
Fife
    as a set of concurrent FSMs, 9-3
FIFO
    interface (deprecated in favor of mkFIFOF), 7-8
FIFOF
    interface, 7-8
    interface methods, 7-8
    mkBypassFIFOF
        module (constructor), 7-9
    mkFIFOF
        instantiation, 7-9
        module (constructor), 7-9
        reset value, 7-9
    mkPipelineFIFOF
        module (constructor), 7-9
    mkSizedFIFOF
        module (constructor), 7-9
    pop method, 7-8
    strongly-typed, 7-10
    type of stored value, 7-8

`FIFOF_I`
interface transformer from `FIFOF`, 7-11
Semi-FIFO interface (`enq` subset/view of `FIFOF`), 7-10
`FIFOF_O`
interface transformer from `FIFOF`, 7-11
`pop_o` method, 7-10
Semi-FIFO interface (`first`/`deq` subset/view of `FIFOF`), 7-10
Finite State Machines, 9-1
First class types, 10-6
`Fmt`
formatted object, 11-4
Formal verification, 12-4
Formatted output using `Fmt` objects, 11-4
`FShow`
Standard Typeclass containing the `fshow()` function, 11-3
`fshow`
Standard overloaded function producing `Fmt` objects for various types, 11-3
FSMs, 9-1
communication between concurrent FSMS, 9-3
communication between states, 9-3
concurrent *vs.*. sequential, 9-2
sequential *vs.*. concurrent, 9-2
`Stmt`: type of argument to FSM module constructors, 9-6
Fully-qualified imported names, 3-7
functions
always inlined, 4-10
any argument and result type, 4-11
application, 4-13
definition, 4-9
for circuit computation, 4-10
for circuit structure, 4-10
higher-order, 4-11, 10-6
recursive, 4-10
zero argument, 10-6
zero hardware cost, 4-10

Haskell
BH alternate Haskell-like syntax for **BSV**, 1-8
Influence on **BSV**, 1-8, 4-14, 10-6
monadic types similarity, 4-14
Hexadecimal notation for integer literals, 4-2
Higher-order functions, 10-6

Identifer syntax, 4-2
Identifiers
Enum constant: initial uppercase letter, 4-2
First letter lower- or uppercase, 4-2
Ordinary: initial lowercase letter, 4-2
Type: initial uppercase letter, 4-2
if-then-else, 4-17
nested, 4-18
Ordinary expression *vs* `StmtFSM` process, 9-7
`export` statement, 3-7
Importing C and C++ functions, D-1
`Int`
Integer type, 4-8
Integer types `Bit`, `Int`, `UInt`, `Integer`, 4-8
Interface
`FIFOF` FIFO interface, 7-8
`Reg` register interface, 7-5
`RegFile` register file interface, 7-7
type, 7-2
interface
declaration, 3-8
`interface`
declaration (a module's API)
typical components in, 3-8
interface definition
typical components in, 3-12
Interface transformer functions, 7-11
internal behavion: rules, 7-2

Kinds
Numeric, 4-3, 4-21
Value, 4-3

`let`
binding an identifier with implicit type declaration, 4-5, 5-3
in `Action` blocks, 9-6
Literals
Binary integer notation, 4-2
Hexadecimal integer notation, 4-2

Macros
for preprocessor, 4-21
member
of a `struct`, 5-1
Memory, 1-10
access latency (cycles/ticks), 1-11
bandwidth, 1-12
Hierarchy, 1-11
L1 cache, level 1 cache, 1-11
L2 cache, level 2 cache, 1-11
latency (cycles/ticks), 1-11
RAM (Random Access Memory), 1-11
Method
invocation of module method, 7-4
`mkAutoFSM` module in `StmtFSM` library package, 9-8

mkBypassFIFOF
  module (constructor), 7-9
mkConnection for connecting compatible interfaces, 7-11
mkFIFOF
  instantiation, 7-9
  module (constructor), 7-9
  reset value, 7-9
mkPipelineFIFOF
  module (constructor), 7-9
mkReg
  instantiation, 7-5
  module (constructor), 7-5
  reset value, 7-5
mkRegU
  module (constructor), 7-5
mkSizedFIFOF
  module (constructor), 7-9
module, 7-1
  (persisitent) state, 7-1
  behavior, 7-4
  constructor, 7-4
  declaration, 3-8
  instance, 7-4
  instantiation, 7-4
  interaction via methods, 3-14
  interface, 7-1, 7-4
  method invocation, 7-4
  mkBypassFIFOF module (constructor), 7-9
  mkFIFOF module (constructor), 7-9
  mkPipelineFIFOF module (constructor), 7-9
  mkReg module (constructor), 7-5
  mkRegFileFull module (constructor), 7-7
  state, 7-4
module
  declaration
    typical components in, 3-9
module instance hierarchy, 3-13
Monadic types
  Haskell similarity, 4-14
Monomorphic Types (types without type-variables), 7-14
multiplexers, 4-17
  cascaded/serial/priority, 4-17
  parallel, 4-18
MUX, 4-17

Numeric Kind, 4-3, 4-21

Operators on Bit Vectors, 4-7
Overloading: Typeclasses and typeclass instances, 7-13

package, 3-3
packing of struct fields and vector elements, 5-4
parameterization, 4-20
Pattern matching
  match statement for tuples, 5-6
Polymorphic Types, 7-14
  Type variables (identifiers beginning with lower-case letter), 7-14
pop
  FIFOF method, 7-8
pop_o
  FIFOF_O method, 7-10
Preprocessor macros, 4-21
printf-style debugging, 11-2
Propagation delay, 4-13

RAM (Random Access Memory), 1-11
_read: register method, 7-5
RegFile register file interface, 7-7
Register, 7-4
  <= register assignment, 7-6
  implicit register read, 7-6
  mkReg
    instantiation, 7-5
    module (constructor), 7-5
    reset value, 7-5
  mkRegU
    module (constructor), 7-5
  _read method, 7-5
  Reg register interface, 7-5
  strongly-typed, 7-5
  _write method, 7-5
Register file, 7-7
  methods, 7-7
  mkRegFileFull
    instantiation, 7-7
    module (constructor), 7-7
    reset value, 7-7
  RegFile interface, 7-7
  RegFile register file interface, 7-7
  type of index, 7-7
  type of stored value, 7-7
replicate vector library function to create a vector value, 16-11
rule, 9-4
  vs. StmtFSM, 13-10
  Body, of type Action, 13-1
  CAN_FIRE condition (overall condition), 13-2
  Explicit condition, 13-1
  Implicit condition (READY), 13-2
  internal behavior, internal process, 7-2
  overall condition (CAN_FIRE), 13-2

prioritizing explicitly, 16-7
READY condition (implicit condition), 13-2
Semantics of collection of rules, 13-5
Semantics of individual rule, 13-2
Syntax and types, 13-1
the fundamental behavioral construct in **BSV**, 7-12
typical components in, 3-11

Semi-FIFO
`FIFOF_I` semi-FIFO interface, 7-10
`FIFOF_O` semi-FIFO interface, 7-10
State elements, 1-9
static elaboration, 3-13
`StmtFSM`
`for`-loop repetition, 9-8
`mkAutoFSM` module, 9-8
`par` blocks (fork-join concurrency), 9-8
vs. rules, 13-10
an abtstraction of rules, 9-4
`await`: pausing until some condition, 9-7
For testbenches, 4-15
`if-then-else`: process conditional, 9-7
in testbenches, 9-8
`seq .. endseq`: sequences of actions, 9-6
structured process, 9-4
translation into rules, 13-9
`while`-loop repetition, 9-7
struct
entire struct values, 5-2
field assignment/update, 5-3
field selection, 5-3
hetoregeneous collection of values, 5-1
nested, 6-2
packing of fields, 5-4
type declaration, 5-2
`synthesize`
`(* synthesize *)` attribute on modules for Verilog generation, 7-4, 7-14

Testbench, 11-1
Testbenches, 4-15
FSMs, 4-15
with `StmtFSM`, 4-15
`True` (`Bool` constant), 4-2
`truncate`, operation to shrink bit-width, 6-7
Tuples, 5-6
Type variables
Identifiers beginning with lower-case letter, for polymorphism, 7-14
Typeclass
instance of, 4-16, 7-13

Typeclasses, 4-16
BSV's "overloading" mechanism, 7-13
Types
"value types" (not `Action` or `ActionValue`), 4-15
`ActionValue`, 4-14
`Action`, 4-14
`Bit#(n)`, 4-5
`Bool`, 4-8
Data types (sets of values), 4-3
First class, 10-6
interface, 7-2
of combinational circuits, 4-15
synonyms, 4-21
`valueOf`: value of a numeric type, 4-4

`Integer`
Unbounded Integer type (non-synthesizable), 4-8
`UInt`
Unsigned integer type, 4-8

Value Change Dump (VCD) waveform dumping, 11-5
Value Kind, 4-3
`valueOf` pseudo-function from numeric types to numeric values, 4-4
values
abstract entity coded in bits, 1-9
VCD waveform dumping, 11-5
vector
library data type, 16-11
library `replicate` function, 16-11
of `n` bits *vs.* `Bit#(n)`, 16-11
packing of fields, 5-4
representation in bits, 16-11

Waveform dumping (VCDs), 11-5
Wraparound arithmetic for fixed-width integer types, 4-8
`_write`: register method, 7-5

X values
Notation for unassigned values in Verilog (no such concept in BSV), 4-6

Index of RISC-V topics

Address alignment, 5-8
Architectural state, 2-17

Balancing concurrent paths in a pipeline, 16-9
BRAM (Block RAMs in FPGAs, which are SRAMs), 19-15
Branch prediction, 15-2
Bubble in a pipeline, 15-5
Bypassing, 15-6

Cache, 19-15
    hit/miss, 1-11
    Miss penalty, 1-11
    miss penalty, 19-15
    non-blocking, 19-15
cache
    hit-under-miss, 19-2
    non-blocking, 19-2
Commit the side-effects of an instruction, 15-4
CPI: cycles per instructions, 2-16

Decode function (`fn_Decode`), 6-3
Dispatch function (`fn_Dispatch`), 6-8
DMem, Data Memory5-7
DMem (data memory), 5-7
DRAM (Dynamic RAM), 1-12
DRAM (dynamic RAM), 19-15
Drum
    Adding instructions for M, F/D and Custom extensions, 10-15
    Adding instructions for the A-extension, 10-16
    CPU interface, 10-1
    CPU module actions, 10-5
    CPU module behavior, 10-13
    Skeleton module, 10-2

Execute Control function (`fn_EX_Control`), 6-12
Execute Integer function (`fn_EX_Int`), 6-13

Fetch function (`fn_Fetch`), 6-1
Fife
    Skeleton module, 10-2
`fn_Decode` (Decode function), 6-3
`fn_Dispatch` (Dispatch function), 6-8
`fn_EX_Control` (Execute Control function), 6-12
`fn_EX_Int` (Execute Integer function), 6-13
`fn_Fetch` (Fetch function), 6-1
Formal verification, 12-4

Golden Reference Model, 12-2
GPRs: general purpose registers, 8-1
`GPRs_IFC` interface for `mkGPRs`, 8-2

Harvard architecture, 5-7
    Self-modifying code, 5-7
Hazards
    Scoreboard for managing, 15-4

IMem, Instruction Memory5-7
IMem (instruction memory), 5-7
In-flight memory transactions, 16-9
Instruction ordering
    tags for, 15-6
IPC: instructions per cycle, 2-16
ISA
    Architectural State, 2-2
    Assembly Language, 2-2
    Contract between software and hardware implementations, 2-2
    Enables software portability, 2-2
    Evolution of, 2-19
    Extensions to, 2-19
    Formal Specification in Sail, 2-2
    Instruction semantics, 2-2
    Instruction Set, 2-2
    Instruction Set Architecture, 2-1
    Instruction Set encoding in bits, 2-2
    PC, Program Counter, 2-2
    Program Counter, PC, 2-2
    Register File, 2-2
    RV32I base integer instructions, 2-7
    Sail Formal Specification language, 2-2

JIT (Just-in-time compiling), 5-7
Just-in-time compiling (JIT), 5-7

Latency-tolerant design, 10-16

Machine code, 2-2
Memory
  Address alignment, 5-8
  BRAM (Block RAMs in FPGAs, which are SRAMs), 19-15
  Cache, 19-15
    non-blocking, 19-15
  DRAM (Dynamic RAM), 1-12
  DRAM (dynamic RAM), 19-15
  latency, 16-9
  path viewed as a pipeline/queue, 1-12
  Physical address, 19-16
  pipelined, 16-9
  Request, 5-2
  Response, 5-9
  split-phase access, 1-12
  SRAM (Static RAM), 1-12
  SRAM (static RAM), 19-15
  Virtual address, 19-16
  Virtual Memory, 19-16
Micro-architecture, 2-2
  Multi-core, 2-2
  Multi-threaded, 2-2
  Out-of-order, 2-2
  Pipelining, 2-2
  Speculation, 2-2
  Superscalar, 2-2
microarchitecture
  out-of-order, 19-2
  superscalar, 19-2
misprediction
  manage with epochs, 19-9
  penalty, 19-9
  shadow, 19-9
Misprediction (wrong path instructions), 15-2
`mkGPRs` a module wrapper around library `RegFile`, 8-2
`mkGPRs_synth` a module wrapper for `mkGPRs` for synthesizability, 8-3
MMIO (Memory-Mapped Input Output), 15-9

Out-of-order execution, 15-7
out-of-order microarchitecture, 19-2

PC prediction, 15-2
  BTB with hysteresis, 19-11
  BTB, control-flow-indexed, 19-12
  BTBs (Branch Target Buffers), 19-11
  RAS (Return Address Stack), 19-12
  redirection, 15-3
PC-prediction
  misprediction, 19-9
Physical address, 19-16
Pipeline
  balanced and unbalanced, 19-6
  Bubble, 15-5
  slack, 19-6
Pipeline trace, 19-3
Prediction, 15-2

redirection on misprediction, 15-3
Reorder buffers, 15-7
`rg_epoch` register for managing mispredictions, 15-4

Scoreboard, to manage register read/write hazards, 15-4
Short-circuiting (bypassing), 15-6
Simulator
  trusted, functional, 12-2
Speculation of instructions, 15-4
Speculative instruction, 15-4
Split-phase memory access, 1-12
Split-phase memory transaction, 1-12
SRAM (Static RAM), 1-12
SRAM (static RAM), 19-15
Store Buffer, 15-8
superscalar microarchitecture, 19-2

Tags
  `EXEC_TAG_CONTROL`, 15-6
  `EXEC_TAG_DIRECT`, 15-6
  `EXEC_TAG_DMEM`, 15-6
  `EXEC_TAG_IALU`, 15-6
  for proper instruction ordering, 15-6
Tandem verification
  symmetric, 12-8
TCM (tightly coupled memory), 19-15
Timing
  slack, 19-6
TLB
  miss penalty , 19-16
TLB translation lookaside buffer, 19-16
Trusted functional simulator, 12-2

Virtual address, 19-16
Virtual Memory, 19-16
  Page Table, 19-16
  Page Table Walk, 19-16
  Pages, 19-16
  PTW (Page Table Walk), 19-16
  TLB translation lookaside buffer, 19-16

Wrong-path due (mispredicted instructions), 15-2

`x0`: Special “always zero” register, 8-1

Bibliography

[1] F. Baader and T. Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998. ISBN 0 521 45520 0.

[2] Bluespec, Inc. BSV Guide, 2022 (first version 2000).

[3] E. Borin. *An Introduction to Assembly Programming with RISC-V*. 2021 (Revised: May 9, 2022). PDF online: https://riscv-programming.org/book.html.

[4] K. Chandy and J. Misra. *Parallel Program Design: A Foundation*. Addison Wesley, 1988.

[5] P. Dabbelt, M. Clark, and A. Bradbury. RISC-V Assembly Programmer's Manual, Recent update: Jun 29, 2023. Online: https://github.com/riscv-non-isa/riscv-asm-manual/blob/master/riscv-asm.md.

[6] P. David and W. Andrew. *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017. ISBN-10: 0999249118, ISBN-13: 978-0999249116. Available in bookstores.

[7] E. W. Dijkstra. *A Discipline of Programming*. Prentice Hall, 1976.

[8] A. J. Dos Reis. *RISC-V Assembly Language*. 2019. ISBN-10: 1088462006, ISBN-13: 978-1088462003. Available on Amazon.com.

[9] J. L. Hennessy and D. A. Patterson. *Computer Architecture: A Quantitative Approach, 6th Edition*. Morgan Kaufmann, November 23 2017. The Morgan Kaufmann Series in Computer Architecture and Design. eBook ISBN 9780128119068. paperback: 978-0128119051.

[10] IEEE. IEEE Standard VHDL Language Reference Manual, IEEE Std 1076-1993, 2002.

[11] IEEE. IEEE Standard Verilog (R) Hardware Description Language, 2005. IEEE Std 1364-2005.

[12] IEEE. IEEE Standard for Standard SystemC Language Reference Manual, January 9 2012. IEEE Std 1666-2011.

[13] IEEE. IEEE Standard for System Verilog—Unified Hardware Design, Specification and Verification Language, 21 February 2013. IEEE Std 1800-2012.

[14] J. Kamperman. *Compilation of Term Rewriting Systems*. PhD thesis, University of Amsterdam, 1996.

[15] J. Klop. *Term Rewriting Systems*, volume 2, pages 1–116. Oxford University Press, 1992.

[16] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Professional (Pearson Education), 2002.

[17] Metayer, C. and J.-R. Abrial and L. Voisin. The Event-B Language, May 31 2005. http://rodin.cs.ncl.ac.uk/deliverables.htm.

[18] D. A. Patterson and J. L. Hennessy. *Computer Organization and Design RISC-V Edition (2nd Edition): The Hardware Software Interface*. Morgan Kaufman, 2020. The Morgan Kaufmann Series in Computer Architecture and Design. ISBN-10: 0128203315, ISBN-13: 978-0128203316. Available in bookstores.

[19] S. Peyton Jones (Editor). *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press, May 5 2003. haskell.org.

[20] D. L. Rosenband. The Ephemeral History Register: Flexible Scheduling for Rule-Based Designs. In *Proc. MEMOCODE'04*, June 2004.

[21] D. L. Rosenband. *A Performance Driven Approach for Hardware Synthesis of Guarded Atomic Actions*. PhD thesis, Dept. of Electrical Engineering and Computer Science, Massachusetts Institute of Technology, August 2005.

[22] SHAKTI Development Team @ iitm '20 shakti.org.in (Indian Institute of Technology, Madras, India). RISC-V ASSEMBLY LANGUAGE, Programmer Manual Part I, 2020. PDF online: https://shakti.org.in/docs/risc-v-asm-manual.pdf.

[23] Terese. *Term Rewriting Systems*. Cambridge University Press, 2003. Cambridge Tracts in Theoretical Computer Science 55.

[24] Various authors. Bluespec Compiler (BSC) User Guide, 2024 (revised frequently).

[25] Various authors. BSV Language Reference Guide, 2024 (revised frequently).

[26] Various authors. BSV Libraries Reference Guide, 2024 (revised frequently).

[27] A. Waterman and K. Asanović. The RISC-V Instruction Set Manual Volume I: Unprivileged ISA, December 13 2019. Document Version 20191213.. PDF online (and newer versions, if any): https://riscv.org/technical/specifications/.

[28] A. Waterman, K. Asanović, and J. Hauser. The RISC-V Instruction Set Manual Volume II: Privileged Architecture, December 4 2021. Document Version 20211203. PDF online (and newer versions, if any): https://riscv.org/technical/specifications/.